Python Essentials for ICERM@2025 Summer Program

Contents

1 Introduction to Python

Lab Objective: *Python is a powerful general-purpose programming language. It can be used interactively, allowing for very rapid development. Python has many powerful scientific computing tools, making it an ideal language for applied and computational mathematics. In this introductory lab we introduce Python syntax, data types, functions, and control flow tools. These Python basics are an essential part of almost every problem you will solve and almost every program you will write.*

Getting Started

Python is quickly gaining momentum as a fundamental tool in scientific computing.

Python Basics

Arithmetic

Python can be used as a calculator with the regular `+`, `-`, `*`, and `/` operators. Use `**` for exponentiation and `%` for modular division.

```
>>> 3**2 + 2*5                      # Python obeys the order of operations.
19

>>> 13 % 3                          # The modulo operator % calculates the
1                                   # remainder: 13 = (3*4) + 1.
```

In most Python interpreters, the underscore character `_` is a variable with the value of the previous command's output, like the ANS button on many calculators.

```
>>> 12 * 3
36
>>> _ / 4
9.0
```

Data comparisons like < and > act as expected. The == operator checks for numerical equality and the <= and >= operators correspond to $\leq$ and $\geq$, respectively. To connect multiple boolean expressions, use the operators `and`, `or`, and `not`.[1]

```
>>> 3 > 2.99
True
>>> 1.0 <= 1 or 2 > 3
True
>>> 7 == 7 and not 4 < 4
True

>>> True and True and True and True and True and False
False
>>> False or False or False or False or False or True
True
>>> True or not True
True
```

Variables

Variables are used to temporarily store data. A **single** equals sign = assigns one or more values (on the right) to one or more variable names (on the left). A **double** equals sign == is a comparison operator that returns `True` or `False`, as in the previous code block.

Unlike many programming languages, Python does not require a variable's data type to be specified upon initialization. Because of this, Python is called a *dynamically typed* language.

```
>>> x = 12                          # Initialize x with the integer 12.
>>> y = 2 * 6                       # Initialize y with the integer 2*6 = 12.
>>> x == y                          # Compare the two variable values.
True

>>> x, y = 2, 4                     # Give both x and y new values in one line.
>>> x == y
False
```

Functions

To define a function, use the `def` keyword followed by the function name, a parenthesized list of parameters, and a colon. Then indent the function body using exactly **four** spaces.

```
>>> def add(x, y):
...     return x + y                # Indent with four spaces.
```

[1] In many other programming languages, the `and`, `or`, and `not` operators are written as `&&`, `||`, and `!`, respectively. Python's convention is much more readable and does not require parentheses.

Achtung!

Many other languages use the curly braces `{}` to delimit blocks, but Python uses whitespace indentation. In fact, whitespace is essentially the only thing that Python is particularly picky about compared to other languages: **mixing tabs and spaces confuses the interpreter and causes problems**. Most text editors have a setting to set the indentation type to spaces so you can use the tab key on your keyboard to insert four spaces (sometimes called *soft tabs*). For consistency, **never** use tabs; **always** use spaces.

Functions are defined with *parameters* and called with *arguments*, though the terms are often used interchangeably. Below, `width` and `height` are parameters for the function `area()`. The values `2` and `5` are the arguments that are passed when calling the function.

```
>>> def area(width, height):        # Define the function.
...     return width * height
...
>>> area(2, 5)                      # Call the function.
10
```

Python functions can also return multiple values.

```
>>> def arithmetic(a, b):
...     return a - b, a * b         # Separate return values with commas.
...
>>> x, y = arithmetic(5, 2)         # Unpack the returns into two variables.
>>> print(x, y)
3 10
```

The keyword `lambda` is a shortcut for creating one-line functions. For example, the polynomials $f(x) = 6x^3 + 4x^2 - x + 3$ and $g(x, y, z) = x + y^2 - z^3$ can be defined as functions in one line each.

```
# Define the polynomials the usual way using 'def'.
>>> def f(x):
...     return 6*x**3 + 4*x**2 - x + 3
>>> def g(x, y, z):
...     return x + y**2 - z**3

# Equivalently, define the polynomials quickly using 'lambda'.
>>> f = lambda x: 6*x**3 + 4*x**2 - x + 3
>>> g = lambda x, y, z: x + y**2 - z**3
```

Note

Documentation is important in every programming language. Every function should have a *docstring*—a string literal in triple quotes just under the function declaration—that describes the purpose of the function, the expected inputs and return values, and any other notes that

are important to the user. Short docstrings are acceptable for very simple functions, but more complicated functions require careful and detailed explanations.

```
>>> def add(x, y):
...     """Return the sum of the two inputs."""
...     return x + y

>>> def area(width, height):
...     """Return the area of the rectangle with the specified width
...     and height.
...     """
...     return width * height
...
>>> def arithmetic(a, b):
...     """Return the difference and the product of the two inputs."""
...     return a - b, a * b
```

Lambda functions cannot have custom docstrings, so the `lambda` keyword should be only be used as a shortcut for very simple or intuitive functions that need no additional labeling.

Problem 1. The volume of a sphere with radius r is $V = \frac{4}{3}\pi r^3$. Define a function called `sphere_volume()` that accepts a single parameter r. Return the volume of the sphere of radius r, using 3.14159 as an approximation for π (for now). Also write an appropriate docstring for your function.

Achtung!

The `return` statement instantly ends the function call and passes the return value to the function caller. However, functions are not required to have a return statement. A function without a return statement implicitly returns the Python constant `None`, which is similar to the special value `null` of many other languages. Calling `print()` at the end of a function does **not** cause a function to return any values.

```
>>> def oops(i):
...     """Increment i (but forget to return anything)."""
...     print(i + 1)
...
>>> def increment(i):
...     """Increment i."""
...     return i + 1
...
>>> x = oops(1999)                  # x contains 'None' since oops()
2000                                # doesn't have a return statement.
```

```
>>> y = increment(1999)             # However, y contains a value.
>>> print(x, y)
None 2000
```

If you have any intention of using the results of a function, use a `return` statement.

It is also possible to specify *default values* for a function's parameters. In the following example, the function `pad()` has three parameters, and the value of `c` defaults to 0. If it is not specified in the function call, the variable `c` will contain the value 0 when the function is executed.

```
>>> def pad(a, b, c=0):
...     """Print the arguments, plus a zero if c is not specified."""
...     print(a, b, c)
...
>>> pad(1, 2, 3)                # Specify each parameter.
1 2 3
>>> pad(1, 2)                   # Specify only non-default parameters.
1 2 0
```

It's important to note that positional arguments must precede named arguments in a function call. Additionally, parameters without default values must precede parameters with default values in a function definition. For example, `a` and `b` must come before `c` in the function definition of `pad()`. Examine the following code blocks demonstrating how positional and named arguments are used to call a function.

```
# Try defining pad() with a named argument before a positional argument.
>>> def pad(c=0, a, b):
...     print(a, b, c)
...
SyntaxError: non-default argument follows default argument
```

```
# Correctly define pad() with the named argument after positional arguments.
>>> def pad(a, b, c=0):
...     """Print the arguments, plus a zero if c is not specified."""
...     print(a, b, c)
...

# Call pad() with 3 positional arguments.
>>> pad(2, 4, 6)
2 4 6

# Call pad() with 3 named arguments. Note the change in order.
>>> pad(b=3, c=5, a=7)
7 3 5

# Call pad() with 2 named arguments, excluding c.
>>> pad(b=1, a=2)
```

```
2 1 0

# Call pad() with 1 positional argument and 2 named arguments.
>>> pad(1, c=2, b=3)
1 3 2
```

Problem 2. The built-in `print()` function has the useful keyword arguments `sep` and `end`. It accepts any number of positional arguments and prints them out with `sep` inserted between values (defaulting to a space), then prints `end` (defaulting to the *newline character* `'\n'`).

Write a function called `isolate()` that accepts five arguments. The function should print the first three arguments separated by 5 spaces and then print the last two arguments with a single space separating the last three arguments. For example,

```
>>> isolate(1, 2, 3, 4, 5)
1     2     3 4 5
```

ACHTUNG!

In previous versions of Python, `print()` was a *statement* (like `return`), not a function, and could therefore be executed without parentheses. However, it lacked keyword arguments like `sep` and `end`. If you are using Python 2.7, include the following line at the top of the file to turn the `print` statement into the new `print()` function.

```
>>> from __future__ import print_function
```

Data Types and Structures

Numerical Types

Python has four numerical data types: `int`, `long`, `float`, and `complex`. Each stores a different kind of number. The built-in function `type()` identifies an object's data type.

```
>>> type(3)                          # Numbers without periods are integers.
int

>>> type(3.0)                        # Floats have periods (3. is also a float).
float
```

Python has two types of division: integer and float. The `/` operator performs float division (true fractional division), and the `//` operator performs integer division, which rounds the result down to the next integer. If both operands for `//` are integers, the result will be an `int`. If one or both operands are floats, the result will be a `float`. Regular division with `/` always returns a `float`.

```
>>> 15 / 4                          # Float division performs as expected.
3.75
>>> 15 // 4                         # Integer division rounds the result down.
3
>>> 15. // 4
3.0
```

ACHTUNG!

In previous versions of Python, using / with two integers performed integer division, even in cases where the division was not even. This can result in some incredibly subtle and frustrating errors. If you are using Python 2.7, always include a . on the operands or cast at least one as a float when you want float division.

```
# PYTHON 2.7
>>> 15 / 4                            # The answer should be 3.75, but the
3                                     # interpreter does integer division!

>>> 15. / float(4)                    # 15. and float(4) are both floats, so
3.75                                  # the interpreter does float division.
```

Alternatively, including the following line at the top of the file redefines the / and // operators so they are handled the same way as in Python 3.

```
>>> from __future__ import division
```

Python also supports complex numbers computations by pairing two numbers as the real and imaginary parts. Use the letter j, not i, for the imaginary part.

```
>>> x = complex(2, 3)                 # Create a complex number this way...
>>> y = 4 + 5j                        # ...or this way, using j (not i).
>>> x.real                            # Access the real part of x.
2.0
>>> y.imag                            # Access the imaginary part of y.
5.0
```

Strings

In Python, strings are created with either single or double quotes. To concatenate two or more strings, use the + operator between string variables or literals.

```
>>> str1 = "Hello"
>>> str2 = 'world'
>>> my_string = str1 + " " + str2 + '!'
```

```
>>> my_string
'Hello world!'
```

Parts of a string can be accessed using *slicing*, indicated by square brackets `[ ]`. Slicing syntax is `[start:stop:step]`. The parameters `start` and `stop` default to the beginning and end of the string, respectively. The parameter `step` defaults to 1.

```
>>> my_string = "Hello world!"
>>> my_string[4]                # Indexing begins at 0.
'o'
>>> my_string[-1]               # Negative indices count backward from the end.
'!'

# Slice from the 0th to the 5th character (not including the 5th character).
>>> my_string[:5]
'Hello'

# Slice from the 6th character to the end.
>>> my_string[6:]
'world!'

# Slice from the 3rd to the 8th character (not including the 8th character).
>>> my_string[3:8]
'lo wo'

# Get every other character in the string.
>>> my_string[::2]
'Hlowrd'
```

Problem 3. Write two new functions, called `first_half()` and `backward()`.

1. `first_half()` should accept a parameter and return the first half of it, excluding the middle character if there is an odd number of characters.
   (Hint: the built-in function `len()` returns the length of the input.)
2. The `backward()` function should accept a parameter and reverse the order of its characters using slicing, then return the reversed parameter.
   (Hint: The `step` parameter used in slicing can be negative.)

Use IPython to quickly test your syntax for each function.

Lists

A Python `list` is created by enclosing comma-separated values with square brackets `[ ]`. Entries of a list do **not** have to be of the same type. Access entries in a list with the same indexing or slicing operations used with strings.

```
>>> my_list = ["Hello", 93.8, "world", 10]
>>> my_list[0]
'Hello'
>>> my_list[-2]
'world'
>>> my_list[:2]
['Hello', 93.8]
```

Common list methods (functions) include `append()`, `insert()`, `remove()`, `extend()`, and `pop()`. Consult IPython for details on each of these methods using object introspection.

```
>>> my_list = [1, 2]                # Create a simple list of two integers.
>>> my_list.append(4)               # Append the integer 4 to the end.
>>> my_list.insert(2, 3)            # Insert 3 at location 2.
>>> my_list
[1, 2, 3, 4]
>>> my_list.remove(3)               # Remove 3 from the list.
>>> my_list.pop()                   # Remove (and return) the last entry.
4
>>> my_list
[1, 2]
>>> my_list.extend([5, 6])           # Append multiple values at once.
>>> my_list
[1, 2, 5, 6]
```

Slicing is also very useful for replacing values in a list.

```
>>> my_list = [10, 20, 30, 40, 50]
>>> my_list[0] = -1
>>> my_list[3:] = [8, 9]
>>> print(my_list)
[-1, 20, 30, 8, 9]
```

The `in` operator quickly checks if a given value is in a list (or another iterable, including strings).

```
>>> my_list = [1, 2, 3, 4, 5]
>>> 2 in my_list
True
>>> 6 in my_list
False
>>> 'a' in "xylophone"              # 'in' also works on strings.
False
```

Tuples

A Python `tuple` is an ordered collection of elements, created by enclosing comma-separated values with parentheses `(` and `)`. Tuples are similar to lists, but they are much more rigid, have fewer

built-in operations, and cannot be altered after creation. Lists are therefore preferable for managing dynamic ordered collections of objects.

When multiple objects are returned by a function, they are returned as a tuple. For example, recall that the `arithmetic()` function returns two values.

```
>>> x, y = arithmetic(5, 2)                    # Get each value individually,
>>> print(x, y)
3 10
>>> both = arithmetic(5, 2)                    # or get them both as a tuple.
>>> print(both)
(3, 10)
```

Problem 4. Write a function called `list_ops()`. Define a list with the entries `"bear"`, `"ant"`, `"cat"`, and `"dog"`, in that order. Then perform the following operations on the list:

1. Append `"eagle"`.
2. Replace the entry at index 2 with `"fox"`.
3. Remove (or pop) the entry at index 1.
4. Sort the list in reverse alphabetical order.
5. Replace `"eagle"` with `"hawk"`.
   (Hint: the list's `index()` method may be helpful.)
6. Add the string `"hunter"` to the last entry in the list.

Return the resulting list of strings.

Work out (on paper) what the result should be, then check that your function returns the correct list. Consider printing the list at each step to see the intermediate results.

Sets

A Python `set` is an unordered collection of distinct objects. Objects can be added to or removed from a set after its creation. Initialize a set with curly braces `{ }`, separating the values by commas, or use `set()` to create an empty set. Like mathematical sets, Python sets have operations like union, intersection, difference, and symmetric difference.

```
# Initialize some sets. Note that repeats are not added.
>>> gym_members = {"Doe, John", "Doe, John", "Smith, Jane", "Brown, Bob"}
>>> print(gym_members)
{'Doe, John', 'Brown, Bob', 'Smith, Jane'}

>>> gym_members.add("Lytle, Josh")      # Add an object to the set.
>>> gym_members.discard("Doe, John")    # Delete an object from the set.
>>> print(gym_members)
{'Lytle, Josh', 'Brown, Bob', 'Smith, Jane'}
```

```
>>> gym_members.intersection({"Lytle, Josh", "Henriksen, Ian", "Webb, Jared"})
{'Lytle, Josh'}
>>> gym_members.difference({"Brown, Bob", "Sharp, Sarah"})
{'Lytle, Josh', 'Smith, Jane'}
```

Dictionaries

Like a set, a Python `dict` (dictionary) is an unordered data type. A dictionary stores key-value pairs, called *items*. The values of a dictionary are indexed by its keys. Dictionaries are initialized with curly braces, colons, and commas. Use `dict()` or `{}` to create an empty dictionary.

```
>>> my_dictionary = {"business": 4121, "math": 2061, "visual arts": 7321}
>>> print(my_dictionary["math"])
2061

# Add a value indexed by 'science' and delete the 'business' keypair.
>>> my_dictionary["science"] = 6284
>>> my_dictionary.pop("business")       # Use 'pop' or 'popitem' to remove.
4121
>>> print(my_dictionary)
{'math': 2061, 'visual arts': 7321, 'science': 6284}

# Display the keys and values.
>>> my_dictionary.keys()
dict_keys(['math', 'visual arts', 'science'])
>>> my_dictionary.values()
dict_values([2061, 7321, 6284])
```

As far as data access goes, lists are like dictionaries whose keys are the integers $0, 1, \ldots, n-1$, where n is the number of items in the list. The keys of a dictionary need not be integers, but they must be *immutable*, which means that they must be objects that cannot be modified after creation. We will discuss mutability more thoroughly in the Standard Library lab.

Type Casting

The names of each of Python's data types can be used as functions to cast a value as that type. This is particularly useful for converting between integers and floats.

```
# Cast numerical values as different kinds of numerical values.
>>> x = int(3.0)
>>> y = float(3)
>>> z = complex(3)
>>> print(x, y, z)
3 3.0 (3+0j)

# Cast a list as a set and vice versa.
>>> set([1, 2, 3, 4, 4])
{1, 2, 3, 4}
```

```
>>> list({'a', 'a', 'b', 'b', 'c'})
['a', 'c', 'b']

# Cast other objects as strings.
>>> str(['a', str(1), 'b', float(2)])
"['a', '1', 'b', 2.0]"
>>> str(list(set([complex(float(3))])))
'[(3+0j)]'
```

Control Flow Tools

Control flow blocks dictate the order in which code is executed. Python supports the usual control flow statements including `if` statements, `while` loops and `for` loops.

The If Statement

An `if` statement executes the indented code **if** (and only if) the given condition holds. The `elif` statement is short for "else if" and can be used multiple times following an if statement, or not at all. The `else` keyword may be used at most once at the end of a series of `if`/`elif` statements.

```
>>> food = "bagel"
>>> if food == "apple":              # As with functions, the colon denotes
...     print("72 calories")         # the start of each code block.
... elif food == "banana" or food == "carrot":
...     print("105 calories")
... else:
...     print("calorie count unavailable")
...
calorie count unavailable
```

Problem 5. Write a function called `pig_latin()`. Accept a string parameter `word`, translate it into Pig Latin, then return the translation. Specifically, if `word` starts with a vowel, add "hay" to the end; if `word` starts with a consonant, take the first character of `word`, move it to the end, and add "ay".
(Hint: use the `in` operator to check if the first letter is a vowel.)

The While Loop

A `while` loop executes an indented block of code **while** the given condition holds.

```
>>> i = 0
>>> while i < 10:
...     print(i, end=' ')            # Print a space instead of a newline.
...     i += 1                       # Shortcut syntax for i = i+1.
...
```

```
0 1 2 3 4 5 6 7 8 9
```

There are two additional useful statements to use inside of loops:

1. `break` manually exits the loop, regardless of which iteration the loop is on or if the termination condition is met.
2. `continue` skips the current iteration and returns to the top of the loop block if the termination condition is still not met.

```
>>> i = 0
>>> while True:
...     print(i, end=' ')
...     i += 1
...     if i >= 10:
...         break                    # Exit the loop.
...
0 1 2 3 4 5 6 7 8 9

>>> i = 0
>>> while i < 10:
...     i += 1
...     if i % 3 == 0:
...         continue                 # Skip multiples of 3.
...     print(i, end=' ')
1 2 4 5 7 8 10
```

The For Loop

A `for` loop iterates over the items in any *iterable*. Iterables include (but are not limited to) strings, lists, sets, and dictionaries.

```
>>> colors = ["red", "green", "blue", "yellow"]
>>> for entry in colors:
...     print(entry + "!")
...
red!
green!
blue!
yellow!
```

The `break` and `continue` statements also work in for loops, but a `continue` in a for loop will automatically increment the index or item, whereas a `continue` in a while loop makes no automatic changes to any variable.

```
>>> for word in ["It", "definitely", "looks", "pretty", "bad", "today"]:
...     if word == "definitely":
...         continue
```

```
...     elif word == "bad":
...         break
...     print(word, end=' ')
...
It looks pretty
```

In addition, Python has some very useful built-in functions that can be used in conjunction with the `for` statement:

1. `range(start, stop, step)`: Produces a sequence of integers, following slicing syntax. If only one argument is specified, it produces a sequence of integers from 0 up to (but not including) the argument, incrementing by one. This function is used **very** often.
2. `zip()`: Joins multiple sequences in parallel so they can be iterated over simultaneously.
3. `enumerate()`: Yields both a count and a value from the sequence. Typically used to get both the index of an item and the actual item simultaneously.
4. `reversed()`: Reverses the order of the iteration.
5. `sorted()`: Returns a new list of sorted items that can then be used for iteration.

Each of these functions except for `sorted()` returns an *iterator*, an object that is built specifically for looping but not for creating actual lists. To put the items of the sequence in a collection, use `list()`, `set()`, or `tuple()`.

```
# Strings and lists are both iterables.
>>> vowels = "aeiou"
>>> colors = ["red", "yellow", "white", "blue", "purple"]

# Iterate by index.
>>> for i in range(5):
...     print(i, vowels[i], colors[i])
...
0 a red
1 e yellow
2 i white
3 o blue
4 u purple

# Iterate through both sequences at once.
>>> for letter, word in zip(vowels, colors):
...     print(letter, word)
...
a red
e yellow
i white
o blue
u purple

# Get the index and the item simultaneously.
```

```
>>> for i, color in enumerate(colors):  #
...     print(i, color)
...
0 red
1 yellow
2 white
3 blue
4 purple

# Iterate through the list in sorted (alphabetical) order.
>>> for item in sorted(colors):
...     print(item, end=' ')
...
blue purple red white yellow

# Iterate through the list backward.
>>> for item in reversed(colors):
...     print(item, end=' ')
...
purple blue white yellow red

# range() arguments follow slicing syntax.
>>> list(range(10))                 # Integers from 0 to 10, exclusive.
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> list(range(4, 8))               # Integers from 4 to 8, exclusive.
[4, 5, 6, 7]

>>> set(range(2, 20, 3))            # Every third integer from 2 to 20.
{2, 5, 8, 11, 14, 17}
```

Problem 6. This problem originates from `https://projecteuler.net`, an excellent resource for math-related coding problems.

A palindromic number reads the same both ways. The largest palindrome made from the product of two 2-digit numbers is $9009 = 91 \times 99$. Write a function called `palindrome()` that finds and returns the largest palindromic number made from the product of two 3-digit numbers.

List Comprehension

A *list comprehension* uses for loop syntax between square brackets to create a list. This is a powerful, efficient way to build lists. The code is concise and runs quickly.

```
>>> [float(n) for n in range(5)]
[0.0, 1.0, 2.0, 3.0, 4.0]
```

List comprehensions can be thought of as "inverted loops", meaning that the body of the loop comes before the looping condition. The following loop and list comprehension produce the same list, but the list comprehension takes only about two-thirds the time to execute.

```
>>> loop_output = []
>>> for i in range(5):
...     loop_output.append(i**2)
...
>>> list_output = [i**2 for i in range(5)]
```

Tuple, set, and dictionary comprehensions can be done in the same way as list comprehensions by using the appropriate style of brackets on the end.

```
>>> colors = ["red", "blue", "yellow"]
>>> {c[0]:c for c in colors}
{'y': 'yellow', 'r': 'red', 'b': 'blue'}

>>> {"bright " + c for c in colors}
{'bright blue', 'bright red', 'bright yellow'}
```

Problem 7. The alternating harmonic series is defined as follows.

$$\sum_{n=1}^{\infty} \frac{(-1)^{(n+1)}}{n} = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \ldots = \ln(2)$$

Write a function called `alt_harmonic()` that accepts an integer n. Use a list comprehension to quickly compute and sum the first n terms of this series (be careful not to sum only $n-1$ terms). The sum of the first 500,000 terms of this series approximates $\ln(2)$ to five decimal places.
(Hint: consider using Python's built-in `sum()` function.)

Additional Material

Further Reading

Refer back to this and other introductory labs often as you continue getting used to Python syntax and data types. As you continue your study of Python, we strongly recommend the following readings.

- The official Python tutorial: `https://docs.python.org/3/tutorial/introduction.html` (especially chapters 3, 4, and 5).
- Section 1.2 of the SciPy lecture notes: `http://scipy-lectures.github.io/`.
- PEP8 - Python style guide: `http://www.python.org/dev/peps/pep-0008/`.

Advanced List Comprehension

As shown in the previous lab, list comprehensions can be used to create lists in a single line of code. There are several ways to supercharge list comprehensions to make them even more powerful. It all depends on what you would like to control within the list. We can add conditionals (i.e `if-else` statements) to the list comprehension to filter out or change certain elements. Moreover, nested looping is applicable for all list comprehensions. You can combine all of the following methods to create a list comprehension that fits your needs. Note that an iterable is an object that is countable and is meant to be traversed through (e.g. lists, tuples, strings, etc.).

- Nested Looping: Just like a nested `for` loop, you can nest the loops within the syntax of your comprehension. This is useful when you want to iterate over multiple lists at the same time. The syntax for this follows a similar structure to a normal nested `for` loop where the outer loop of the iterable goes before all loops it encompasses.
- Filtering the elements of the iterable: If you want to be able to only look at certain elements of the iterable, you can add an `if` statement following the name of the iterable. Note that this typically only works for simple conditionals. If you require a more complex conditional, you may want to create a function for the conditional and call the function in the list comprehension.
- Changing or filtering the elements going into the list: If your goal is to change or filter the actual elements that will go into the list you are making, you can add an `if` conditional following the expression dictating what the value of the element will be and immediately before the start of the `for` statement. All other conditionals that function like the `elif` statement come after the first `if` statement. However, even if you only have one `if` statement, Python syntax requires that you have an `else` statement at the end of all your conditionals, but before the start of the `for` loop, so the list has a default value it can give the element. Note we say "function like the `elif` statement" because list comprehension does not support the `elif` statement. To make conditionals that function like the `elif` statement, you must first state the `else` statement followed by the expression or value that the element will take, and then by the `if` statement and the condition the element must meet. That is, we use [... `else` *expression* `if` *condition* ...] for however many `elif` statements you need after the first `if` statement and always ensuring we end with the default `else` statement.

```
# Nested Looping
>>> [letter for word in ["Hello", "There"] for letter in word]
['H', 'e', 'l', 'l', 'o', 'T', 'h', 'e', 'r', 'e']
```

```
# Filtering the elements of the iterable
# Make a list of only even numbers from 0 to 9 (range is the iterable)
>>> [i for i in range(10) if i % 2 == 0]
[0, 2, 4, 6, 8]

# Changing or filtering the elements going into the list
# Make a list of the modulus of 3 of all numbers from 0 to 9
# using 'z' for 0, 'o' for 1, and 't' for 2. Note 'o' is
# the elif and 't' is the default if no other condition is met
>>> ['z' if i%3==0 else 'o' if i%3==1 else 't' for i in range(10)]
['z', 'o', 't', 'z', 'o', 't', 'z', 'o', 't', 'z']

# Combination of all three (notice the difficulty in code readability)
>>> iterable = ["General", "Kenobi"]
>>> [letter if letter != 'K' else 0 for word in iterable for letter in word if ↩
    letter not in "aeiou"]
['G', 'n', 'r', 'l', 0, 'n', 'b']
```

List comprehensions are actually faster than normal `for` loops. This is because when you append or create lists, Python has to perform a few operations to check that the given variable is a list and then append the new element to the list. This is not the case with list comprehensions. You can view this video for more information on list comprehensions. While list comprehensions are faster, they are not always the best option. They can be hard to read and understand, especially when they use various conditionals and nesting, and cannot give you all the control you may need when doing a normal for loop. So be sure to use them wisely and not overuse or overcomplicate them.

Generalized Function Input

On rare occasion, it is necessary to define a function without knowing exactly what the parameters will be like or how many there will be. This is usually done by defining the function with the parameters `*args` and `**kwargs`. Here `*args` is a list of the positional arguments and `**kwargs` is a dictionary mapping the keywords to their argument. This is the most general form of a function definition.

```
>>> def report(*args, **kwargs):
...     for i, arg in enumerate(args):
...         print("Argument " + str(i) + ":", arg)
...     for key in kwargs:
...         print("Keyword", key, "-->", kwargs[key])
...
>>> report("TK", 421, exceptional=False, missing=True)
Argument 0: TK
Argument 1: 421
Keyword missing --> True
Keyword exceptional --> False
```

See `https://docs.python.org/3/tutorial/controlflow.html` for more on this topic.

Function Decorators

A *function decorator* is a special function that "wraps" other functions. It takes in a function as input and returns a new function that pre-processes the inputs or post-processes the outputs of the original function.

```
>>> def typewriter(func):
...     """Decorator for printing the type of output a function returns"""
...     def wrapper(*args, **kwargs):
...         output = func(*args, **kwargs)      # Call the decorated function.
...         print("output type:", type(output)) # Process before finishing.
...         return output                       # Return the function output.
...     return wrapper
```

The outer function, `typewriter()`, returns the new function `wrapper()`. Since `wrapper()` accepts `*args` and `**kwargs` as arguments, the input function `func()` could accept any number of positional or keyword arguments.

Apply a decorator to a function by tagging the function's definition with an @ symbol and the decorator name.

```
>>> @typewriter
... def combine(a, b, c):
...     return a*b // c
```

Placing the tag above the definition is equivalent to adding the following line of code after the function definition:

```
>>> combine = typewriter(combine)
```

Now calling `combine()` actually calls `wrapper()`, which then calls the original `combine()`.

```
>>> combine(3, 4, 6)
output type: <class 'int'>
2
>>> combine(3.0, 4, 6)
output type: <class 'float'>
2.0
```

Function decorators can also be customized with arguments. This requires another level of nesting: the outermost function must define and return a decorator that defines and returns a wrapper.

```
>>> def repeat(times):
...     """Decorator for calling a function several times."""
...     def decorator(func):
...         def wrapper(*args, **kwargs):
...             for _ in range(times):
...                 output = func(*args, **kwargs)
...             return output
```

```
...         return wrapper
...     return decorator
...
>>> @repeat(3)
... def hello_world():
...     print("Hello, world!")
...
>>> hello_world()
Hello, world!
Hello, world!
Hello, world!
```

See `https://www.python.org/dev/peps/pep-0318/` for more details.

2 The Standard Library

Lab Objective: *Python is designed to make it easy to implement complex tasks with little code. To that end, every Python distribution includes several built-in functions for accomplishing common tasks. In addition, Python is designed to import and reuse code written by others. A Python file with code that can be imported is called a* module*. All Python distributions include a collection of modules for accomplishing a variety of tasks, collectively called the* Python Standard Library*. In this lab we explore some built-in functions, learn how to create, import, and use modules, and become familiar with the standard library.*

Built-in Functions

Python has several built-in functions that may be used at any time. IPython's object introspection feature makes it easy to learn about these functions: start IPython from the command line and use `?` to bring up technical details on each function.

```
In [1]: min?
Docstring:
min(iterable, *[, default=obj, key=func]) -> value
min(arg1, arg2, *args, *[, key=func]) -> value

With a single iterable argument, return its smallest item. The
default keyword-only argument specifies an object to return if
the provided iterable is empty.
With two or more arguments, return the smallest argument.
Type:      builtin_function_or_method

In [2]: len?
Signature: len(obj, /)
Docstring: Return the number of items in a container.
Type:      builtin_function_or_method
```

Function	Returns
`abs()`	The absolute value of a real number, or the magnitude of a complex number.
`min()`	The smallest element of a single iterable, or the smallest of several arguments. Strings are compared based on lexicographical order: numerical characters first, then upper-case letters, then lower-case letters.
`max()`	The largest element of a single iterable, or the largest of several arguments.
`len()`	The number of items of a sequence or collection.
`round()`	A float rounded to a given precision in decimal digits.
`sum()`	The sum of a sequence of numbers.

Table 2.1: Common built-in functions for numerical calculations.

```
# abs() can be used with real or complex numbers.
>>> print(abs(-7), abs(3 + 4j))
7 5.0

# min() and max() can be used on a list, string, or several arguments.
# String characters are ordered lexicographically.
>>> print(min([4, 2, 6]), min("aXbYcZ"), min('1', 'a', 'A'))
2 X 1
>>> print(max([4, 2, 6]), max("aXbYcZ"), max('1', 'a', 'A'))
6 c a

# len() can be used on a string, list, set, dict, tuple, or other iterable.
>>> print(len([2, 7, 1]), len("abcdef"), len({1, 'a', 'a'}))
3 6 2

# sum() can be used on iterables containing numbers, but not strings.
>>> my_list = [1, 2, 3]
>>> my_tuple = (4, 5, 6)
>>> my_set = {7, 8, 9}
>>> sum(my_list) + sum(my_tuple) + sum(my_set)
45
>>> sum([min(my_list), max(my_tuple), len(my_set)])
10

# round() is particularly useful for formatting data to be printed.
>>> round(3.14159265358979323, 2)
3.14
```

See `https://docs.python.org/3/library/functions.html` for more detailed documentation on all of Python's built-in functions.

Problem 1. Write a function that accepts a list L and returns the minimum, maximum, and average of the entries of L in that order as multiple values (separated by a comma). Can you implement this function in one return statement?

Namespaces

Whenever a Python object—a number, data structure, function, or other entity—is created, it is stored somewhere in computer memory. A *name* (or variable) is a reference to a Python object, and a *namespace* is a dictionary that maps names to Python objects.

```
# The number 4 is the object; 'number_of_students' is the name.
>>> number_of_students = 4

# The list is the object, and 'beatles' is the name.
>>> beatles = ["John", "Paul", "George", "Ringo"]

# Python statements defining a function also form an object.
# The name for this function is 'add_numbers'.
>>> def add_numbers(a, b):
...     return a + b
...
```

A single equals sign assigns a name to an object. If a name is assigned to another name, that new name refers to the same object as the original name.

```
>>> beatles = ["John", "Paul", "George", "Ringo"]
>>> band_members = beatles             # Assign a new name to the list.
>>> print(band_members)
['John', 'Paul', 'George', 'Ringo']
```

To see all of the names in the current namespace, use the built-in function `dir()`. To delete a name from the namespace, use the `del` keyword (**with caution!**).

```
# Add 'stem' to the namespace.
>>> stem = ["Science", "Technology", "Engineering", "Mathematics"]
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'stem']

# Remove 'stem' from the namespace.
>>> del stem
>>> "stem" in dir()
False
>>> print(stem)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'stem' is not defined
```

NOTE

Many programming languages distinguish between *variables* and *pointers*. A pointer refers to a variable by storing the address in memory where the corresponding object is stored. Python names are essentially pointers, and traditional pointer operations and cleanup are done automatically. For example, Python automatically deletes objects in memory that have no names assigned to them (no pointers referring to them). This feature is called *garbage collection*.

Mutability

Every Python object type falls into one of two categories: a *mutable* object, which may be altered at any time, or an *immutable* object, which cannot be altered once created. Attempting to change an immutable object creates a new object in memory. If two names refer to the same mutable object, any changes to the object are reflected in both names since they still both refer to that same object. On the other hand, if two names refer to the same immutable object and one of the values is "changed," then one name will refer to the original object, and the other will refer to a new object in memory.

ACHTUNG!

Failing to correctly copy mutable objects can cause subtle problems. For example, consider a dictionary that maps items to their base prices. To make a similar dictionary that accounts for a small sales tax, we might try to make a copy by assigning a new name to the first dictionary.

```
>>> holy = {"moly": 1.99, "hand_grenade": 3, "grail": 1975.41}
>>> tax_prices = holy              # Try to make a copy for processing.
>>> for item, price in tax_prices.items():
...     # Add a 7 percent tax, rounded to the nearest cent.
...     tax_prices[item] = round(1.07 * price, 2)
...
# Now the base prices have been updated to the total price.
>>> print(tax_prices)
{'moly': 2.13, 'hand_grenade': 3.21, 'grail': 2113.69}

# However, dictionaries are mutable, so 'holy' and 'tax_prices' actually
# refer to the same object. The original base prices have been lost.
>>> print(holy)
{'moly': 2.13, 'hand_grenade': 3.21, 'grail': 2113.69}
```

To avoid this problem, explicitly create a copy of the object by casting it as a new structure. Changes made to the copy will not change the original object, since they are distinct objects in memory. To fix the above code, replace the second line with the following:

```
>>> tax_prices = dict(holy)
```

Then, after running the same procedure, the two dictionaries will be different.

Problem 2. Determine which Python object types are mutable and which are immutable by repeating the following experiment for an `int`, `str`, `list`, `tuple`, and `set`.

1. Create an object of the given type and assign a name to it.
2. Assign a new name to the first name.
3. Alter the object via only one of the names (for tuples, use `my_tuple += (1,)`).
4. Check to see if the two names are equal. If they are, then changing one name also changes the other. Thus, both names refer to the same object and the object type is mutable. Otherwise, the names refer to different objects—meaning a new object was created in step 2—and therefore the object type is immutable.

For example, the following experiment shows that `dict` is a mutable type.

```
>>> dict_1 = {1: 'x', 2: 'b'}          # Create a dictionary.
>>> dict_2 = dict_1                    # Assign it a new name.
>>> dict_2[1] = 'a'                    # Change the 'new' dictionary.
>>> dict_1 == dict_2                   # Compare the two names.
True                                   # Both names changed!
```

Print a statement of your conclusions that clearly indicates which object types are mutable and which are immutable.

Achtung!

Mutable objects cannot be put into Python sets or used as keys in Python dictionaries. However, the values of a dictionary may be mutable or immutable.

```
>>> a_dict = {"key": "value"}              # Dictionaries are mutable.
>>> broken = {1, 2, 3, a_dict, a_dict}     # Try putting a dict in a set.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'dict'

>>> okay = {1: 2, "3": a_dict}             # Try using a dict as a value.
```

Modules

A *module* is a Python file containing code that is meant to be used in some other setting, and not necessarily run directly.[1] The `import` statement loads code from a specified Python file. Importing a module containing some functions, classes, or other objects makes those functions, classes, or objects available for use by adding their names to the current namespace.

All import statements should occur at the top of the file, below the header but before any other code. There are several ways to use `import`:

1. `import <module>` makes the specified module available under the alias of its own name.

```
>>> import math                 # The name 'math' now gives
>>> math.sqrt(2)                # access to the math module.
1.4142135623730951
```

2. `import <module> as <name>` creates an alias for an imported module. The alias is added to the current namespace, but the module name itself is not.

```
>>> import numpy as np          # The name 'np' gives access to the numpy
>>> np.sqrt(2)                  # module, but the name 'numpy' does not.
1.4142135623730951
>>> numpy.sqrt(2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'numpy' is not defined
```

3. `from <module> import <object>` loads the specified object into the namespace without loading anything else in the module or the module name itself. This is used most often to access specific functions from a module. The `as` statement can also be tacked on to create an alias.

```
>>> from random import randint  # The name 'randint' gives access to the
>>> r = randint(0, 10000)       # randint() function, but the rest of
>>> random.seed(r)              # the random module is unavailable.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'random' is not defined
```

In each case, the final word of the import statement is the name that is added to the namespace.

Running and Importing

Consider the following simple Python module, saved as `example1.py`.

```
# example1.py

data = list(range(4))
```

[1] Python files that are primarily meant to be executed, not imported, are often called *scripts*.

```
def display():
    print("Data:", data)

if __name__ == "__main__":
    display()
    print("This file was executed from the command line or an interpreter.")
else:
    print("This file was imported.")
```

Executing the file from the command line executes the file line by line, including the code under the `if __name__ == "__main__"` clause.

```
$ python example1.py
Data: [0, 1, 2, 3]
This file was executed from the command line or an interpreter.
```

Executing the file with IPython's special `%run` command executes each line of the file and also adds the module's names to the current namespace. **This is the quickest way to test individual functions via IPython**.

```
In [1]: %run example1.py
Data: [0, 1, 2, 3]
This file was executed from the command line or an interpreter.

In [2]: display()
Data: [0, 1, 2, 3]
```

Importing the file also executes each line,[2] but only adds the indicated alias to the namespace. Also, code under the `if __name__ == "__main__"` clause is **not** executed when a file is imported.

```
In [1]: import example1 as ex
This file was imported.

# The module's names are not directly available...
In [2]: display()
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-795648993119> in <module>()
----> 1 display()

NameError: name 'display' is not defined

# ...unless accessed via the module's alias.
In [3]: ex.display()
Data: [0, 1, 2, 3]
```

[2] Try importing the `this` or `antigravity` modules. Importing these modules actually executes some code.

Problem 3. Create a module called `calculator.py`. Write a function `sum()` that returns the sum of two arguments and a function `product()` that returns the product of two arguments. Also use `import` to add the `sqrt()` function from the `math` module to the namespace. When this file is either run or imported, nothing should be executed.

In your solutions file, import your new custom module. Write a function that accepts two numbers representing the lengths of the sides of a right triangle. Using only the functions from `calculator.py`, calculate and return the length of the hypotenuse of the triangle.

Achtung!

If a module has been imported in IPython and the source code then changes, using `import` again does **not** refresh the name in the IPython namespace. Use `run` instead to correctly refresh the namespace. Consider this example where we test the function `sum_of_squares()`, saved in the file `example2.py`.

```
# example2.py

def sum_of_squares(x):
    """Return the sum of the squares of all positive integers
    less than or equal to x.
    """
    return sum([i**2 for i in range(1, x)])
```

In IPython, run the file and test `sum_of_squares()`.

```
# Run the file, adding the function sum_of_squares() to the namespace.
In [1]: %run example2

In [2]: sum_of_squares(3)
Out[2]: 5                           # Should be 14!
```

Since $1^2+2^2+3^2 = 14$, not 5, something has gone wrong. Modify the source file to correct the mistake, then run the file again in IPython.

```
# example2.py

def sum_of_squares(x):
    """Return the sum of the squares of all positive integers
    less than or equal to x.
    """
    return sum([i**2 for i in range(1, x+1)])    # Include the final term.
```

```
# Run the file again to refresh the namespace.
In [3]: %run example2
```

```
# Now sum_of_squares() is updated to the new, corrected version.
In [4]: sum_of_squares(3)
Out[4]: 14                          # It works!
```

Remember that running or importing a file executes any freestanding code snippets, but any code under an `if __name__ == "__main__"` clause will **only** be executed when the file is run (not when it is imported).

The Python Standard Library

All Python distributions include a collection of modules for accomplishing a variety of common tasks, collectively called the *Python standard library*. Some commonly standard library modules are listed below, and the complete list is at `https://docs.python.org/3/library/`.

Module	Description
`cmath`	Mathematical functions for complex numbers.
`itertools`	Tools for iterating through sequences in useful ways.
`math`	Standard mathematical functions and constants.
`random`	Random variable generators.
`string`	Common string literals.
`sys`	Tools for interacting with the interpreter.
`time`	Time value generation and manipulation.

Use IPython's object introspection to quickly learn about how to use the various modules and functions in the standard library. Use `?` or `help()` for information on the module or one of its names. To see the entire module's namespace, use the `tab` key.

```
In [1]: import math

In [2]: math?
Type:        module
String form: <module 'math' (built-in)>
Docstring:
This module provides access to the mathematical functions
defined by the C standard.

# Type the module name, a period, then press tab to see the module's namespace.
In [3]: math.   # Press 'tab'.
    acos()      cos()       factorial() isclose()   log2()      tan()
    acosh()     cosh()      floor()     isfinite()  modf()      tanh()
    asin()      degrees()   fmod()      isinf()     nan         tau
    asinh()     e           frexp()     isnan()     pi          trunc()
    atan()      erf()       fsum()      ldexp()     pow()
    atan2()     erfc()      gamma()     lgamma()    radians()
    atanh()     exp()       gcd()       log()       sin()
    ceil()      expm1()     hypot()     log10()     sinh()
```

```
    copysign()  fabs()      inf         log1p()     sqrt()

In [3]: math.sqrt?
Signature: math.sqrt(x, /)
Docstring: Return the square root of x.
Type:      builtin_function_or_method
```

The Itertools Module

The `itertools` module makes it easy to iterate over one or more collections in specialized ways.

Function	Description
`chain()`	Iterate over several iterables in sequence.
`cycle()`	Iterate over an iterable repeatedly.
`combinations()`	Return successive combinations of elements in an iterable.
`permutations()`	Return successive permutations of elements in an iterable.
`product()`	Iterate over the Cartesian product of several iterables.

```
>>> from itertools import chain, cycle              # Import multiple names.

>>> list(chain("abc", ['d', 'e'], ('f', 'g')))     # Join several
['a', 'b', 'c', 'd', 'e', 'f', 'g']                 # sequences together.

>>> for i,number in enumerate(cycle(range(4))):    # Iterate over a single
...     if i > 10:                                 # sequence over and over.
...         break
...     print(number, end=' ')
...
0 1 2 3 0 1 2 3 0 1 2
```

A *k-combination* is a set of k elements from a collection where the ordering is unimportant. Thus the combination (a, b) and (b, a) are equivalent because they contain the same elements. One the other hand, a *k-permutation* is a sequence of k elements from a collection where the ordering matters. Even though (a, b) and (b, a) contain the same elements, they are counted as different permutations.

```
>>> from itertools import combinations, permutations

# Get all combinations of length 2 from the iterable "ABC".
>>> list(combinations("ABC", 2))
[('A', 'B'), ('A', 'C'), ('B', 'C')]

# Get all permutations of length 2 from "ABC". Note that order matters here.
>>> list(permutations("ABC", 2))
[('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
```

Problem 4. The *power set* of a set A, denoted $\mathcal{P}(A)$ or 2^A, is the set of all subsets of A, including the empty set $\emptyset$ and A itself. For example, the power set of the set $A = \{a, b, c\}$ is $2^A = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$.

Write a function that accepts an iterable A. Use an `itertools` function to compute and return the power set of A as a list of sets (why couldn't it be a set of sets in Python?). The empty set should be returned as `set()`.

The Random Module

Many real-life events can be simulated by taking random samples from a probability distribution. For example, a coin flip can be simulated by randomly choosing between the integers 1 (for heads) and 0 (for tails). The `random` module includes functions for sampling from probability distributions and generating random data.

Function	Description
`choice()`	Choose a random element from a non-empty sequence, such as a list.
`randint()`	Choose a random integer over a closed interval.
`random()`	Pick a float from the interval [0, 1).
`sample()`	Choose several unique random elements from a non-empty sequence.
`seed()`	Seed the random number generator.
`shuffle()`	Randomize the ordering of the elements in a list.

Some of the most common `random` utilities involve picking random elements from iterables.

```
>>> import random

>>> numbers = list(range(1, 11))     # Get the integers from 1 to 10.
>>> print(numbers)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

>>> random.shuffle(numbers)          # Mix up the ordering of the list.
>>> print(numbers)                   # Note that shuffle() returns nothing.
[5, 9, 1, 3, 8, 4, 10, 6, 2, 7]

>>> random.choice(numbers)           # Pick a single element from the list.
5

>>> random.sample(numbers, 4)        # Pick 4 unique elements from the list.
[5, 8, 3, 2]

>>> random.randint(1, 10)            # Pick a random number between 1 and 10.
10
```

The Time Module

The `time` module in the standard library include functions for dealing with time. In particular, the `time()` function measures the number of seconds from a fixed starting point, called "the Epoch" (January 1, 1970 for Unix machines).

```
>>> import time
>>> time.time()
1495243696.645818
```

The `time()` function is useful for measuring how long it takes for code to run: record the time just before and just after the code in question, then subtract the first measurement from the second to get the number of seconds that have passed.

```
>>> def time_for_loop(iters):
...     """Time how long it takes to iterate 'iters' times."""
...     start = time.time()         # Clock the starting time.
...     for _ in range(int(iters)):
...         pass
...     end = time.time()           # Clock the ending time.
...     return end - start          # Report the difference.
...
>>> time_for_loop(1e5)              # 1e5 = 100000.
0.005570173263549805
>>> time_for_loop(1e7)              # 1e7 = 10000000.
0.26819777488708496
```

The Sys Module

The `sys` (system) module includes methods for interacting with the Python interpreter. For our purposes, we care about the instance that you call a .py file in the command line using 'python' followed by the name of a .py file or in your ipython terminal using '%run' followed by the name of a .py file. Remember, to run a .py file in the command line, use cd to navigate to the folder containing the file before trying to run it. One command we will use is `sys.argv`; which returns a list containing the .py file name and all arguments that were passed to the interpreter. This way we can interact with these initial arguments and execute certain parts of our code if the arguments satisfy certain conditions. This will be implemented in problem 5. Note, command line arguments are passed in after the name of the .py file and are separated by spaces like so:

```
# the 'python' command followed by a .py file followed by any arguments
$ python yourProgram.py argument1 argument2 argument3

# the equivalent form in an ipython terminal
In[1]: %run yourProgram.py argument1 argument2 argument3
```

Now we can execute code based on the command line arguments by inserting the use of the `sys` module in our .py file:

```
# example3.py
"""If there are two command line arguments  after the .py file  name, print a descriptive statment."""
import sys

if len(sys.argv) == 3:  # if there's two arguments and the file name
    print("the first command line argument is " + sys.argv[1])
    print("the second command line argument is " + sys.argv[2])
else:
    print("um... I need two after the .py file. This is not two:")
    print(sys.argv)
```

Now provide command line arguments for the program to process.

```
# No extra command line arguments.
$ python example3.py
Output:
um... I need two after the .py file. This is not two:
['example3.py']

# With two arguments the if statement executes.
$ python example3.py crunchy juicy
Output:
the first command line argument is crunchy
the second command line argument is juicy
```

Note that the first command line argument is always the filename, so that is always the first element of the sys.argv list. This is why we have the if statement execute if the length of the list is 3 even though we want it to execute when there are 2 command line arguments. Also, `sys.argv` is always a list of strings. If a number is provided on the command line, it is converted to a string when it is stored in `sys.argv`. In IPython, command line arguments are specified after the `%run` command.

```
In [1]: %run example3.py 42 too many
Output:
um... I need two after the .py file. This is not two:
['example3.py', '42', 'too', 'many']
```

Another way to get input from the program user is to prompt the user for text. The built-in function `input()` pauses the program and waits for the user to type something. Like command line arguments, the user's input is parsed as a string.

```
>>> x = input("Enter a value for x: ")
Enter a value for x: 20             # Type '20' and press 'enter.'

>>> x
'20'                                # Note that x contains a string.
```

```
>> y = int(input("Enter an integer for y: "))
Enter an integer for y: 16          # Type '16' and press 'enter.'

>>> y
16                                  # Note that y contains an integer.
```

Problem 5. *Shut the box* is a popular British pub game that is used to help children learn arithmetic. The player starts with the numbers 1 through 9, and the goal of the game is to eliminate as many of these numbers as possible. At each turn the player rolls two dice, then chooses a set of integers from the remaining numbers that sum up to the sum of the dice roll. These numbers are removed, and the dice are then rolled again. If the sum of the remaining numbers is 6 or less, then only one die is rolled. The game ends when none of the remaining integers can be combined to the sum of the dice roll, and the player's final score is the sum of the numbers that could not be eliminated. For a demonstration, see `https://www.youtube.com/watch?v=mwURQC7mjDI`.

Modify your solutions file so that when the file is run with the correct command line arguments (but **not** when it is imported), the user plays a game of shut the box. The provided module `box.py` contains two functions that will be useful in your implementation of the game. You do not need to understand exactly how the functions work, but you do need to be able to import and use them correctly. Their functionality is outlined at the beginning of each function declaration. Your game should match the following specifications:

- Require three total command line arguments: the file name (included by default), the player's name, and a time limit in seconds. If there are not exactly three command line arguments, do not start the game.
- Track the player's remaining numbers, starting with 1 through 9.
- Use the `random` module to simulate rolling two six-sided dice. However, if the sum of the player's remaining numbers is 6 or less, roll only one die.
- The player wins if they have no numbers left, and they lose if they are out of time or if they cannot choose numbers to match the dice roll.
- If the game is not over, print the player's remaining numbers, the sum of the dice roll, and the number of seconds remaining. Prompt the user for numbers to eliminate. The input should be one or more of the remaining integers, separated by spaces. If the user's input is invalid, prompt them for input again before rolling the dice again.
  (Hint: use `round()` to format the number of seconds remaining nicely.)
- When the game is over, display the player's name, their score, and the total number of seconds since the beginning of the game. Congratulate or mock the player appropriately.

(Hint: **Before you start coding**, write an outline for the entire program, adding one feature at a time. Only start implementing the game after you are completely finished designing it.)

Your game should look similar to the following examples. The characters in red are typed inputs from the user.

```
$ python standard_library.py LuckyDuke 60

Numbers left: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Roll: 12
Seconds left: 60.0
Numbers to eliminate: 3 9

Numbers left: [1, 2, 4, 5, 6, 7, 8]
Roll: 9
Seconds left: 53.51
Numbers to eliminate: 8 1

Numbers left: [2, 4, 5, 6, 7]
Roll: 7
Seconds left: 51.39
Numbers to eliminate: 7

Numbers left: [2, 4, 5, 6]
Roll: 2
Seconds left: 48.24
Numbers to eliminate: 2

Numbers left: [4, 5, 6]
Roll: 11
Seconds left: 45.16
Numbers to eliminate: 5 6

Numbers left: [4]
Roll: 4
Seconds left: 42.76
Numbers to eliminate: 4

Score for player LuckyDuke: 0 points
Time played: 21.82 seconds
Congratulations!! You shut the box!
```

The next two examples show different ways that a player could lose (which they usually do), as well as examples of invalid user input. Use the `box` module's `parse_input()` to detect invalid input.

```
$ python standard_library.py ShakySteve 10

Numbers left: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Roll: 7
```

```
Seconds left: 10.0
Numbers to eliminate: Seven             # Must enter a number.
Invalid input

Seconds left: 7.64
Numbers to eliminate: 1, 2, 4           # Do not use commas.
Invalid input

Seconds left: 4.55
Numbers to eliminate: 1 2 3             # Numbers don't sum to the roll.
Invalid input

Seconds left: 2.4
Numbers to eliminate: 1 2 4

Numbers left: [3, 5, 6, 7, 8, 9]
Roll: 8
Seconds left: 0.31
Numbers to eliminate: 8
Game over!                              # Time is up!

Score for player ShakySteve: 30 points
Time played: 11.77 seconds
Better luck next time >:)
```

```
$ python standard_library.py SnakeEyesTom 10000

Numbers left: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Roll: 2
Seconds left: 10000.0
Numbers to eliminate: 2

Numbers left: [1, 3, 4, 5, 6, 7, 8, 9]
Roll: 2
Game over!                              # Numbers cannot match roll.

Score for player SnakeEyesTom: 43 points
Time played: 1.53 seconds
Better luck next time >:)
```

Additional Material

More Built-in Functions

The following built-in functions are worth knowing, especially for working with iterables and writing very readable conditional statements.

Function	Description
`all()`	Return `True` if `bool(entry)` evaluates to `True` for *every* entry in the input iterable.
`any()`	Return `True` if `bool(entry)` evaluates to `True` for *any* entry in the input iterable.
`bool()`	Evaluate a single input object as `True` or `False`.
`eval()`	Execute a string as Python code and return the output.
`map()`	Apply a function to every item of the input iterable and return an iterable of the results.

```
>>> from random import randint
# Get 5 random numbers between 1 and 10, inclusive.
>>> numbers = [randint(1, 10) for _ in range(5)]

# If all of the numbers are less than 8, print the list.
>>> if all([num < 8 for num in numbers]):
...     print(numbers)
...
[1, 5, 6, 3, 3]

# If none of the numbers are divisible by 3, print the list.
>>> if not any([num % 3 == 0 for num in numbers]):
...     print(numbers)
...
```

Two-Player Shut the Box

Consider modifying your shut the box program so that it pits two players against each other (one player tries to shut the box while the other tries to keep it open). The first player plays a regular round as described in Problem 5. Suppose he or she eliminates every number but 2, 3, and 6. The second player then begins a round with the numbers 1, 4, 5, 7, 8, and 9, the numbers that the first player had eliminated. If the second player loses, the first player gets another round to try to shut the box with the numbers that the second player had eliminated. Play continues until one of the players eliminates their entire list. In addition, each player should have their own time limit that only ticks down during their turn. If time runs out on your turn, you lose no matter what.

Python Packages

Large programming projects often have code spread throughout several folders and files. In order to get related files in different folders to communicate properly, the associated directories must be organized into a Python *packages*.

A package is simply a folder that contains a file called `__init__.py`. This file is always executed first whenever the package is used. A package must also have a file called `__main__.py` in order to be executable. Executing the package will run `__init__.py` and then `__main__.py`, but importing the package will only run `__init__.py`.

Use the regular syntax to import a module or subpackage that is in the current package, and use `from <subpackage.module> import <object>` to load a module within a subpackage. Once a name has been loaded into a package's `__init__.py`, other files in the same package can load the same name with `from . import <object>`. To access code in the directory one level above the current directory, use the syntax `from .. import <object>` This tells the interpreter to go up one level and import the object from there. This is called an *explicit relative import* and cannot be done in files that are executed directly (like `__main__.py`).

Finally, to execute a package, run Python from the shell with the flag `-m` (for "module-name") and exclude the extension `.py`.

```
$ python -m package_name
```

See `https://docs.python.org/3/tutorial/modules.html#packages` for examples and more details.

3 Introduction to NumPy

Lab Objective: *NumPy is a powerful Python package for manipulating data with multi-dimensional vectors. Its versatility and speed makes Python an ideal language for applied and computational mathematics. In this lab we introduce basic NumPy data structures and operations as a first step to numerical computing in Python.*

Arrays

In many algorithms, data can be represented mathematically as a *vector* or a *matrix*. Conceptually, a vector is just a list of numbers and a matrix is a two-dimensional list of numbers (a list of lists). However, even basic linear algebra operations like matrix multiplication are cumbersome to implement and slow to execute when data is stored this way. The *NumPy* module[1] [**?**, **?**, **?**] offers a much better solution.

The basic object in NumPy is the *array*, which is conceptually similar to a matrix. The NumPy array class is called `ndarray` (for "n-dimensional array"). The simplest way to explicitly create a 1-D `ndarray` is to define a list, then cast that list as an `ndarray` with NumPy's `array()` function.

```
>>> import numpy as np

# Create a 1-D array by passing a list into NumPy's array() function.
>>> np.array([8, 4, 6, 0, 2])
array([8, 4, 6, 0, 2])

# The string representation has no commas or an array() label.
>>> print(np.array([1, 3, 5, 7, 9]))
[1 3 5 7 9]
```

The alias "`np`" is standard in the Python community.

An `ndarray` can have arbitrarily many dimensions. A 2-D array is a 1-D array of 1-D arrays (like a list of lists), a 3-D array is a 1-D array of 2-D arrays (a list of lists of lists), and, more generally, an n-dimensional array is a 1-D array of $(n-1)$-dimensional arrays (a list of lists of lists of lists...). Each dimension is called an *axis*. For a 2-D array, the 0-axis indexes the rows and the 1-axis indexes the columns. Elements are accessed using brackets and indices, with the axes separated by commas.

[1] NumPy is *not* part of the standard library, but it is included in most Python distributions.

```
# Create a 2-D array by passing a list of lists into array().
>>> A = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(A)
[[1 2 3]
 [4 5 6]]

# Access elements of the array with brackets.
>>> print(A[0, 1], A[1, 2])
2 6

# The elements of a 2-D array are 1-D arrays.
>>> A[0]
array([1, 2, 3])
```

Problem 1. There are two main ways to perform matrix multiplication in NumPy: with NumPy's `dot()` function (`np.dot(A, B)`), or with the `@` operator (`A @ B`). Write a function that defines the following matrices as NumPy arrays.

$$A = \begin{bmatrix} 3 & -1 & 4 \\ 1 & 5 & -9 \end{bmatrix} \qquad B = \begin{bmatrix} 2 & 6 & -5 & 3 \\ 5 & -8 & 9 & 7 \\ 9 & -3 & -2 & -3 \end{bmatrix}$$

Return the matrix product AB.

For examples of array initialization and matrix multiplication, use object introspection in IPython to look up the documentation for `np.ndarray`, `np.array()` and `np.dot()`.

```
In [1]: import numpy as np

In [2]: np.array?       # press 'enter'
```

ACHTUNG!

The `@` operator was not introduced until Python 3.5. It triggers the `__matmul__()` magic method,[a] which for the `ndarray` is essentially a wrapper around `np.dot()`. If you are using a previous version of Python, always use `np.dot()` to perform basic matrix multiplication.

[a] See the lab on Object Oriented Programming for an overview of magic methods.

Basic Array Operations

NumPy arrays behave differently with respect to the binary arithmetic operators `+` and `*` than Python lists do. For lists, `+` concatenates two lists and `*` replicates a list by a scalar amount (strings also behave this way).

```
# Addition concatenates lists together.
>>> [1, 2, 3] + [4, 5, 6]
[1, 2, 3, 4, 5, 6]

# Mutliplication concatenates a list with itself a given number of times.
>>> [1, 2, 3] * 4
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

NumPy arrays act like mathematical vectors and matrices: `+` and `*` perform component-wise addition or multiplication.

```
>>> x, y = np.array([1, 2, 3]), np.array([4, 5, 6])

# Addition or multiplication by a scalar acts on each element of the array.
>>> x + 10                          # Add 10 to each entry of x.
array([11, 12, 13])
>>> x * 4                           # Multiply each entry of x by 4.
array([4, 8, 12])

# Add two arrays together (component-wise).
>>> x + y
array([5, 7, 9])

# Multiply two arrays together (component-wise).
>>> x * y
array([4, 10, 18])
```

Problem 2. Write a function that defines the following matrix as a NumPy array.

$$A = \begin{bmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \\ -5 & 3 & 1 \end{bmatrix}$$

Return the matrix $-A^3 + 9A^2 - 15A$.

In this context, $A^2 = AA$ (the matrix product, not the component-wise square). The somewhat surprising result is a demonstration of the Cayley-Hamilton theorem.

Array Attributes

An `ndarray` object has several attributes, some of which are listed below.

Attribute	Description
`dtype`	The type of the elements in the array.
`ndim`	The number of axes (dimensions) of the array.
`shape`	A tuple of integers indicating the size in each dimension.
`size`	The total number of elements in the array.

```
>>> A = np.array([[1, 2, 3], [4, 5, 6]])

# 'A' is a 2-D array with 2 rows, 3 columns, and 6 entries.
>>> print(A.ndim, A.shape, A.size)
2 (2, 3) 6
```

Note that `ndim` is the number of entries in `shape`, and that the `size` of the array is the product of the entries of `shape`.

Array Creation Routines

In addition to casting other structures as arrays via `np.array()`, NumPy provides efficient ways to create certain commonly-used arrays.

Function	Returns
`arange()`	Array of sequential integers (like `list(range())`).
`eye()`	2-D array with ones on the diagonal and zeros elsewhere.
`ones()`	Array of given shape and type, filled with ones.
`ones_like()`	Array of ones with the same shape and type as a given array.
`zeros()`	Array of given shape and type, filled with zeros.
`zeros_like()`	Array of zeros with the same shape and type as a given array.
`full()`	Array of given shape and type, filled with a specified value.
`full_like()`	Full array with the same shape and type as a given array.

Each of these functions accepts the keyword argument `dtype` to specify the data type. Common types include `np.bool_`, `np.int64`, `np.float64`, and `np.complex128`. To learn more about NumPy datatypes, see `https://numpy.org/devdocs/user/basics.types.html`.

```
# A 1-D array of 5 zeros.
>>> np.zeros(5)
array([0.,  0.,  0.,  0.,  0.])

# A 2x5 matrix (2-D array) of integer ones.
>>> np.ones((2, 5), dtype=np.int64)    # The shape is specified as a tuple.
array([[1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1]])

# The 2x2 identity matrix.
>>> I = np.eye(2)
>>> print(I)
[[ 1.  0.]
 [ 0.  1.]]

# Array of 3s the same size as 'I'.
>>> np.full_like(I, 3)              # Equivalent to np.full(I.shape, 3).
array([[ 3.,  3.],
       [ 3.,  3.]])
```

Unlike native Python data structures, **all elements of a NumPy array must be of the same data type**. To change an existing array's data type, use the array's `astype()` method.

```
# A list of integers becomes an array of integers.
>>> x = np.array([0, 1, 2, 3, 4])
>>> print(x)
[0 1 2 3 4]
>>> x.dtype
dtype('int64')

# Change the data type to one of NumPy's float types.
>>> x = x.astype(np.float64)        # Equivalent to x = np.float64(x).
>>> print(x)
[ 0.  1.  2.  3.  4.]               # Floats are displayed with periods.
>>> x.dtype
dtype('float64')
```

The following functions are for dealing with the diagonal, upper, or lower portion of an array.

Function	Description
`diag()`	Extract a diagonal or construct a diagonal array.
`tril()`	Get the lower-triangular portion of an array by replacing entries above the diagonal with zeros.
`triu()`	Get the upper-triangular portion of an array by replacing entries below the diagonal with zeros.

```
>>> A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Get only the upper triangular entries of 'A'.
>>> np.triu(A)
array([[1, 2, 3],
       [0, 5, 6],
       [0, 0, 9]])

# Get the diagonal entries of 'A' as a 1-D array.
>>> np.diag(A)
array([1, 5, 9])

# diag() can also be used to create a diagonal matrix from a 1-D array.
>>> np.diag([1, 11, 111])
array([[  1,   0,   0],
       [  0,  11,   0],
       [  0,   0, 111]])
```

See `http://docs.scipy.org/doc/numpy/reference/routines.array-creation.html` for the official documentation on NumPy's array creation routines.

Problem 3. Write a function that defines the following matrices as NumPy arrays using the functions presented in this section (not `np.array()`). Calculate the matrix product ABA. Change the data type of the resulting matrix to `np.int64`, then return it.

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad B = \begin{bmatrix} -1 & 5 & 5 & 5 & 5 & 5 & 5 \\ -1 & -1 & 5 & 5 & 5 & 5 & 5 \\ -1 & -1 & -1 & 5 & 5 & 5 & 5 \\ -1 & -1 & -1 & -1 & 5 & 5 & 5 \\ -1 & -1 & -1 & -1 & -1 & 5 & 5 \\ -1 & -1 & -1 & -1 & -1 & -1 & 5 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

Data Access

Array Slicing

Indexing for a 1-D NumPy array uses the slicing syntax `x[start:stop:step]`. If there is no colon, a single entry of that dimension is accessed. With a colon, a range of values is accessed. For multi-dimensional arrays, use a comma to separate slicing syntax for each axis.

```
# Make an array of the integers from 0 to 10 (exclusive).
>>> x = np.arange(10)
>>> x
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Access elements of the array with slicing syntax.
>>> x[3]                            # The element at index 3.
3
>>> x[:3]                           # Everything up to index 3 (exclusive).
array([0, 1, 2])
>>> x[3:]                           # Everything from index 3 on.
array([3, 4, 5, 6, 7, 8, 9])
>>> x[3:8]                          # The elements from index 3 to 8.
array([3, 4, 5, 6, 7])

>>> A = np.array([[0, 1, 2, 3, 4], [5, 6, 7, 8, 9]])
>>> A
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])

# Use a comma to separate the dimensions for multi-dimensional arrays.
>>> A[1, 2]                         # The element at row 1, column 2.
7
>>> A[:, 2:]                        # All of the rows, from column 2 on.
array([[2, 3, 4],
       [7, 8, 9]])
```

> **Note**
>
> Indexing and slicing operations return a *view* of the array. Changing a view of an array also changes the original array. In other words, **arrays are mutable**. To create a copy of an array, use `np.copy()` or the array's `copy()` method. Changes to a copy of an array does not affect the original array, but copying an array uses more time and memory than getting a view.

Fancy Indexing

So-called *fancy indexing* is a second way to access or change the elements of an array. Instead of using slicing syntax, provide either an array of indices or an array of boolean values (called a *mask*) to extract specific elements.

```
>>> x = np.arange(0, 50, 10)        # The integers from 0 to 50 by tens.
>>> x
array([0, 10, 20, 30, 40])

# An array of integers extracts the entries of 'x' at the given indices.
>>> index = np.array([3, 1, 4])     # Get the 3rd, 1st, and 4th elements.
>>> x[index]                        # Same as np.array([x[i] for i in index]).
array([30, 10, 40])

# A boolean array extracts the elements of 'x' at the same places as 'True'.
>>> mask = np.array([True, False, False, True, False])
>>> x[mask]                         # Get the 0th and 3rd entries.
array([0, 30])
```

Fancy indexing is especially useful for extracting or changing the values of an array that meet some sort of criterion. Use comparison operators like `<` and `==` to create masks.

```
>>> y = np.arange(10, 20, 2)        # Every other integer from 10 to 20.
>>> y
array([10, 12, 14, 16, 18])

# Extract the values of 'y' larger than 15.
>>> mask = y > 15                   # Same as np.array([i > 15 for i in y]).
>>> mask
array([False, False, False, True, True], dtype=bool)
>>> y[mask]                         # Same as y[y > 15]
array([16, 18])

# Change the values of 'y' that are larger than 15 to 100.
>>> y[mask] = 100
>>> print(y)
[10 12 14 100 100]
```

While indexing and slicing always return a view, fancy indexing always returns a copy.

Problem 4. Write a function that accepts a single array as input. Make a copy of the array, then use fancy indexing to set all negative entries of the copy to 0. Return the resulting array.

Array Manipulation

Shaping

An array's `shape` attribute describes its dimensions. Use `np.reshape()` or the array's `reshape()` method to give an array a new shape. The total number of entries in the old array and the new array must be the same in order for the shaping to work correctly. Using a `-1` in the new shape tuple makes the specified dimension as long as necessary.

```
>>> A = np.arange(12)              # The integers from 0 to 12 (exclusive).
>>> print(A)
[ 0  1  2  3  4  5  6  7  8  9 10 11]

# 'A' has 12 entries, so it can be reshaped into a 3x4 matrix.
>>> A.reshape((3, 4))               # The new shape is specified as a tuple.
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

# Reshape 'A' into an array with 2 rows and the appropriate number of columns.
>>> A.reshape((2, -1))
array([[ 0,  1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10, 11]])
```

Use `np.ravel()` to flatten a multi-dimensional array into a 1-D array and `np.transpose()` or the `T` attribute to transpose a 2-D array in the matrix sense.

```
>>> A = np.arange(12).reshape((3, 4))
>>> A
array([[0, 1,  2,  3],
       [4, 5,  6,  7],
       [8, 9, 10, 11]])

# Flatten 'A' into a one-dimensional array.
>>> np.ravel(A)                     # Equivalent to A.reshape(A.size)
array([0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])

# Transpose the matrix 'A'.
>>> A.T                             # Equivalent to np.transpose(A).
array([[0, 4,  8],
       [1, 5,  9],
       [2, 6, 10],
       [3, 7, 11]])
```

Note

By default, all NumPy arrays that can be represented by a single dimension, including column slices, are automatically reshaped into "flat" 1-D arrays. For example, by default an array will have 10 elements instead of 10 arrays with one element each. Though we usually represent vectors vertically in mathematical notation, NumPy methods such as `dot()` are implemented to purposefully work well with 1-D "row arrays".

```
>>> A = np.arange(10).reshape((2, 5))
>>> A
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])

# Slicing out a column of A still produces a "flat" 1-D array.
>>> x = A[:, 1]                     # All of the rows, column 1.
>>> x
array([1, 6])                       # Not array([[1],
>>> x.shape                         #             [6]])
(2,)
>>> x.ndim
1
```

However, it is occasionally necessary to change a 1-D array into a "column array". Use `np.reshape()`, `np.vstack()`, or slice the array and put `np.newaxis` on the second axis. Note that `np.transpose()` does not alter 1-D arrays.

```
>>> x = np.arange(3)
>>> x
array([0, 1, 2])

>>> x.reshape((-1, 1))               # Or x[:,np.newaxis] or np.vstack(x).
array([[0],
       [1],
       [2]])
```

Do not force a 1-D vector to be a column vector unless necessary.

Stacking

NumPy has functions for *stacking* two or more arrays with similar dimensions into a single block matrix. Each of these methods takes in a single tuple of arrays to be stacked in sequence.

Function	Description
`concatenate()`	Join a sequence of arrays along an existing axis
`hstack()`	Stack arrays in sequence horizontally (column wise).
`vstack()`	Stack arrays in sequence vertically (row wise).
`column_stack()`	Stack 1-D arrays as columns into a 2-D array.

```
>>> A = np.arange(6).reshape((2, 3))
>>> B = np.zeros((4, 3))

# vstack() stacks arrays vertically (row-wise).
>>> np.vstack((A, B, A))
array([[ 0.,  1.,  2.],            # A
       [ 3.,  4.,  5.],
       [ 0.,  0.,  0.],            # B
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  0.,  0.],
       [ 0.,  1.,  2.],            # A
       [ 3.,  4.,  5.]])

>>> A = A.T
>>> B = np.ones((3, 4))

# hstack() stacks arrays horizontally (column-wise).
>>> np.hstack((A, B, A))
array([[ 0.,  3.,  1.,  1.,  1.,  1.,  0.,  3.],
       [ 1.,  4.,  1.,  1.,  1.,  1.,  1.,  4.],
       [ 2.,  5.,  1.,  1.,  1.,  1.,  2.,  5.]])

# column_stack() stacks arrays horizontally, including 1-D arrays.
>>> np.column_stack((A, np.zeros(3), np.ones(3), np.full(3, 2)))
array([[ 0.,  3.,  0.,  1.,  2.],
       [ 1.,  4.,  0.,  1.,  2.],
       [ 2.,  5.,  0.,  1.,  2.]])
```

See `http://docs.scipy.org/doc/numpy-1.10.1/reference/routines.array-manipulation.html` for more array manipulation routines and documentation.

Problem 5. Write a function that defines the following matrices as NumPy arrays.

$$A = \begin{bmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \end{bmatrix} \qquad B = \begin{bmatrix} 3 & 0 & 0 \\ 3 & 3 & 0 \\ 3 & 3 & 3 \end{bmatrix} \qquad C = \begin{bmatrix} -2 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix}$$

Use NumPy's stacking functions to create and return the block matrix:

$$\begin{bmatrix} \mathbf{0} & A^\mathsf{T} & I \\ A & \mathbf{0} & \mathbf{0} \\ B & \mathbf{0} & C \end{bmatrix},$$

where I is the 3×3 identity matrix and each $\mathbf{0}$ is a matrix of all zeros of appropriate size.

A block matrix of this form is used in the interior point method for linear optimization.

Array Broadcasting

Many matrix operations make sense only when the two operands have the same shape, such as element-wise addition. *Array broadcasting* extends such operations to accept some (but not all) operands with different shapes, and occurs automatically whenever possible.

Suppose, for example, that we would like to add different values to the columns of an $m \times n$ matrix A. Adding a 1-D array x with the n entries to A will automatically do this correctly. To add different values to the different rows of A, first reshape a 1-D array of m values into a column array. Broadcasting then correctly takes care of the operation.

Broadcasting can also occur between two 1-D arrays, once they are reshaped appropriately.

```
>>> A = np.arange(12).reshape((4, 3))
>>> x = np.arange(3)
>>> A
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> x
array([0, 1, 2])

# Add the entries of 'x' to the corresponding columns of 'A'.
>>> A + x
array([[ 0,  2,  4],
       [ 3,  5,  7],
       [ 6,  8, 10],
       [ 9, 11, 13]])

>>> y = np.arange(0, 40, 10).reshape((4, 1))
>>> y
array([[ 0],
       [10],
       [20],
       [30]])

# Add the entries of 'y' to the corresponding rows of 'A'.
>>> A + y
array([[ 0,  1,  2],
       [13, 14, 15],
       [26, 27, 28],
       [39, 40, 41]])

# Add 'x' and 'y' together with array broadcasting.
>>> x + y
array([[ 0,  1,  2],
       [10, 11, 12],
       [20, 21, 22],
       [30, 31, 32]])
```

Numerical Computing with NumPy

Universal Functions

A *universal function* is one that operates on an entire array element-wise. Universal functions are significantly more efficient than using a loop to operate individually on each element of an array.

Function	Description
`abs()` or `absolute()`	Calculate the absolute value element-wise.
`exp()` / `log()`	Exponential (e^x) / natural log element-wise.
`maximum()` / `minimum()`	Element-wise maximum / minimum of two arrays.
`sqrt()`	The positive square-root, element-wise.
`sin()`, `cos()`, `tan()`, etc.	Element-wise trigonometric operations.

```
>>> x = np.arange(-2, 3)
>>> print(x, np.abs(x))             # Like np.array([abs(i) for i in x]).
[-2 -1  0  1  2] [2 1 0 1 2]

>>> np.sin(x)                       # Like np.array([math.sin(i) for i in x]).
array([-0.90929743, -0.84147098,  0.        ,  0.84147098,  0.90929743])
```

See `http://docs.scipy.org/doc/numpy/reference/ufuncs.html#available-ufuncs` for a more comprehensive list of universal functions.

Achtung!

The `math` module has many useful functions for numerical computations. However, most of these functions can only act on single numbers, not on arrays. NumPy functions can act on either scalars or entire arrays, but `math` functions tend to be a little faster for acting on scalars.

```
>>> import math

# Math and NumPy functions can both operate on scalars.
>>> print(math.exp(3), np.exp(3))
20.085536923187668 20.0855369232

# However, math functions cannot operate on arrays.
>>> x = np.arange(-2, 3)
>>> np.tan(x)
array([ 2.18503986, -1.55740772,  0.        ,  1.55740772, -2.18503986])
>>> math.tan(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: only length-1 arrays can be converted to Python scalars
```

Always use universal NumPy functions, not the `math` module, when working with arrays.

Other Array Methods

The `np.ndarray` class itself has many useful methods for numerical computations.

Method	Returns
`all()`	`True` if all elements evaluate to `True`.
`any()`	`True` if any elements evaluate to `True`.
`argmax()`	Index of the maximum value.
`argmin()`	Index of the minimum value.
`argsort()`	Indices that would sort the array.
`clip()`	restrict values in an array to fit within a given range
`max()`	The maximum element of the array.
`mean()`	The average value of the array.
`min()`	The minimum element of the array.
`sort()`	Return nothing; sort the array in-place.
`std()`	The standard deviation of the array.
`sum()`	The sum of the elements of the array.
`var()`	The variance of the array.

Each of these `np.ndarray` methods has an equivalent NumPy function. For example, `A.max()` and `np.max(A)` operate the same way. The one exception is the `sort()` function: `np.sort()` returns a sorted copy of the array, while `A.sort()` sorts the array in-place and returns nothing.

Every method listed can operate *along an axis* via the keyword argument `axis`. If `axis` is specified for a method on an n-D array, the return value is an $(n-1)$-D array, the specified axis having been collapsed in the evaluation process. If `axis` is not specified, the return value is usually a scalar. Refer to the NumPy Visual Guide in the appendix for more visual examples.

```
>>> A = np.arange(9).reshape((3, 3))
>>> A
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])

# Find the maximum value in the entire array.
>>> A.max()
8

# Find the minimum value of each column.
>>> A.min(axis=0)                  # np.array([min(A[:, i]) for i in range(3)↩
    ])
array([0, 1, 2])

# Compute the sum of each row.
>>> A.sum(axis=1)                  # np.array([sum(A[i, :]) for i in range(3)↩
    ])
array([3, 12, 21])
```

See `http://docs.scipy.org/doc/numpy/reference/generated/numpy.ndarray.html` for a more comprehensive list of array methods.

Problem 6. A matrix is called *row-stochastic*[a] if its rows each sum to 1. Stochastic matrices are fundamentally important for finite discrete random processes and some machine learning algorithms.

Write a function that accepts a matrix (as a 2-D NumPy array). Divide each row of the matrix by the row sum and return the new row-stochastic matrix. Use array broadcasting and the `axis` argument instead of a loop.

[a]Similarly, a matrix is called *column-stochastic* if its columns each sum to 1.

Problem 7. This problem comes from `https://projecteuler.net`.

In the 20×20 grid below, four numbers along a diagonal line have been marked in red.

08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 26 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48

The product of these numbers is $26 \times 63 \times 78 \times 14 = 1788696$. Write a function that returns the greatest product of four adjacent numbers in the same direction (up, down, left, right, or diagonally) in the grid.

For convenience, this array has been saved in the file `grid.npy`. Use the following syntax to extract the array:

```
>>> grid = np.load("grid.npy")
```

One way to approach this problem is to iterate through the rows and columns of the array, checking small slices of the array at each iteration and updating the current largest product. Array slicing, however, provides a much more efficient solution.

The naïve method for computing the greatest product of four adjacent numbers in a horizontal row might be as follows:

```
>>> winner = 0
>>> for i in range(20):
...     for j in range(17):
...         winner = max(np.prod(grid[i, j:j+4]), winner)
...
>>> winner
48477312
```

Instead, use array slicing to construct a single array where the (i, j)th entry is the product of the four numbers to the right of the (i, j)th entry in the original grid. Then find the largest element in the new array.

```
>>> np.max(grid[:, :-3] * grid[:, 1:-2] * grid[:, 2:-1] * grid[:, 3:])
48477312
```

Use slicing to similarly find the greatest products of four vertical, right diagonal, and left diagonal adjacent numbers.
(Hint: Consider drawing the portions of the grid that each slice in the above code covers, like the examples in the visual guide. Then draw the slices that produce vertical, right diagonal, or left diagonal sequences, and translate the pictures into slicing syntax.)

Achtung!

All of the examples in this lab use NumPy arrays, objects of type `np.ndarray`. NumPy also has a "matrix" data structure called `np.matrix` that was built specifically for MATLAB users who are transitioning to Python and NumPy. It behaves slightly differently than the regular array class, and can cause some unexpected and subtle problems.

For consistency (and your sanity), **never** use a NumPy matrix; **always** use NumPy arrays. If necessary, cast a matrix object as an array with `np.array()`.

Additional Material

Random Sampling

The submodule `np.random` holds many functions for creating arrays of random values chosen from probability distributions such as the uniform, normal, and multinomial distributions. It also contains some utility functions for getting non-distributional random samples, such as random integers or random samples from a given array.

Function	Description
`choice()`	Take random samples from a 1-D array.
`random()`	Uniformly distributed floats over [0, 1).
`randint()`	Random integers over a half-open interval.
`randn()`	Sample from the standard normal distribution.
`permutation()`	Randomly permute a sequence / generate a random sequence.

Function	Distribution
`beta()`	Beta distribution over [0, 1].
`binomial()`	Binomial distribution.
`exponential()`	Exponential distribution.
`gamma()`	Gamma distribution.
`geometric()`	Geometric distribution.
`multinomial()`	Multivariate generalization of the binomial distribution.
`multivariate_normal()`	Multivariate generalization of the normal distribution.
`normal()`	Normal / Gaussian distribution.
`poisson()`	Poisson distribution.
`uniform()`	Uniform distribution.

Note that many of these functions have counterparts in the standard library's `random` module. These NumPy functions, however, are much better suited for working with large collections of random samples.

```
# 5 uniformly distributed values in the interval [0, 1).
>>> np.random.random(5)
array([ 0.21845499,  0.73352537,  0.28064456,  0.66878454,  0.44138609])

# A 2x5 matrix (2-D array) of integers in the interval [10, 20).
>>> np.random.randint(10, 20, (2, 5))
array([[17, 12, 13, 13, 18],
       [16, 10, 12, 18, 12]])
```

Saving and Loading Arrays

It is often useful to save an array as a file for later use. NumPy provides several easy methods for saving and loading array data.

Function	Description
`save()`	Save a single array to a `.npy` file.
`savez()`	Save multiple arrays to a `.npz` file.
`savetxt()`	Save a single array to a `.txt` file.
`load()`	Load and return an array or arrays from a `.npy` or `.npz` file.
`loadtxt()`	Load and return an array from a text file.

```
# Save a 100x100 matrix of uniformly distributed random values.
>>> x = np.random.random((100, 100))
>>> np.save("uniform.npy", x)       # Or np.savetxt("uniform.txt", x).

# Read the array from the file and check that it matches the original.
>>> y = np.load("uniform.npy")      # Or np.loadtxt("uniform.txt").
>>> np.allclose(x, y)               # Check that x and y are close entry-wise.
True
```

To save several arrays to a single file, specify a keyword argument for each array in `np.savez()`. Then `np.load()` will return a dictionary-like object with the keyword parameter names from the save command as the keys.

```
# Save two 100x100 matrices of normally distributed random values.
>>> x = np.random.randn(100, 100)
>>> y = np.random.randn(100, 100)
>>> np.savez("normal.npz", first=x, second=y)

# Read the arrays from the file and check that they match the original.
>>> arrays = np.load("normal.npz")
>>> np.allclose(x, arrays["first"])
True
>>> np.allclose(y, arrays["second"])
True
```

4 Object-oriented Programming

Lab Objective: *Python is a class-based language. A* class *is a blueprint for an object that binds together specified variables and routines. Creating and using custom classes is often a good way to write clean, efficient, well-designed programs. In this lab we learn how to define and use Python classes. In subsequent labs we will often create customized classes for use in algorithms.*

Classes

A Python *class* is a code block that defines a custom object and determines its behavior. The `class` key word defines and names a new class. Other statements follow, indented below the class name, to determine the behavior of objects instantiated by the class.

A class needs a method called a *constructor* that is called whenever the class instantiates a new object. The constructor specifies the initial state of the object. In Python, a class's constructor is always named `__init__()`. For example, the following code defines a class for storing information about backpacks.

```
class Backpack:
    """A Backpack object class. Has a name and a list of contents.

    Attributes:
        name (str): the name of the backpack's owner.
        contents (list): the contents of the backpack.
    """
    def __init__(self, name):           # This function is the constructor.
        """Set the name and initialize an empty list of contents.

        Parameters:
            name (str): the name of the backpack's owner.
        """
        self.name = name                # Initialize some attributes.
        self.contents = []
```

An *attribute* is a variable stored within an object. The `Backpack` class has two attributes: `name` and `contents`. In the body of the class definition, attributes are assigned and accessed via

the identifier `self`. The identifier `self` refers to the object internally once it has been created. In the previous example, the line `self.name = name` stores the input argument `name` to the attribute `self.name`.

Instantiation

The `class` code block above only defines a blueprint for backpack objects. To create an actual backpack object, call the class name like a function. This triggers the constructor and returns a new *instance* of the class, an object whose type is the class.

```
# Import the Backpack class and instantiate an object called 'my_backpack'.
>>> from object_oriented import Backpack
>>> my_backpack = Backpack("Fred")
>>> type(my_backpack)
<class 'object_oriented.Backpack'>

# Access the object's attributes with a period and the attribute name.
>>> print(my_backpack.name, my_backpack.contents)
Fred []

# The object's attributes can be modified after instantiation.
>>> my_backpack.name = "George"
>>> print(my_backpack.name, my_backpack.contents)
George []
```

> **Note**
>
> Every object in Python has some built-in attributes. For example, modules have a `__name__` attribute that identifies the scope in which it is being executed. If the module is being run directly, and not imported, then `__name__` is set to `"__main__"`. Therefore, any commands under an `if __name__ == "__main__":` clause are ignored when the module is imported.

Methods

In addition to storing variables as attributes, classes can have functions attached to them. A function that belongs to a specific class is called a *method*.

```
class Backpack:
    # ...
    def put(self, item):
        """Add an item to the backpack's list of contents."""
        self.contents.append(item)  # Use 'self.contents', not just 'contents'.

    def take(self, item):
        """Remove an item from the backpack's list of contents."""
        self.contents.remove(item)
```

The first argument of each method must be `self`, to give the method access to the attributes and other methods of the class. The `self` argument is only included in the declaration of the class methods, **not** when calling the methods on an instantiation of the class.

```
# Add some items to the backpack object.
>>> my_backpack.put("notebook")         # my_backpack is passed implicitly to
>>> my_backpack.put("pencils")          # Backpack.put() as the first argument.
>>> my_backpack.contents
['notebook', 'pencils']

# Remove an item from the backpack.     # This is equivalent to
>>> my_backpack.take("pencils")         # Backpack.take(my_backpack, "pencils")
>>> my_backpack.contents
['notebook']
```

Problem 1. Expand the `Backpack` class to match the following specifications.

1. Modify the constructor so that it accepts three total arguments: `name`, `color`, and `max_size` (in that order). Make `max_size` a keyword argument that defaults to 5. Store each input as an attribute.

2. Modify the `put()` method to check that the backpack does not go over capacity. If there are already `max_size` items or more, print "No Room!" and do not add the item to the contents list.

3. Write a new method called `dump()` that resets the contents of the backpack to an empty list. This method should not receive any arguments (except `self`).

4. Documentation is especially important in classes so that the user knows what an object's attributes represent and how to use methods appropriately. Update (or write) the docstrings for the `__init__()`, `put()`, and `dump()` methods, as well as the actual class docstring (under `class` but before `__init__()`) to reflect the changes from parts 1-3 of this problem.

To ensure that your class works properly, write a test function outside of the `Backpack` class that instantiates and analyzes a `Backpack` object.

```
def test_backpack():
    testpack = Backpack("Barry", "black")       # Instantiate the object.
    if testpack.name != "Barry":                # Test an attribute.
        print("Backpack.name assigned incorrectly")
    for item in ["pencil", "pen", "paper", "computer"]:
        testpack.put(item)                      # Test a method.
    print("Contents:", testpack.contents)
    # ...
```

Inheritance

To create a new class that is similar to one that already exists, it is often better to *inherit* the methods and attributes from an existing class rather than create a new class from scratch. This creates a *class hierarchy*: a class that inherits from another class is called a *subclass*, and the class that a subclass inherits from is called a *superclass*. To define a subclass, add the name of the superclass as an argument at the end of the `class` declaration.

For example, since a knapsack is a kind of backpack (but not all backpacks are knapsacks), we create a special `Knapsack` subclass that inherits the structure and behaviors of the `Backpack` class and adds some extra functionality.

```
# Inherit from the Backpack class in the class definition.
class Knapsack(Backpack):
    """A Knapsack object class. Inherits from the Backpack class.
    A knapsack is smaller than a backpack and can be tied closed.

    Attributes:
        name (str): the name of the knapsack's owner.
        color (str): the color of the knapsack.
        max_size (int): the maximum number of items that can fit inside.
        contents (list): the contents of the backpack.
        closed (bool): whether or not the knapsack is tied shut.
    """
    def __init__(self, name, color, max_size=3):
        """Use the Backpack constructor to initialize the name, color,
        and max_size attributes. A knapsack only holds 3 item by default.

        Parameters:
            name (str): the name of the knapsack's owner.
            color (str): the color of the knapsack.
            max_size (int): the maximum number of items that can fit inside.
        """
        Backpack.__init__(self, name, color, max_size)
        self.closed = True
```

A subclass may have new attributes and methods that are unavailable to the superclass, such as the `closed` attribute in the `Knapsack` class. If methods from the superclass need to be changed for the subclass, they can be overridden by defining them again in the subclass. New methods can be included normally.

```
class Knapsack(Backpack):
    # ...
    def put(self, item):            # Override the put() method.
        """If the knapsack is untied, use the Backpack.put() method."""
        if self.closed:
            print("I'm closed!")
        else:                       # Use Backpack's original put().
            Backpack.put(self, item)
```

```
    def take(self, item):           # Override the take() method.
        """If the knapsack is untied, use the Backpack.take() method."""
        if self.closed:
            print("I'm closed!")
        else:
            Backpack.take(self, item)

    def weight(self):               # Define a new method just for knapsacks.
        """Calculate the weight of the knapsack by counting the length of the
        string representations of each item in the contents list.
        """
        return sum(len(str(item)) for item in self.contents)
```

Since `Knapsack` inherits from `Backpack`, a knapsack object **is** a backpack object. All methods defined in the `Backpack` class are available as instances of the `Knapsack` class. For example, the `dump()` method is available even though it is not defined explicitly in the `Knapsack` class.

The built-in function `issubclass()` shows whether or not one class is derived from another. Similarly, `isinstance()` indicates whether or not an object belongs to a specified class hierarchy. Finally, `hasattr()` shows whether or not a class or object **has** a specified attribute or method.

```
>>> from object_oriented import Knapsack
>>> my_knapsack = Knapsack("Brady", "brown")

# A Knapsack is a Backpack, but a Backpack is not a Knapsack.
>>> print(issubclass(Knapsack, Backpack), issubclass(Backpack, Knapsack))
True False
>>> isinstance(my_knapsack, Knapsack) and isinstance(my_knapsack, Backpack)
True

# The put() and take() method now require the knapsack to be open.
>>> my_knapsack.put('compass')
I'm closed!

# Open the knapsack and put in some items.
>>> my_knapsack.closed = False
>>> my_knapsack.put("compass")
>>> my_knapsack.put("pocket knife")
>>> my_knapsack.contents
['compass', 'pocket knife']

# The Knapsack class has a weight() method, but the Backpack class does not.
>>> print(hasattr(my_knapsack, 'weight'), hasattr(my_backpack, 'weight'))
True False

# The dump method is inherited from the Backpack class.
>>> my_knapsack.dump()
>>> my_knapsack.contents
[]
```

Problem 2. Write a `Jetpack` class that inherits from the `Backpack` class.

1. Override the constructor so that in addition to a name, color, and maximum size, it also accepts an amount of fuel. Change the default value of `max_size` to 2, and set the default value of fuel to 10. Store the fuel as an attribute.
2. Add a `fly()` method that accepts an amount of fuel to be burned and decrements the fuel attribute by that amount. If the user tries to burn more fuel than remains, print "Not enough fuel!" and do not decrement the fuel.
3. Override the `dump()` method so that both the contents and the fuel tank are emptied.
4. Write clear, detailed docstrings for the class and each of its methods.

NOTE

All classes are subclasses of the built-in `object` class, even if no parent class is specified in the class definition. In fact, the syntax "`class ClassName(object):`" is not uncommon (or incorrect) for the class declaration, and is equivalent to the simpler "`class ClassName:`".

Magic Methods

A *magic method* is a special method used to make an object behave like a built-in data type. Magic methods begin and end with two underscores, like the constructor `__init__()`. Every Python object is automatically endowed with several magic methods, which can be revealed through IPython.

```
In [1]: %run object_oriented.py

In [2]: b = Backpack("Oscar", "green")

In [3]: b.          # Press 'tab' to see standard methods and attributes.
          color    max_size take()
          contents name
          dump()   put()

In [3]: b.__        # Press 'tab' to see magic methods and hidden attributes.
          __add__()           __getattribute__    __new__()
          __class__           __gt__              __reduce__()
          __delattr__         __hash__            __reduce_ex__()
          __dict__            __init__()          __repr__
          __dir__()           __init_subclass__() __setattr__
          __doc__             __le__              __sizeof__()
          __eq__              __lt__()            __str__
          __format__()        __module__          __subclasshook__()
          __ge__              __ne__              __weakref__
```

> **Note**
>
> Many programming languages distinguish between *public* and *private* variables. In Python, all attributes are public, period. However, attributes that start with an underscore are hidden from the user, which is why magic methods do not show up at first in the preceding code box.

The more common magic methods define how an object behaves with respect to addition and other binary operations. For example, how should addition be defined for backpacks? A simple option is to add the number of contents. Then if backpack `A` has 3 items and backpack `B` has 5 items, `A + B` should return 8. To incorporate this idea, we implement the `__add__()` magic method.

```
class Backpack:
    # ...
    def __add__(self, other):
        """Add the number of contents of each Backpack."""
        return len(self.contents) + len(other.contents)
```

Using the `+` binary operator on two `Backpack` objects calls the class's `__add__()` method. The object on the left side of the `+` is passed in to `__add__()` as `self` and the object on the right side of the `+` is passed in as `other`.

```
>>> pack1 = Backpack("Rose", "red")
>>> pack2 = Backpack("Carly", "cyan")

# Put some items in the backpacks.
>>> pack1.put("textbook")
>>> pack2.put("water bottle")
>>> pack2.put("snacks")

# Add the backpacks together.
>>> pack1 + pack2                   # Equivalent to pack1.__add__(pack2).
3
```

Comparisons

Magic methods also facilitate object comparisons. For example, the `__lt__()` method corresponds to the `<` operator. Suppose one backpack is considered "less" than another if it has fewer items in its list of contents.

```
class Backpack(object)
    # ...
    def __lt__(self, other):
        """If 'self' has fewer contents than 'other', return True.
        Otherwise, return False.
        """
        return len(self.contents) < len(other.contents)
```

Using the `<` binary operator on two `Backpack` objects calls `__lt__()`. As with addition, the object on the left side of the `<` operator is passed to `__lt__()` as `self`, and the object on the right is passed in as `other`.

```
>>> pack1, pack2 = Backpack("Maggy", "magenta"), Backpack("Yolanda", "yellow")
>>> pack1 < pack2                    # Equivalent to pack1.__lt__(pack2).
False

>>> pack2.put('pencils')
>>> pack1 < pack2
True
```

Comparison methods should return either `True` or `False`, while methods like `__add__()` might return a numerical value or another kind of object.

Method	Arithmetic Operator	Method	Comparison Operator
`__add__()`	`+`	`__lt__()`	`<`
`__sub__()`	`-`	`__le__()`	`<=`
`__mul__()`	`*`	`__gt__()`	`>`
`__pow__()`	`**`	`__ge__()`	`>=`
`__truediv__()`	`/`	`__eq__()`	`==`
`__floordiv__()`	`//`	`__ne__()`	`!=`

Table 4.1: Common magic methods for arithmetic and comparisons. What each of these operations do is up to the programmer and should be carefully documented. For more methods and details, see `https://docs.python.org/3/reference/datamodel.html#special-method-names`.

Problem 3. Endow the `Backpack` class with two additional magic methods:

1. The `__eq__()` magic method is used to determine if two objects are equal, and is invoked by the `==` operator. Implement the `__eq__()` magic method for the `Backpack` class so that two `Backpack` objects are equal if and only if they have the same name, color, and number of contents.

2. The `__str__()` magic method returns the string representation of an object. This method is invoked by `str()` and used by `print()`. Implement the `__str__()` method in the `Backpack` class so that printing a `Backpack` object yields the following output (that is, construct and return the following string).

   ```
   Owner:      <name>
   Color:      <color>
   Size:       <number of items in contents>
   Max Size:   <max_size>
   Contents:   [<item1>, <item2>, ...]
   ```

   (Hint: Use the tab and newline characters `'\t'` and `'\n'` to align output nicely.)

Achtung!

Magic methods for comparison are **not** automatically related. For example, even though the `Backpack` class implements the magic methods for `<` and `==`, two `Backpack` objects cannot respond to the `<=` operator unless `__le__()` is explicitly defined. The exception to this rule is the `!=` operator: as long as `__eq__()` is defined, `A!=B` is `False` if and only if `A==B` is `True`.

Problem 4. Write a `ComplexNumber` class from scratch.

1. Complex numbers are denoted $a+bi$ where $a, b \in \mathbb{R}$ and $i = \sqrt{-1}$. Write the constructor so it accepts two numbers. Store the first as `self.real` and the second as `self.imag`.
2. The *complex conjugate* of $a+bi$ is defined as $\overline{a+bi} = a-bi$. Write a `conjugate()` method that returns the object's complex conjugate as a new `ComplexNumber` object.
3. Add the following magic methods:
   (a) Implement `__str__()` so that $a+bi$ is printed out as `(a+bj)` for $b \geq 0$ and `(a-bj)` for $b < 0$.
   (b) The *magnitude* of $a+bi$ is $|a+bi| = \sqrt{a^2+b^2}$. The `__abs__()` magic method determines the output of the built-in `abs()` function (absolute value). Implement `__abs__()` so that it returns the magnitude of the complex number.
   (c) Two `ComplexNumber` objects are equal if and only if they have the same real and imaginary parts. Implement `__eq__()` so that it compares the `ComplexNumber` object with another `ComplexNumber` object and returns a bool indicating whether they are equal.
   (d) Implement `__add__()`, `__sub__()`, `__mul__()`, and `__truediv__()` appropriately. Each of these should return a new `ComplexNumber` object.

Write a function to test your class by comparing it to Python's built-in `complex` type.

```
def test_ComplexNumber(a, b):
    py_cnum, my_cnum = complex(a, b), ComplexNumber(a, b)

    # Validate the constructor.
    if my_cnum.real != a or my_cnum.imag != b:
        print("__init__() set self.real and self.imag incorrectly")

    # Validate conjugate() by checking the new number's imag attribute.
    if py_cnum.conjugate().imag != my_cnum.conjugate().imag:
        print("conjugate() failed for", py_cnum)

    # Validate __str__().
    if str(py_cnum) != str(my_cnum):
        print("__str__() failed for", py_cnum)
```

```
    # ...
```

Additional Material

Static Attributes

Attributes that are accessed through `self` are called *instance* attributes because they are bound to a particular instance of the class. In contrast, a *static* attribute is one that is shared between all instances of the class. To make an attribute static, declare it inside of the `class` block but outside of any of the class's methods, and do not use `self`. Since the attribute is not tied to a specific instance of the class, it may be accessed or changed via the class name without even instantiating the class at all.

```
class Backpack:
    # ...
    brand = "Adidas"                # Backpack.brand is a static attribute.
```

```
>>> pack1, pack2 = Backpack("Bill", "blue"), Backpack("William", "white")
>>> print(pack1.brand, pack2.brand, Backpack.brand)
Adidas Adidas Adidas

# Change the brand name for the class to change it for all class instances.
>>> Backpack.brand = "Nike"
>>> print(pack1.brand, pack2.brand, Backpack.brand)
Nike Nike Nike
```

Static Methods

Individual class methods can also be static. A static method cannot be dependent on the attributes of individual instances of the class, so there can be no references to `self` inside the body of the method and `self` is **not** listed as an argument in the function definition. Thus static methods only have access to static attributes and other static methods. Include the tag `@staticmethod` above the function definition to designate a method as static.

```
class Backpack:
    # ...
    @staticmethod
    def origin():                   # Do not use 'self' as a parameter.
        print("Manufactured by " + Backpack.brand + ", inc.")
```

```
# Static methods can be called without instantiating the class.
>>> Backpack.origin()
Manufactured by Nike, inc.

# The method can also be accessed by individual class instances.
>>> pack = Backpack("Larry", "lime")
>>> pack.origin()
Manufactured by Nike, inc.
```

To practice these principles, consider adding a static attribute to the `Backpack` class to serve as a counter for a unique ID. In the constructor for the `Backpack` class, add an instance variable called `self.ID`. Set this ID based on the static ID variable, then increment the static ID so that the next `Backpack` object will have a different ID.

More Magic Methods

Consider how the following methods might be implemented for the `Backpack` class. These methods are particularly important for custom data structure classes.

Method	Operation	Trigger Function
`__bool__()`	Truth value	`bool()`
`__len__()`	Object length or size	`len()`
`__repr__()`	Object representation	`repr()`
`__getitem__()`	Indexing and slicing	`self[index]`
`__setitem__()`	Assignment via indexing	`self[index] = x`
`__iter__()`	Iteration over the object	`iter()`
`__reversed__()`	Reverse iteration over the object	`reversed()`
`__contains__()`	Membership testing	`in`

See `https://docs.python.org/3/reference/datamodel.html#special-method-names` for more details and documentation on all magic methods.

Hashing

A *hash function* is a method that maps objects to identifiers which can be numeric or alphanumeric strings. These identifiers are known as *hash values*. The built-in `hash()` function calculates an object's hash value by calling its `__hash__()` magic method. While an object has one unique hash value, it is possible that two distinct objects may share the same hash value. This is called a *collision*. A good hash function is one that minimizes the probability of collisions occurring.

In Python, the built-in `set` and `dict` structures use hash values to store and retrieve objects in memory quickly. If an object is *unhashable*, it cannot be put in a set or be used as a key in a dictionary. See `https://docs.python.org/3/glossary.html#term-hashable` for details. If the `__hash__()` method is not defined, the default hash value is the object's memory address (accessible via the built-in function `id()`) divided by 16, rounded down to the nearest integer. However, two objects that compare as equal via the `__eq__()` magic method must have the same hash value. The following simple `__hash__()` method for the `Backpack` class conforms to this rule and returns an integer.

```
class Backpack:
    # ...
    def __hash__(self):
        return hash(self.name) ^ hash(self.color) ^ hash(len(self.contents))
```

The caret operator `^` is a bitwise XOR (exclusive or). The bitwise AND operator `&` and the bitwise OR operator `|` are also good choices to use.

See `https://docs.python.org/3/reference/datamodel.html#object.__hash__` for more on hashing.

5 Introduction to Matplotlib

Lab Objective: *Matplotlib is the most commonly used data visualization library in Python. Being able to visualize data helps to determine patterns and communicate results and is a key component of applied and computational mathematics. In this lab we introduce techniques for visualizing data in 1, 2, and 3 dimensions. The plotting techniques presented here will be used in the remainder of the labs in the manual.*

Line Plots

Raw numerical data is rarely helpful unless it can be visualized. The quickest way to visualize a simple 1-dimensional array is with a *line plot*. The following code creates an array of outputs of the function $f(x) = x^2$, then visualizes the array using the `matplotlib` module[1] [?].

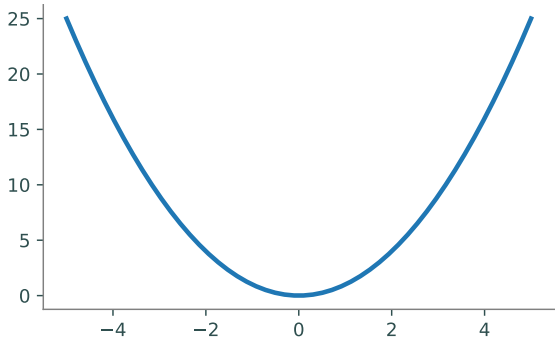
```
>>> import numpy as np
>>> from matplotlib import pyplot as plt

>>> y = np.arange(-5, 6)**2
>>> y
array([25, 16,  9,  4,  1,  0,  1,  4,  9, 16, 25])

# Visualize the plot.
>>> plt.plot(y)                     # Draw the line plot.
[<matplotlib.lines.Line2D object at 0x1084762d0>]
>>> plt.show()                      # Reveal the resulting plot.
```

The result is shown in Figure 5.1a. Just as `np` is a standard alias for NumPy, `plt` is a standard alias for `matplotlib.pyplot` in the Python community.

The call `plt.plot(y)` creates a figure and draws straight lines connecting the entries of `y` relative to the y-axis. The x-axis is (by default) the index of the array, which in this case is the integers from 0 to 10. Calling `plt.show()` then displays the figure.

[1] Like NumPy, Matplotlib is *not* part of the Python standard library, but it is included in most Python distributions. See `https://matplotlib.org/` for the complete Matplotlib documentation.

(a) `plt.plot(y)` uses the indices of the array for the x-axis.



(b) `plt.plot(x,y)` specifies both the domain and the range.

Figure 5.1: Plots of $f(x) = x^2$ over the interval $[-5, 5]$.

Problem 1. NumPy's `random` module has tools for sampling from probability distributions. For instance, `np.random.normal()` draws samples from the normal (Gaussian) distribution. The `size` parameter specifies the shape of the resulting array.

```
>>> np.random.normal(size=(2, 3))    # Get a 2x3 array of samples.
array([[ 1.65896515, -0.43236783, -0.99390897],
       [-0.35753688, -0.76738306,  1.29683025]])
```

Write a function that accepts an integer n as input.

1. Use `np.random.normal()` to create an $n \times n$ array of values randomly sampled from the standard normal distribution.
2. Compute the mean of each row of the array.
   (Hint: Use `np.mean()` and specify the `axis` keyword argument.)
3. Return the variance of these means.
   (Hint: Use `np.var()` to calculate the variance).

Define another function that creates an array of the results of the first function with inputs $n = 100,\ 200,\ \ldots,\ 1000$. Plot (and show) the resulting array.

Specifying a Domain

An obvious problem with Figure 5.1a is that the x-axis does not correspond correctly to the y-axis for the function $f(x) = x^2$ that is being drawn. To correct this, define an array `x` for the domain, then use it to calculate the image `y = f(x)`. The command `plt.plot(x,y)` plots `x` against `y` by drawing a line between the consecutive points `(x[i], y[i])`.

Another problem with Figure 5.1a is its poor resolution: the curve is visibly bumpy, especially near the bottom of the curve. NumPy's `linspace()` function makes it easy to get a higher-resolution domain. Recall that `np.arange()` returns an array of evenly-spaced values in a given interval, where

the **spacing** between the entries is specified. In contrast, `np.linspace()` creates an array of evenly-spaced values in a given interval where the **number of elements** is specified.

```
# Get 4 evenly-spaced values between 0 and 32 (including endpoints).
>>> np.linspace(0, 32, 4)
array([  0.        ,  10.66666667,  21.33333333,  32.        ])

# Get 50 evenly-spaced values from -5 to 5 (including endpoints).
>>> x = np.linspace(-5, 5, 50)
>>> y = x**2                          # Calculate the image of f(x) = x**2.
>>> plt.plot(x, y)
>>> plt.show()
```

The resulting plot is shown in Figure 5.1b. This time, the x-axis correctly matches up with the y-axis. The resolution is also much better because `x` and `y` have 50 entries each instead of only 10.

Subsequent calls to `plt.plot()` modify the same figure until `plt.show()` is executed, which displays the current figure and resets the system. This behavior can be altered by specifying separate figures or axes, which we will discuss shortly.

Note

Plotting can seem a little mystical because the actual plot doesn't appear until `plt.show()` is executed. Matplotlib's *interactive mode* allows the user to see the plot be constructed one piece at a time. Use `plt.ion()` to turn interactive mode on and `plt.ioff()` to turn it off. This is very useful for quick experimentation. Try executing the following commands in IPython:

```
In [1]: import numpy as np
In [2]: from matplotlib import pyplot as plt

# Turn interactive mode on and make some plots.
In [3]: plt.ion()
In [4]: x = np.linspace(1, 4, 100)
In [5]: plt.plot(x, np.log(x))
In [6]: plt.plot(x, np.exp(x))

# Clear the figure, then turn interactive mode off.
In [7]: plt.clf()
In [8]: plt.ioff()
```

Use interactive mode **only** with IPython. Using interactive mode in a non-interactive setting may freeze the window or cause other problems.

Problem 2. Write a function that plots the functions $\sin(x)$, $\cos(x)$, and $\arctan(x)$ on the domain $[-2\pi, 2\pi]$ (use `np.pi` for π). Make sure the domain is refined enough to produce a figure with good resolution.

Plot Customization

`plt.plot()` receives several keyword arguments for customizing the drawing. For example, the color and style of the line are specified by the following string arguments.

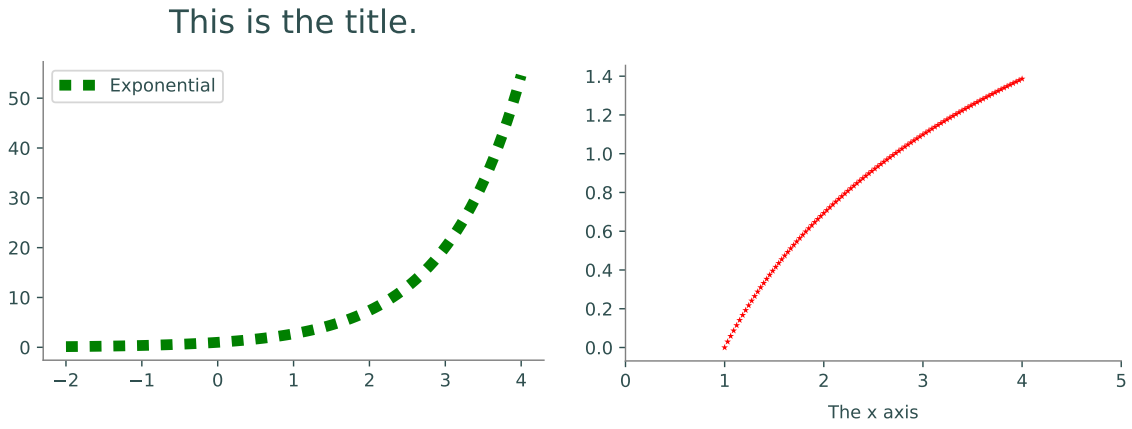
Key	Color
`'b'`	blue
`'g'`	green
`'r'`	red
`'c'`	cyan
`'k'`	black

Key	Style
`'-'`	solid line
`'--'`	dashed line
`'-.'`	dash-dot line
`':'`	dotted line
`'o'`	circle marker

Specify one or both of these string codes as the third argument to `plt.plot()` to change from the default color and style. Other `plt` functions further customize a figure.

Function	Description
`legend()`	Place a legend in the plot
`title()`	Add a title to the plot
`xlim()` / `ylim()`	Set the limits of the x- or y-axis
`xlabel()` / `ylabel()`	Add a label to the x- or y-axis

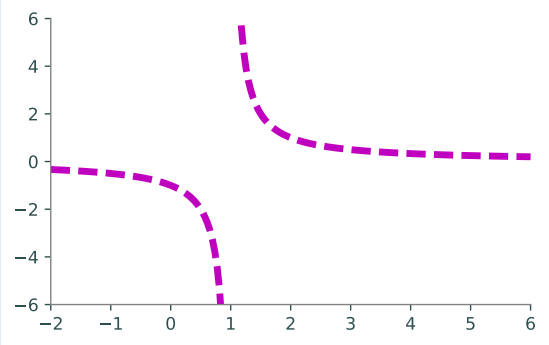
```
>>> x1 = np.linspace(-2, 4, 100)
>>> plt.plot(x1, np.exp(x1), 'g:', linewidth=6, label="Exponential")
>>> plt.title("This is the title.", fontsize=18)
>>> plt.legend(loc="upper left")    # plt.legend() uses the 'label' argument of
>>> plt.show()                      # plt.plot() to create a legend.

>>> x2 = np.linspace(1, 4, 100)
>>> plt.plot(x2, np.log(x2), 'r*', markersize=4)
>>> plt.xlim(0, 5)                  # Set the visible limits of the x axis.
>>> plt.xlabel("The x axis")        # Give the x axis a label.
>>> plt.show()
```



Problem 3. Write a function to plot the curve $f(x) = \frac{1}{x-1}$ on the domain $[-2, 6]$.

1. Although $f(x)$ has a discontinuity at $x = 1$, a single call to `plt.plot()` in the usual way will make the curve look continuous. Split up the domain into $[-2, 1)$ and $(1, 6]$. Plot the two sides of the curve separately so that the graph looks discontinuous at $x = 1$.
2. Plot both curves with a dashed magenta line. Set the keyword argument `linewidth` (or `lw`) of `plt.plot()` to 4 to make the line a little thicker than the default setting.
3. Use `plt.xlim()` and `plt.ylim()` to change the range of the x-axis to $[-2, 6]$ and the range of the y-axis to $[-6, 6]$.

The plot should resemble the figure below.



Figures, Axes, and Subplots

The window that `plt.show()` reveals is called a *figure*, stored in Python as a `plt.Figure` object. A space on a figure where a plot is drawn is called an *axes*, a `plt.Axes` object. A figure can have multiple axes, and a single program may create several figures. There are several ways to create or grab figures and axes with `plt` functions.

Function	Description
`axes()`	Add an axes to the current figure
`figure()`	Create a new figure or grab an existing figure
`gca()`	Get the current axes
`gcf()`	Get the current figure
`subplot()`	Add a single subplot to the current figure
`subplots()`	Create a figure and add several subplots to it

Usually when a figure has multiple axes, they are organized into non-overlapping *subplots*. The command `plt.subplot(nrows, ncols, plot_number)` creates an axes in a subplot grid where `nrows` is the number of rows of subplots in the figure, `ncols` is the number of columns, and `plot_number` specifies which subplot to modify. If the inputs for `plt.subplot()` are all integers, the commas between the entries can be omitted. For example, `plt.subplot(3,2,2)` can be shortened to `plt.subplot(322)`.

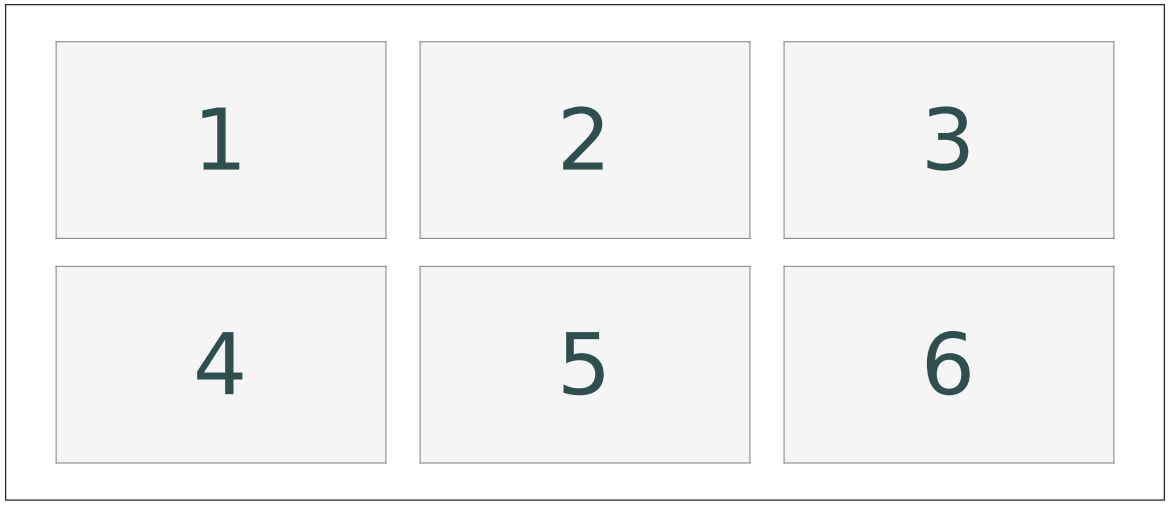
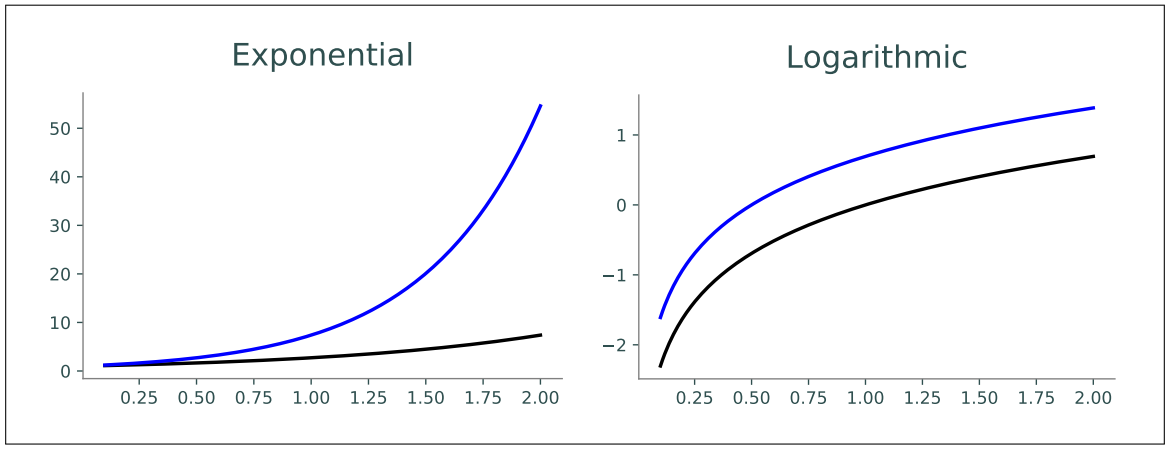


Figure 5.3: The layout of subplots with `plt.subplot(2,3,i)` (2 rows, 3 columns), where `i` is the index pictured above. The outer border is the figure that the axes belong to.

```
>>> x = np.linspace(.1, 2, 200)
# Create a subplot to cover the left half of the figure.
>>> ax1 = plt.subplot(121)
>>> ax1.plot(x, np.exp(x), 'k', lw=2)
>>> ax1.plot(x, np.exp(2*x), 'b', lw=2)
>>> plt.title("Exponential", fontsize=18)

# Create another subplot to cover the right half of the figure.
>>> ax2 = plt.subplot(122)
>>> ax2.plot(x, np.log(x), 'k', lw=2)
>>> ax2.plot(x, np.log(2*x), 'b', lw=2)
>>> ax2.set_title("Logarithmic", fontsize=18)
>>> plt.show()
```



Note

Plotting functions such as `plt.plot()` are shortcuts for accessing the current axes on the current figure and calling a method on that `Axes` object. Calling `plt.subplot()` changes the current axis, and calling `plt.figure()` changes the current figure. Use `plt.gca()` to get the current axes and `plt.gcf()` to get the current figure. Compare the following equivalent strategies for producing a figure with two subplots.

```
>>> x = np.linspace(-5, 5, 100)

# 1. Use plt.subplot() to switch the current.
>>> plt.subplot(121)
>>> plt.plot(x, 2*x)
>>> plt.subplot(122)
>>> plt.plot(x, x**2)

# 2. Use plt.subplot() to explicitly grab the two subplot axes.
>>> ax1 = plt.subplot(121)
>>> ax1.plot(x, 2*x)
>>> ax2 = plt.subplot(122)
>>> ax2.plot(x, x**2)

# 3. Use plt.subplots() to get the figure and all subplots simultaneously.
>>> fig, axes = plt.subplots(1, 2)
>>> axes[0].plot(x, 2*x)
>>> axes[1].plot(x, x**2)
```

Problem 4. Write a function that plots the functions $\sin(x)$, $\sin(2x)$, $2\sin(x)$, and $2\sin(2x)$ on the domain $[0, 2\pi]$, each in a separate subplot of a single figure.

1. Arrange the plots in a 2×2 grid of subplots.
2. Set the limits of each subplot to $[0, 2\pi] \times [-2, 2]$.
   (Hint: Consider using `plt.axis([xmin, xmax, ymin, ymax])` instead of `plt.xlim()` and `plt.ylim()` to set all boundaries simultaneously.)
3. Use `plt.title()` or `ax.set_title()` to give each subplot an appropriate title.
4. Use `plt.suptitle()` or `fig.suptitle()` to give the overall figure a title.
5. Use the following colors and line styles.

   $\sin(x)$: green solid line. $\sin(2x)$: red dashed line.

   $2\sin(x)$: blue dashed line. $2\sin(2x)$: magenta dotted line.

Achtung!

Be careful not to mix up the following functions.

1. `plt.axes()` creates a new place to draw on the figure, while `plt.axis()` (or `ax.axis()`) sets properties of the x- and y-axis in the current axes, such as the x and y limits.
2. `plt.subplot()` (singular) returns a single subplot belonging to the current figure, while `plt.subplots()` (plural) creates a new figure and adds a collection of subplots to it.

Other Kinds of Plots

Line plots are not always the most illuminating choice of graph to describe a set of data. Matplotlib provides several other easy ways to visualize data.

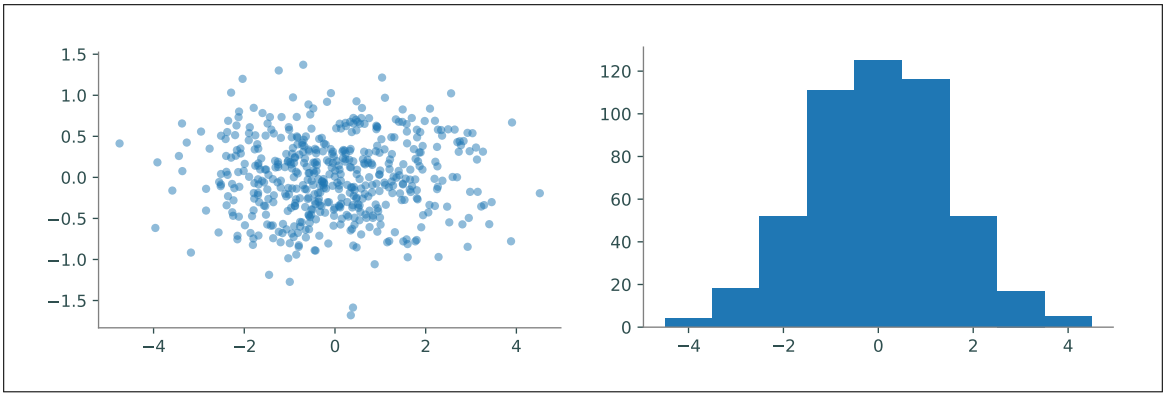
- A *scatter plot* plots two 1-dimensional arrays against each other without drawing lines between the points. Scatter plots are particularly useful for data that is not correlated or ordered.

  To create a scatter plot, use `plt.plot()` and specify a point marker (such as `'o'` or `'*'`) for the line style, or use `plt.scatter()` (or `ax.scatter()`). Beware that `plt.scatter()` has slightly different arguments and syntax than `plt.plot()`.

- A *histogram* groups entries of a 1-dimensional data set into a given number of intervals, called *bins*. Each bin has a bar whose height indicates the number of values that fall in the range of the bin. Histograms are best for displaying distributions, relating data values to frequency.

  To create a histogram, use `plt.hist()` (or `ax.hist()`). Use the argument `bins` to specify the edges of the bins or to choose a number of bins. The `range` argument specifies the outer limits of the first and last bins.

```
# Get 500 random samples from two normal distributions.
>>> x = np.random.normal(scale=1.5, size=500)
>>> y = np.random.normal(scale=0.5, size=500)

# Draw a scatter plot of x against y, using transparent circle markers.
>>> ax1 = plt.subplot(121)
>>> ax1.plot(x, y, 'o', markersize=5, alpha=.5)

# Draw a histogram to display the distribution of the data in x.
>>> ax2 = plt.subplot(122)
>>> ax2.hist(x, bins=np.arange(-4.5, 5.5))      # Or, equivalently,
#   ax2.hist(x, bins=9, range=[-4.5, 4.5])

>>> plt.show()
```



Problem 5. The Fatality Analysis Reporting System (FARS) is a nationwide census that provides yearly data regarding fatal injuries suffered in motor vehicle traffic crashes.[a] The array contained in `FARS.npy` is a small subset of the FARS database from 2010–2014. Each of the 148,206 rows in the array represents a different car crash; the columns represent the hour (in military time, as an integer), the longitude, and the latitude, in that order.

Write a function to visualize the data in `FARS.npy`. Use `np.load()` to load the data, then create a single figure with two subplots:

1. A scatter plot of longitudes against latitudes. Because of the large number of data points, use black pixel markers (use `"k,"` as the third argument to `plt.plot()`). Label both axes using `plt.xlabel()` and `plt.ylabel()` (or `ax.set_xlabel()` and `ax.set_ylabel()`). (Hint: Use `plt.axis("equal")` or `ax.set_aspect("equal")` so that the x- and y-axis are scaled the same way.

2. A histogram of the hours of the day, with one bin per hour. Set the limits of the x-axis appropriately. Label the x-axis. You should be able to clearly see which hours of the day experience more traffic.

[a] See `http://www.nhtsa.gov/FARS`.

Matplotlib also has tools for creating other kinds of plots for visualizing 1-dimensional data, including bar plots and box plots. See the Matplotlib Appendix for examples and syntax.

Visualizing 3-D Surfaces

Line plots, histograms, and scatter plots are good for visualizing 1- and 2-dimensional data, including the domain and range of a function $f : \mathbb{R} \to \mathbb{R}$. However, visualizing 3-dimensional data or a function $g : \mathbb{R}^2 \to \mathbb{R}$ (two inputs, one output) requires a different kind of plot. The process is similar to creating a line plot but requires slightly more setup: first construct an appropriate domain, then calculate the image of the function on that domain.

NumPy's `np.meshgrid()` function is the standard tool for creating a 2-dimensional domain in the Cartesian plane. Given two 1-dimensional coordinate arrays, `np.meshgrid()` creates two corresponding coordinate matrices. See Figure 5.6.

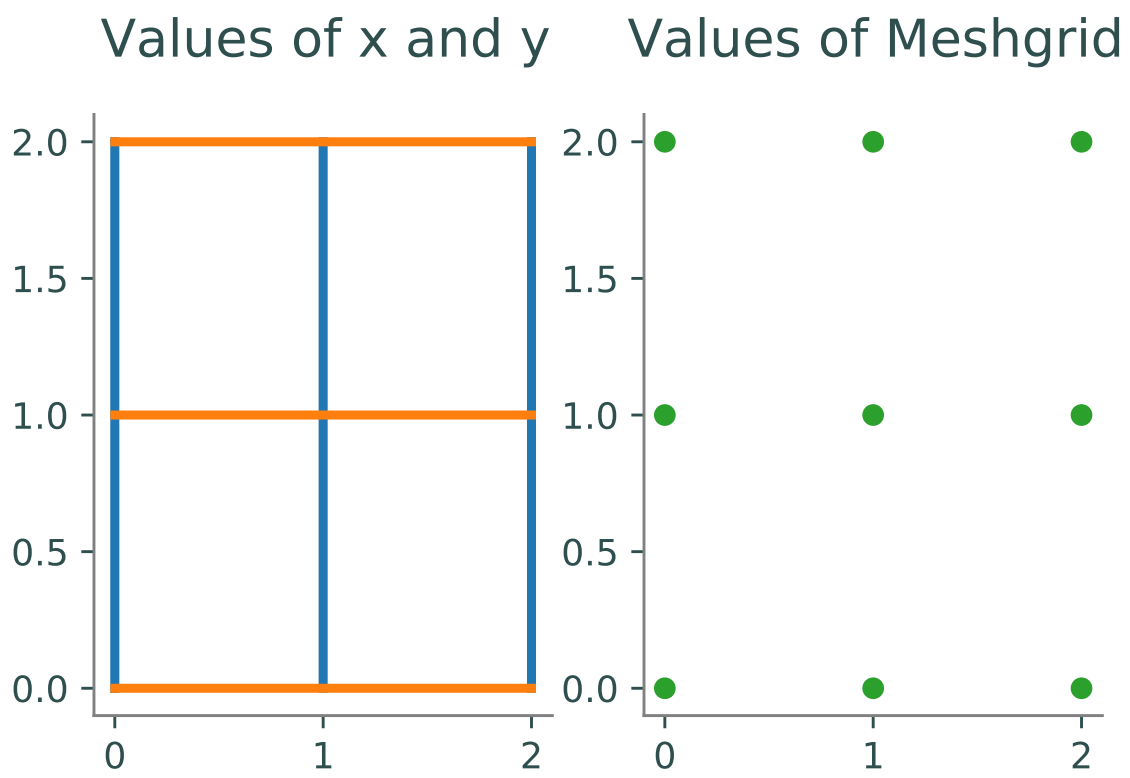


Figure 5.6: In the left plot, we have two arrays where x and y have the values `x = y = [0,1,2]`. The command `np.meshgrid(x, y)` returns the arrays $X = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix}$ and $Y = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix}$. These give the x- and y-coordinates of the points in the grid formed by `x` and `y` as seen in the right plot and satisfy the relation `(X[i,j], Y[i,j]) = (x[j],y[i])`.

```
>>> x, y = [0, 1, 2], [3, 4, 5]      # A rough domain over [0,2]x[3,5].
>>> X, Y = np.meshgrid(x, y)         # Combine the 1-D data into 2-D data.
>>> for xrow, yrow in zip(X, Y):
...     print(xrow, yrow, sep='\t')
...
[0 1 2]    [3 3 3]
[0 1 2]    [4 4 4]
[0 1 2]    [5 5 5]
```

With a 2-dimensional domain, $g(x, y)$ is usually visualized with two kinds of plots.

- A *heat map* assigns a color to each point in the domain, producing a 2-dimensional colored picture describing a 3-dimensional shape. Darker colors typically correspond to lower values while lighter colors typically correspond to higher values.

  Use `plt.pcolormesh()` to create a heat map. You can add an optional argument for the shading type; this determines the layout and fill style of the heat map. This argument defaults to

`shading='auto'`, and will automatically choose a fill method suited to the data being graphed.

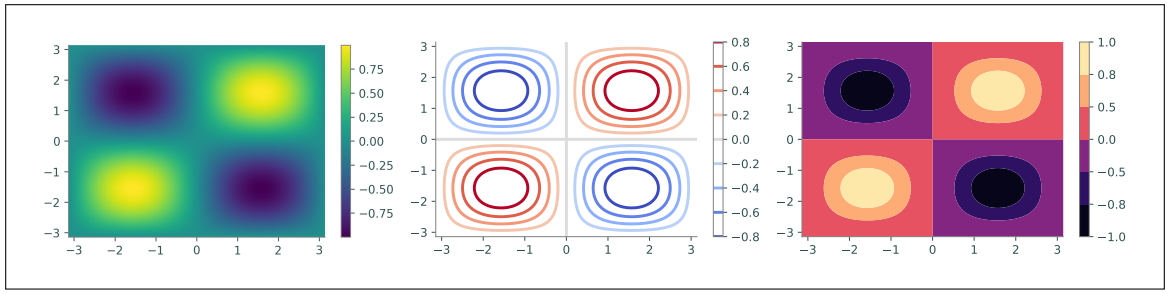
- A *contour map* draws several *level curves* of g on the 2-dimensional domain. A level curve corresponding to the constant c is the collection of points $\{(x, y) \mid c = g(x, y)\}$. Coloring the space between the level curves produces a discretized version of a heat map. Including more and more level curves makes a filled contour plot look more and more like the complete, blended heat map.

  Use `plt.contour()` to create a contour plot and `plt.contourf()` to create a filled contour plot. Specify either the number of level curves to draw, or a list of constants corresponding to specific level curves.

These functions each receive the keyword argument `cmap` to specify a color scheme (some of the better schemes are `"viridis"`, `"magma"`, and `"coolwarm"`). For the list of all Matplotlib color schemes, see `http://matplotlib.org/examples/color/colormaps_reference.html`.

Finally, `plt.colorbar()` draws the color scale beside the plot to indicate how the colors relate to the values of the function.

```
# Create a 2-D domain with np.meshgrid().
>>> x = np.linspace(-np.pi, np.pi, 100)
>>> y = x.copy()
>>> X, Y = np.meshgrid(x, y)
>>> Z = np.sin(X) * np.sin(Y)       # Calculate g(x,y) = sin(x)sin(y).

# Plot the heat map of f over the 2-D domain.
>>> plt.subplot(131)
>>> plt.pcolormesh(X, Y, Z, cmap="viridis", shading="auto")
>>> plt.colorbar()
>>> plt.xlim(-np.pi, np.pi)
>>> plt.ylim(-np.pi, np.pi)

# Plot a contour map of f with 10 level curves.
>>> plt.subplot(132)
>>> plt.contour(X, Y, Z, 10, cmap="coolwarm")
>>> plt.colorbar()

# Plot a filled contour map, specifying the level curves.
>>> plt.subplot(133)
>>> plt.contourf(X, Y, Z, [-1, -.8, -.5, 0, .5, .8, 1], cmap="magma")
>>> plt.colorbar()
>>> plt.show()
```



Problem 6. Write a function to plot $g(x, y) = \frac{\sin(x)\sin(y)}{xy}$ on the domain $[-2\pi, 2\pi] \times [-2\pi, 2\pi]$.

1. Create 2 subplots: one with a heat map of g, and one with a contour map of g. Choose an appropriate number of level curves, or specify the curves yourself.
2. Set the limits of each subplot to $[-2\pi, 2\pi] \times [-2\pi, 2\pi]$.
3. Choose a non-default color scheme.
4. Include a color scale bar for each subplot.

Additional Material

Further Reading and Tutorials

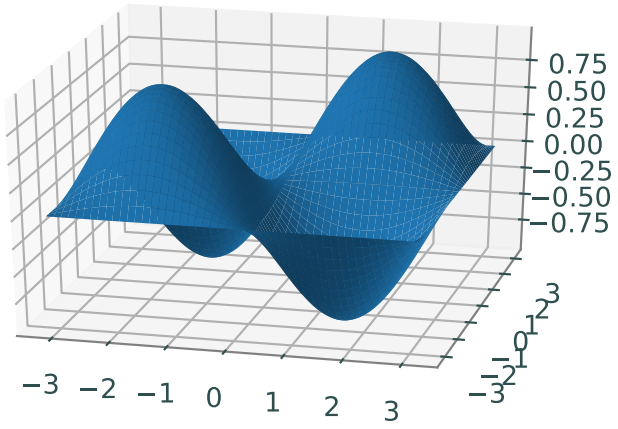
Plotting takes some getting used to. See the following materials for more examples.

- `https://www.labri.fr/perso/nrougier/teaching/matplotlib/`.
- `https://matplotlib.org/stable/tutorials/introductory/pyplot.html`.
- `http://scipy-lectures.org/intro/matplotlib/`.
- The Matplotlib Appendix in this manual.

3-D Plotting

Matplotlib can also be used to plot 3-dimensional surfaces. The following code produces the surface corresponding to $g(x, y) = \sin(x)\sin(y)$.

```
# Create the domain and calculate the range like usual.
>>> x = np.linspace(-np.pi, np.pi, 200)
>>> y = np.copy(x)
>>> X, Y = np.meshgrid(x, y)
>>> Z = np.sin(X) * np.sin(Y)

# Draw the corresponding 3-D plot using some extra tools.
>>> fig = plt.figure()
>>> ax = fig.add_subplot(1, 1, 1, projection='3d')
>>> ax.plot_surface(X, Y, Z)
>>> plt.show()
```



Animations

Lines and other graphs can be altered dynamically to produce animations. Follow these steps to create a Matplotlib animation:

1. Calculate all data that is needed for the animation.
2. Define a figure explicitly with `plt.figure()` and set its window boundaries.
3. Draw empty objects that can be altered dynamically.
4. Define a function to update the drawing objects.
5. Use `matplotlib.animation.FuncAnimation()`.

The submodule `matplotlib.animation` contains the tools for putting together and managing animations. The function `matplotlib.animation.FuncAnimation()` accepts the figure to animate, the function that updates the figure, the number of frames to show before repeating, and how fast to run the animation (lower numbers mean faster animations).

```
from matplotlib.animation import FuncAnimation

def sine_animation():
    # Calculate the data to be animated.
    x = np.linspace(0, 2*np.pi, 200)[:-1]
    y = np.sin(x)

    # Create a figure and set the window boundaries of the axes.
    fig = plt.figure()
    plt.xlim(0, 2*np.pi)
    plt.ylim(-1.2, 1.2)

    # Draw an empty line. The comma after 'drawing' is crucial.
    drawing, = plt.plot([], [])

    # Define a function that updates the line data.
    def update(index):
        drawing.set_data(x[:index], y[:index])
        return drawing,                    # Note the comma!

    a = FuncAnimation(fig, update, frames=len(x), interval=10)
    plt.show()
```

Try using the following function in place of `update()`. Can you explain why this animation is different from the original?

```
def wave(index):
    drawing.set_data(x, np.roll(y, index))
    return drawing,
```

To animate multiple objects at once, define the objects separately and make sure the update function returns both objects.

```
def sine_cosine_animation():
    x = np.linspace(0, 2*np.pi, 200)[:-1]
    y1, y2 = np.sin(x), np.cos(x)

    fig = plt.figure()
    plt.xlim(0, 2*np.pi)
    plt.ylim(-1.2, 1.2)

    sin_drawing, = plt.plot([], [])
    cos_drawing, = plt.plot([], [])

    def update(index):
        sin_drawing.set_data(x[:index], y1[:index])
        cos_drawing.set_data(x[:index], y2[:index])
        return sin_drawing, cos_drawing,

    a = FuncAnimation(fig, update, frames=len(x), interval=10)
    plt.show()
```

Animations can also be 3-dimensional. The only major difference is an extra operation to set the 3-dimensional component of the drawn object. The code below animates the space curve parametrized by the following equations:

$$x(\theta) = \cos(\theta)\cos(6\theta), \qquad y(\theta) = \sin(\theta)\cos(6\theta), \qquad z(\theta) = \tfrac{\theta}{10}$$

```
def rose_animation_3D():
    theta = np.linspace(0, 2*np.pi, 200)
    x = np.cos(theta) * np.cos(6*theta)
    y = np.sin(theta) * np.cos(6*theta)
    z = theta / 10

    fig = plt.figure()
    ax = fig.add_subplot(projection='3d')           # Make the figure 3-D.
    ax.set_xlim3d(-1.2, 1.2)                # Use ax instead of plt.
    ax.set_ylim3d(-1.2, 1.2)
    ax.set_aspect("equal")

    drawing, = ax.plot([], [], [])            # Provide 3 empty lists.

    # Update the first 2 dimensions like usual, then update the 3-D component.
    def update(index):
        drawing.set_data(x[:index], y[:index])
        drawing.set_3d_properties(z[:index])
        return drawing,

    a = FuncAnimation(fig, update, frames=len(x), interval=10, repeat=False)
    plt.show()
```

6 Exceptions and File Input/Output

Lab Objective: *In Python, an* exception *is an error detected during execution. Exceptions are important for regulating program usage and for correctly reporting problems to the programmer and end user. An understanding of exceptions is essential to safely read data from and write data to external files. Being able to interact with external files is important for analyzing data and communicating results. In this lab we learn exception syntax and file interaction protocols.*

Exceptions

An *exception* formally indicates an error and terminates the program early. Some of the more common exception types are listed below, along with the kinds of problems they typically indicate.

Exception	Indication
`AttributeError`	An attribute reference or assignment failed.
`ImportError`	An `import` statement failed.
`IndexError`	A sequence subscript was out of range.
`NameError`	A local or global name was not found.
`TypeError`	An operation or function was applied to an object of inappropriate type.
`ValueError`	An operation or function received an argument that had the right type but an inappropriate value.
`ZeroDivisionError`	The second argument of a division or modulo operation was zero.

```
>>> print(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined

>>> [1, 2, 3].fly()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'list' object has no attribute 'fly'
```

Raising Exceptions

Most exceptions are due to coding mistakes and typos. However, exceptions can also be used intentionally to indicate a problem to the user or programmer. To create an exception, use the keyword `raise`, followed by the name of the exception class. As soon as an exception is raised, the program stops running unless the exception is handled properly.

```
>>> if 7 is not 7.0:                # Raise an exception with an error message.
...     raise Exception("ints and floats are different!")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: ints and floats are different!

>>> for x in range(10):
...     if x > 5:                   # Raise a specific kind of exception.
...         raise ValueError("'x' should not exceed 5.")
...     print(x, end=' ')
...
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
ValueError: 'x' should not exceed 5.
0 1 2 3 4 5
```

Problem 1. Consider the following arithmetic "magic" trick.

1. Choose a 3-digit number where the first and last digits differ by 2 or more (say, 123).
2. Reverse this number by reading it backwards (321).
3. Calculate the positive difference of these numbers ($321 - 123 = 198$).
4. Add the reverse of the result to itself ($198 + 891 = 1089$).

The result of the last step will always be 1089, regardless of the original number chosen in step 1 (can you explain why?).

The following function prompts the user for input at each step of the magic trick, but does not check that the user's inputs are correct.

```
def arithmagic():
    step_1 = input("Enter a 3-digit number where the first and last "
                                          "digits differ by 2 or more: ")
    step_2 = input("Enter the reverse of the first number, obtained "
                                            "by reading it backwards: ")
    step_3 = input("Enter the positive difference of these numbers: ")
    step_4 = input("Enter the reverse of the previous result: ")
    print(str(step_3), "+", str(step_4), "= 1089 (ta-da!)")
```

Modify `arithmagic()` so that it verifies the user's input at each step. Raise a `ValueError` with an informative error message if any of the following occur:

- The first number (`step_1`) is not a 3-digit number.
- The first number's first and last digits differ by less than 2.
- The second number (`step_2`) is not the reverse of the first number.
- The third number (`step_3`) is not the positive difference of the first two numbers.
- The fourth number (`step_4`) is not the reverse of the third number.

(Hint: `input()` always returns a string, so each variable is a string initially. Use `int()` to cast the variables as integers when necessary. The built-in function `abs()` may also be useful.)

Handling Exceptions

To prevent an exception from halting the program, it must be handled by placing the problematic lines of code in a `try` block. An `except` block then follows with instructions for what to do in the event of an exception.

```
# The 'try' block should hold any lines of code that might raise an exception.
>>> try:
...     print("Entering try block...")
...     raise Exception("for no reason")
...     print("No problem!")        # This line gets skipped.
... # The 'except' block is executed just after the exception is raised.
... except Exception as e:
...     print("There was a problem:", e)
...
Entering try block...
There was a problem: for no reason
>>> # The program then continues on.
```

In this example, the name `e` represents the exception within the `except` block. Printing `e` displays its error message. If desired, `e` can be raised again with `raise e` or just `raise`.

The try-except control flow can be expanded with two other blocks, forming a code structure similar to a sequence of `if-elif-else` blocks.

1. The `try` block is executed until an exception is raised (if at all).
2. An `except` statement specifying the same kind of exception that was raised in the try block "catches" the exception, and the block is then executed. There may be multiple except blocks following a single try block (similiar to having several `elif` statements following a single `if` statement), and a single except statement may specify multiple kinds of exceptions to catch.
3. The `else` block is executed if an exception was **not** raised in the try block.
4. The `finally` block is always executed if it is included.

```
>>> try:
...     print("Entering try block...", end='')
...     house_on_fire = False
...     raise ValueError("The house is on fire!")
... # Check for multiple kinds of exceptions using parentheses.
... except (ValueError, TypeError) as e:
...     print("caught an exception.")
...     house_on_fire = True
... else:                           # Skipped due to the exception.
...     print("no exceptions raised.")
... finally:
...     print("The house is on fire:", house_on_fire)
...
Entering try block...caught an exception.
The house is on fire: True

>>> try:
...     print("Entering try block...", end='')
...     house_on_fire = False
... except ValueError as e:         # Skipped because there was no exception.
...     print("caught a ValueError.")
...     house_on_fire = True
... except TypeError as e:          # Also skipped.
...     print("caught a TypeError.")
...     house_on_fire = True
... else:
...     print("no exceptions raised.")
... finally:
...     print("The house is on fire:", house_on_fire)
...
Entering try block...no exceptions raised.
The house is on fire: False
```

The code in the `finally` block is always executed, even if a `return` statement or an uncaught exception occurs in any block following the try statement.

```
>>> def implode():
...     try:                       # Try to return immediately...
...         return
...     finally:                   # ...but 'finally' goes before 'return'.
...         print("Goodbye, world!")
...
>>> implode()
Goodbye, world!
```

See `https://docs.python.org/3/tutorial/errors.html` for more examples.

Achtung!

An `except` statement with no specified exception type catches **any** exception raised in the corresponding `try` block. This approach can mistakenly mask unexpected errors. Always be specific about the kinds of exceptions you expect to encounter.

```
>>> def divider(x, y):
...     try:
...         return x / yy           # The mispelled yy raises a NameError.
...     except:                     # Catch ANY exception.
...         print("y must not equal zero!")
...
>>> divider(2, 3)
y must not equal zero!

>>> def divider(x, y):
...     try:
...         return x / yy
...     except ZeroDivisionError:   # Specify an exception type.
...         print("y must not equal zero!")
...
>>> divider(2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in divider
NameError: name 'yy' is not defined     # Now the mistake is obvious.
```

Problem 2. A *random walk* is a path created by a sequence of random steps. The following function simulates a random walk by repeatedly adding or subtracting 1 to a running total.

```
from random import choice

def random_walk(max_iters=1e12):
    walk = 0
    directions = [1, -1]
    for i in range(int(max_iters)):
        walk += choice(directions)
    return walk
```

A `KeyboardInterrupt` is a special exception that can be triggered at any time by entering `ctrl+c` (on most systems) in the keyboard. Modify `random_walk()` so that if the user raises a `KeyboardInterrupt` by pressing `ctrl+c` while the program is running, the function catches the exception and prints "Process interrupted at iteration i". If no `KeyboardInterrupt` is raised, print "Process completed". In both cases, return `walk` as before.

Note

The built-in exceptions are organized into a class hierarchy. For example, the `ValueError` class inherits from the generic `Exception` class. Thus, a `ValueError` **is** an `Exception`, but an `Exception` is **not** a `ValueError`.

```
>>> try:
...     raise ValueError("caught!")
... except Exception as e:          # A ValueError is an Exception.
...     print(e)
...
caught!                             # The exception was caught.

>>> try:
...     raise Exception("not caught!")
... except ValueError as e:         # A Exception is not a ValueError.
...     print(e)
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: not caught!            # The exception wasn't caught!
```

See `https://docs.python.org/3/library/exceptions.html` for the complete list of built-in exceptions and the exception class hierarchy.

Debugging Exceptions

Exceptions are an essential and informative tool for debugging your code. As such, it's important to understand what an exception is saying and how to properly debug (or fix) the issue causing it.

The Stack Trace

The stack trace is the printed-out statement given to the user or programmer whenever an exception is raised. It contains important information about the type of exception and the location where the exception occured.

```
>>> x = 1/0       # Stack trace below
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

Exceptions can have more complicated stack traces if the error occurs inside of a function call, among other scenarios. As stated at the top of the stack trace, the most recent function call before the exception occurred is output last. This means that the stack trace could contain several lines indicating where the error occured.

```
>>> def make_error():     # Function to raise an error
```

```
...     return 1/0
...
>>> x = make_error()     # Calling the function
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in make_error
ZeroDivisionError: division by zero   # The exception occurred in the function!
```

Additionally, most modern IDE's display the line of code that caused the error (and may even point to the specific part of the line that is believed to be at fault). If the last function called was a function you did not write, this can mean that unfamiliar code is displayed to you which might not be helpful in debugging your code. As such, it's important to first look at the type of exception raised (listed at the bottom of the stack trace) and then move up the stack trace to identify code that may have caused the issue.

```
# Stack trace as displayed by the terminal
Traceback (most recent call last):
  File "<filepath>", line 4, in <module>
    x = make_error()
  File "<filepath>", line 2, in make_error
    return 1/0
ZeroDivisionError: division by zero
```

Print Debugging

One of the most basic (yet effective) methods for debugging is using print statements. This involves printing out information in the lines leading up to where the exception was raised (which you can find using the stack trace). For example, if a variable being passed into a function is causing an exception, it can be helpful to print out the value of that variable on the line before the function is executed.

```
>>> def divide(a,b):           # Normal function to divide variables
...     return a/b
...
>>> x = 1
>>> y = 0
>>> print(f"{x=} and {y=}")    # Print out the variables that we'll be using
x=1 and y=0
>>> z = divide(x,y)            # Our input values cause an error
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in divide
ZeroDivisionError: division by zero
```

This method is simple to implement and can help identify problems in the code that occured before an exception was raised. It can even help identify problems with function outputs that don't result in an exception (such as unexpected results or values).

Debuggers

While print statements can be a quick solution to small bugs, most IDE's (including VS Code) include a helpful tool called a debugger to aid in finding and fixing errors in code. This is a separate runtime environment to running the code normally and is generally referred to as runing in debug mode. Debuggers have many useful features, including the ability to add special markers called *break points* to your code.

These markers (when running in debug mode) temporarily pause execution just before the line specified and display the values of all variables, as well as other useful information. After execution is paused, it will not resume until the programmer indicates. Additionally, code can be executed line-by-line in this paused state, and variables will display their new values as they update.

There are many more tools available in the debugger, so it's highly encouraged to test out the version specific to your IDE to learn best how it can help you debug your code.

File Input and Output

A file object acts as an interface to a file stream, meaning, it allows a program to read from or write to external files. The built-in function `open()` creates a file object. It accepts the name of the file to open and an editing mode. The mode determines the kind of access that the user has to the file. There are four common modes:

`'r'`: **read**. Open an existing file for reading. The file must already exist, or `open()` raises a `FileNotFoundError`. This is the default mode.

`'w'`: **write**. Create a new file or **overwrite an existing file** (careful!) and open it for writing.

`'x'`: **write new**. Create a new file and open it for writing. If the file already exists, `open()` raises a `FileExistsError`. This is a safer form of `'w'` because it never overwrites existing files.

`'a'`: **append**. Open a file for writing and append new data to the end of the file if it already exists.

```
>>> myfile = open("hello_world.txt", 'r')   # Open a file for reading.
>>> print(myfile.read())                    # Print the contents of the file.
Hello,                                      # (it's a really small file.)
World!

>>> myfile.close()                          # Close the file connection.
```

The With Statement

An `IOError` indicates that some input or output operation has failed. A simple `try-finally` control flow can ensure that a file stream is closed safely.

The `with` statement provides an alternative method for safely opening and closing files. Use `with open(<filename>, <mode>) as <alias>:` to create an indented block in which the file is open and available under the specified alias. At the end of the block, the file is automatically and safely closed, even in the event of an exception. This is the preferred file-reading method when a file only needs to be accessed briefly.

```
>>> myfile = open("hello_world.txt", 'r')   # Open a file for reading.
>>> try:
...     contents = myfile.readlines()       # Read in the content by line.
... finally:
...     myfile.close()                      # Explicitly close the file.

# Equivalently, use a 'with' statement to take care of errors.
>>> with open("hello_world.txt", 'r') as myfile:
...    contents = myfile.readlines()
...                                        # The file is closed automatically.
```

In both cases, if the file `hello_world.txt` does not exist in the current directory, `open()` raises a `FileNotFoundError`. However, errors in the `try` or `with` blocks do not prevent the file from being safely closed.

Reading and Writing

Open file objects have an implicit *cursor* that determines the location in the file to read from or write to. After the entire file has been read once, either the file must be closed and reopened, or the cursor must be reset to the beginning of the file with `seek(0)` before it can be read again.

Some of the more important file object attributes and methods are listed below.

Attribute	Description
`closed`	`True` if the object is closed.
`mode`	The access mode used to open the file object.
`name`	The name of the file.
Method	Description
`close()`	Close the connection to the file.
`read()`	Read a given number of bytes; with no input, read the rest of the file.
`readline()`	Read a line of the file, including the newline character at the end.
`readlines()`	Call `readline()` repeatedly and return a list of the resulting lines.
`seek()`	Move the cursor to a new position.
`tell()`	Report the current position of the cursor.
`write()`	Write a single string to the file (spaces are **not** added automatically).
`writelines()`	Write a list of strings to the file (newline characters are **not** added automatically).

Only strings can be written to files; to write a non-string type, first cast it as a string with `str()`. Be mindful of spaces and newlines to separate the data.

```
>>> with open("out.txt", 'w') as outfile:   # Open 'out.txt' for writing.
...     for i in range(10):
...         outfile.write(str(i**2)+' ')    # Write some strings (and spaces).
...
>>> outfile.closed                          # The file is closed automatically.
True
```

Problem 3. Define a class called `ContentFilter`. Implement the constructor so that it accepts the name of a file to be read.

1. If the file name is invalid in any way, prompt the user for another filename using `input()`. Continue prompting the user until they provide a valid filename.

```
>>> cf1 = ContentFilter("hello_world.txt")  # File exists.
>>> cf2 = ContentFilter("not-a-file.txt")   # File doesn't exist.
Please enter a valid file name: still-not-a-file.txt
Please enter a valid file name: hello_world.txt
>>> cf3 = ContentFilter([1, 2, 3])          # Not even a string.
Please enter a valid file name: hello_world.txt
```

   (Hint: `with open()` might raise a `FileNotFoundError`, a `TypeError`, or an `OSError`.)

2. Read the file and store its name and contents as attributes (store the contents as a single string). Make sure the file is securely closed.

String Formatting

The `str` class has several useful methods for parsing and formatting strings. They are particularly useful for processing data from a source file and for preparing data to be written to an external file.

Method	Returns
`count()`	The number of times a given substring occurs within the string.
`find()`	The lowest index where a given substring is found.
`isalpha()`	`True` if all characters in the string are alphabetic (a, b, c, ...).
`isdigit()`	`True` if all characters in the string are digits (0, 1, 2, ...).
`isspace()`	`True` if all characters in the string are whitespace (`" "`, `'\t'`, `'\n'`).
`join()`	The concatenation of the strings in a given iterable with a specified separator between entries.
`lower()`	A copy of the string converted to lowercase.
`upper()`	A copy of the string converted to uppercase.
`replace()`	A copy of the string with occurrences of a given substring replaced by a different specified substring.
`split()`	A list of segments of the string, using a given character or string as a delimiter.
`strip()`	A copy of the string with leading and trailing whitespace removed.

The `join()` method translates a list of strings into a single string by concatenating the entries of the list and placing the principal string between the entries. Conversely, `split()` translates the principal string into a list of substrings, with the separation determined by a single input.

```
# str.join() puts the string between the entries of a list.
>>> words = ["state", "of", "the", "art"]
>>> "-".join(words)
'state-of-the-art'
```

```
# str.split() creates a list out of a string, given a delimiter.
>>> "One fish\nTwo fish\nRed fish\nBlue fish\n".split('\n')
['One fish', 'Two fish', 'Red fish', 'Blue fish', '']

# If no delimiter is provided, the string is split by whitespace characters.
>>> "One fish\nTwo fish\nRed fish\nBlue fish\n".split()
['One', 'fish', 'Two', 'fish', 'Red', 'fish', 'Blue', 'fish']
```

Can you tell the difference between the following routines?

```
>>> with open("hello_world.txt", 'r') as myfile:
...     contents = myfile.readlines()
...
>>> with open("hello_world.txt", 'r') as myfile:
...     contents = myfile.read().split('\n')
```

Variables within Strings

In addition to formatting strings using the methods above there may also be times when you wish to insert generalized variable values into the middle of strings. There are two ways to do this: using .format() or using f-strings. Of the two, f-strings are generally more readable, concise, faster, and less prone to error. As a result, an explanation of how to use .format() has been banished to the "Additional Materials" section, which we know is rarely read, and a short explanation of f-strings will be included here.

We define an f-string with an 'f' preceding the string declaration (i.e f'your words here', or f"your words here"). Within the f-string, curly brackets can be used along with variable names to insert the variable value into the string. Consider the following code which puts an argument defaulting to 'bread and butter pickles' into the sentence: "I think that (blank) are an abomination." Note that f-strings can also be used with multiple variables at once by using multiple sets of curly brackets.

```
# defining function
>>> def abomination(userOpinion="Bread and Butter Pickles"):
...         # notice the use of the f-string
...         return f"I think that {userOpinion} are an abomination"

# calling the function
>>> abomination()
'I think that Bread and Butter Pickles are an abomination.'
```

Problem 4. Add the following methods to the `ContentFilter` class for writing the contents of the original file to new files. Each method should accept the name of a file to write to and a keyword argument `mode` that specifies the file access mode, defaulting to `'w'`. If `mode` is not `'w'`, `'x'`, or `'a'`, raise a `ValueError` with an informative message.

1. `uniform()`: write the data to the outfile with uniform case. Include a keyword argument `case` that defaults to `"upper"`.

   If `case="upper"`, write the data in upper case. If `case="lower"`, write the data in lower case. If `case` is not one of these two values, raise a `ValueError`.

2. `reverse()`: write the data to the outfile in reverse order. Include a keyword argument `unit` that defaults to `"line"`.

   If `unit="word"`, reverse the ordering of the words in each line, but write the lines in the same order as the original file. If `unit="line"`, reverse the ordering of the lines, but do not change the ordering of the words on each individual line. If `unit` is not one of these two values, raise a `ValueError`.

3. `transpose()`: write a "transposed" version of the data to the outfile. That is, write the first word of each line of the data to the first line of the new file, the second word of each line of the data to the second line of the new file, and so on. Viewed as a matrix of words, the rows of the input file then become the columns of the output file, and vice versa. You may assume that there are an equal number of words on each line of the input file.

4. `__str__()`: Also implement the `__str__()` magic method so that printing a `ContentFilter` object yields the following output. You may want to calculate these statistics in the constructor. (Note: Using f-strings will also make this implementation much simpler).

```
Source file:              <filename>
Total characters:         <The total number of characters in file>
Alphabetic characters:    <The number of letters>
Numerical characters:     <The number of digits>
Whitespace characters:    <The number of spaces, tabs, and newlines>
Number of lines:          <The number of lines>
```

(Hint: list comprehensions are **very** useful for some of these functions. For example, what does `[line[::-1] for line in lines]` do? What about `sum([s.isspace() for s in data])`?)

Compare your class to the following example.

```
# cf_example1.txt
A b C
d E f
```

```
>>> cf = ContentFilter("cf_example1.txt")
>>> cf.uniform("uniform.txt", mode='w', case="upper")
>>> cf.uniform("uniform.txt", mode='a', case="lower")
>>> cf.reverse("reverse.txt", mode='w', unit="word")
>>> cf.reverse("reverse.txt", mode='a', unit="line")
>>> cf.transpose("transpose.txt", mode='w')
```

```
# uniform.txt
A B C
D E F
a b c
d e f
```

```
# reverse.txt
C b A
f E d
d E f
A b C
```

```
# transpose.txt
A d
b E
C f
```

Achtung!

When parsing though the datafiles be sure to account for trailing whitespaces at the end of lines. Each file may or may not contain them, which can lead to unexpected behavior. Consider using `strip()` to avoid bugs in your code.

Additional Material

Custom Exception Classes

Custom exceptions can be defined by writing a class that inherits from some existing exception class. The generic `Exception` class is typically the parent class of choice.

```
>>> class TooHardError(Exception):
...     pass
...
>>> raise TooHardError("This lab is impossible!")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
__main__.TooHardError: This lab is impossible!
```

This may seem like a trivial extension of the `Exception` class, but it is useful to do because the interpreter never automatically raises a `TooHardError`. Any `TooHardError` must have originated from a hand-written `raise` command, making it easier to identify the exact source of the problem.

Chaining Exceptions

Sometimes, especially in large programs, it is useful to raise one kind of exception just after catching another. The two exceptions can be linked together using the `from` statement. This syntax makes it possible to see where the error originated from and to "pass it up" to another part of the program.

```
>>> try:
...     raise TooHardError("This lab is impossible!")
... except TooHardError as e:
...     raise NotImplementedError("Lab is incomplete") from e
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
__main__.TooHardError: This lab is impossible!

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
NotImplementedError: Lab is incomplete
```

More String Formatting Tools

Concatenating string values with non-string values can be cumbersome and tedious. The `str` class's `format()` method makes it easier to insert non-string values into the middle of a string. Write the desired output in its entirety, replacing non-string values with curly braces `{}`. Then use the `format()` method, entering each replaced value in order.

```
# Join the data using string concatenation.
>>> day, month, year = 10, "June", 2017
```

```
>>> print("Is today", day, str(month) + ',', str(year) + "?")
Is today 10 June, 2017?

# Join the data using str.format().
>>> print("Is today {} {}, {}?".format(day, month, year))
Is today 10 June, 2017?

# Join the data more easily using an f-string.
>>> print(f"Is today {day} {month}, {year}?")
Is today 10 June, 2017?
```

This method is extremely flexible and provides many convenient ways to format string output nicely. Consider the following code for printing out a simple progress bar from within a loop.

```
>>> iters = int(1e7)
>>> chunk = iters // 20
>>> for i in range(iters):
...     print("\r[{:<20}] i = {}".format('='*((i//chunk)+1), i),
...                                                     end='', flush=True)
...
```

Here the string `"\r[{:<20}]"` used in conjunction with the `format()` method tells the cursor to go back to the beginning of the line, print an opening bracket, then print the first argument of `format()` left-aligned with at least 20 total spaces before printing the closing bracket. The `end` parameter can change the default newline added to the output to another ending. Setting to an empty string, `end=''`, the output ends without any whitespace. The `flush` parameter defaults to `False` and does not need to be changed if the `end` parameter is not changed. As shown in the example above, setting the parameter equal to `True` forces the output to be printed on the terminal before it is complete. If it is `False` with the `end=''` the `print()` function will first build the output with the set ending before printing each iteration rather than printing incrementally.

Printing at each iteration dramatically slows down the progression through the loop. How does the following code solve that problem?

```
>>> for i in range(iters):
...     if not i % chunk:
...         print("\r[{:<20}] i = {}".format('='*((i//chunk)+1),i),
...                                                     end='', flush=True)
...
```

See `https://docs.python.org/3/library/string.html#format-string-syntax` for more examples and specific syntax for using `str.format()`. For a more robust progress bar printer, research the `tqdm` module.

Standard Library Modules for I/O

The standard library has other tools for input and output operations. For details on each module, see `https://docs.python.org/3/library`.

Module	Description
`csv`	CSV (comma separated value) file writing and parsing.
`io`	Support for file objects and `open()`.
`os`	Communication with the operating system.
`os.path`	Common path operations such as checking for file existence.
`pickle`	Create portable serialized representations of Python objects.

7 Data Visualization

Lab Objective: *This lab demonstrates how to communicate information through clean, concise, and honest data visualization. We recommend completing the exercises in a Jupyter Notebook.*

The Importance of Visualizations

Visualizations of data can reveal insights that are not immediately obvious from simple statistics. The data set in the following exercise is known as *Anscombe's quartet*. It is famous for demonstrating the importance of data visualization.

Problem 1. The file `anscombe.npy` contains the quartet of data points shown in the table below. For each section of the quartet,

- Plot the data as a scatter plot on the box $[0, 20] \times [0, 13]$.
- Use `scipy.stats.linregress()` to calculate the slope and intercept of the least squares regression line for the data and its correlation coefficient (the first three return values).
- Plot the least squares regression line over the scatter plot on the domain $x \in [0, 20]$.
- Report (print) the mean and variance in x and y, the slope and intercept of the regression line, and the correlation coefficient. Compare these statistics to those of the other sections.
- Describe how the section is similar to the others and how it is different.

I		II		III		IV	
x	y	x	y	x	y	x	y
10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56
7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89

Improving Specific Types of Visualizations

Effective data visualizations show specific comparisons and relationships in the data. Before designing a visualization, decide what to look for or what needs to be communicated. Then choose the visual scheme that makes sense for the data. The following sections demonstrate how to improve commonly used plots to communicate information visually.

Line Plots

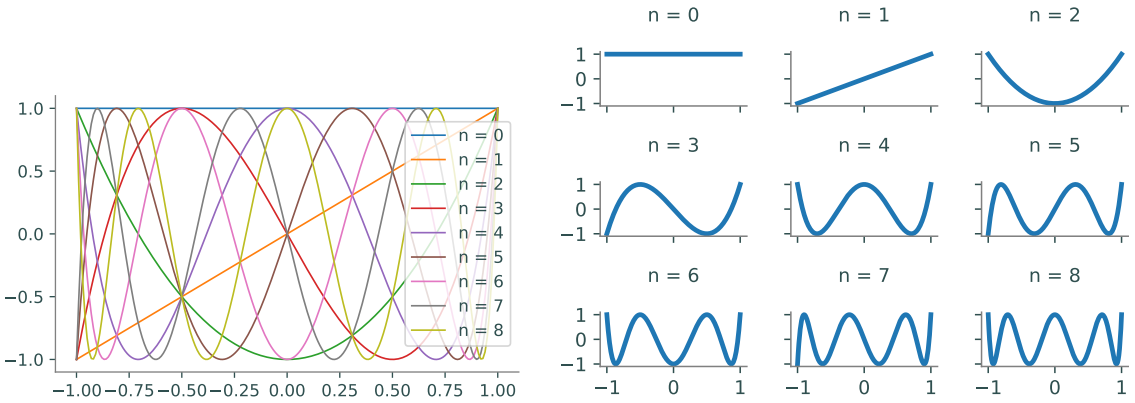


Figure 7.1: Line plots can be used to visualize and compare mathematical functions. For example, this figure shows the first nine Chebyshev polynomials in one plot (left) and small multiples (right). Using small multiples makes comparison easy and shows how each polynomial changes as n increases.

```
>>> import numpy as np
>>> from matplotlib import pyplot as plt

# Plot the first 9 Chebyshev polynomials in the same plot.
>>> T = np.polynomial.Chebyshev.basis
>>> x = np.linspace(-1, 1, 200)
>>> for n in range(9):
...     plt.plot(x, T(n)(x), label="n = "+str(n))
...
>>> plt.axis([-1.1, 1.1, -1.1, 1.1])        # Set the window limits.
>>> plt.legend(loc="right")
>>> plt.show()
```

A line plot connects ordered (x, y) points with straight lines, and is best for visualizing one or two ordered arrays, such as functional outputs over an ordered domain or a sequence of values over time. Sometimes, plotting multiple lines on the same plot helps the viewer compare two different data sets. However, plotting several lines on top of each other makes the visualization difficult to read, even with a legend. For example, Figure 7.1 shows the first nine *Chebyshev polynomials*, a family of orthogonal polynomials that satisfies the recursive relation

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_{n+1} = 2xT_n(x) - T_{n-1}(x).$$

The plot on the right makes comparison easier by using *small multiples*. Instead of using a legend, the figure makes a separate subplot with a title for each polynomial. Adjusting the figure size and the line thickness also makes the information easier to read.

Note

Matplotlib titles and annotations can be formatted with LaTeX, a system for creating technical documents.[a] To do so, use an `r` before the string quotation mark and surround the text with dollar signs. For example, add the following line of code to the loop from the previous example.

```
...     plt.title(rf"$T_{n}(x)$")
```

The f-string inserts the input n at the curly braces. The title of the sixth subplot, instead of being "n = 5," will then be "$T_5(x)$."

[a] See `http://www.latex-project.org/` for more information.

Problem 2. The $n+1$ Bernstein basis polynomials of degree n are defined as follows:

$$b_{v,n}(x) = \binom{n}{v} x^v (1-x)^{n-v}, \qquad v = 0,\ 1,\ \ldots,\ n$$

Plot the first 10 Bernstein basis polynomials ($n = 0,\ 1,\ 2,\ 3$) as small multiples on the domain $[0,1] \times [0,1]$. Label the subplots for clarity, adjust tick marks and labels for simplicity, and set the window limits of each plot to be the same. Consider arranging the subplots so that the rows correspond with n and the columns with v.
Hint: The constant $\binom{n}{v} = \frac{n!}{v!(n-v)!}$ is called the *binomial coefficient* and can be efficiently computed with `scipy.special.comb()`.

Bar Charts

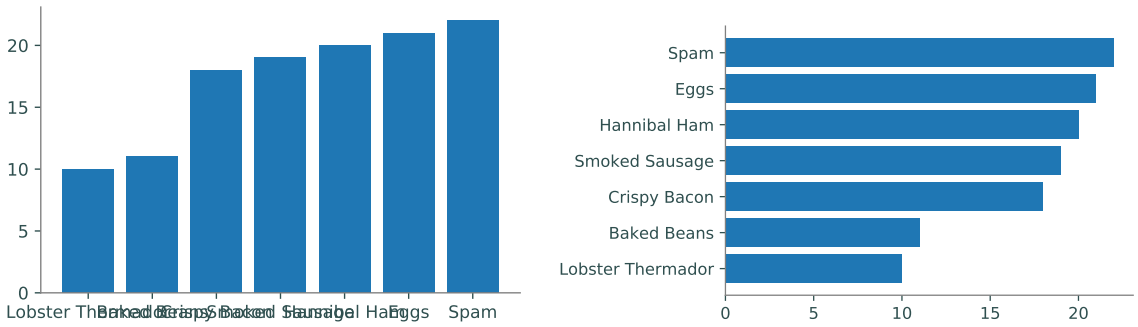


Figure 7.2: Bar charts are used to compare quantities between categorical variables. The labels on the vertical bar chart (left) are more difficult to read than the labels on the horizontal bar chart (right). Although the labels can be rotated, horizontal text is much easier to read than vertical text.

```
>>> labels = ["Lobster Thermador", "Baked Beans", "Crispy Bacon",
...             "Smoked Sausage", "Hannibal Ham", "Eggs", "Spam"]
>>> values = [10, 11, 18, 19, 20, 21, 22]
>>> positions = np.arange(len(labels))

>>> plt.bar(positions, values, align="center")  # Vertical bar chart.
>>> plt.xticks(positions, labels)
>>> plt.show()

>>> plt.barh(positions, values, align="center") # Horizontal bar char (better).
>>> plt.yticks(positions, labels)
>>> plt.tight_layout()
>>> plt.show()
```

A bar chart plots categorical data in a sequence of bars. They are best for small, discrete, one-dimensional data sets. In Matplotlib, `plt.bar()` creates a vertical bar chart or `plt.barh()` creates a horizontal bar chart. These functions receive the locations of each bar followed by the height of each bar (as lists or arrays). In most situations, horizontal bar charts are preferable to vertical bar charts because horizontal labels are easier to read than vertical labels. Data in a bar chart should also be sorted in a logical way, such as alphabetically, by size, or by importance.

Histograms

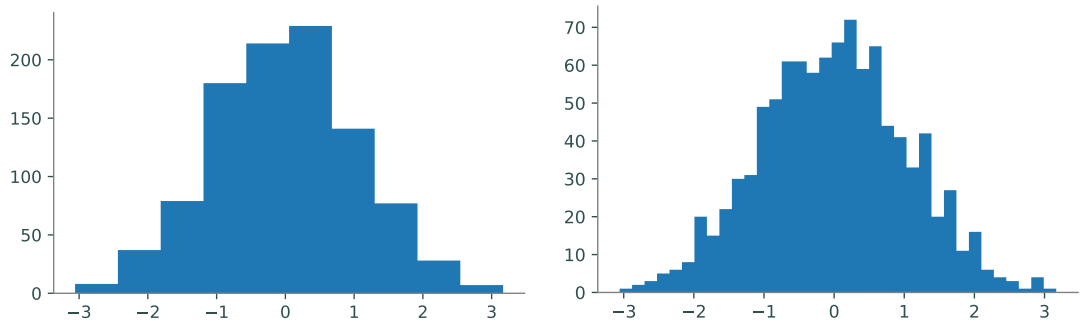


Figure 7.3: Histograms are used to show the distribution of one-dimensional data. Experimenting with different values for the bin size is important when plotting a histogram. Using only 10 bins (left) doesn't give a good sense for how the randomly generated data is distributed. However, using 35 bins (right) reveals the shape of a normal distribution.

```
>>> data = np.random.normal(size=10000)
>>> fig, ax = plt.subplots(1, 2)
>>> ax[0].hist(data, bins=10)
>>> ax[1].hist(data, bins=35)
>>> plt.show()
```

A histogram partitions an interval into a number of bins and counts the number of values that fall into each bin. Histograms are ideal for visualizing how unordered data in a single array is distributed over an interval. For example, if data are drawn from a probability distribution, a histogram approximates the distribution's probability density function. Use `plt.hist()` to create a histogram. The arguments `bins` and `range` specify the number of bins to draw and over what domain. A histogram with too few or too many bins will not give a clear view of the distribution.

Scatter Plots

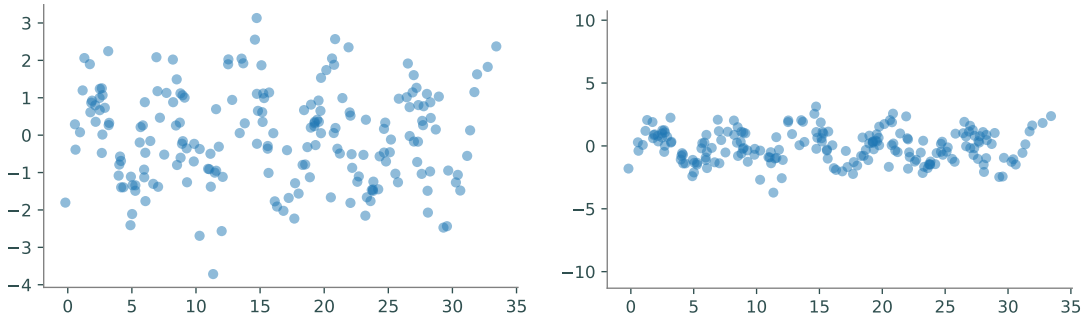


Figure 7.4: Scatter plots show correlations between variables by plotting markers at coordinate points. The figure above displays randomly perturbed data that is visualized using two scatter plots with `alpha=.5` and `edgecolor='none'`. The default (left) makes it harder to see correlation and pattern whereas making the axes equal better reveals the oscillatory behavior in the perturbed sine wave.

```
>>> np.random.seed(0)
>>> x = np.linspace(0, 10*np.pi, 200) + np.random.normal(size=200)
>>> y = np.sin(x) + np.random.normal(size=200)

>>> plt.scatter(x, y, alpha=.5, edgecolor='none')
>>> plt.show()

>>> plt.scatter(x, y, alpha=.5, edgecolor='none')
>>> plt.axis('equal')
>>> plt.show()
```

A scatter plot draws (x, y) points without connecting them. Scatter plots are best for displaying data sets without a natural order, or where each point is a distinct, individual instance. They are frequently used to show correlation between variables in a data set. Use `plt.scatter()` to create a scatter plot.[1]

Similar data points in a scatter plot may overlap, as in Figure 7.4. Specifying an *alpha value* reveals overlapping data by making the markers transparent (see Figure 7.5 for an example). The keyword `alpha` accepts values between 0 (completely transparent) and 1 (completely opaque). When

[1]Scatter plots can also be drawn with `plt.plot()` by specifying a point marker such as `'.'`, `','`, `'o'`, or `'+'`. The keywords `markersize` and `color` can be used to change the marker size and marker color, respectively.

plotting lots of overlapping points, the outlines on the markers can make the visualization look cluttered. Setting the edgecolor keyword to zero removes the outline and improves the visualization.

Problem 3. The file `MLB.npy` contains measurements from over 1,000 recent Major League Baseball players, compiled by UCLA.[a] Each row in the array represents a player; the columns are the player's height (in inches), weight (in pounds), and age (in years), in that order.

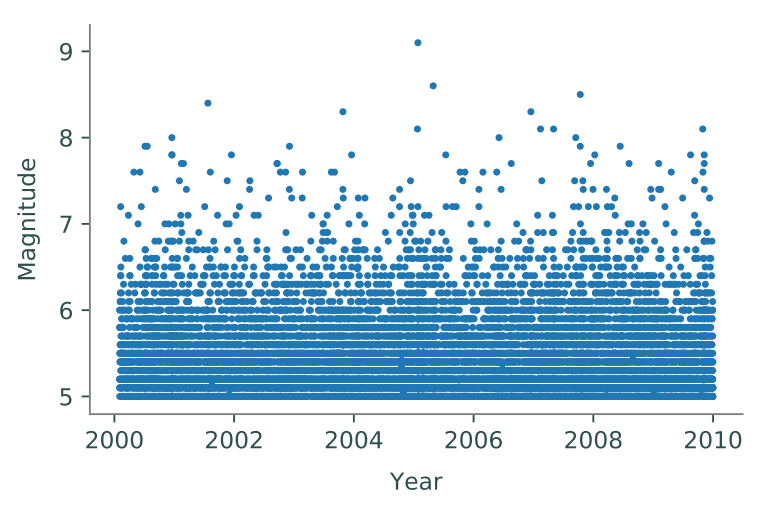
Create several visualizations to show the correlations between height, weight, and age in the MLB data set. Use at least one scatter plot. Adjust the marker size, plot a regression line, change the window limits, and use small multiples where appropriate.

[a] See `http://wiki.stat.ucla.edu/socr/index.php/SOCR_Data_MLB_HeightsWeights`.

Problem 4. The file `earthquakes.npy` contains data from over 17,000 earthquakes from the beginning of 2000 to the end of 2009 that were at least a 5 on the Richter scale.[a] Each row in the array represents an earthquake; the columns are the earthquake's date (as a fraction of the year, so March 14 2001 would be 2001.2), magnitude (on the Richter scale), longitude, and latitude, in that order.

Because each earthquake is a distinct event, a good way to start visualizing this data might be a scatter plot of the years versus the magnitudes of each earthquake.

```
>>> year, magnitude, longitude, latitude = np.load("earthquakes.npy").T
>>> plt.plot(year, magnitude, '.')
>>> plt.xlabel("Year")
>>> plt.ylabel("Magnitude")
```



Unfortunately, this plot communicates very little information because the data is so cluttered. Describe the data with at least two better visualizations. Include line plots, scatter plots, and histograms as appropriate. Your plots should answer the following questions:

1. How many earthquakes happened every year?
2. How often do stronger earthquakes happen compared to weaker ones?
3. Where do earthquakes happen? Where do the strongest earthquakes happen? (Hint: Use `plt.axis("equal")` or `ax.set_aspect("equal")` to fix the aspect ratio, which may improve comparisons between longitude and latitude.)

[a] See `http://earthquake.usgs.gov/earthquakes/search/`.

Hexbins

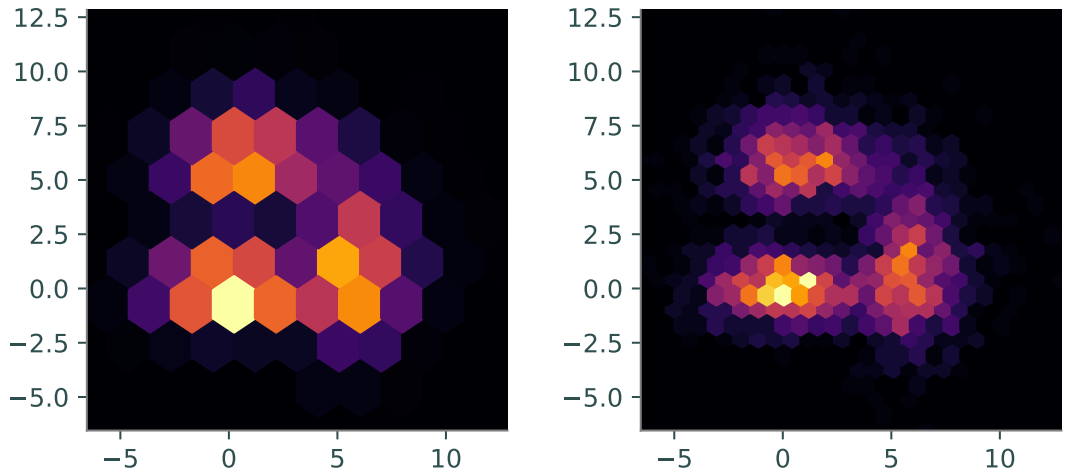


Figure 7.5: Hexbins can be used instead of using a three-dimensional histogram to show the distribution of two-dimensional data. Choosing the right gridsize will give a better picture of the distribution. The figure above shows random data plotted as hexbins with a gridsize of 10 (left) and 25 (right). Hexbins use color to show height via a colormap and both histograms above use the `'inferno'` colormap.

```
# Add random draws from various distributions in two dimensions.
>>> a = np.random.exponential(size=1000) + np.random.normal(size=1000) + 5
>>> b = np.random.exponential(size=1000) + 2*np.random.normal(size=1000)
>>> x = np.hstack((a, b, 2*np.random.normal(size=1000)))
>>> y = np.hstack((b, a,   np.random.normal(size=1000)))

# Plot the samples with hexbins of gridsize 10 and 25.
>>> fig, axes = plt.subplots(1, 2)
>>> window = [x.min(), x.max(), y.min(), y.max()]
>>> for ax, size in zip(axes, [10, 25]):
...     ax.hexbin(x, y, gridsize=size, cmap='inferno')
...     ax.axis(window)
...     ax.set_aspect("equal")
...
```

```
>>> plt.show()
```

A *hexbin* is a way of representing the frequency of occurrences in a two-dimensional plane. Similar to a histogram, which sorts one-dimensional data into bins, a hexbin sorts two-dimensional data into hexagonal bins arranged in a grid and uses color instead of height to show frequency. Creating an effective hexbin relies on choosing an appropriate `gridsize` and colormap. The *colormap* is a function that assigns data points to an ordering of colors. Use `plt.hexbin()` to create a hexbin and use the `cmap` keyword to specify the colormap.

Heat Maps and Contour Plots

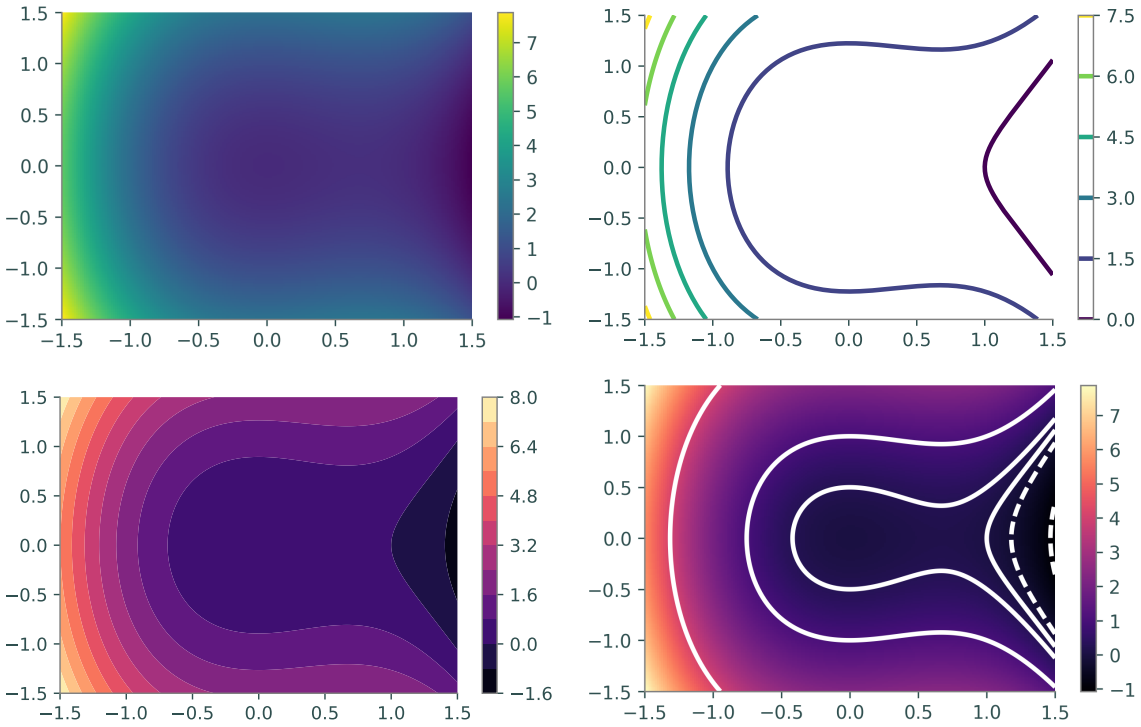


Figure 7.6: Heat maps visualize three-dimensional functions or surfaces by using color to represent the value in one dimension. With continuous data, it can be hard to identify regions of interest. Contour plots solve this problem by visualizing the level curves of the surface. Top left: heat map. Top right: contour plot. Bottom left: filled contour map. Bottom right: contours plotted on a heat map.

```
# Construct a 2-D domain with np.meshgrid() and calculate f on the domain.
>>> x = np.linspace(-1.5, 1.5, 200)
>>> X, Y = np.meshgrid(x, x)
>>> Z = Y**2 - X**3 + X**2

# Plot f using a heat map, a contour map, and a filled contour map.
>>> fig, ax = plt.subplots(2, 2)
```

```
>>> ax[0, 0].pcolormesh(X, Y, Z, cmap="viridis")     # Heat map.
>>> ax[0, 1].contour(X, Y, Z, 6, cmap="viridis")     # Contour map.
>>> ax[1, 0].contourf(X, Y, Z, 12, cmap="magma")   # Filled contour map.


# Plot specific level curves and a heat map with a colorbar.
>>> ax[1, 1].contour(X, Y, Z, [-1, -.25, 0, .25, 1, 4], colors="white")
>>> cax = ax[1, 1].pcolormesh(X, Y, Z, cmap="magma")
>>> fig.colorbar(cax, ax=ax[1, 1])

>>> plt.show()
```

Let $f : \mathbb{R}^2 \to \mathbb{R}$ be a scalar-valued function on a 2-dimensional domain. A *heat map* of f assigns a color to each (x, y) point in the domain based on the value of $f(x, y)$, while a contour plot is a drawing of the *level curves* of f. The level curve corresponding to the constant c is the set $\{(x, y) \mid c = f(x, y)\}$. A filled contour plot colors in the sections between the level curves and is a discretized version of a heat map. The values of c corresponding to the level curves are automatically chosen to be evenly spaced over the range of values of f on the domain. However, it is sometimes better to strategically specify the curves by providing a list of c constants.

Consider the function $f(x, y) = y^2 - x^3 + x^2$ on the domain $[-\frac{3}{2}, \frac{3}{2}] \times [-\frac{3}{2}, \frac{3}{2}]$. A heat map of f reveals that it has a large basin around the origin. Since $f(0, 0) = 0$, choosing several level curves close to 0 more closely describes the topography of the basin. The fourth subplot in 7.6 uses the curves with $c = -1,\ -\frac{1}{4},\ 0,\ \frac{1}{4},\ 1$, and 4.

When plotting hexbins, heat maps, and contour plots, be sure to choose a colormap that best represents the data. Avoid using spectral or rainbow colormaps like `"jet"` because they are not *perceptually uniform*, meaning that the rate of change in color is not constant. Because of this, data points may appear to be closer together or farther apart than they actually are. This creates visual false positives or false negatives in the visualization and can affect the interpretation of the data. As a default, we recommend using the sequential colormaps `"viridis"` or `"inferno"` because they are designed to be perceptually uniform and colorblind friendly. For the complete list of Matplotlib color maps, see `http://matplotlib.org/examples/color/colormaps_reference.html`.

Problem 5. The *Rosenbrock function* is defined as

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

The minimum value of f is 0, which occurs at the point $(1, 1)$ at the bottom of a steep, banana-shaped valley of the function.

Use a heat map and a contour plot to visualize the Rosenbrock function. Also plot the minimizer $(1, 1)$. Use a different sequential colormap for each visualization.

Best Practices

Good scientific visualizations make comparison easy and clear. The eye is very good at detecting variation in one dimension and poor in two or more dimensions. For example, consider Figure 7.7. Despite the difficulty, most people can probably guess which slice of a pie chart is the largest or smallest. However, it's almost impossible to confidently answer the question *by how much?* The bar

charts may not be as aesthetically pleasing but they make it much easier to precisely compare the data. Avoid using pie charts as well as other visualizations that make accurate comparison difficult, such as radar charts, bubble charts, and stacked bar charts.

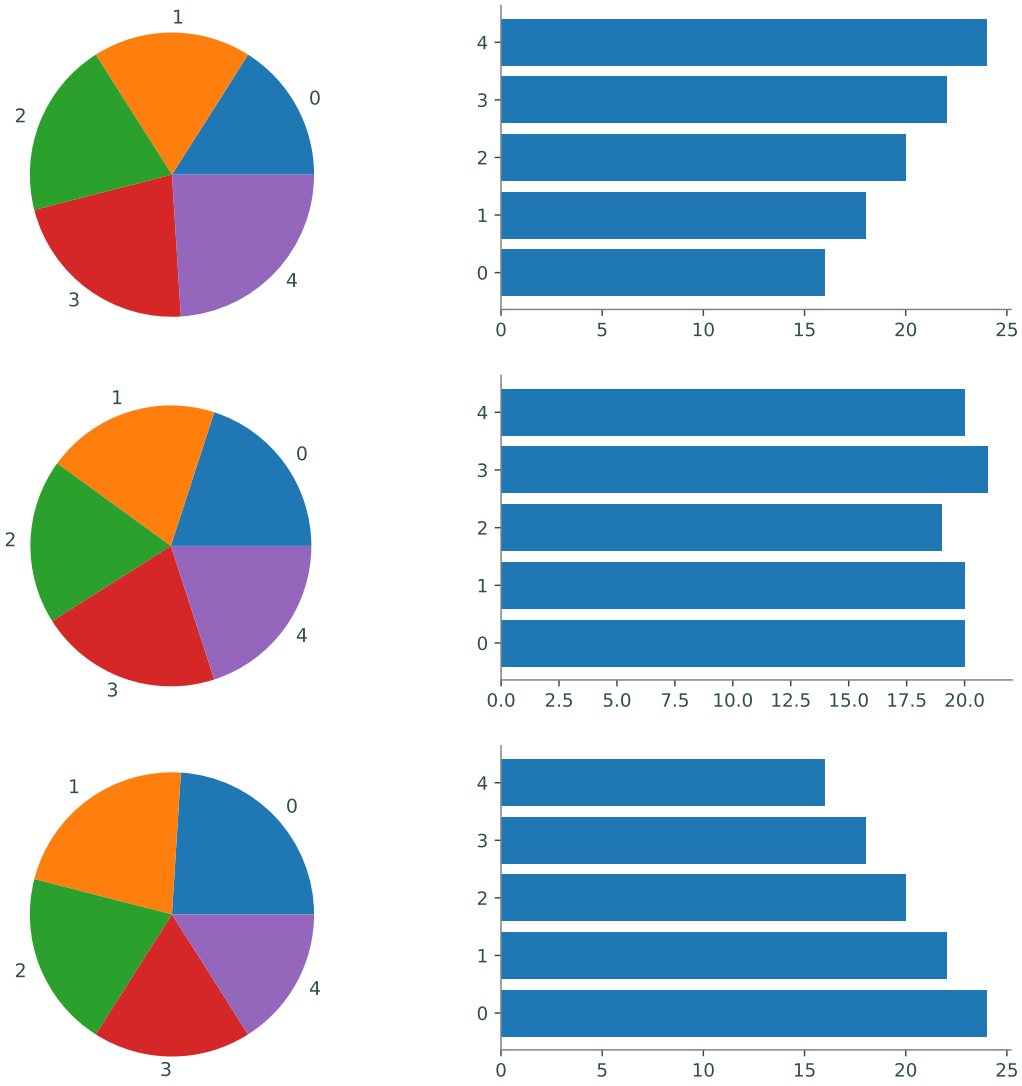


Figure 7.7: The pie charts on the left may be more colorful but it's extremely difficult to quantify the difference between each slice. Instead, the horizontal bar charts on the right make it very easy to see the difference between each variable.

No visualization perfectly represents data, but some are better than others. Finding the best visualization for a data set is an iterative process. Experiment with different visualizations by adjusting their parameters: color, scale, size, shape, position, and length. It may be necessary to use a data transformation or visualize various subsets of the data. As you iterate, keep in mind the saying attributed to George Box: "All models are wrong, but some are useful." Do whatever is needed to make the visualization useful and effective.

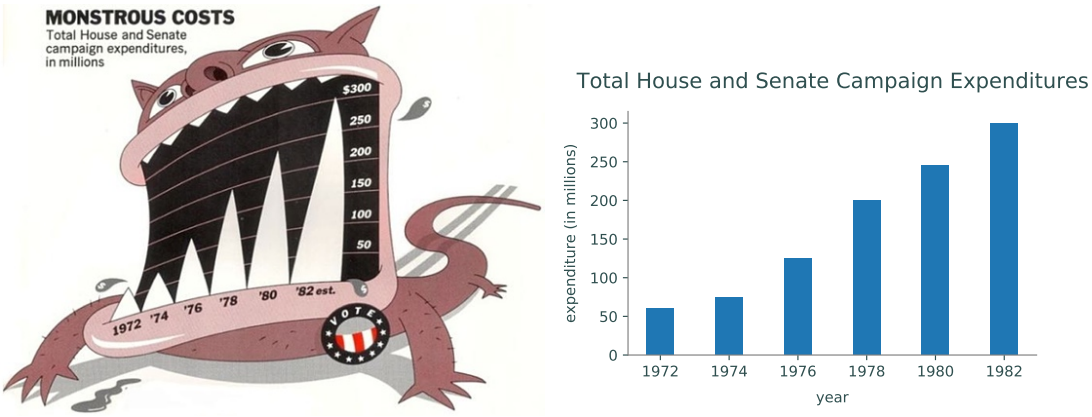


Figure 7.8: Chartjunk refers to anything that does not communicate data. In the image on the left, the cartoon monster distorts the bar chart and manipulates the feelings of the viewer to think negatively about the results. The image on the right shows the same data without chartjunk, making it simple and very easy to interpret the data objectively.

Good visualizations are as simple as possible and no simpler. Edward Tufte coined the term *chartjunk* to mean anything (pictures, icons, colors, and text) that does not represent data or is distracting. Though chartjunk might appear to make data graphics more memorable than plain visualizations, **it is more important to be clear and precise in order to prevent misinterpretation**. The physicist Richard Feynman said, "For a successful technology, reality must take precedence over public relations, for Nature cannot be fooled." Remove chartjunk and anything that prevents the viewer from objectively interpreting the data.

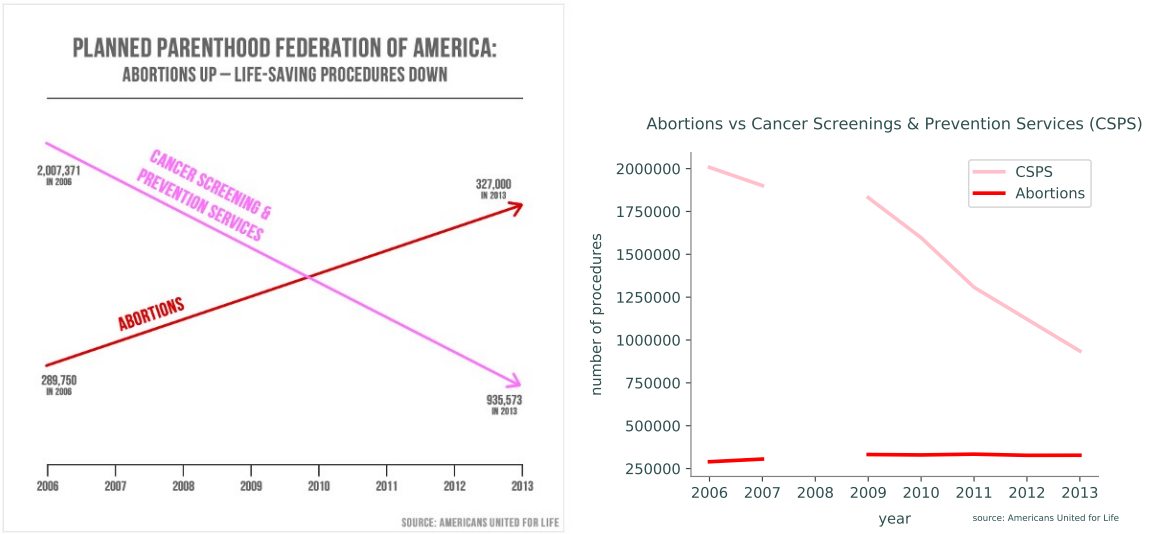


Figure 7.9: The chart on the left is an example of a dishonest graphic shown at a United States congressional hearing in 2015. The chart on the right shows a more accurate representation of the data by showing the y-axis and revealing the missing data from 2008. Source: PolitiFact.

Visualizations should be honest. Figure 7.9 shows how visualizations can be dishonest. The misleading graphic on the left was used as evidence in a United States congressional hearing in 2015.

With the y-axis completely removed, it is easy to miss that each line is shown on a different y-axis even though they are measured in the same units. Furthermore, the chart fails to indicate that data is missing from the year 2008. The graphic on the right shows a more accurate representation of the data.[2]

Never use data visualizations to deceive or manipulate. Always present information on who created it, where the data came from, how it was collected, whether it was cleaned or transformed, and whether there are conflicts of interest or possible biases present. Use specific titles and axis labels, and include units of measure. Choose an appropriate window size and use a legend or other annotations where appropriate.

Problem 6. The file `countries.npy` contains information from 20 different countries. Each row in the array represents a different country; the columns are the 2015 population (in millions of people), the 2015 GDP (in billions of US dollars), the average male height (in centimeters), and the average female height (in centimeters), in that order.[a]

The countries corresponding are listed below in order.

```
countries = ["Austria", "Bolivia", "Brazil", "China",
             "Finland", "Germany", "Hungary", "India",
             "Japan", "North Korea", "Montenegro", "Norway",
             "Peru", "South Korea", "Sri Lanka", "Switzerland",
             "Turkey", "United Kingdom", "United States", "Vietnam"]
```

Visualize this data set with at least four plots, using at least one scatter plot, one histogram, and one bar chart. List the major insights that your visualizations reveal.
(Hint: consider using `np.argsort()` and fancy indexing to sort the data for the bar chart.)

[a] See `https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)`, `https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population`, and `http://www.averageheight.co/`.

For more about data visualization, we recommend the following books and websites.

- *How to Lie with Statistics* by Darrell Huff (1954).
- *The Visual Display of Quantitative Information* by Edward Tufte (2nd edition).
- *Visual Explanations* by Edward Tufte.
- *Envisioning Information* by Edward Tufte.
- *Beautiful Evidence* by Edward Tufte.
- *The Functional Art* by Alberto Cairo.
- *Visualization Analysis and Design* by Tamara Munzner.
- *Designing New Default Colormaps*: `https://bids.github.io/colormap/`.

[2] For more information about this graphic, visit `http://www.politifact.com/truth-o-meter/statements/2015/oct/01/jason-chaffetz/chart-shown-planned-parenthood-hearing-misleading-/`.

8 SQL 1: Introduction

Lab Objective: *Being able to store and manipulate large data sets quickly is a fundamental part of data science. The SQL language is the classic database management system for working with tabular data. In this lab we introduce the basics of SQL, including creating, reading, updating, and deleting SQL tables, all via Python's standard SQL interaction modules.*

Relational Databases

A *relational database* is a collection of tables called *relations*. A single row in a table, called a *tuple*, corresponds to an individual instance of data. The columns, called *attributes* or *features*, are data values of a particular category. The collection of column headings is called the *schema* of the table, which describes the kind of information stored in each entry of the tuples.

For example, suppose a database contains demographic information for M individuals. If a table had the schema `(Name, Gender, Age)`, then each row of the table would be a 3-tuple corresponding to a single individual, such as `(Jane Doe, F, 20)` or `(Samuel Clemens, M, 74.4)`. The table would therefore be $M \times 3$ in shape. Note that including a person's age in a database means that the data would quickly be outdated since people get older every year. A better choice would be to use birth year. Another table with the schema `(Name, Income)` would be $M \times 2$ if it included all M individuals.

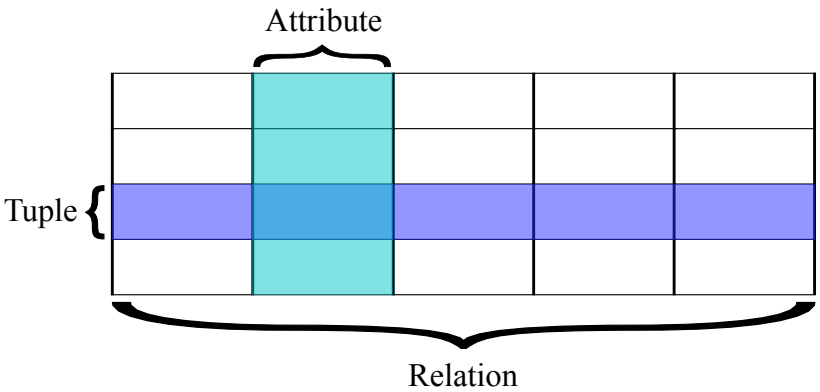


Figure 8.1: See `https://en.wikipedia.org/wiki/Relational_database`.

SQLite

The most common database management systems (DBMS) for relational databases are based on *Structured Query Language*, commonly called SQL (pronounced[1] "sequel"). Though SQL is a language in and of itself, most programming languages have tools for executing SQL routines. In Python, the most common variant of SQL is *SQLite*, implemented as the `sqlite3` module in the standard library, which we use in this lab.

Achtung!

The goal of this lab is to introduce the language SQL, not to teach a specific implementation of SQL. In this lab, we will use the `sqlite3` library, which allows us to interact with a *SQLite* database file. *SQLite* is lightweight, and does not require additional setup, making it great for small projects like this lab and databases which will only be used by a single program at once. For example, Android applications frequently use *SQLite* databases for local storage. However, it is not suitable for many data science applications, since it is difficult to scale across computers and programs. There are a variety of databases that use SQL-like languages designed for different purposes, including *MySQL*, which scales much better and manages concurrent connections efficiently but requires a server and a significant amount of overhead to run.

Although these databases have minor differences in SQL syntax, the concepts taught in this lab are extremely transferrable across all SQL-like derivatives. Remember that you will need to change the Python library you use based on which database type you are connecting to.

A SQL database is stored in an external file, usually marked with the file extension `db` or `mdf`. These files should **not** be opened in Python with `open()` like text files; instead, any interactions with the database—creating, reading, updating, or deleting data—should occur as follows.

1. Create a connection to the database with `sqlite3.connect()`. This creates a database file if one does not already exist.
2. Get a *cursor*, an object that manages the actual traversal of the database, with the connection's `cursor()` method.
3. Alter or read data with the cursor's `execute()` method, which accepts an actual SQL command as a string.
4. Save any changes with the cursor's `commit()` method, or revert changes with `rollback()`.
5. Close the connection.

```
>>> import sqlite3 as sql

# Establish a connection to a database file or create one if it doesn't exist.
>>> conn = sql.connect("my_database.db")
>>> try:
...     cur = conn.cursor()                          # Get a cursor object.
```

[1] See `https://english.stackexchange.com/questions/7231/how-is-sql-pronounced` for a brief history of the somewhat controversial pronunciation of SQL.

```
...     cur.execute("SELECT * FROM MyTable")        # Execute a SQL command.
... except sql.Error:                               # If there is an error,
...     conn.rollback()                             #  revert the changes
...     raise                                       #  and raise the error.
... else:                                           # If there are no errors,
...     conn.commit()                               #  save the changes.
... finally:
...     conn.close()                                # Close the connection.
```

> ACHTUNG!
>
> Some changes, such as creating and deleting tables, are automatically committed to the database as part of the cursor's `execute()` method. Be **extremely cautious** when deleting tables, as the action is immediate and permanent. Most changes, however, do not take effect in the database file until the connection's `commit()` method is called. Be careful not to close the connection before committing desired changes, or those changes will not be recorded.

The `with` statement can be used with `open()` so that file streams are automatically closed, even in the event of an error. Likewise, combining the `with` statement with `sql.connect()` automatically rolls back changes if there is an error and commits them otherwise. However, the actual database connection is **not** closed automatically. With this strategy, the previous code block can be reduced to the following.

```
>>> try:
...     with sql.connect("my_database.db") as conn:
...         cur = conn.cursor()                     # Get the cursor.
...         cur.execute("SELECT * FROM MyTable")    # Execute a SQL command.
... finally:                                        # Commit or revert, then
...     conn.close()                                #  close the connection.
```

Managing Database Tables

SQLite uses five native data types (relatively few compared to other SQL systems) that correspond neatly to native Python data types.

Python Type	SQLite Type
`None`	`NULL`
`int`	`INTEGER`
`float`	`REAL`
`str`	`TEXT`
`bytes`	`BLOB`

The `CREATE TABLE` command, together with a table name and a schema, adds a new table to a database. The schema is a comma-separated list where each entry specifies the column name, the

column data type,[2] and other optional parameters. For example, the following code adds a table called `MyTable` with the schema `(Name, ID, Age)` with appropriate data types.

```
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     cur.execute("CREATE TABLE MyTable (Name TEXT, ID INTEGER, Age REAL)")
...
>>> conn.close()
```

The `DROP TABLE` command deletes a table. However, using `CREATE TABLE` to try to create a table that already exists or using `DROP TABLE` to remove a nonexistent table raises an error. Use `DROP TABLE IF EXISTS` to remove a table without raising an error if the table doesn't exist. See Table 8.1 for more table management commands.

Operation	SQLite Command
Create a new table	`CREATE TABLE <table> (<schema>);`
Delete a table	`DROP TABLE <table>;`
Delete a table if it exists	`DROP TABLE IF EXISTS <table>;`
Add a new column to a table	`ALTER TABLE <table> ADD <column> <dtype>`
Remove an existing column	`ALTER TABLE <table> DROP COLUMN <column>;`
Rename an existing column	`ALTER TABLE <table> ALTER COLUMN <column> <dtype>;`

Table 8.1: SQLite commands for managing tables and columns.

Note

SQL commands like `CREATE TABLE` are often written in all caps to distinguish them from other parts of the query, like the table name. This is only a matter of style: SQLite, along with most other versions of SQL, is case insensitive. In Python's SQLite interface, the trailing semicolon is also unnecessary. However, most other database systems require it, so it's good practice to include the semicolon in Python.

Problem 1. Write a function that accepts the name of a database file. Connect to the database (and create it if it doesn't exist). Drop the tables `MajorInfo`, `CourseInfo`, `StudentInfo`, and `StudentGrades` from the database **if** they exist. Next, add the following tables to the database with the specified column names and types.

- `MajorInfo`: `MajorID` (integers) and `MajorName` (strings).
- `CourseInfo`: `CourseID` (integers) and `CourseName` (strings).
- `StudentInfo`: `StudentID` (integers), `StudentName` (strings), and `MajorID` (integers).
- `StudentGrades`: `StudentID` (integers), `CourseID` (integers), and `Grade` (strings).

[2] Though SQLite does not force the data in a single column to be of the same type, most other SQL systems enforce uniform column types, so it is good practice to specify data types in the schema.

Remember to commit and close the database. You should be able to execute your function more than once with the same input without raising an error.

To check the database, use the following commands to get the column names of a specified table. Assume here that the database file is called `students.db`.

```
>>> with sql.connect("students.db") as conn:
...     cur = conn.cursor()
...     cur.execute("SELECT * FROM StudentInfo;")
...     print([d[0] for d in cur.description])
...
['StudentID', 'StudentName', 'MajorID']
```

Inserting, Removing, and Altering Data

Tuples are added to SQLite database tables with the `INSERT INTO` command.

```
# Add the tuple (Samuel Clemens, 1910421, 74.4) to MyTable in my_database.db.
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     cur.execute("INSERT INTO MyTable "
...                 "VALUES("Samuel Clemens", 1910421, 74.4);")
```

With this syntax, SQLite assumes that values match sequentially with the schema of the table. The schema of the table can also be written explicitly for clarity.

```
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     cur.execute("INSERT INTO MyTable(Name, ID, Age) "
...                 "VALUES("Samuel Clemens", 1910421, 74.4);")
```

Achtung!

Never use Python's string operations to construct a SQL query from variables. Doing so makes the program susceptible to a *SQL injection attack.*[a] Instead, use parameter substitution to construct dynamic commands: use a `?` character within the command, then provide the sequence of values as a second argument to `execute()`.

```
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     values = ("Samuel Clemens", 1910421, 74.4)
...     # Don't piece the command together with string operations!
...     # cur.execute("INSERT INTO MyTable VALUES " + str(values)) # BAD!
...     # Instead, use parameter substitution.
...     cur.execute("INSERT INTO MyTable VALUES(?,?,?);", values)  # Good.
```

[a] See `https://xkcd.com/327/` for an example.

To insert several rows at a time to the same table, use the cursor object's `executemany()` method and parameter substitution with a list of tuples. This is typically much faster than using `execute()` repeatedly.

```
# Insert (Samuel Clemens, 1910421, 74.4) and (Jane Doe, 123, 20) to MyTable.
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     rows = [("John Smith", 456, 40.5), ("Jane Doe", 123, 20)]
...     cur.executemany("INSERT INTO MyTable VALUES(?,?,?);", rows)
```

Problem 2. Expand your function from Problem 1 so that it populates the tables with the data given in Tables 8.2a–8.2d.

MajorID	MajorName
1	Math
2	Science
3	Writing
4	Art

(a) MajorInfo

CourseID	CourseName
1	Calculus
2	English
3	Pottery
4	History

(b) CourseInfo

StudentID	StudentName	MajorID
401767594	Michelle Fernandez	1
678665086	Gilbert Chapman	NULL
553725811	Roberta Cook	2
886308195	Rene Cross	3
103066521	Cameron Kim	4
821568627	Mercedes Hall	NULL
206208438	Kristopher Tran	2
341324754	Cassandra Holland	1
262019426	Alfonso Phelps	NULL
622665098	Sammy Burke	2

(c) StudentInfo

StudentID	CourseID	Grade
401767594	4	C
401767594	3	B–
678665086	4	A+
678665086	3	A+
553725811	2	C
678665086	1	B
886308195	1	A
103066521	2	C
103066521	3	C–
821568627	4	D
821568627	2	A+
821568627	1	B
206208438	2	A
206208438	1	C+
341324754	2	D–
341324754	1	A–
103066521	4	A
262019426	2	B
262019426	3	C
622665098	1	A
622665098	2	A–

(d) StudentGrades

Table 8.2: Student database.

The `StudentInfo` and `StudentGrades` tables are also recorded in `student_info.csv` and `student_grades.csv`, respectively, with `NULL` values represented as -1 (we'll leave them as -1 for now). A CSV (**c**omma-**s**eparated **v**alues) file can be read like a normal text file or with the `csv` module.

```
>>> import csv
>>> with open("student_info.csv", 'r') as infile:
```

```
...     rows = list(csv.reader(infile))
```

To validate your database, use the following command to retrieve the rows from a table.

```
>>> with sql.connect("students.db") as conn:
...     cur = conn.cursor()
...     for row in cur.execute("SELECT * FROM MajorInfo;"):
...         print(row)
(1, 'Math')
(2, 'Science')
(3, 'Writing')
(4, 'Art')
```

Problem 3. The data file `us_earthquakes.csv`[a] contains data from about 3,500 earthquakes in the United States since the 1769. Each row records the year, month, day, hour, minute, second, latitude, longitude, and magnitude of a single earthquake (in that order). Note that latitude, longitude, and magnitude are floats, while the remaining columns are integers.

Write a function that accepts the name of a database file. Drop the table `USEarthquakes` if it already exists, then create a new `USEarthquakes` table with schema `(Year, Month, Day, Hour, Minute, Second, Latitude, Longitude, Magnitude)`. Populate the table with the data from `us_earthquakes.csv`. Remember to commit the changes and close the connection. (Hint: using `executemany()` is much faster than using `execute()` in a loop.)

[a] Retrieved from `https://datarepository.wolframcloud.com/resources/Sample-Data-US-Earthquakes`.

The WHERE Clause

Deleting or altering existing data in a database requires some searching for the desired row or rows. The `WHERE` clause is a *predicate* that filters the rows based on a boolean condition. The operators `==`, `!=`, `<`, `>`, `<=`, `>=`, `AND`, `OR`, and `NOT` all work as expected to create search conditions.

```
>>> with sql.connect("my_database.db") as conn:
...     cur = conn.cursor()
...     # Delete any rows where the Age column has a value less than 30.
...     cur.execute("DELETE FROM MyTable WHERE Age < 30;")
...     # Change the Name of "Samuel Clemens" to "Mark Twain".
...     cur.execute("UPDATE MyTable SET Name="Mark Twain" WHERE ID==1910421;")
```

If the `WHERE` clause were omitted from either of the previous commands, every record in `MyTable` would be affected. **Always** use a very specific `WHERE` clause when removing or updating data.

Operation	SQLite Command
Add a new row to a table	`INSERT INTO table VALUES(<values>);`
Remove rows from a table	`DELETE FROM <table> WHERE <condition>;`
Change values in existing rows	`UPDATE <table> SET <column1>=<value1>, ... WHERE <condition>;`

Table 8.3: SQLite commands for inserting, removing, and updating rows.

Note

SQLite treats `=` and `==` as equivalent operators. For clarity, in this lab we will always use `==` when comparing two values, such as in a `WHERE` clause. The only time we use `=` is in `SET` statements. Be aware that some flavors of SQL (such as MySQL) have no concept of `==`, and typing it in a query will return an error.

Problem 4. Modify your function from Problems 1 and 2 so that in the `StudentInfo` table, values of -1 in the `MajorID` column are replaced with `NULL` values.

Also modify your function from Problem 3 in the following ways.

1. Remove rows from `USEarthquakes` that have a value of 0 for the `Magnitude`.
2. Replace 0 values in the `Day`, `Hour`, `Minute`, and `Second` columns with `NULL` values.

Reading and Analyzing Data

Constructing and managing databases is fundamental, but most time in SQL is spent analyzing existing data. A *query* is a SQL command that reads all or part of a database without actually modifying the data. Queries start with the `SELECT` command, followed by column and table names and additional (optional) conditions. The results of a query, called the *result set*, are accessed through the cursor object. After calling `execute()` with a SQL query, use `fetchall()` or another cursor method from Table 8.4 to get the list of matching tuples.

Method	Description
`execute()`	Execute a single SQL command
`executemany()`	Execute a single SQL command over different values
`executescript()`	Execute a SQL script (multiple SQL commands)
`fetchone()`	Return a single tuple from the result set
`fetchmany(n)`	Return the next n rows from the result set as a list of tuples
`fetchall()`	Return the entire result set as a list of tuples

Table 8.4: Methods of database cursor objects.

```
>>> conn = sql.connect("students.db")
>>> cur = conn.cursor()
```

```
# Get tuples of the form (StudentID, StudentName) from the StudentInfo table.
>>> cur.execute("SELECT StudentID, StudentName FROM StudentInfo;")
>>> cur.fetchone()          # List the first match (a tuple).
(401767594, 'Michelle Fernandez')

>>> cur.fetchmany(3)        # List the next three matches (a list of tuples).
[(678665086, 'Gilbert Chapman'),
 (553725811, 'Roberta Cook'),
 (886308195, 'Rene Cross')]

>>> cur.fetchall()          # List the remaining matches.
[(103066521, 'Cameron Kim'),
 (821568627, 'Mercedes Hall'),
 (206208438, 'Kristopher Tran'),
 (341324754, 'Cassandra Holland'),
 (262019426, 'Alfonso Phelps'),
 (622665098, 'Sammy Burke')]

# Use * in place of column names to get all of the columns.
>>> cur.execute("SELECT * FROM MajorInfo;").fetchall()
[(1, 'Math'), (2, 'Science'), (3, 'Writing'), (4, 'Art')]

>>> conn.close()
```

The `WHERE` predicate can also refine a `SELECT` command. If the condition depends on a column in a different table from the data that is being a selected, create a *table alias* with the `AS` command to specify columns in the form `table.column`.

```
>>> conn = sql.connect("students.db")
>>> cur = conn.cursor()

# Get the names of all math majors.
>>> cur.execute("SELECT SI.StudentName "
...             "FROM StudentInfo AS SI, MajorInfo AS MI "
...             "WHERE SI.MajorID == MI.MajorID AND MI.MajorName == 'Math'")
# The result set is a list of 1-tuples; extract the entry from each tuple.
>>> [t[0] for t in cur.fetchall()]
['Cassandra Holland', 'Michelle Fernandez']

# Get the names and grades of everyone in English class.
>>> cur.execute("SELECT SI.StudentName, SG.Grade "
                "FROM StudentInfo AS SI, StudentGrades AS SG "
                "WHERE SI.StudentID == SG.StudentID AND CourseID == 2;")
>>> cur.fetchall()
[('Roberta Cook', 'C'),
 ('Cameron Kim', 'C'),
 ('Mercedes Hall', 'A+'),
 ('Kristopher Tran', 'A'),
 ('Cassandra Holland', 'D-'),
```

```
 ('Alfonso Phelps', 'B'),
 ('Sammy Burke', 'A-')]

>>> conn.close()
```

Problem 5. Write a function that accepts the name of a database file. Assuming the database to be in the format of the one created in Problems 1 and 2, query the database for all tuples of the form (`StudentName`, `CourseName`) where that student has an "A" or "A+" grade in that course. Return the list of tuples.

Aggregate Functions

A result set can be analyzed in Python using tools like NumPy, but SQL itself provides a few tools for computing a few very basic statistics: `AVG()`, `MIN()`, `MAX()`, `SUM()`, and `COUNT()` are *aggregate functions* that compress the columns of a result set into the desired quantity.

```
>>> conn = sql.connect("students.db")
>>> cur = conn.cursor()

# Get the number of students and the lowest ID number in StudentInfo.
>>> cur.execute("SELECT COUNT(StudentName), MIN(StudentID) FROM StudentInfo;")
>>> cur.fetchall()
[(10, 103066521)]
```

Problem 6. Write a function that accepts the name of a database file. Assuming the database to be in the format of the one created in Problem 3, query the `USEarthquakes` table for the following information.

- The magnitudes of the earthquakes during the 19th century (1800–1899).
- The magnitudes of the earthquakes during the 20th century (1900–1999).
- The average magnitude of all earthquakes in the database.

Create a single figure with two subplots: a histogram of the magnitudes of the earthquakes in the 19th century, and a histogram of the magnitudes of the earthquakes in the 20th century. Show the figure, then return the average magnitude of all of the earthquakes in the database. Be sure to return an actual number, not a list or a tuple.
(Hint: use `np.ravel()` to convert a result set of 1-tuples to a 1-D array.)

Note

Problem 6 raises an interesting question: are the number of earthquakes in the United States

increasing with time, and if so, how drastically? A closer look shows that only 3 earthquakes were recorded (in this data set) from 1700–1799, 208 from 1800–1899, and a whopping 3049 from 1900–1999. Is the increase in earthquakes due to there actually being more earthquakes, or to the improvement of earthquake detection technology? The best answer without conducting additional research is "probably both." Be careful to question the nature of your data—how it was gathered, what it may be lacking, what biases or lurking variables might be present—before jumping to strong conclusions.

See the following for more info on the `sqlite3` and SQL in general.

- `https://docs.python.org/3/library/sqlite3.html`
- `https://www.w3schools.com/sql/`
- `https://en.wikipedia.org/wiki/SQL_injection`

Additional Material

Shortcuts for WHERE Conditions

Complicated `WHERE` conditions can be simplified with the following commands.

- `IN`: check for equality to one of several values quickly, similar to Python's `in` operator. In other words, the following SQL commands are equivalent.

```
SELECT * FROM StudentInfo WHERE MajorID == 1 OR MajorID == 2;
SELECT * FROM StudentInfo WHERE MajorID IN (1,2);
```

- `BETWEEN`: check two (inclusive) inequalities quickly. The following are equivalent.

```
SELECT * FROM MyTable WHERE AGE >= 20 AND AGE <= 60;
SELECT * FROM MyTable WHERE AGE BETWEEN 20 AND 60;
```

9 SQL 2 (The Sequel)

Lab Objective: *Since SQL databases contain multiple tables, retrieving information about the data can be complicated. In this lab we discuss joins, grouping, and other advanced SQL query concepts to facilitate rapid data retrieval.*

We will use the following database as an example throughout this lab, found in `students.db`.

MajorID	MajorName
1	Math
2	Science
3	Writing
4	Art

(a) MajorInfo

CourseID	CourseName
1	Calculus
2	English
3	Pottery
4	History

(b) CourseInfo

StudentID	StudentName	MajorID
401767594	Michelle Fernandez	1
678665086	Gilbert Chapman	NULL
553725811	Roberta Cook	2
886308195	Rene Cross	3
103066521	Cameron Kim	4
821568627	Mercedes Hall	NULL
206208438	Kristopher Tran	2
341324754	Cassandra Holland	1
262019426	Alfonso Phelps	NULL
622665098	Sammy Burke	2

(c) StudentInfo

StudentID	CourseID	Grade
401767594	4	C
401767594	3	B–
678665086	4	A+
678665086	3	A+
553725811	2	C
678665086	1	B
886308195	1	A
103066521	2	C
103066521	3	C–
821568627	4	D
821568627	2	A+
821568627	1	B
206208438	2	A
206208438	1	C+
341324754	2	D–
341324754	1	A–
103066521	4	A
262019426	2	B
262019426	3	C
622665098	1	A
622665098	2	A–

(d) StudentGrades

Table 9.1: Student database.

Joining Tables

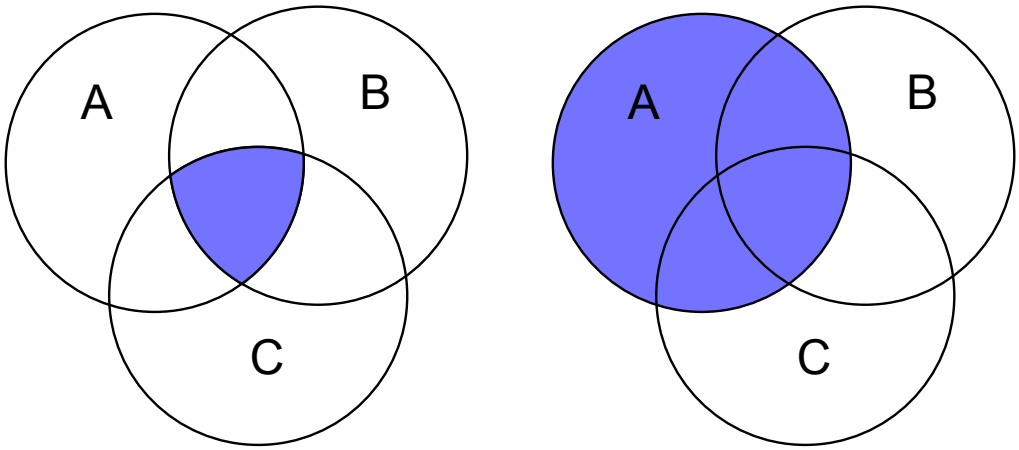
A *join* combines rows from different tables in a database based on common attributes. In other words, a join operation creates a new, temporary table containing data from 2 or more existing tables. Join commands in SQLite have the following general syntax.

```
SELECT <alias.column, ...>
    FROM <table> AS <alias> JOIN <table> AS <alias>, ...
    ON <alias.column> == <alias.column>, ...
    WHERE <condition>;
```

The `ON` clause tells the query how to join tables together. Typically if there are N tables being joined together, there should be $N - 1$ conditions in the `ON` clause.

Inner Joins

An *inner join* creates a temporary table with the rows that have exact matches on the attribute(s) specified in the `ON` clause. Inner joins **intersect** two or more tables, as in Figure 9.1a.



(a) An inner join of A, B, and C.

(b) A left outer join of A with B and C.

Figure 9.1

For example, Table 9.1c (`StudentInfo`) and Table 9.1a (`MajorInfo`) both have a `MajorID` column, so the tables can be joined by pairing rows that have the same `MajorID`. Such a join temporarily creates the following table.

StudentID	StudentName	MajorID	MajorID	MajorName
401767594	Michelle Fernandez	1	1	Math
553725811	Roberta Cook	2	2	Science
886308195	Rene Cross	3	3	Writing
103066521	Cameron Kim	4	4	Art
206208438	Kristopher Tran	2	2	Science
341324754	Cassandra Holland	1	1	Math
622665098	Sammy Burke	2	2	Science

Table 9.2: An inner join of `StudentInfo` and `MajorInfo` on `MajorID`.

Notice that this table is missing the rows where `MajorID` was `NULL` in the `StudentInfo` table. This is because there was no match for `NULL` in the `MajorID` column of the `MajorInfo` table, so the inner join throws those rows away.

Because joins deal with multiple tables at once, it is important to assign table aliases with the `AS` command. Join statements can also be supplemented with `WHERE` clauses like regular queries.

```
>>> import sqlite3 as sql
>>> conn = sql.connect("students.db")
>>> cur = conn.cursor()

>>> cur.execute("SELECT * "
...             "FROM StudentInfo AS SI INNER JOIN MajorInfo AS MI "
...             "ON SI.MajorID == MI.MajorID;").fetchall()
[(401767594, 'Michelle Fernandez', 1, 1, 'Math'),
 (553725811, 'Roberta Cook', 2, 2, 'Science'),
 (886308195, 'Rene Cross', 3, 3, 'Writing'),
 (103066521, 'Cameron Kim', 4, 4, 'Art'),
 (206208438, 'Kristopher Tran', 2, 2, 'Science'),
 (341324754, 'Cassandra Holland', 1, 1, 'Math'),
 (622665098, 'Sammy Burke', 2, 2, 'Science')]

# Select the names and ID numbers of the math majors.
>>> cur.execute("SELECT SI.StudentName, SI.StudentID "
...             "FROM StudentInfo AS SI INNER JOIN MajorInfo AS MI "
...             "ON SI.MajorID == MI.MajorID "
...             "WHERE MI.MajorName == 'Math';").fetchall()
[('Cassandra Holland', 341324754), ('Michelle Fernandez', 401767594)]
```

Problem 1. Write a function that accepts the name of a database file. Assuming the database to be in the format of Tables 9.1a–9.1d, query the database for the list of the names of students who have a B grade in any course (not a B– or a B+). Be sure to return a list of strings, not a list of tuples of strings.

Outer Joins

A *left outer join*, sometimes called a *left join*, creates a temporary table with **all** of the rows from the first (left-most) table, and all the "matched" rows on the given attribute(s) from the other relations. Rows from the left table that don't match up with the columns from the other tables are supplemented with `NULL` values to fill extra columns. Compare the following table and code to Table 9.2.

StudentID	StudentName	MajorID	MajorID	MajorName
401767594	Michelle Fernandez	1	1	Math
678665086	Gilbert Chapman	NULL	NULL	NULL
553725811	Roberta Cook	2	2	Science
886308195	Rene Cross	3	3	Writing
103066521	Cameron Kim	4	4	Art
821568627	Mercedes Hall	NULL	NULL	NULL
206208438	Kristopher Tran	2	2	Science
341324754	Cassandra Holland	1	1	Math
262019426	Alfonso Phelps	NULL	NULL	NULL
622665098	Sammy Burke	2	2	Science

Table 9.3: A left outer join of `StudentInfo` and `MajorInfo` on `MajorID`.

```
>>> cur.execute("SELECT * "
...             "FROM StudentInfo AS SI LEFT OUTER JOIN MajorInfo AS MI "
...             "ON SI.MajorID == MI.MajorID;").fetchall()
[(401767594, 'Michelle Fernandez', 1, 1, 'Math'),
 (678665086, 'Gilbert Chapman', None, None, None),
 (553725811, 'Roberta Cook', 2, 2, 'Science'),
 (886308195, 'Rene Cross', 3, 3, 'Writing'),
 (103066521, 'Cameron Kim', 4, 4, 'Art'),
 (821568627, 'Mercedes Hall', None, None, None),
 (206208438, 'Kristopher Tran', 2, 2, 'Science'),
 (341324754, 'Cassandra Holland', 1, 1, 'Math'),
 (262019426, 'Alfonso Phelps', None, None, None),
 (622665098, 'Sammy Burke', 2, 2, 'Science')]
```

Some flavors of SQL also support the `RIGHT OUTER JOIN` command, but `sqlite3` does not recognize the command since `T1 RIGHT OUTER JOIN T2` is equivalent to `T2 LEFT OUTER JOIN T1`.

Joining Multiple Tables

Complicated queries often join several different relations. If the same kind of join is being used, the relations and conditional statements can be put in list form. For example, the following code selects courses that Kristopher Tran has taken, and the grades that he got in those courses, by joining three tables together. Note that 2 conditions are required in the `ON` clause in this case.

```
>>> cur.execute("SELECT CI.CourseName, SG.Grade "
...             "FROM StudentInfo AS SI "           # Join 3 tables.
...                 "INNER JOIN CourseInfo AS CI, StudentGrades SG "
...             "ON SI.StudentID==SG.StudentID AND CI.CourseID==SG.CourseID "
...             "WHERE SI.StudentName == 'Kristopher Tran';").fetchall()
[('Calculus', 'C+'), ('English', 'A')]
```

To use different kinds of joins in a single query, append one join statement after another. The join closest to the beginning of the statement is executed first, creating a temporary table, and the next join attempts to operate on that table. The following example performs an additional join on Table 9.3 to find the name and major of every student who got a C in a class.

```
# Do an inner join on the results of the left outer join.
>>> cur.execute("SELECT SI.StudentName, MI.MajorName "
...              "FROM StudentInfo AS SI LEFT OUTER JOIN MajorInfo AS MI "
...              "ON SI.MajorID == MI.MajorID "
...              "INNER JOIN StudentGrades AS SG "
...              "ON SI.StudentID == SG.StudentID "
...              "WHERE SG.Grade == 'C';").fetchall()
[('Michelle Fernandez', 'Math'),
 ('Roberta Cook', 'Science'),
 ('Cameron Kim', 'Art'),
 ('Alfonso Phelps', None)]
```

In this last example, note carefully that Alfonso Phelps would have been excluded from the result set if an inner join was performed first instead of an outer join (since he lacks a major).

Problem 2. Write a function that accepts the name of a database file. Query the database for all tuples of the form `(Name, MajorName, Grade)` where `Name` is a student's name and `Grade` is their grade in Calculus. Only include results for students that are actually taking Calculus, but be careful not to exclude students who haven't declared a major.

Grouping Data

Many data sets can be naturally sorted into groups. The `GROUP BY` command gathers rows from a table and groups them by a certain attribute. The groups are then combined by one of the *aggregate functions* `AVG()`, `MIN()`, `MAX()`, `SUM()`, or `COUNT()`. Each of these functions accepts the name of the column to be operated on. Note that the first four of these require the column to hold numerical data. Since our database has no such data, we will delay examples of using the functions other than `COUNT()` until later.

The following code groups the rows in Table 9.1d by `studentID` and counts the number of entries in each group.

```
>>> cur.execute("SELECT StudentID, COUNT(*) "   # * means "all of the rows".
...              "FROM StudentGrades "
...              "GROUP BY StudentID;").fetchall()
[(103066521, 3),
 (206208438, 2),
 (262019426, 2),
 (341324754, 2),
 (401767594, 2),
 (553725811, 1),
 (622665098, 2),
 (678665086, 3),
 (821568627, 3),
 (886308195, 1)]
```

`GROUP BY` can also be used in conjunction with joins. The join creates a temporary table like Tables 9.2 or 9.3, the results of which can then be grouped.

```
>>> cur.execute("SELECT SI.StudentName, COUNT(*) "
...             "FROM StudentGrades AS SG INNER JOIN StudentInfo AS SI "
...             "ON SG.StudentID == SI.StudentID "
...             "GROUP BY SG.StudentID;").fetchall()
[('Cameron Kim', 3),
 ('Kristopher Tran', 2),
 ('Alfonso Phelps', 2),
 ('Cassandra Holland', 2),
 ('Michelle Fernandez', 2),
 ('Roberta Cook', 1),
 ('Sammy Burke', 2),
 ('Gilbert Chapman', 3),
 ('Mercedes Hall', 3),
 ('Rene Cross', 1)]
```

Just like the `WHERE` clause chooses rows in a relation, the `HAVING` clause chooses groups from the result of a `GROUP BY` based on some criteria related to the groupings. For this particular command, it is often useful (but not always necessary) to create an alias for the columns of the result set with the `AS` operator. For instance, the result set of the previous example can be filtered down to only contain students who are taking 3 courses.

```
>>> cur.execute("SELECT SI.StudentName, COUNT(*) as num_courses "   # Alias.
...             "FROM StudentGrades AS SG INNER JOIN StudentInfo AS SI "
...             "ON SG.StudentID == SI.StudentID "
...             "GROUP BY SG.StudentID "
...             "HAVING num_courses == 3;").fetchall()   # Refer to alias later↩
    .
[('Cameron Kim', 3), ('Gilbert Chapman', 3), ('Mercedes Hall', 3)]

# Alternatively, get just the student names.
>>> cur.execute("SELECT SI.StudentName "                             # No alias.
...             "FROM StudentGrades AS SG INNER JOIN StudentInfo AS SI "
...             "ON SG.StudentID == SI.StudentID "
...             "GROUP BY SG.StudentID "
...             "HAVING COUNT(*) == 3;").fetchall()
[('Cameron Kim',), ('Gilbert Chapman',), ('Mercedes Hall',)]
```

Other Miscellaneous Commands

Ordering Result Sets

The `ORDER BY` command sorts a result set by one or more attributes. Sorting can be done in ascending or descending order with `ASC` or `DESC`, respectively. This is always the very last statement in a query.

```
>>> cur.execute("SELECT SI.StudentName, COUNT(*) AS num_courses "   # Alias.
```

```
...             "FROM StudentGrades AS SG INNER JOIN StudentInfo AS SI "
...             "ON SG.StudentID == SI.StudentID "
...             "GROUP BY SG.StudentID "
...             "ORDER BY num_courses DESC, SI.StudentName ASC;").fetchall()
[('Cameron Kim', 3),                # The results are now ordered by the
 ('Gilbert Chapman', 3),            # number of courses each student is in,
 ('Mercedes Hall', 3),              # then alphabetically by student name.
 ('Alfonso Phelps', 2),
 ('Cassandra Holland', 2),
 ('Kristopher Tran', 2),
 ('Michelle Fernandez', 2),
 ('Sammy Burke', 2),
 ('Rene Cross', 1),
 ('Roberta Cook', 1)]
```

Problem 3. Write a function that accepts a database file. Query the given database for tuples of the form `(MajorName, N)` where `N` is the number of students in the specified major. Sort the results in descending order by the count `N`, and then in alphabetic order by MajorName. Include **Null** majors.

Searching Text with Wildcards

The `LIKE` operator within a `WHERE` clause matches patterns in a `TEXT` column. The special characters `%` and `_` and called *wildcards* that match any number of characters or a single character, respectively. For instance, `%Z_` matches any string of characters ending in a Z then another character, and `%i%` matches any string containing the letter i.

```
>>> results = cur.execute("SELECT StudentName FROM StudentInfo "
...                       "WHERE StudentName LIKE '%i%';").fetchall()
>>> [r[0] for r in results]
['Michelle Fernandez', 'Gilbert Chapman', 'Cameron Kim', 'Kristopher Tran']
```

Case Expressions

A case expression maps the values in a column using boolean logic. There are two forms of a case expression: simple and searched. A *simple case expression* matches and replaces specified attributes.

```
# Replace the values MajorID with new custom values.
>>> cur.execute("SELECT StudentName, CASE MajorID "
...                 "WHEN 1 THEN 'Mathematics' "
...                 "WHEN 2 THEN 'Soft Science' "
...                 "WHEN 3 THEN 'Writing and Editing' "
...                 "WHEN 4 THEN 'Fine Arts' "
...                 "ELSE 'Undeclared' END "
...             "FROM StudentInfo "
```

```
...             "ORDER BY StudentName ASC;").fetchall()
[('Alfonso Phelps', 'Undeclared'),
 ('Cameron Kim', 'Fine Arts'),
 ('Cassandra Holland', 'Mathematics'),
 ('Gilbert Chapman', 'Undeclared'),
 ('Kristopher Tran', 'Soft Science'),
 ('Mercedes Hall', 'Undeclared'),
 ('Michelle Fernandez', 'Mathematics'),
 ('Rene Cross', 'Writing and Editing'),
 ('Roberta Cook', 'Soft Science'),
 ('Sammy Burke', 'Soft Science')]
```

A *searched case expression* involves using a boolean expression at each step, instead of listing all of the possible values for an attribute.

```
# Change NULL values in MajorID to 'Undeclared' and non-NULL to 'Declared'.
>>> cur.execute("SELECT StudentName, CASE "
...                 "WHEN MajorID IS NULL THEN 'Undeclared' "
...                 "ELSE 'Declared' END "
...             "FROM StudentInfo "
...             "ORDER BY StudentName ASC;").fetchall()
[('Alfonso Phelps', 'Undeclared'),
 ('Cameron Kim', 'Declared'),
 ('Cassandra Holland', 'Declared'),
 ('Gilbert Chapman', 'Undeclared'),
 ('Kristopher Tran', 'Declared'),
 ('Mercedes Hall', 'Undeclared'),
 ('Michelle Fernandez', 'Declared'),
 ('Rene Cross', 'Declared'),
 ('Roberta Cook', 'Declared'),
 ('Sammy Burke', 'Declared')]
```

Chaining Queries

The result set of any SQL query is really just another table with data from the original database. Separate queries can be made from result sets by enclosing the entire query in parentheses. For these sorts of operations, it is very important to carefully label the columns resulting from a subquery.

```
# Count how many declared and undeclared majors there are
# The subquery changes NULL values in MajorID to 'Undeclared' and
#    non-NULL to 'Declared'.
>>> cur.execute("SELECT majorstatus, COUNT(*) AS majorcount "
...             "FROM ( "                                   # Begin subquery.
...                 "SELECT StudentName, CASE "
...                 "WHEN MajorID IS NULL THEN 'Undeclared' "
...                 "ELSE 'Declared' END AS majorstatus "
...                 "FROM StudentInfo) "                    # End subquery.
...             "GROUP BY majorstatus "
```

```
...             "ORDER BY majorcount DESC;").fetchall()
[('Declared', 7), ('Undeclared', 3)]
```

Subqueries can also be joined with other tables, as in the following example. Note also that a subquery can be used to create numerical data out of non-numerical data, which can then be passed into any of the aggregate functions.

```
# Find the proportion of classes each student has an A+, A, or A- in.
# The inner query creates a column 'gradeisa' which is 1 if the student's grade
#    is A+, A, or A-, and 0 otherwise.
>>> cur.execute("SELECT SI.StudentName, AVG(SG.gradeisa) "
...             "FROM ("
...                 "SELECT StudentID, CASE Grade "
...                     "WHEN 'A+' THEN 1 "
...                     "WHEN 'A' THEN 1 "
...                     "WHEN 'A-' THEN 1 "
...                     "ELSE 0 END AS gradeisa "
...                 "FROM StudentGrades) AS SG "
...             "INNER JOIN StudentInfo AS SI "
...             "ON SG.StudentID == SI.StudentID "
...             "GROUP BY SG.StudentID;").fetchall()
[('Cameron Kim', 0.3333333333333333),
 ('Kristopher Tran', 0.5),
 ('Alfonso Phelps', 0.0),
 ('Cassandra Holland', 0.5),
 ('Michelle Fernandez', 0.0),
 ('Roberta Cook', 0.0),
 ('Sammy Burke', 1.0),
 ('Gilbert Chapman', 0.6666666666666666),
 ('Mercedes Hall', 0.3333333333333333),
 ('Rene Cross', 1.0)]
```

Problem 4. Write a function that accepts the name of a database file. Query the database for tuples of the form `(StudentName, N, GPA)` where `N` is the number of courses that the specified student is enrolled in and `GPA` is their grade point average based on the following point system:

A+, A = 4.0	B = 3.0	C = 2.0	D = 1.0
A– = 3.7	B– = 2.7	C– = 1.7	D– = 0.7
B+ = 3.4	C+ = 2.4	D+ = 1.4	

Order the results from greatest GPA to least.

Problem 5. The file `mystery_database.db` contains 4 tables called `table_1`, `table_2`, `table_3`, and `table_4` which contain information on over 5000 subjects. Hidden within these subjects is an obvious outlier. Use what you've learned about SQL to identify the outlier in

this database. Return the outlier's name, ID number, eye color, and height as strings in a list. Hint: you may find that joining the tables is more difficult than it's worth; instead, try finding one clue at a time. Most of these subjects lived a long time ago in a galaxy far, far away... so a good place to start might be to find a subject who doesn't meet that criteria. Interesting information about subjects is often included in the `description` column. Also, recall that the following commands can be used to get the column names of a specified table:

```
>>> with sql.connect("database.db") as conn:
...     cur = conn.cursor()
...     cur.execute("SELECT * FROM specified_table;")
...     print([d[0] for d in cur.description])
...
['column_1', 'column_2', 'column_3']
```

The following command might be of use: `cur.execute("PRAGMA case_sensitive_like = TRUE;")`. This command will make the `LIKE` operator case sensitive from the point of execution, until the case sensitivity is turned off.

```
>>> with sql.connect("database.db") as conn:
...     cur = conn.cursor()
...     # Make the LIKE operator hereafter case sensitive
...     cur.execute("PRAGMA case_sensitive_like = TRUE;")
...     cur.execute("SELECT * FROM specified_table "
...                 "WHERE column_1 LIKE 'value';")
```

Achtung!

The goal of this lab is to introduce the language SQL, not to teach a specific implementation of SQL. In this lab, we will use the `sqlite3` library, which allows us to interact with a *SQLite* database file. *SQLite* is lightweight, and does not require additional setup, making it great for small projects like this lab and databases which will only be used by a single program at once. For example, Android applications frequently use *SQLite* databases for local storage. However, it is not suitable for many data science applications, since it is difficult to scale across computers and programs. There are a variety of databases that use SQL-like languages designed for different purposes, including *MySQL*, which scales much better and manages concurrent connections efficiently but requires a server and a significant amount of overhead to run.

Although these databases have minor differences in SQL syntax, the concepts taught in this lab are extremely transferrable across all SQL-like derivatives. Remember that you will need to change the Python library you use based on which database type you are connecting to.

10 Regular Expressions

Lab Objective: *Cleaning and formatting data are fundamental problems in data science. Regular expressions are an important tool for working with text carefully and efficiently, and are useful for both gathering and cleaning data. This lab introduces regular expression syntax and common practices, including an application to a data cleaning problem. This link may be helpful as a reference: https://docs.python.org/3/library/re.html*

A *regular expression* or *regex* is a string of characters that follows a certain syntax to specify a pattern, like generalized shorthand for strings. Strings that follow the pattern are said to *match* the expression (and vice versa). A single regular expression can match a large set of strings, such as the set of all valid email addresses.

> ACHTUNG!
>
> There are some universal standards for regular expression syntax, but the exact syntax varies slightly depending on the program or language. However, the syntax presented in this lab (for Python) is sufficiently similar to any other regex system. Consider learning to use regular expressions in Vim or your favorite text editor, keeping in mind that there will be slight syntactic differences from what is presented here.

Regular Expression Syntax in Python

The `re` module implements regular expressions in Python. The function `re.compile()` takes in a regular expression string and returns a corresponding *pattern* object, which has methods for determining if and how other strings match the pattern. You can think of the re.compile object as a box with a certain shape cut out of the bottom. When a lot of differently shaped objects are put into the box and shaken around only the objects with the same exact shape as the one cut out of the box will fall out. One method that `re.compile()` uses is the `search()` method, which returns `None` for a string that doesn't match, and a *match* object for a string that does match.

The `match()` method is different than the match object mentioned previously. The match method only matches strings that satisfy the pattern **at the beginning** of the string. To answer the question "does any part of my target string match this regular expression?" always use the `search()` method.

```
>>> import re
>>> pattern = re.compile("cat")      # Make a pattern object for finding 'cat'.
>>> bool(pattern.search("cat"))      # 'cat' matches 'cat', of course.
True
>>> bool(pattern.match("catfish"))   # 'catfish' starts with 'cat'.
True
>>> bool(pattern.match("fishcat"))   # 'fishcat' doesn't start with 'cat'.
False
>>> bool(pattern.search("fishcat")) # but it does contain 'cat'.
True
>>> bool(pattern.search("hat"))      # 'hat' does not contain 'cat'.
False
```

Most of the functions in the `re` module are shortcuts for compiling a pattern object and calling one of its methods. Using `re.compile()` is good practice because the resulting object (analagously, the box you made) is reusable, while each call to `re.search()` compiles a new (but redundant) pattern object. For example, the following lines of code are equivalent.

```
>>> bool(re.compile("cat").search("catfish"))
True
>>> bool(re.search("cat", "catfish"))
True
```

Assigning re.compile("cat").search("catfish") or re.search("cat","catfish") without the bool() around them to variables will create match objects.

Problem 1. Write a function that compiles and returns a regular expression pattern object with the pattern string `"python"`.

Literal Characters and Metacharacters

The following string characters (separated by spaces) are *metacharacters* in Python's regular expressions, meaning they have special significance in a pattern string:
`. ^ $ * + ? { } [ ] \ | ( )`.

A regular expression that matches strings with one or more metacharacters requires two things.

1. Use *raw strings* instead of regular Python strings by prefacing the string with an `r`, such as `r"cat"`. The resulting string interprets backslashes as actual backslash characters, rather than the start of an escape sequence like `\n` or `\t`.

2. Preface any metacharacters with a backslash to indicate a literal character. For example, to match the string `"$3.99? Thanks."`, use `r"\$3\.99\? Thanks\."`.

Without raw strings, every backslash has to be written as a double backslash, which makes many regular expression patterns hard to read (`"\\$3\\.99\\? Thanks\\."`).

Problem 2. Write a function that compiles and returns a regular expression pattern object that matches the string `"^{@}(?)[%]{.}(*)[_]{&}$"`.

Hint: There are online sites like `https://regex101.com/` that can help check answers. Consider building regex expressions one character at a time at this website.

The regular expressions of Problems 1 and 2 only match strings that are or include the exact pattern. The metacharacters allow regular expressions to have much more flexibility and control so that a single pattern can match a wide variety of strings, or a very specific set of strings. The *line anchor* metacharacters `^` and `$` are used to match the **start** and the **end** of a line of text, respectively. This shrinks the matching set, even when using the `search()` method instead of the `match()` method. For example, the only single-line string that the expression `'^x$'` matches is `'x'`, whereas the expression `'x'` can match any string with an `'x'` in it.

The *pipe* character `|` is a logical `OR` in a regular expression: `A|B` matches `A` or `B`. The parentheses `()` create a *group* in a regular expression. A group establishes an order of operations in an expression. For example, in the regex `"^one|two fish$"`, precedence is given to the invisible string concatenation between `"two"` and `"fish"`, while `"^(one|two) fish$"` gives precedence to the `'|'` metacharacter. Notice that the *pipe* is inside the *group*.

```
>>> fish = re.compile(r"^(one|two) fish$")
>>> for test in ["one fish", "two fish", "red fish", "one two fish"]:
...     print(test + ':', bool(fish.search(test)))
...
one fish: True
two fish: True
red fish: False
one two fish: False
```

Problem 3. Write a function that compiles and returns a regular expression pattern object that matches the following strings, and no other strings, even with `re.search()`.

```
"Book store"      "Mattress store"      "Grocery store"
"Book supplier"   "Mattress supplier"   "Grocery supplier"
```

Hint: The naive way to do this is create a very long chain of `or` operators with the exact phrases as options. Instead, think about dividing it into two groups of `or` operators where the first group picks the first word and the second groups picks the second word.

There is a file called `test_regular_expressions.py` that contains some prewritten unit tests to help you test your function for this problem.

Character Classes

The hard bracket metacharacters `[` and `]` are used to create *character classes*, a part of a regular expression that can match a variety of characters. For example, the pattern `[abc]` matches any of the characters `a`, `b`, or `c`. This is different than a group delimited by parentheses: a group can match multiple characters, while a character class matches only one character. For instance, `[abc]` does not match `ab` or `abc`, and `(abc)` matches `abc` but not `ab` or even `a`.

Within character classes, there are two additional metacharacters. When `^` appears **as the first character** in a character class, right after the opening bracket `[`, the character class matches anything **not** specified instead. In other words, `^` is the set complement operation on the character class. Additionally, the dash `-` specifies a range of values. For instance, `[0-9]` matches any digit, and `[a-z]` matches any lowercase letter. Thus `[^0-9]` matches any character **except** for a digit, and `[^a-z]` matches any character **except** for lowercase letters. Keep in mind that the dash `-`, when at the beginning or end of the character class, will match the literal `'-'`. Note that `[0-27-9]` acts like `[(0-2)|(7-9)]`.

```
>>> p1, p2 = re.compile(r"^[a-z][^0-7]$"), re.compile(r"^[^abcA-C][0-27-9]$")
>>> for test in ["d8", "aa", "E9", "EE", "d88"]:
...     print(test + ':', bool(p1.search(test)), bool(p2.search(test)))
...
d8: True True
aa: True False                      # a is not in [^abcA-C] or [0-27-9].
E9: False True                      # E is not in [a-z].
EE: False False                     # E is not in [a-z] or [0-27-9].
d88: False False                    # Too many characters.
```

There are also a variety of shortcuts that represent common character classes, listed in Table 10.1. Familiarity with these shortcuts makes some regular expressions significantly more readable.

Character	Description
`\b`	Matches the empty string, but only at the start or end of a word.
`\s`	Matches any whitespace character; equivalent to `[ \t\n\r\f\v]`.
`\S`	Matches any non-whitespace character; equivalent to `[^\s]`.
`\d`	Matches any decimal digit; equivalent to `[0-9]`.
`\D`	Matches any non-digit character; equivalent to `[^\d]`.
`\w`	Matches any alphanumeric character; equivalent to `[a-zA-Z0-9_]`.
`\W`	Matches any non-alphanumeric character; equivalent to `[^\w]`.

Table 10.1: Character class shortcuts.

Any of the character class shortcuts can be used within other custom character classes. For example, `[_A-Z\s]` matches an underscore, capital letter, or whitespace character.

Finally, a period `.` matches **any** character except for a line break. This is a very powerful metacharacter; be careful to only use it when part of the regular expression really should match **any** character.

```
# Match any three-character string with a digit in the middle.
>>> pattern = re.compile(r"^.\d.$")
>>> for test in ["a0b", "888", "n2%", "abc", "cat"]:
...     print(test + ':', bool(pattern.search(test)))
...
a0b: True
888: True
n2%: True
abc: False
cat: False
```

```
# Match two letters followed by a number and two non-newline characters.
>>> pattern = re.compile(r"^[a-zA-Z][a-zA-Z]\d..$")
>>> for test in ["tk421", "bb8!?", "JB007", "Boba?"]:
...     print(test + ':', bool(pattern.search(test)))
..
tk421: True
bb8!?: True
JB007: True
Boba?: False
```

The following table is a useful recap of some common regular expression metacharacters.

Character	Description
`.`	Matches any character except a newline.
`^`	Matches the start of the string.
`$`	Matches the end of the string or just before the newline at the end of the string.
`|`	`A|B` creates an regular expression that will match either `A` or `B`.
`[...]`	Indicates a set of characters. A `^` as the first character indicates a complementing set.
`(...)`	Matches the regular expression inside the parentheses. The contents can be retrieved or matched later in the string.

Table 10.2: Standard regular expression metacharacters in Python.

Repetition

The remaining metacharacters are for matching a specified number of characters. This allows a single regular expression to match strings of varying lengths.

Character	Description
`*`	Matches 0 or more repetitions of the preceding regular expression.
`+`	Matches 1 or more repetitions of the preceding regular expression.
`?`	Matches 0 or 1 of the preceding regular expression.
`{m,n}`	Matches from m to n repetitions of the preceding regular expression.
`*?`, `+?`, `??`, `{m,n}?`	Non-greedy versions of the previous four special characters.

Table 10.3: Repetition metacharacters for regular expressions in Python.

Each of the repetition operators acts on the expression immediately preceding it. This could be a single character, a group, or a character class. For instance, `(abc)+` matches `abc`, `abcabc`, `abcabcabc`, and so on, but not `aba` or `cba`. On the other hand, `[abc]*` matches any sequence of `a`, `b`, and `c`, including `abcabc` and `aabbcc`.

The curly braces `{}` specify a custom number of repetitions allowed. `{,n}` matches **up to** n instances, `{m,}` matches **at least** m instances, `{k}` matches **exactly** k instances, and `{m,n}` matches from m to n instances. Thus the `?` operator is equivalent to `{,1}` and `+` is equivalent to `{1,}`.

```
# Match exactly 3 'a' characters.
>>> pattern = re.compile(r"^a{3}$")
```

```
>>> for test in ["aa", "aaa", "aaaa", "aba"]:
...     print(test + ':', bool(pattern.search(test)))
...
aa: False                           # Too few.
aaa: True
aaaa: False                         # Too many.
aba: False
```

Be aware that line anchors are especially important when using repetition operators. Consider the following (bad) example and compare it to the previous example.

```
# Match exactly 3 'a' characters, hopefully.
>>> pattern = re.compile(r"a{3}")
>>> for test in ["aaa", "aaaa", "aaaaa", "aaaab"]:
...     print(test + ':', bool(pattern.search(test)))
...
aaa: True
aaaa: True                          # Should be too many!
aaaaa: True                         # Should be too many!
aaaab: True                         # Too many, and even with the 'b'?
```

The unexpected matches occur because `"aaa"` is at the beginning of each of the test strings. With the line anchors `^` and `$`, the search truly only matches the exact string `"aaa"`.

Problem 4. A *valid Python identifier* (a valid variable name) is any string starting with an alphabetic character or an underscore, followed by any (possibly empty) sequence of alphanumeric characters and underscores.

A *valid python parameter definition* is defined as the concatenation of the following strings:

- any valid python identifier
- any number of spaces
- (optional) an equals sign followed by any number of spaces and ending with one of the following three things: any real number, a single quote followed by any number of non-single-quote characters followed by a single quote (ex: 'example'), or any valid python identifier

Define a function that compiles and returns a regular expression pattern object that matches any valid Python parameter definition.
(Hint: Use the `\w` character class shortcut to keep your regular expression clean.)

To help in debugging, the following examples may be useful. These test cases are a good start, but are not exhaustive. The first table should match valid Python identifiers. The second should match a valid python parameter definition, as defined in this problem. Note that some strings which would be valid in python will not be for this problem.

Matches:	`"Mouse"`	`"_num = 2.3"`	`"arg_ = 'hey'"`	`"__x__"`	`"var24"`
Non-matches:	`"3rats"`	`"_num = 2.3.2"`	`"arg_ = 'one'two"`	`"sq(x)"`	`" x"`

Matches:	`"max=total"`	`"string= '"`	`"num_guesses"`
Non-matches:	`"max=2total"`	`"is_4=(value==4)"`	`"pattern = r'^one|two fish$'"`

Hint: It may seem more efficient to keep the equals sign part of the expression outisde of your `or` group (parentheses with pipelines) but that is a really tricky way to do it. It is easier to include the equals sign and space in each case individually. For example, $(= \backslash s^*...| = \backslash s^*...|...)$ instead of $(= \backslash s^*(...|...|...))$.
`Note:` The equals sign is not a special character, but is matching the character '=' exactly. The example above matches an equal sign followed by any number of whitespace.

Unit Test

There is a file called `test_regular_expressions.py` that contains prewritten unit tests for Problem 3. There is a place for you to add your own unit tests for your function in Problem 4 which will be graded.

Manipulating Text with Regular Expressions

So far we have been solely concerned with whether or not a regular expression and a string match, but the power of regular expressions comes with what can be done with a match. In addition to the `search()` method, regular expression pattern objects have the following useful methods.

Method	Description
`match()`	Match a regular expression pattern to the beginning of a string.
`fullmatch()`	Match a regular expression pattern to all of a string.
`search()`	Search a string for the presence of a pattern.
`sub()`	Substitute occurrences of a pattern found in a string.
`subn()`	Same as sub, but also return the number of substitutions made.
`split()`	Split a string by the occurrences of a pattern.
`findall()`	Find all occurrences of a pattern in a string.
`finditer()`	Return an iterator yielding a match object for each match.

Table 10.4: Methods of regular expression pattern objects.

Some substitutions require remembering part of the text that the regular expression matches. Groups are useful here: each group in the regular expression can be represented in the substitution string by `\n`, where n is an integer (starting at 1) specifying which group to use.

```
# Find words that start with 'cat', remembering what comes after the 'cat'.
>>> pig_latin = re.compile(r"\bcat(\w*)")
>>> target = "Let's catch some catfish for the cat"

>>> pig_latin.sub(r"at\1clay", target)  # \1 = (\w*) from the expression.
"Let's atchclay some atfishclay for the atclay"
```

The repetition operators `?`, `+`, `*`, and `{m,n}` are *greedy*, meaning that they match the largest string possible. On the other hand, the operators `??`, `+?`, `*?`, and `{m,n}?` are *non-greedy*, meaning they match the smallest strings possible. This is very often the desired behavior for a regular expression.

```
>>> target = "<abc> <def> <ghi>"

# Match angle brackets and anything in between.
>>> greedy = re.compile(r"^<.*>$")  # Greedy *
>>> greedy.findall(target)
['<abc> <def> <ghi>']               # The entire string matched!

# Try again, using the non-greedy version.
>>> nongreedy = re.compile(r"<.*?>")# Non-greedy *?
>>> nongreedy.findall(target)
['<abc>', '<def>', '<ghi>']         # Each <> set is an individual match.
```

Finally, there are a few customizations that make searching larger texts manageable. Each of these *flags* can be used as keyword arguments to `re.compile()`.

Flag	Description
`re.DOTALL`	`.` matches any character at all, including the newline.
`re.IGNORECASE`	Perform case-insensitive matching.
`re.MULTILINE`	`^` matches the beginning of lines (after a newline) as well as the string; `$` matches the end of lines (before a newline) as well as the end of the string.

Table 10.5: Regular expression flags.

A benefit of using `'^'` and `'$'` is that they allow you to search across multiple lines. For example, how would we match `"World"` in the string `"Hello\nWorld"`? Using `re.MULTILINE` in the `re.search` function will allow us to match at the beginning of each new line, instead of just the beginning of the string. The following shows how to implement multiline searching:

```
>>>pattern1 = re.compile("^W")
>>>pattern2 = re.compile("^W", re.MULTILINE)
>>>bool(pattern1.search("Hello\nWorld"))
False
>>>bool(pattern2.search("Hello\nWorld"))
True
```

Problem 5. A Python *block* is composed of several lines of code with the same indentation level. Blocks are delimited by key words and expressions, followed by a colon. Possible key words are `if`, `elif`, `else`, `for`, `while`, `try`, `except`, `finally`, `with`, `def`, and `class`. Some of these keywords require an expression to precede the colon (`if`, `elif`, `for`, etc.). Some require no expressions to precede the colon (`else`, `finally`), and `except` may or may not have an expression before the colon.

Write a function that accepts a string of Python code and uses regular expressions to place

colons in the appropriate spots. Assume that every colon is missing in the input string and that any keyword that should have an expression after it does. (Note that this will simplify your regex expression since you won't have to design it to handle cases where some colons are present or to detect the key words that need expressions versus ones that don't.) Return the string of code with colons in the correct places.

```
"""
k, i, p = 999, 1, 0
while k > i
    i *= 2
    p += 1
    if k != 999
        print("k should not have changed")
    else
        pass
print(p)
"""

# The string given above should become this string.
"""
k, i, p = 999, 1, 0
while k > i:
    i *= 2
    p += 1
    if k != 999:
        print("k should not have changed")
    else:
        pass
print(p)
"""
```

Extracting Text with Regular Expressions

Regular expressions are useful for locating and extracting information that matches a certain format. The method `pattern.findall(string)` returns a list containing all non-overlapping matches of `pattern` found in `string`. The method scans the string from left to right and returns the matches in that order. If two matches overlap, the match that begins first is returned.

When at least one group, indicated by `()`, is present in the pattern, then only information contained in a group is returned. Each match is returned as a tuple containing the part of the string that matches each group in the pattern.

```
>>> pattern = re.compile("\w* fish")

# Without any groups, the entirety of each match is returned.
>>> pattern.findall("red fish, blue fish, one fish, two fish")
['red fish', 'blue fish', 'one fish', 'two fish']
```

```
# When a group is present, only information contained in a group is returned.
>>> pattern2 = re.compile("(\w*) (fish|dish)")
>>> pattern2.findall("red dish, blue dish, one fish, two fish")
[('red', 'dish'), ('blue', 'dish'), ('one', 'fish'), ('two', 'fish')]
```

If you wish to extract the characters that match some groups, but not others, you can choose to exclude a group from being returned using the syntax `(?:)`

```
>>> pattern = re.compile("(\w*) (?:fish|dish)")
>>> pattern.findall("red dish, blue dish, one fish, two fish")
['red', 'blue', 'one', 'two']
```

Problem 6. The file `fake_contacts.txt` contains poorly formatted contact data for 2000 fictitious individuals. Each line of the file contains data for one person, including their name and possibly their birthday, email address, and/or phone number. The formatting of the data is not consistent, and much of it is missing. Each contact name includes a first and last name. Some names have middle initials, in the form `Jane C. Doe`. Each birthday lists the month, then the day, and then the year, though the format varies from `1/1/11`, `1/01/2011`, etc. If century is not specified for birth year, as in `1/01/XX`, birth year is assumed to be `20XX`. Remember, not all information is listed for each contact.

Use regular expressions to extract the necessary data and format it uniformly, writing birthdays as `mm/dd/yyyy` and phone numbers as `(xxx)xxx-xxxx`. Return a dictionary where the key is the name of an individual and the value is another dictionary containing their information. Each of these inner dictionaries should have the keys `"birthday"`, `"email"`, and `"phone"`. In the case of missing data, map the key to `None`.

The first two entries of the completed dictionary are given below.

```
{
    "John Doe": {
        "birthday": "01/01/2099",
        "email": "john_doe90@hopefullynotarealaddress.com",
        "phone": "(123)456-7890"
        },
    "Jane Smith": {
        "birthday": None,
        "email": None,
        "phone": "(222)111-3333"
        },
# ...
}
```

Hint: Think about creating a separate re.compile() object to 'catch' each piece of identifying information. Extract and clean each picece individually and build your dictionary incrementally. If the piece of information exists for a specific person, reformat and assign it to that pesron's dictionary. If the information doesn't exist assign it to be `None`.

Additional Material

Regular Expressions in the Unix Shell

As we have seen,, regular expressions are very useful when we want to match patterns. Regular expressions can be used when matching patterns in the Unix Shell. Though there are many Unix commands that take advantage of regular expressions, we will focus on `grep` and `awk`.

Regular Expressions and grep

Recall from Lab 1 that `grep` is used to match patterns in files or output. It turns out we can use regular expressions to define the pattern we wish to match.

In general, we use the following syntax:

```
$ grep 'regexp' filename
```

We can also use regular expressions when piping output to `grep`.

```
# List details of directories within current directory.
$ ls -l | grep ^d
```

Regular Expressions and awk

By incorporating regular expressions, the `awk` command becomes much more robust. Before GUI spreedsheet programs like Microsoft Excel, `awk` was commonly used to visualize and query data from a file.

Including `if` statements inside `awk` commands gives us the ability to perform actions on lines that match a given pattern. The following example prints the filenames of all files that are owned by `freddy`.

```
$ ls -l | awk ' {if ($3 ~ /freddy/) print $9} '
```

Because there is a lot going on in this command, we will break it down piece-by-piece. The output of `ls -l` is getting piped to `awk`. Then we have an `if` statement. The syntax here means if the condition inside the parenthesis holds, print field 9 (the field with the filename). The condition is where we use regular expressions. The `~` checks to see if the contents of field 3 (the field with the username) matches the regular expression found inside the forward slashes. To clarify, `freddy` is the regular expression in this example and the expression must be surrounded by forward slashes.

Consider a similar example. In this example, we will list the names of the directories inside the current directory. (This replicates the behavior of the Unix command `ls -d */`)

```
$ ls -l | awk ' {if ($1 ~ /^d/) print $9} '
```

Notice in this example, we printed the names of the directories, whereas in one of the example using `grep`, we printed all the details of the directories as well.

Achtung!

Some of the definitions for character classes we used earlier in this lab will not work in the Unix Shell. For example, `\w` and `\d` are not defined. Instead of `\w`, use `[[:alnum:]]`. Instead of `\d`, use `[[:digit:]]`. For a complete list of similar character classes, search the internet for *POSIX Character Classes* or *Bracket Character Classes.*

11 Web Scraping

Lab Objective: *Web Scraping is the process of gathering data from websites on the internet. Since almost everything rendered by an internet browser as a web page uses HTML, the first step in web scraping is being able to extract information from HTML. In this lab, we introduce the requests library for scraping web pages, BeautifulSoup: Python's canonical tool for efficiently and cleanly navigating and parsing HTML, and how to use Pandas to extract data from HTML tables.*

HTTP and Requests

HTTP stands for Hypertext Transfer Protocol, which is an application layer networking protocol. It is a higher level protocol than TCP, which we used to build a server in the Web Technologies lab, but uses TCP protocols to manage connections and provide network capabilities. The HTTP protocol is centered around a request and response paradigm, in which a client makes a request to a server and the server replies with a response. There are several methods, or *requests*, defined for HTTP servers, the three most common of which are GET, POST, and PUT. A GET request asks for information from the server, a POST request modifies the state of the server, and a PUT request adds new pieces of data to the server.

The standard way to get the source code of a website using Python is via the `requests` library.[1] Calling `requests.get()` sends an HTTP GET request to a specified website. The website returns a response code, which indicates whether or not the request was received, understood, and accepted. If the response code is good, typically `200`[2], then the response will also include the website source code as an HTML file.

```
>>> import requests

# Make a request and check the result. A status code of 200 is good.
>>> response = requests.get("http://www.byu.edu")
>>> print(response.status_code, response.ok, response.reason)
200 True OK
```

[1] Though `requests` is not part of the standard library, it is recognized as a standard tool in the data science community. See `http://docs.python-requests.org/`.

[2] See `https://en.wikipedia.org/wiki/List_of_HTTP_status_codes` for explanation of specific response codes.

```
# The HTML of the website is stored in the "text" attribute.
>>> print(response.text)
<!DOCTYPE html>
<html lang="en" dir="ltr" prefix="content: http://purl.org/rss/1.0/modules/↩
    content/  dc: http://purl.org/dc/terms/  foaf: http://xmlns.com/foaf/0.1/  ↩
    og: http://ogp.me/ns#  rdfs: http://www.w3.org/2000/01/rdf-schema#  schema:↩
     http://schema.org/  sioc: http://rdfs.org/sioc/ns#  sioct: http://rdfs.org↩
    /sioc/types#  skos: http://www.w3.org/2004/02/skos/core#  xsd: http://www.↩
    w3.org/2001/XMLSchema# " class=" ">

  <head>
    <meta charset="utf-8" />
# ...
```

Attribute	Description
`text`	Content of the response, in unicode
`content`	Content of the response, in bytes
`encoding`	Encoding used to decode response.content
`raw`	Raw socket response, which allows applications to send and obtain packets of information
`status_code`	Numeric code that shows how the action was successful
`ok`	Boolean that is `True` if the `status_code` is less than 400, or `False` if not
`reason`	Text corresponding to the status code

Table 11.1: Different content attributes of the request class.

Note that some websites aren't built to handle large amounts of traffic or many repeated requests. Most are built to identify web scrapers or crawlers that initiate many consecutive GET requests without pauses, and retaliate or block them. When web scraping, always make sure to store the data that you receive in a file and include error checks to prevent retrieving the same data unnecessarily. We won't spend much time on that in this lab, but it's especially important in larger applications.

Problem 1. Use the `requests` library to get the HTML source for the website `http://www.example.com`. Save the source as a file called `"example.html"`. If the file already exists, make sure not to scrape the website or overwrite the file. You will use this file later in the lab.

Hint: When writing responses to file you need to use the use the `open()` function with the appropriate file write mode ((either `mode='w'` for plain write mode, or `mode="wb"` for binary write mode). Python interpreters will write the example file the same way whether you use `response.text` or `response.content`.

ACHTUNG!

Scraping copyrighted information without the consent of the copyright owner can have severe

legal consequences. Many websites, in their terms and conditions, prohibit scraping parts or all of the site. Websites that do allow scraping usually have a file called `robots.txt` (for example, `www.google.com/robots.txt`) that specifies which parts of the website are off-limits and how often requests can be made according to the *robots exclusion standard*.[a]

Be careful and considerate when doing any sort of scraping, and take care when writing and testing code to avoid unintended behavior. It is up to the programmer to create a scraper that respects the rules found in the terms and conditions and in `robots.txt`.[b]

We will cover this more in the next lab.

[a] See `www.robotstxt.org/orig.html` and `en.wikipedia.org/wiki/Robots_exclusion_standard`.

[b] Python provides a parsing library called `urllib.robotparser` for reading `robot.txt` files. For more information, see `https://docs.python.org/3/library/urllib.robotparser.html`.

HTML

Hyper Text Markup Language, or *HTML*, is the standard *markup language*—a language designed for the processing, definition, and presentation of text—for creating webpages. It structures a document using pairs of *tags* that surround and define content. Opening tags have a tag name surrounded by angle brackets (`<tag-name>`). The companion closing tag looks the same, but with a forward slash before the tag name (`</tag-name>`). A list of all current HTML tags can be found at `http://htmldog.com/reference/htmltags`.

Most tags can be combined with *attributes* to include more data about the content, help identify individual tags, and make navigating the document much simpler. In the following example, the `<a>` tag has `id` and `href` attributes.

```
<html>                                  <!-- Opening tags -->
   <body>
       <p>
           Click <a id='info' href='http://www.example.com'>here</a>
           for more information.
       </p>                             <!-- Closing tags -->
   </body>
</html>
```

In HTML, `href` stands for *hypertext reference*, a link to another website. Thus the above example would be rendered by a browser as a single line of text, with `here` being a clickable link to `http://www.example.com`:

Click here for more information.

Unlike Python, HTML does not enforce indentation (or any whitespace rules), though indentation generally makes HTML more readable. The previous example can be written in a single line.

```
<html><body><p>Click <a id='info' href='http://www.example.com/info'>here</a>
  for more information.</p></body></html>
```

Special tags, which don't contain any text or other tags, are written without a closing tag and in a single pair of brackets. A forward slash is included between the name and the closing bracket. Examples of these include `<hr/>`, which describes a horizontal line, and `<img/>`, the tag for representing an image.

> **Note**
>
> You can open .html files using a text editor or any web browser. In a browser, you can inspect the source code associated with specific elements. Right click the element and select `Inspect`. If you are using Safari, you may first need to enable "Show Develop menu" in "Preferences" under the "Advanced" tab.

BeautifulSoup

BeautifulSoup (`bs4`) is a package[3] that makes it simple to navigate and extract data from HTML documents. See `http://www.crummy.com/software/BeautifulSoup/bs4/doc/index.html` for the full documentation.

The `bs4.BeautifulSoup` class accepts two parameters to its constructor: a string of HTML code and an HTML parser to use under the hood. The HTML parser is technically a keyword argument, but the constructor prints a warning if one is not specified. The standard choice for the parser is `"html.parser"`, which means the object uses the standard library's `html.parser` module as the engine behind the scenes.

> **Note**
>
> Depending on project demands, a parser other than `"html.parser"` may be useful. A couple of other options are `"lxml"`, an extremely fast parser written in C, and `"html5lib"`, a slower parser that treats HTML in much the same way a web browser does, allowing for irregularities. Both must be installed independently; see `https://www.crummy.com/software/BeautifulSoup/bs4/doc/#installing-a-parser` for more information.

A `BeautifulSoup` object represents an HTML document as a tree. In the tree, each tag is a *node* with nested tags and strings as its *children*. The `prettify()` method returns a string that can be printed to represent the BeautifulSoup object in a readable format that reflects the tree structure.

```
>>> from bs4 import BeautifulSoup

>>> small_example_html = """
<html><body><p>
    Click <a id='info' href='http://www.example.com'>here</a>
    for more information.
</p></body></html>
"""

>>> small_soup = BeautifulSoup(small_example_html, "html.parser")
>>> print(small_soup.prettify())
<html>
  <body>
```

[3] BeautifulSoup is not part of the standard library; install it with `pip install beautifulsoup4`.

```
    <p>
      Click
      <a href="http://www.example.com" id="info">
         here
      </a>
      for more information.
    </p>
  </body>
</html>
```

Each tag in a `BeautifulSoup` object's HTML code is stored as a `bs4.element.Tag` object, with actual text stored as a `bs4.element.NavigableString` object. Tags are accessible directly through the `BeautifulSoup` object.

```
# Get the <p> tag (and everything inside of it).
>>> small_soup.p
<p>
    Click <a href="http://www.example.com" id="info">here</a>
    for more information.
</p>

# Get the <a> sub-tag of the <p> tag.
>>> a_tag = small_soup.p.a
>>> print(a_tag, type(a_tag), sep='\n')
<a href="http://www.example.com" id="info">here</a>
<class 'bs4.element.Tag'>

# Get just the name, attributes, and text of the <a> tag.
>>> print(a_tag.name, a_tag.attrs, a_tag.string, sep="\n")
a
{'id': 'info', 'href': 'http://www.example.com'}
here
```

Attribute	Description
`name`	The name of the tag
`attrs`	A dictionary of the attributes
`string`	The single string contained in the tag
`strings`	Generator for strings of children tags
`stripped_strings`	Generator for strings of children tags, stripping whitespace
`text`	Concatenation of strings from all children tags

Table 11.2: Data attributes of the `bs4.element.Tag` class.

Problem 2. The `BeautifulSoup` class has a `find_all()` method that, when called with `True` as the only argument, returns a list of all tags in the HTML source code.

Write a function that accepts a string of HTML code as an argument. Use BeautifulSoup to return a list of the **names** of the tags in the code.

Navigating the Tree Structure

Not all tags are easily accessible from a `BeautifulSoup` object. Consider the following example.

```
>>> pig_html = """
<html><head><title>Three Little Pigs</title></head>
<body>
<p class="title"><b>The Three Little Pigs</b></p>
<p class="story">Once upon a time, there were three little pigs named
<a href="http://example.com/larry" class="pig" id="link1">Larry,</a>
<a href="http://example.com/mo" class="pig" id="link2">Mo</a>, and
<a href="http://example.com/curly" class="pig" id="link3">Curly.</a>
<p>The three pigs had an odd fascination with experimental construction.</p>
<p>...</p>
</body></html>
"""

>>> pig_soup = BeautifulSoup(pig_html, "html.parser")
>>> pig_soup.p
<p class="title"><b>The Three Little Pigs</b></p>

>>> pig_soup.a
<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>
```

Since the HTML in this example has several `<p>` and `<a>` tags, only the **first** tag of each name is accessible directly from `pig_soup`. The other tags can be accessed by manually navigating through the HTML tree.

Every HTML tag (except for the topmost tag, which is usually `<html>`) has a *parent* tag. Each tag also has zero or more *sibling* and *children* tags or text. Following a true tree structure, every `bs4.element.Tag` in a soup has multiple attributes for accessing or iterating through parent, sibling, or child tags.

Attribute	Description
`parent`	The parent tag
`parents`	Generator for the parent tags up to the top level
`next_sibling`	The tag immediately after to the current tag
`next_siblings`	Generator for sibling tags after the current tag
`previous_sibling`	The tag immediately before the current tag
`previous_siblings`	Generator for sibling tags before the current tag
`contents`	A list of the immediate children tags
`children`	Generator for immediate children tags
`descendants`	Generator for all children tags (recursively)

Table 11.3: Navigation attributes of the `bs4.element.Tag` class.

```
# Start at the first <a> tag in the soup.
>>> a_tag = pig_soup.a
>>> a_tag
<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>

# Get the names of all of <a>'s parent tags, traveling up to the top.
# The name "[document]" means it is the top of the HTML code.
>>> [par.name for par in a_tag.parents]     # <a>'s parent is <p>, whose
['p', 'body', 'html', '[document]']         # parent is <body>, and so on.

# Get the next siblings of <a>.
>>> a_tag.next_sibling
'\n'                                        # The first sibling is just text.
>>> a_tag.next_sibling.next_sibling         # The second sibling is a tag.
<a class="pig" href="http://example.com/mo" id="link2">Mo</a>
```

Note carefully that newline characters are considered to be children of a parent tag. Therefore iterating through children or siblings often requires checking which entries are tags and which are just text. In the next example, we use a tag's `attrs` attribute to access specific attributes within the tag (see Table 11.2).

```
# Get to the <p> tag that has class="story" using these commands.
>>> p_tag = pig_soup.body.p.next_sibling.next_sibling
>>> p_tag.attrs["class"]                # Make sure it's the right tag.
['story']

# Iterate through the child tags of <p> and print hrefs whenever they exist.
>>> for child in p_tag.children:
...     # Skip the children that are not bs4.element.Tag objects
...     # These don't have the attribute "attrs"
...     if hasattr(child, "attrs") and "href" in child.attrs:
...         print(child.attrs["href"])
http://example.com/larry
http://example.com/mo
http://example.com/curly
```

Note that the `"class"` attribute of the `<p>` tag is a list. This is because the `"class"` attribute can take on several values at once; for example, the tag `<p class="story book">` is of class `"story"` and of class `"book"`.

The behavior of the `string` attribute of a `bs4.element.Tag` object depends on the structure of the corresponding HTML tag.

1. If the tag has a string of text and no other child elements, then `string` is just that text.
2. If the tag has exactly one child tag and the child tag has only a string of text, then the tag has the same `string` as its child tag.
3. If the tag has more than one child, then `string` is `None`. In this case, use `strings` to iterate through the child strings. Alternatively, the `get_text()` method returns all text belonging to

a tag and to all of its descendants. In other words, it returns anything inside a tag that isn't another tag.

```
>>> pig_soup.head
<head><title>Three Little Pigs</title></head>

# Case 1: the <title> tag's only child is a string.
>>> pig_soup.head.title.string
'Three Little Pigs'

# Case 2: The <head> tag's only child is the <title> tag.
>>> pig_soup.head.string
'Three Little Pigs'

# Case 3: the <body> tag has several children.
>>> pig_soup.body.string is None
True
>>> print(pig_soup.body.get_text().strip())
The Three Little Pigs
Once upon a time, there were three little pigs named
Larry,
Mo, and
Curly.
The three pigs had an odd fascination with experimental construction.
...
```

Problem 3. Write a function that reads a file of the same format as the file generated from Problem 1 and load it into BeautifulSoup. Find the first `<a>` tag, and return its text along with a boolean value indicating whether or not it has a hyperlink (`href` attribute).

Searching for Tags

Navigating the HTML tree manually can be helpful for gathering data out of lists or tables, but these kinds of structures are usually buried deep in the tree. The `find()` and `find_all()` methods of the `BeautifulSoup` class identify tags that have distinctive characteristics, making it much easier to jump straight to a desired location in the HTML code. The `find()` method only returns the **first** tag that matches a given criteria, while `find_all()` returns a list of all matching tags. Tags can be matched by name, attributes, and/or text.

```
# Find the first <b> tag in the soup.
>>> pig_soup.find(name='b')
<b>The Three Little Pigs</b>

# Find all tags with a class attribute of "pig".
# Since "class" is a Python keyword, use "class_" as the argument.
>>> pig_soup.find_all(class_="pig")
```

```
[<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>,
 <a class="pig" href="http://example.com/mo" id="link2">Mo</a>,
 <a class="pig" href="http://example.com/curly" id="link3">Curly.</a>]

# Find the first tag that matches several attributes.
>>> pig_soup.find(attrs={"class": "pig", "href": "http://example.com/mo"})
<a class="pig" href="http://example.com/mo" id="link2">Mo</a>

# Find the first tag whose text is "Mo".
>>> pig_soup.find(string="Mo")
'Mo'                                    # The result is the actual string,
>>> pig_soup.find(string="Mo").parent   # so go up one level to get the tag.
<a class="pig" href="http://example.com/mo" id="link2">Mo</a>
```

Problem 4. The file `san_diego_weather.html` contains the HTML source for an old page from Weather Underground.[a] Write a function that reads the file and loads it into BeautifulSoup.

Return a list of the following tags:

1. The tag containing the date "Thursday, January 1, 2015".
2. The tags which contain the **links** "Previous Day" and "Next Day".

   Hint: the tags that contain the links do not explicitly have "Previous Day" or "Next Day," so you cannot find the tags based on a string. Instead, open the HTML file manually and look at the attributes of the tags that contain the links. Then inspect them to see if there is a better attribute with which to conduct a search.
3. The tag which contains the number associated with the Max Actual Temperature.

   Hint: Manually inspect the HTML document in both a browser and in a text editor. Look for a good attribute shared by the Actual Temperatures with which you can conduct a search. Note that the Max Actual Temperature tag will not be the first tag found. Make sure you return the correct tag.

[a] See `http://www.wunderground.com/history/airport/KSAN/2015/1/1/DailyHistory.html?req_city=San+Diego&req_state=CA&req_statename=California&reqdb.zip=92101&reqdb.magic=1&reqdb.wmo=99999&MR=1`

Advanced Search Techniques: Regular Expressions

Consider the problem of finding the tag that is a link to the URL `http://example.com/curly`.

```
>>> pig_soup.find(href="http://example.com/curly")
<a class="pig" href="http://example.com/curly" id="link3">Curly.</a>
```

This approach works, but it requires entering in the entire URL. To perform generalized searches, the `find()` and `find_all()` method also accept compiled regular expressions from the `re` module. This way, the methods locate tags whose name, attributes, and/or string matches a pattern.

```
>>> import re

# Find the first tag with an href attribute containing "curly".
>>> pig_soup.find(href=re.compile(r"curly"))
<a class="pig" href="http://example.com/curly" id="link3">Curly.</a

# Find the first tag with a string that starts with "Cu".
>>> pig_soup.find(string=re.compile(r"^Cu")).parent
<a class="pig" href="http://example.com/curly" id="link3">Curly.</a>

# Find all tags with text containing "Three".
>>> [tag.parent for tag in pig_soup.find_all(string=re.compile(r"Three"))]
[<title>Three Little Pigs</title>, <b>The Three Little Pigs</b>]
```

Finally, to find a tag that has a particular attribute, regardless of the actual value of the attribute, use `True` in place of search values.

```
# Find all tags with an "id" attribute.
>>> pig_soup.find_all(id=True)
[<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>,
 <a class="pig" href="http://example.com/mo" id="link2">Mo</a>,
 <a class="pig" href="http://example.com/curly" id="link3">Curly.</a>]

# Find the names all tags WITHOUT an "id" attribute.
>>> [tag.name for tag in pig_soup.find_all(id=False)]
['html', 'head', 'title', 'body', 'p', 'b', 'p', 'p', 'p']
```

It is important to note that using Regular Expressions to locate tags will only find tags whose name, attributes, and/or string match exactly. If searching through an HTML file with a Regular expression to find all the tags with an `"id"` attribute with labels that you specify, it will only return the tags found with those labels, not all instances of the labels. Sometimes that is very useful in simplifying Regular Expression searches.

Advanced Search Techniques: CSS Selectors

`BeautifulSoup` also supports the use of CSS selectors. CSS (Cascading Style Sheet) describes the style and layout of a webpage, and CSS selectors provide a useful way to navigate HTML code. Use the method `soup.select()` to find all elements matching an argument. The general format for an argument is `tag-name[attribute-name = 'attribute value']`. The table below lists symbols you can use to more precisely locate various elements.

Symbol	Meaning
=	Matches an attribute value exactly
*=	Partially matches an attribute value
^=	Matches the beginning of an attribute value
$=	Matches the end of an attribute value
+	Next sibling of matching element
>	Search an element's children

Table 11.4: CSS symbols for use with Selenium

You can do many other useful things with CSS selectors. A helpful guide can be found at `https://www.w3schools.com/cssref/css_selectors.asp`. The code below gives an example using arguments described above.

```
# Find all <a> tags with id="link1"
>>> pig_soup.select("[id='link1']")
[<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>]

# Find all tags with an href attribute containing "curly".
>>> pig_soup.select("[href*='curly']")
[<a class="pig" href="http://example.com/curly" id="link3">Curly.</a>]

# Find all <a> tags with an href attribute
>>> pig_soup.select("a[href]")
[<a class="pig" href="http://example.com/larry" id="link1">Larry,</a>,
 <a class="pig" href="http://example.com/mo" id="link2">Mo</a>,
 <a class="pig" href="http://example.com/curly" id="link3">Curly.</a>]

 # Find all <b> tags within a <p> tag with class="title"
 >>> pig_soup.select("p[class='title'] b")
 [<b>The Three Little Pigs</b>]

 # Use a comma to find elements matching one of two arguments
 >>> pig_soup.select("a[href$='mo'],[id='link3']")
[<a class="pig" href="http://example.com/mo" id="link2">Mo</a>,
 <a class="pig" href="http://example.com/curly" id="link3">Curly.</a>]
```

Problem 5. The file `large_banks_index.html` is an index of data about large banks, as recorded by the Federal Reserve.[a] Write a function that reads the file and loads the source into BeautifulSoup. Return a list of the `<a>` tags containing the links to bank data from September 30, 2003 to December 31, 2014, where the dates are in reverse chronological order.

[a] See `https://www.federalreserve.gov/releases/lbr/`.

Incorporating Pandas with HTML Tables

The Pandas `pd.read_html` method is a neat way to automatically read tables into DataFrames. It works similarly to `BeautifulSoup`, but is streamlined to only scrape all the tables from HTML pages.

```
>>> import pandas as pd
#Read in "file.html"
>>> tables = pd.read_html("file.html")
# Choose the correct table that pd.read_html found as a DataFrame
>>> df = tables[0]
# The DataFrame is now like any other Pandas DataFrame that needs a bit of data↩
    cleaning
>>> df["column_1"]
0       0
1       2
2       .
3       6
4       8
       ..
15     30
16     32
17     .
18     .
19     38
```

Like any other DataFrame table, if there are non-numerical values in a column, the column type will default to strings, so make sure when sorting values by number, set the column data type to numeric with `pd.to_numeric`.

For more help with `pd.read_html` see `https://pandas.pydata.org/docs/reference/api/pandas.read_html.html`.

Problem 6. The file `large_banks_data.html` is one of the pages from the index in Problem 5.[a] Read the specified file and load it into Pandas.

Create a single figure with two subplots:

1. A sorted bar chart of the seven banks with the most domestic branches.
2. A sorted bar chart of the seven banks with the most foreign branches.

[a] See `http://www.federalreserve.gov/releases/lbr/20030930/default.htm`.

12 Pandas 1: Introduction

Lab Objective: *Though NumPy and SciPy are powerful tools for numerical computing, they lack some of the high-level functionality necessary for many data science applications. Python's* pandas *library, built on NumPy, is designed specifically for data management and analysis. In this lab we introduce pandas data structures and syntax, and we explore the capabilities of pandas for quickly analyzing and presenting data.*

Pandas Basics

Pandas is a python library used primarily to analyze data. It combines functionality of NumPy, MatPlotLib, and SQL to create an easy to understand library that allows for the manipulation of data in various ways. In this lab we focus on the use of Pandas to analyze and manipulate data in ways similar to NumPy and SQL.

Pandas Data Structures

Series

The first pandas data structure is a `Series`. A `Series` is a one-dimensional array that can hold any datatype, similar to an `ndarray`. However, a `Series` has an explicitly defined `index` that gives a label to each entry. An `index` generally is used to label the data.

Typically a `Series` contains information about one feature of the data. For example, the data in a `Series` might show a class's grades on a test and the `Index` would indicate each student in the class. To initialize a `Series`, the first parameter is the data and the second is the index.

```
>>> import pandas as pd
>>> import numpy as np
# Initialize Series of student grades
>>> math = pd.Series(np.random.randint(0, 100, 4), ["Mark", "Barbara",
...     "Eleanor", "David"])
>>> english = pd.Series(np.random.randint(0, 100, 5), ["Mark", "Barbara",
...     "David", "Greg", "Lauren"])
```

DataFrame

The second key pandas data structure is a `DataFrame`. A `DataFrame` is a collection of multiple `Series`. It can be thought of as a 2-dimensional array, where each row is a separate datapoint and each column is a feature of the data. The rows are labeled with an `index` (as in a `Series`) and the columns are labeled in the attribute `columns`.

There are many different ways to initialize a `DataFrame`. One way to initialize a `DataFrame` is by passing in a dictionary as the data of the `DataFrame`. The keys of the dictionary will become the labels in `columns` and the values are the `Series` associated with the label.

```
# Create a DataFrame of student grades
>>> grades = pd.DataFrame({"Math": math, "English": english})
>>> grades
          Math  English
Barbara   52.0     73.0
David     10.0     39.0
Eleanor   35.0      NaN
Greg       NaN     26.0
Lauren     NaN     99.0
Mark      81.0     68.0
```

Notice that `pd.DataFrame` automatically lines up data from both `Series` that have the same index. If the data only appears in one of the `Series`, the corresponding entry for the other `Series` is `NaN`.

We can also initialize a `DataFrame` with a NumPy array. With this method, the data is passed in as a 2-dimensional NumPy array, while the column labels and the index are passed in as parameters. The first column label goes with the first column of the array, the second with the second, and so forth. The index works similarly.

```
# Initialize DataFrame with NumPy array. This is identical to the grades ↩
    DataFrame above.
>>> data = np.array([[52.0, 73.0], [10.0, 39.0], [35.0, np.nan],
...     [np.nan, 26.0], [np.nan, 99.0], [81.0, 68.0]])
>>> grades = pd.DataFrame(data, columns = ["Math", "English"], index =
...     ["Barbara", "David", "Eleanor", "Greg", "Lauren", "Mark"])

# View the columns
>>> grades.columns
Index(["Math", "English"], dtype="object")

# View the Index
>>> grades.index
Index(["Barbara", "David", "Eleanor", "Greg", "Lauren", "Mark"], dtype="object"↩
    )
```

A `DataFrame` can also be viewed as a NumPy array using the attribute `values`.

```
# View the DataFrame as a NumPy array
>>> grades.values
array([[ 52.,  73.],
       [ 10.,  39.],
       [ 35.,  nan],
       [ nan,  26.],
       [ nan,  99.],
       [ 81.,  68.]])
```

Data I/O

The pandas library has functions that make importing and exporting data simple. The functions allow for a variety of file formats to be imported and exported, including CSV, Excel, HDF5, SQL, JSON, HTML, and pickle files.

Method	Description
`to_csv()`	Write the index and entries to a CSV file
`read_csv()`	Read a csv and convert into a DataFrame
`to_json()`	Convert the object to a JSON string
`to_pickle()`	Serialize the object and store it in an external file
`to_sql()`	Write the object data to an open SQL database
`read_html()`	Read a table in an html page and convert to a DataFrame

Table 12.1: Methods for exporting data in a pandas `Series` or `DataFrame`.

The CSV (comma separated values) format is a simple way of storing tabular data in plain text. Because CSV files are one of the most popular file formats for exchanging data, we will explore the `read_csv()` function in more detail. Some frequently-used keyword arguments include the following:

- `delimiter`: The character that separates data fields. It is often a comma or a whitespace character.
- `header`: The row number (0 indexed) in the CSV file that contains the column names.
- `index_col`: The column (0 indexed) in the CSV file that is the index for the `DataFrame`.
- `skiprows`: If an integer n, skip the first n rows of the file, and then start reading in the data. If a list of integers, skip the specified rows.
- `names`: If the CSV file does not contain the column names, or you wish to use other column names, specify them in a list.

Another particularly useful function is `read_html()`, which is useful when scraping data. It takes in a url or html file and an optional argument `match`, a string or regex, and returns a list of the tables that match the `match` in a DataFrame.

Data Manipulation

Accessing Data

In general, the best way to access data in a `Series` or `DataFrame` is through the indexers `loc` and `iloc`. While array slicing can be used, it is more efficient to use these indexers. Accessing `Series` and `DataFrame` objects using these indexing operations is more efficient than slicing because the bracket indexing has to check many cases before it can determine how to slice the data structure. Using `loc` or `iloc` explicitly bypasses these extra checks. The `loc` index selects rows and columns based on their labels, while `iloc` selects them based on their integer position. With these indexers, the first and second arguments refer to the rows and columns, respectively, just as array slicing.

```
# Use loc to select the Math scores of David and Greg
>>> grades.loc[["David", "Greg"], "Math"]
David    10.0
Greg      NaN
Name: Math, dtype: float64

# Use iloc to select the Math scores of David and Greg
>>> grades.iloc[[1,3], 0]
David    10.0
Greg      NaN
```

To access an entire column of a `DataFrame`, the most efficient method is to use only square brackets and the name of the column, without the indexer. This syntax can also be used to create a new column or reset the values of an entire column.

```
# Create a new History column with array of random values
>>> grades["History"] = np.random.randint(0, 100, 6)
>>> grades["History"]
Barbara     4
David      92
Eleanor    25
Greg       79
Lauren     82
Mark       27
Name: History, dtype: int64

# Reset the column such that everyone has a 100
>>> grades["History"] = 100.0
>>> grades
         Math  English  History
Barbara  52.0     73.0    100.0
David    10.0     39.0    100.0
Eleanor  35.0      NaN    100.0
Greg      NaN     26.0    100.0
Lauren    NaN     99.0    100.0
Mark     81.0     68.0    100.0
```

Datasets can often be very large and thus difficult to visualize. Pandas has various methods to make this easier. The methods `head` and `tail` will show the first or last n data points, respectively, where n defaults to 5. The method `sample` will draw n random entries of the dataset, where n defaults to 1.

```
# Use head to see the first n rows
>>> grades.head(n=2)
         Math  English  History
Barbara  52.0     73.0    100.0
David    10.0     39.0    100.0

# Use sample to sample a random entry
>>> grades.sample()
        Math  English  History
Lauren   NaN     99.0    100.0
```

It may also be useful to re-order the columns or rows or sort according to a given column.

```
# Re-order columns
>>> grades.reindex(columns=["English", "Math", "History"])
         English  Math  History
Barbara     73.0  52.0    100.0
David       39.0  10.0    100.0
Eleanor      NaN  35.0    100.0
Greg        26.0   NaN    100.0
Lauren      99.0   NaN    100.0
Mark        68.0  81.0    100.0

# Sort descending according to Math grades
>>> grades.sort_values("Math", ascending=False)
         Math  English  History
Mark     81.0     68.0    100.0
Barbara  52.0     73.0    100.0
Eleanor  35.0      NaN    100.0
David    10.0     39.0    100.0
Greg      NaN     26.0    100.0
Lauren    NaN     99.0    100.0
```

Other methods used for manipulating `DataFrame` and `Series` panda structures can be found in Table 12.2.

Method	Description
`append()`	Concatenate two or more `Series`.
`drop()`	Remove the entries with the specified label or labels
`drop_duplicates()`	Remove duplicate values
`dropna()`	Drop null entries
`fillna()`	Replace null entries with a specified value or strategy
`reindex()`	Replace the index
`sample()`	Draw a random entry
`shift()`	Shift the index
`unique()`	Return unique values

Table 12.2: Methods for managing or modifying data in a pandas `Series` or `DataFrame`.

Problem 1. The file `budget.csv` contains the budget of a college student over the course of 4 years. Write a function that performs the following operations in this order:

1. Read in `budget.csv` as a `DataFrame` with the index as column 0. Hint: Use `index_col=0` to set the first column as the index when reading in the csv.
2. Reindex the columns such that amount spent on groceries is the first column and all other columns maintain the same ordering.
3. Sort the `DataFrame` in descending order by how much money was spent on `Groceries`.
4. Reset all values in the `"Rent"` column to `800.0`.
5. Reset all values in the first 5 data points to `0.0`.

Return the values of the updated `DataFrame` as a NumPy array.

Basic Data Manipulation

Because the primary pandas data structures are based off of `ndarray`, most NumPy functions work with pandas structures. For example, basic vector operations work as would be expected:

```
# Sum history and english grades of all students
>>> grades["English"] + grades["History"]
Barbara    173.0
David      139.0
Eleanor      NaN
Greg       126.0
Lauren     199.0
Mark       168.0
dtype: float64

# Double all Math grades
>>>  grades["Math"]*2
Barbara    104.0
David       20.0
```

```
Eleanor     70.0
Greg         NaN
Lauren       NaN
Mark       162.0
Name: Math, dtype: float64
```

In addition to arithmetic, `Series` has a variety of other methods similar to NumPy arrays. A collection of these methods is found in Table 12.3.

Method	Returns
`abs()`	Object with absolute values taken (of numerical data)
`idxmax()`	The index label of the maximum value
`idxmin()`	The index label of the minimum value
`count()`	The number of non-null entries
`cumprod()`	The cumulative product over an axis
`cumsum()`	The cumulative sum over an axis
`max()`	The maximum of the entries
`mean()`	The average of the entries
`median()`	The median of the entries
`min()`	The minimum of the entries
`mode()`	The most common element(s)
`prod()`	The product of the elements
`sum()`	The sum of the elements
`var()`	The variance of the elements

Table 12.3: Numerical methods of the `Series` and `DataFrame` pandas classes.

Basic Statistical Functions

The pandas library allows us to easily calculate basic summary statistics of our data, which can be useful when we want a quick description of the data. The `describe()` function outputs several such summary statistics for each column in a `DataFrame`:

```
# Use describe to better understand the data
>>> grades.describe()
            Math   English  History
count   4.000000   5.00000      6.0
mean   44.500000  61.00000    100.0
std    29.827281  28.92231      0.0
min    10.000000  26.00000    100.0
25%    28.750000  39.00000    100.0
50%    43.500000  68.00000    100.0
75%    59.250000  73.00000    100.0
max    81.000000  99.00000    100.0
```

Functions for calculating means and variances, the covariance and correlation matrices, and other basic statistics are also available.

```
# Find the average grade for each student
```

```
>>> grades.mean(axis=1)
Barbara    75.000000
David      49.666667
Eleanor    67.500000
Greg       63.000000
Lauren     99.500000
Mark       83.000000
dtype: float64

# Give correlation matrix between subjects
>>> grades.corr()
            Math  English  History
Math     1.00000  0.84996      NaN
English  0.84996  1.00000      NaN
History      NaN      NaN      NaN
```

The method `rank()` can be used to rank the values in a data set, either within each entry or with each column. This function defaults ranking in ascending order: the least will be ranked 1 and the greatest will be ranked the highest number.

```
# Rank each student's performance in their classes in descending order
# (best to worst)
# The method keyword specifies what rank to use when ties occur.
>>> grades.rank(axis=1, method="max", ascending=False)
         Math  English  History
Barbara   3.0      2.0      1.0
David     3.0      2.0      1.0
Eleanor   2.0      NaN      1.0
Greg      NaN      2.0      1.0
Lauren    NaN      2.0      1.0
Mark      2.0      3.0      1.0
```

These methods can be very effective in interpreting data. For example, the `rank()` example above shows use that Barbara does best in History, then English, and then Math.

Dealing with Missing Data

Missing data is a ubiquitous problem in data science. Fortunately, pandas is particularly well-suited to handling missing or anomalous data. As we have already seen, the pandas default for a missing value is `NaN`. In basic arithmetic operations, if one of the operands is `NaN`, then the output is also `NaN`. If we are not interested in the missing values, we can simply drop them from the data altogether, or we can fill them with some other value, such as the mean. `NaN` might also mean something specific, such as some default value, which should inform what to do with `NaN` values.

```
# Grades with all NaN values dropped
>>> grades.dropna()
         Math  English  History
Barbara  52.0     73.0    100.0
David    10.0     39.0    100.0
```

```
Mark      81.0     68.0    100.0

# fill missing data with 50.0
>>> grades.fillna(50.0)
          Math  English  History
Barbara   52.0     73.0    100.0
David     10.0     39.0    100.0
Eleanor   35.0     50.0    100.0
Greg      50.0     26.0    100.0
Lauren    50.0     99.0    100.0
Mark      81.0     68.0    100.0
```

When dealing with missing data, make sure you are aware of the behavior of the pandas functions you are using. For example, `sum()` and `mean()` ignore NaN values in the computation.

ACHTUNG!

Always consider missing data carefully when analyzing a dataset. It may not always be helpful to drop the data or fill it in with a random number. Consider filling the data with the mean of surrounding data or the mean of the feature in question. Overall, the choice for how to fill missing data should make sense with the dataset.

Problem 2. Write a function which uses `budget.csv` to answer the questions "Which category affects living expenses the most? Which affects other expenses the most?" Perform the following manipulations:

1. Fill all `NaN` values with `0.0`.
2. Create two new columns, `"Living Expenses"` and `"Other"`. Set the value of `"Living Expenses"` to be the sum of the columns `"Rent"`, `"Groceries"`, `"Gas"` and `"Utilities"`. Set the value of `"Other"` to be the sum of the columns `"Dining Out"`, `"Out With Friends"` and `"Netflix"`.
3. Identify which column, other than `"Living Expenses"`, correlates most with `"Living Expenses"` and which column, other than `"Other"`, correlates most with `"Other"`. This can indicate which columns in the budget affect the overarching categories the most.

Return the names of each of those columns as a tuple. The first should be of the column corresponding to `"Living Expenses"` and the second to `"Other"`.

Complex Operations in Pandas

Often times, the data that we have is not exactly the data we want to analyze. In cases like this we use more complex data manipulation tools to access only the data that we need.

For the examples below, we will use the following data:

```
>>> name = ["Mylan", "Regan", "Justin", "Jess", "Jason", "Remi", "Matt",
...     "Alexander", "JeanMarie"]
>>> sex = ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'M', 'F']
>>> age = [20, 21, 18, 22, 19, 20, 20, 19, 20]
>>> rank = ["Sp", "Se", "Fr", "Se", "Sp", 'J', 'J', 'J', "Se"]
>>> ID = range(9)
>>> aid = ['y', 'n', 'n', 'y', 'n', 'n', 'n', 'y', 'n']
>>> GPA = [3.8, 3.5, 3.0, 3.9, 2.8, 2.9, 3.8, 3.4, 3.7]
>>> mathID = [0, 1, 5, 6, 3]
>>> mathGd = [4.0, 3.0, 3.5, 3.0, 4.0]
>>> major = ['y', 'n', 'y', 'n', 'n']
>>> studentInfo = pd.DataFrame({"ID": ID, "Name": name, "Sex": sex, "Age": age,
...     "Class": rank})
>>> otherInfo = pd.DataFrame({"ID": ID, "GPA": GPA, "Financial_Aid": aid})
>>> mathInfo = pd.DataFrame({"ID": mathID, "Grade": mathGd, "Math_Major":
...     major})
```

Before querying our data, it is helpful to know some of its basic properties, such as number of columns, number of rows, and the datatypes of the columns. This can be done by simply calling the `info()` method on the desired `DataFrame`:

```
>>> mathInfo.info()
<class "pandas.core.frame.DataFrame">
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   ID          5 non-null      int64
 1   Grade       5 non-null      float64
 2   Math_Major  5 non-null      object
dtypes: float64(1), int64(1), object(1)
memory usage: 248.0+ bytes
```

Masks

Sometimes, we only want to access data from a single column. For example if we want to only access the `ID` of the students in the `studentInfo DataFrame`, then we would use the following syntax.

```
# Get the ID column from studentInfo
>>> studentInfo.ID
0    0
1    1
2    2
3    3
4    4
5    5
6    6
7    7
```

```
8    8
Name: ID, dtype: int64
```

If we want to access multiple columns at once we can use a list of column names.

```
# Get the ID and Age columns.
>>> studentInfo[["ID", "Age"]]
   ID  Age
0   0   20
1   1   21
2   2   18
3   3   22
4   4   19
5   5   20
6   6   20
7   7   19
8   8   20
```

Now we can access the specific columns that we want. However, some of these columns may still contain data points that we don't want to consider. In this case we can build a mask. Each mask that we build will return a pandas `Series` object with a `bool` value at each index indicating if the condition is satisfied.

```
# Create a mask for all student receiving financial aid.
>>> mask = otherInfo["Financial_Aid"] == 'y'
# Access other info where the mask is true and display the ID and GPA columns.
>>> otherInfo[mask][["ID", "GPA"]]
   ID  GPA
0   0  3.8
3   3  3.9
7   7  3.4
```

We can also create compound masks with multiple statements. We do this using the same syntax you would use for a compound mask in a normal NumPy array. Useful operators are `&`, the `AND` operator; `|`, the `OR` operator; and $\sim$, the `NOT` operator.

```
# Get all student names where Class = 'J' OR Class = "Sp".
>>>  mask = (studentInfo.Class == 'J') | (studentInfo.Class == "Sp")
>>> studentInfo[mask].Name
0        Mylan
4        Jason
5         Remi
6         Matt
7    Alexander
Name: Name, dtype: object
# This can also be acomplished with the following command:
# studentInfo[studentInfo["Class"].isin(['J', "Sp"])]["Name"]
```

Problem 3. Read in the file `crime_data.csv` as a pandas object. The file contains data on types of crimes in the U.S. from 1960 to 2016. Set the index as the column `"Year"`. Answer the following questions using the pandas methods learned in this lab. The answer of each question should be saved as indicated. Return the answers to all three questions as a tuple (i.e. `(answer_1,answer_2,answer_3)`).

1. Identify the three crimes that have a mean yearly number of occurences over 1,500,000. Of these three crimes, which two are very correlated? Which of these two crimes has a greater maximum value? Save the title of this column as a variable to return as the answer. Hint: Consider using `idxmax()` to extract the title.
2. Examine the data from 2000 and later. Sort this data (in ascending order) according to number of murders. Find the years where aggravated assault is greater than 850,000. Save the indices (the years) of the masked and reordered `DataFrame` as a NumPy array to return as the answer.
3. What year had the highest crime rate? In this year, which crime was committed the most? What percentage of the total crime that year was it? Return this percentage as the answer in decimal form (e.g. 50% would be returned as 0.5). Note that the crime rate is #crimes/population.

Working with Dates and Times

The `datetime` module in the standard library provides a few tools for representing and operating on dates and times. The `datetime.datetime` object represents a *time stamp*: a specific time of day on a certain day. Its constructor accepts a four-digit year, a month (starting at 1 for January), a day, and, optionally, an hour, minute, second, and microsecond. Each of these arguments must be an integer, with the hour ranging from 0 to 23.

```
>>> from datetime import datetime

# Represent November 18th, 1991, at 2:01 PM.
>>> bday = datetime(1991, 11, 18, 14, 1)
>>> print(bday)
1991-11-18 14:01:00

# Find the number of days between 11/18/1991 and 11/9/2017.
>>> dt = datetime(2017, 11, 9) - bday
>>> dt.days
9487
```

The `datetime.datetime` object has a parser method, `strptime()`, that converts a string into a new `datetime.datetime` object. The parser is flexible so the user must specify the format that the dates are in. For example, if the dates are in the format `"Month/Day//Year::Hour"`, specify `format"=%m/%d//%Y::%H"` to parse the string appropriately. See Table 12.4 for formatting options.

Pattern	Description
`%Y`	4-digit year
`%y`	2-digit year
`%m`	1- or 2-digit month
`%d`	1- or 2-digit day
`%H`	Hour (24-hour)
`%I`	Hour (12-hour)
`%M`	2-digit minute
`%S`	2-digit second

Table 12.4: Formats recognized by `datetime.strptime()`

```
>>> print(datetime.strptime("1991-11-18 / 14:01", "%Y-%m-%d / %H:%M"),
...       datetime.strptime("1/22/1996", "%m/%d/%Y"),
...       datetime.strptime("19-8, 1998", "%d-%m, %Y"), sep='\n')
1991-11-18 14:01:00                 # The date formats are now standardized.
1996-01-22 00:00:00                 # If no hour/minute/seconds data is given,
1998-08-19 00:00:00                 # the default is midnight.
```

Converting Dates to an Index

The `TimeStamp` class is the pandas equivalent to a `datetime.datetime` object. A pandas index composed of `TimeStamp` objects is a `DatetimeIndex`, and a `Series` or `DataFrame` with a `DatetimeIndex` is called a *time series*. The function `pd.to_datetime()` converts a collection of dates in a parsable format to a `DatetimeIndex`. The format of the dates is inferred if possible, but it can be specified explicitly with the same syntax as `datetime.strptime()`.

```
>>> import pandas as pd

# Convert some dates (as strings) into a DatetimeIndex.
>>> dates = ["2010-1-1", "2010-2-1", "2012-1-1", "2012-1-2"]
>>> pd.to_datetime(dates)
DatetimeIndex(['2010-01-01', '2010-02-01', '2012-01-01', '2012-01-02'],
              dtype='datetime64[ns]', freq=None)

# Create a time series, specifying the format for the DatetimeIndex.
>>> dates = ["1/1, 2010", "1/2, 2010", "1/1, 2012", "1/2, 2012"]
>>> date_index = pd.to_datetime(dates, format="%m/%d, %Y")
>>> pd.Series([x**2 for x in range(4)], index=date_index)
2010-01-01    0
2010-01-02    1
2012-01-01    4
2012-01-02    9
dtype: int64
```

Problem 4. The file `DJIA.csv` contains daily closing values of the Dow Jones Industrial Average from 2006–2016. Read the data into a `Series` or `DataFrame` with a `DatetimeIndex` as the index. Drop any rows without numerical values, cast the `"VALUE"` column to floats, then return the updated DataFrame.

Hint: You can change the column type the same way you'd change a numpy array type.

Generating Time-based Indices

Some time series datasets come without explicit labels but have instructions for deriving timestamps. For example, a list of bank account balances might have records from the beginning of every month, or heart rate readings could be recorded by an app every 10 minutes. Use `pd.date_range()` to generate a `DatetimeIndex` where the timestamps are equally spaced. The function is analogous to `np.arange()` and has the following parameters:

Parameter	Description
`start`	Starting date
`end`	End date
`periods`	Number of dates to include
`freq`	Amount of time between consecutive dates
`normalize`	Normalizes the start and end times to midnight

Table 12.5: Parameters for `pd.date_range()`.

Exactly three of the parameters `start`, `end`, `periods`, and `freq` must be specified to generate a range of dates. The `freq` parameter accepts a variety of string representations, referred to as *offset aliases*. See Table 12.6 for a sampling of some of the options. For a complete list of the options, see `https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#timeseries-offset-aliases`.

Parameter	Description
`'D'`	calendar daily (default)
`'B'`	business daily (every business day)
`'H'`	hourly
`'T'`	minutely
`'S'`	secondly
`"MS"`	first day of the month (Month Start)
`"BMS"`	first business day of the month (Business Month Start)
`"W-MON"`	every Monday (Week-Monday)
`"WOM-3FRI"`	every 3rd Friday of the month (Week of the Month - 3rd Friday)

Table 12.6: Options for the `freq` parameter to `pd.date_range()`.

```
# Create a DatetimeIndex for 5 consecutive days starting on September 28, 2016.
>>> pd.date_range(start="9/28/2016 16:00", periods=5)
DatetimeIndex(['2016-09-28 16:00:00', '2016-09-29 16:00:00',
```

```
               '2016-09-30 16:00:00', '2016-10-01 16:00:00',
               '2016-10-02 16:00:00'],
              dtype='datetime64[ns]', freq='D')

# Create a DatetimeIndex with the first weekday of every other month in 2016.
>>> pd.date_range(start="1/1/2016", end="1/1/2017", freq="2BMS" )
DatetimeIndex(['2016-01-01', '2016-03-01', '2016-05-02', '2016-07-01',
               '2016-09-01', '2016-11-01'],
              dtype='datetime64[ns]', freq='2BMS')


# Create a DatetimeIndex for 10 minute intervals between 4:00 PM and 4:30 PM on↩
    September 9, 2016.
>>> pd.date_range(start="9/28/2016 16:00",
           end="9/28/2016 16:30", freq="10T")
DatetimeIndex(['2016-09-28 16:00:00', '2016-09-28 16:10:00',
               '2016-09-28 16:20:00', '2016-09-28 16:30:00'],
              dtype='datetime64[ns]', freq='10T')

# Create a DatetimeIndex for 2 hour 30 minute intervals between 4:30 PM and ↩
   2:30 AM on September 29, 2016.
>>> pd.date_range(start="9/28/2016 16:30", periods=5, freq="2h30min")
DatetimeIndex(['2016-09-28 16:30:00', '2016-09-28 19:00:00',
               '2016-09-28 21:30:00', '2016-09-29 00:00:00',
               '2016-09-29 02:30:00'],
              dtype='datetime64[ns]', freq='150T')
```

Problem 5. The file `paychecks.csv` contains values of an hourly employee's last 93 paychecks. Paychecks are given every other Friday, starting on March 14, 2008, and the employee started working on March 13, 2008.

Read in the data, using `pd.date_range()` to generate the `DatetimeIndex`. Set this as the new index of the `DataFrame` and return the `DataFrame`.

Elementary Time Series Analysis

Shifting

`DataFrame` and `Series` objects have a `shift()` method that allows you to move data up or down relative to the index. When dealing with time series data, we can also shift the `DatetimeIndex` relative to a time offset.

```
>>> df = pd.DataFrame(dict(VALUE=np.random.rand(5)),
                      index=pd.date_range("2016-10-7", periods=5, freq='D'))
>>> df
                VALUE
2016-10-07  0.127895
```

```
2016-10-08  0.811226
2016-10-09  0.656711
2016-10-10  0.351431
2016-10-11  0.608767

>>> df.shift(1)
               VALUE
2016-10-07       NaN
2016-10-08  0.127895
2016-10-09  0.811226
2016-10-10  0.656711
2016-10-11  0.351431

>>> df.shift(-2)
               VALUE
2016-10-07  0.656711
2016-10-08  0.351431
2016-10-09  0.608767
2016-10-10       NaN
2016-10-11       NaN

>>> df.shift(14, freq='D')
               VALUE
2016-10-21  0.127895
2016-10-22  0.811226
2016-10-23  0.656711
2016-10-24  0.351431
2016-10-25  0.608767
```

Shifting data makes it easy to gather statistics about changes from one timestamp or period to the next.

```
# Find the changes from one period/timestamp to the next
>>> df - df.shift(1)            # Equivalent to df.diff().
               VALUE
2016-10-07       NaN
2016-10-08  0.683331
2016-10-09 -0.154516
2016-10-10 -0.305279
2016-10-11  0.257336
```

Problem 6. Compute the following information about the DJIA dataset from Problem 4 that has a DateTimeIndex.

- The single day with the largest gain.
- The single day with the largest loss.

Return the DateTimeIndex of the day with the largest gain and the day with the largest loss as a tuple.
(Hint: Call your function from Problem 4 to get the DataFrame already cleaned and with DatetimeIndex).

More information on how to use `datetime` with Pandas is in the additional material section. This includes working with `Periods` and more analysis with time series.

Additional Material

SQL Operations in pandas

`DataFrames` are tabular data structures bearing an obvious resemblance to a typical relational database table. SQL is the standard for working with relational databases; however, pandas can accomplish many of the same tasks as SQL. The SQL-like functionality of pandas is one of its biggest advantages, eliminating the need to switch between programming languages for different tasks. Within pandas, we can handle both the querying *and* data analysis.

For the examples below, we will use the following data:

```
>>> name = ["Mylan", "Regan", "Justin", "Jess", "Jason", "Remi", "Matt",
...     "Alexander", "JeanMarie"]
>>> sex = ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'M', 'F']
>>> age = [20, 21, 18, 22, 19, 20, 20, 19, 20]
>>> rank = ["Sp", "Se", "Fr", "Se", "Sp", 'J', 'J', 'J', "Se"]
>>> ID = range(9)
>>> aid = ['y', 'n', 'n', 'y', 'n', 'n', 'n', 'y', 'n']
>>> GPA = [3.8, 3.5, 3.0, 3.9, 2.8, 2.9, 3.8, 3.4, 3.7]
>>> mathID = [0, 1, 5, 6, 3]
>>> mathGd = [4.0, 3.0, 3.5, 3.0, 4.0]
>>> major = ['y', 'n', 'y', 'n', 'n']
>>> studentInfo = pd.DataFrame({"ID": ID, "Name": name, "Sex": sex, "Age": age,
...     "Class": rank})
>>> otherInfo = pd.DataFrame({"ID": ID, "GPA": GPA, "Financial_Aid": aid})
>>> mathInfo = pd.DataFrame({"ID": mathID, "Grade": mathGd, "Math_Major":
...     major})
```

SQL SELECT statements can be done by column indexing. WHERE statements can be included by adding masks (just like in a NumPy array). The method `isin()` can also provide a useful WHERE statement. This method accepts a list, dictionary, or `Series` containing possible values of the `DataFrame` or `Series`. When called upon, it returns a `Series` of booleans, indicating whether an entry contained a value in the parameter passed into `isin()`.

```
# SELECT ID, Age FROM studentInfo
>>> studentInfo[["ID", "Age"]]
   ID  Age
0   0   20
1   1   21
2   2   18
3   3   22
4   4   19
5   5   20
6   6   20
7   7   19
8   8   20

# SELECT ID, GPA FROM otherInfo WHERE Financial_Aid = 'y'
>>> mask = otherInfo["Financial_Aid"] == 'y'
>>> otherInfo[mask][["ID", "GPA"]]
```

```
   ID  GPA
0   0  3.8
3   3  3.9
7   7  3.4

# SELECT Name FROM studentInfo WHERE Class = 'J' OR Class = "Sp"
>>>  studentInfo[studentInfo["Class"].isin(['J', "Sp"])]["Name"]
0        Mylan
4        Jason
5         Remi
6         Matt
7    Alexander
Name: Name, dtype: object
```

Next, let's look at JOIN statements. In pandas, this is done with the `merge` function. `merge` takes the two `DataFrame` objects to join as parameters, as well as keyword arguments specifying the column on which to join, along with the type (left, right, inner, outer).

```
# SELECT * FROM studentInfo INNER JOIN mathInfo ON studentInfo.ID = mathInfo.ID
>>> pd.merge(studentInfo, mathInfo, on="ID")
   ID   Name Sex  Age Class  Grade Math_Major
0   0  Mylan   M   20    Sp    4.0          y
1   1  Regan   F   21    Se    3.0          n
2   3   Jess   F   22    Se    4.0          n
3   5   Remi   F   20     J    3.5          y
4   6   Matt   M   20     J    3.0          n

# SELECT GPA, Grade FROM otherInfo FULL OUTER JOIN mathInfo ON otherInfo.
# ID = mathInfo.ID
>>> pd.merge(otherInfo, mathInfo, on="ID", how="outer")[["GPA", "Grade"]]
   GPA  Grade
0  3.8    4.0
1  3.5    3.0
2  3.0    NaN
3  3.9    4.0
4  2.8    NaN
5  2.9    3.5
6  3.8    3.0
7  3.4    NaN
8  3.7    NaN
```

More Datetime with Pandas

Periods

A pandas `Timestamp` object represents a precise moment in time on a given day. Some data, however, is recorded over a time interval, and it wouldn't make sense to place an exact timestamp on any of the measurements. For example, a record of the number of steps walked in a day, box

office earnings per week, quarterly earnings, and so on. This kind of data is better represented with the pandas `Period` object and the corresponding `PeriodIndex`.

The `Period` class accepts a `value` and a `freq`. The `value` parameter indicates the label for a given `Period`. This label is tied to the **end** of the defined `Period`. The `freq` indicates the length of the `Period` and in some cases can also indicate the offset of the `Period`. The default value for `freq` is 'M' for months. The `freq` parameter accepts the majority, but not all, of frequencies listed in Table 12.6.

```
# Creates a period for month of Oct, 2016.
>>> p1 = pd.Period("2016-10")
>>> p1.start_time                    # The start and end times of the period
Timestamp('2016-10-01 00:00:00')     # are recorded as Timestamps.
>>> p1.end_time
Timestamp('2016-10-31 23:59:59.999999999')

# Represent the annual period ending in December that includes 10/03/2016.
>>> p2 = pd.Period("2016-10-03", freq="A-DEC")
>>> p2.start_time
Timestamp('2016-01-01 00:00:00')
> p2.end_time
Timestamp('2016-12-31 23:59:59.999999999')

# Get the weekly period ending on a Saturday that includes 10/03/2016.
>>> print(pd.Period("2016-10-03", freq="W-SAT"))
2016-10-02/2016-10-08
```

Like the `pd.date_range()` method, the `pd.period_range()` method is useful for generating a `PeriodIndex` for unindexed data. The syntax is essentially identical to that of `pd.date_range()`. When using `pd.period_range()`, remember that the `freq` parameter marks the end of the period. After creating a `PeriodIndex`, the `freq` parameter can be changed via the `asfreq()` method.

```
# Represent quarters from 2008 to 2010, with Q4 ending in December.
>>> pd.period_range(start="2008", end="2010-12", freq="Q-DEC")
PeriodIndex(['2008Q1', '2008Q2', '2008Q3', '2008Q4', '2009Q1', '2009Q2',
             '2009Q3', '2009Q4', '2010Q1', '2010Q2', '2010Q3', '2010Q4'],
            dtype='period[Q-DEC]')

# Get every three months form March 2010 to the start of 2011.
>>> p = pd.period_range("2010-03", "2011", freq="3M")
>>> p
PeriodIndex(['2010-03', '2010-06', '2010-09', '2010-12'], dtype='period[3M]')

# Change frequency to be quarterly.
>>> q = p.asfreq("Q-DEC")
>>> q
PeriodIndex(['2010Q2', '2010Q3', '2010Q4', '2011Q1'], dtype='period[Q-DEC]')
```

The bounds of a `PeriodIndex` object can be shifted by adding or subtracting an integer. `PeriodIndex` will be shifted by $n \times$ `freq`.

```
# Shift index by 1
>>> q -= 1
>>> q
PeriodIndex(['2010Q1', '2010Q2', '2010Q3', '2010Q4'], dtype='period[Q-DEC]')
```

If for any reason you need to switch from periods to timestamps, pandas provides a very simple method to do so. The `how` parameter can be `start` or `end` and determines if the timestamp is the beginning or the end of the period. Similarly, you can switch from timestamps to periods.

```
# Convert to timestamp (last day of each quarter)
>>> q = q.to_timestamp(how="end")
>>> q
DatetimeIndex(['2010-03-31', '2010-06-30', '2010-09-30', '2010-12-31'],
                dtype='datetime64[ns]', freq='Q-DEC')

>>> q.to_period("Q-DEC")
PeriodIndex(['2010Q1', '2010Q2', '2010Q3', '2010Q4'],
            dtype='int64', freq='Q-DEC')
```

Operations on Time Series

There are certain operations only available to Series and DataFrames that have a `DatetimeIndex`. A sampling of this functionality is described throughout the remainder of this lab.

Slicing

Slicing is much more flexible in pandas for time series. We can slice by year, by month, or even use traditional slicing syntax to select a range of dates.

```
# Select all rows in a given year
>>> df["2010"]
                   0         1
2010-01-01  0.566694  1.093125
2010-02-01 -0.219856  0.852917
2010-03-01  1.511347 -1.324036

# Select all rows in a given month of a given year
>>> df["2012-01"]
                   0         1
2012-01-01  0.212141  0.859555
2012-01-02  1.483123 -0.520873
2012-01-03  1.436843  0.596143

# Select a range of dates using traditional slicing syntax
>>> df["2010-1-2": "2011-12-31"]
                   0         1
2010-02-01 -0.219856  0.852917
```

```
2010-03-01  1.511347 -1.324036
2011-01-01  0.300766  0.934895
```

Resampling

Some datasets do not have datapoints at a fixed frequency. For example, a dataset of website traffic has datapoints that occur at irregular intervals. In situations like these, *resampling* can help provide insight on the data.

The two main forms of resampling are *downsampling*, aggregating data into fewer intervals, and *upsampling*, adding more intervals.

To downsample, use the `resample()` method of the `Series` or `DataFrame`. This method is similar to `groupby()` in that it groups different entries together. Then aggregation produces a new data set. The first parameter to `resample()` is an offset string from Table 12.6: `'D'` for daily, `'H'` for hourly, and so on.

```
>>> import numpy as np

# Get random data for every day from 2000 to 2010.
>>> dates = pd.date_range(start="2000-1-1", end="2009-12-31", freq='D')
>>> df = pd.Series(np.random.random(len(dates)), index=dates)
>>> df
2000-01-01    0.559
2000-01-02    0.874
2000-01-03    0.774
                ...
2009-12-29    0.837
2009-12-30    0.472
2009-12-31    0.211
Freq: D, Length: 3653, dtype: float64

# Group the data by year.
>>> years = df.resample('A')        # 'A' for "annual".
>>> years.agg(len)                  # Number of entries per year.
2000-12-31    366.0
2001-12-31    365.0
2002-12-31    365.0
                ...
2007-12-31    365.0
2008-12-31    366.0
2009-12-31    365.0
Freq: A-DEC, dtype: float64

>>> years.mean()                    # Average entry by year.
2000-12-31    0.491
2001-12-31    0.514
2002-12-31    0.484
                ...
2007-12-31    0.508
```

```
2008-12-31    0.521
2009-12-31    0.523
Freq: A-DEC, dtype: float64

# Group the data by month.
>>> months = df.resample('M')
>>> len(months.mean())              # 12 months x 10 years = 120 months.
120
```

Elementary Time Series Analysis

Rolling Functions and Exponentially-Weighted Moving Functions

Many time series are inherently noisy. To analyze general trends in data, we use *rolling functions* and *exponentally-weighted moving (EWM)* functions. Rolling functions, or *moving window functions*, perform a calculation on a window of data. There are a few rolling functions that come standard with pandas.

Rolling Functions (Moving Window Functions)

One of the most commonly used rolling functions is the *rolling average*, which takes the average value over a window of data.

```
import matplotlib.pyplot as plt

# Generate a time series using random walk from a uniform distribution.
seed = 42
N = 10000
bias = 0.01

rng = np.random.default_rng(seed)
s = np.zeros(N)
s[1:] = rng.uniform(low=-1, high=1, size=N-1) + bias
s = pd.Series(s.cumsum(),
              index=pd.date_range("2015-10-20", freq='H', periods=N))

# Plot the original data together with a rolling average.
ax1 = plt.subplot(121)
s.plot(color="gray", lw=0.3, ax=ax1)
s.rolling(window=200).mean().plot(color='r', lw=1, ax=ax1)
ax1.legend(["Actual", "Rolling"], loc="lower right")
ax1.set_title("Rolling Average")
```

The function call `s.rolling(window=200)` creates a `pd.core.rolling.Window` object that can be aggregated with a function like `mean()`, `std()`, `var()`, `min()`, `max()`, and so on.

Exponentially-Weighted Moving (EWM) Functions

Whereas a moving window function gives equal weight to the whole window, an *exponentially-weighted moving* function gives more weight to the most recent data points.

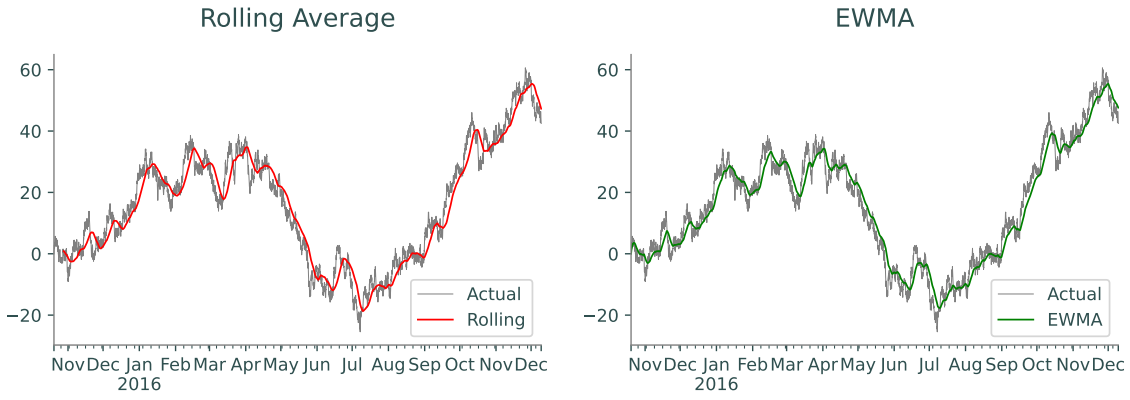
In the case of a *exponentially-weighted moving average* (EWMA), each data point is calculated as follows.

$$z_i = \alpha \bar{x}_i + (1 - \alpha) z_{i-1},$$

where z_i is the value of the EWMA at time i, $\bar{x}_i$ is the average for the i-th window, and α is the decay factor that controls the importance of previous data points. Notice that $\alpha = 1$ reduces to the rolling average.

More commonly, the decay is expressed as a function of the window size. In fact, the `span` for an EWMA is nearly analogous to `window` size for a rolling average.

Notice the syntax for EWM functions is very similar to that of rolling functions.



(a) (b)

Figure 12.1: Rolling average and EWMA.

```
ax2 = plt.subplot(122)
s.plot(color="gray", lw=0.3, ax=ax2)
s.ewm(span=200).mean().plot(color='g', lw=1, ax=ax2)
ax2.legend(["Actual", "EWMA"], loc="lower right")
ax2.set_title("EWMA")
```

13 Pandas 2: Plotting

Lab Objective: *Clear, insightful visualizations are a crucial part of data analysis. To facilitate quick data visualization, pandas includes several tools that wrap around matplotlib. These tools make it easy to compare different parts of a data set, explore the data as a whole, and spot patterns and correlations in the data.*

> **Note**
>
> This lab is designed to be more flexible than other labs. You will be asked to make visualizations customized to your liking. As long as your plots clearly visualize the data, are properly labeled, etc., you will be graded generously.

Overview of Plotting Tools

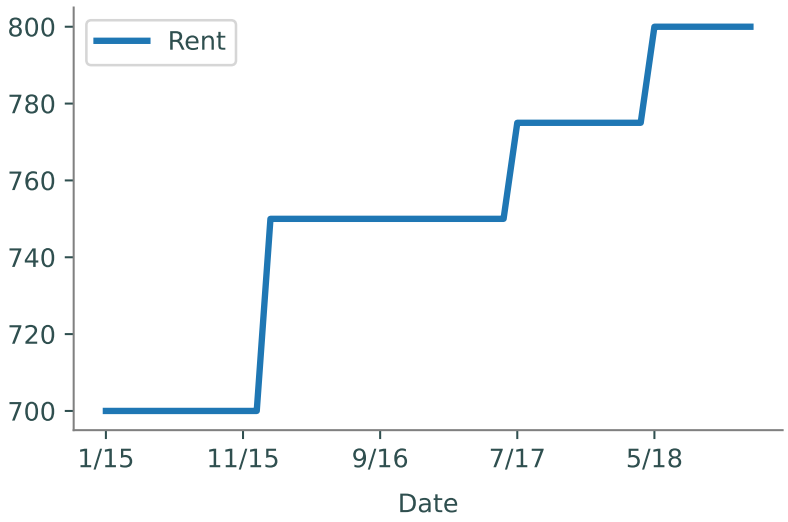
The main tool for visualization in pandas is the `plot()` method for `Series` and `DataFrames`. The method has a keyword argument `kind` that specifies the type of plot to draw. The valid options for `kind` are detailed below.

Plot Type	`plot()` ID	Uses and Advantages
Line plot	`"line"`	Show trends ordered in data; easy to compare multiple data sets
Scatter plot	`"scatter"`	Compare exactly two data sets, independent of ordering
Bar plot	`"bar"`, `"barh"`	Compare categorical or sequential data
Histogram	`"hist"`	Show frequencies of one set of values, independent of ordering
Box plot	`"box"`	Display min, median, max, and quartiles; compare data distributions
Hexbin plot	`"hexbin"`	2D histogram; reveal density of cluttered scatter plots

Table 13.1: Types of plots in pandas. The plot ID is the value of the keyword argument `kind`. That is, `df.plot(kind="scatter")` creates a scatter plot. The default `kind` is `"line"`.

The `plot()` method calls `plt.plot()`, `plt.hist()`, `plt.scatter()`, and other matplotlib plotting functions, but it also assigns axis labels, tick marks, legends, and a few other things based on the index and the data. Most calls to `plot()` specify the `kind` of plot and which `Series` to use as the `x` and `y` axes. By default, the `index` of the `Series` or `DataFrame` is used for the `x` axis.

```
>>> import pandas as pd
>>> from matplotlib import pyplot as plt

>>> budget = pd.read_csv("budget.csv", index_col="Date")
>>> budget.plot(y="Rent")  # Plot rent against the index (date).
```



In this case, the call to the `plot()` method is essentially equivalent to the following code.

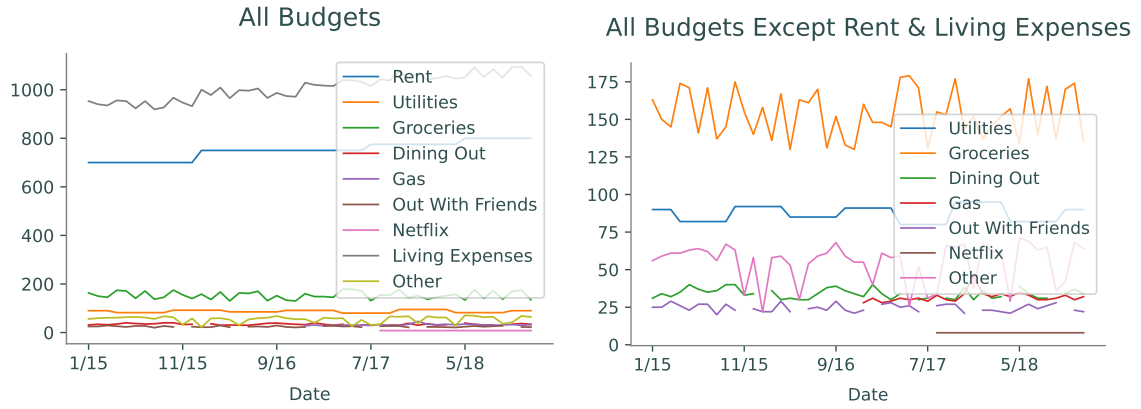
```
>>> plt.plot(budget.index, budget["Rent"], label="Rent")
>>> plt.xlabel(budget.index.name)
>>> plt.xlim(min(budget.index), max(budget.index))
>>> plt.legend(loc="best")
```

The `plot()` method also takes in many keyword arguments for matplotlib plotting and annotation functions. For example, setting `legend=False` disables the legend, providing a value for `title` sets the figure title, `grid=True` turns a grid on, and so on. For more customizations, see `https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.plot.html`.

Visualizing an Entire Data Set

A good way to start analyzing an unfamiliar data set is to visualize as much of the data as possible to determine which parts are most important or interesting. For example, since the columns in a `DataFrame` share the same index, the columns can all be graphed together using the index as the x-axis. By default, the `plot()` method attempts to plot **every** `Series` (column) in a `DataFrame`. This is especially useful with sequential data, like the budget data set.

```
# Plot all columns together against the index.
>>> budget.plot(title="All Budgets", linewidth=1)
>>> budget.drop(["Rent", "Living Expenses"], axis=1).plot(linewidth=1, title="↩
    All Budgets Except Rent & Living Expenses")
```



(a) All columns of the budget data set on the same figure, using the index as the x-axis.

(b) All columns of the budget data set except `"Living Expenses"` and `"Rent"`.

Figure 13.1

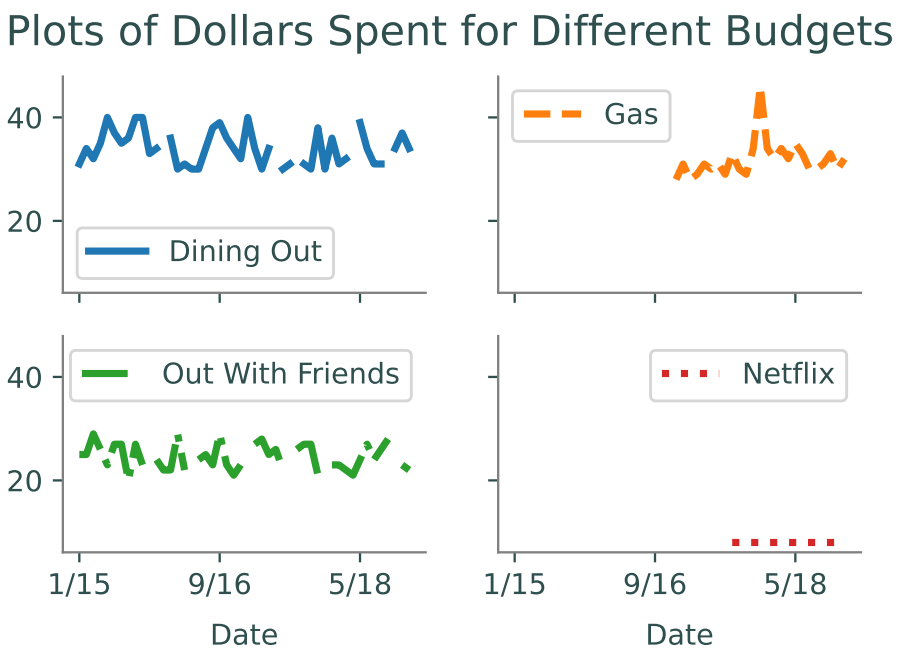
While plotting every `Series` at once can give an overview of all the data, the resulting plot is often difficult for the reader to understand. For example, the budget data set has 9 columns, so the resulting figure, Figure 13.1a, is fairly cluttered.

One way to declutter a visualization is to examine less data. For example, the columns `"Living Expenses"` and `"Rent"` have values that are much larger than the other columns. Dropping these columns gives a better overview of the remaining data, as shown in Figure 13.1b.

> **Achtung!**
>
> Often plotting all data at once is unwise because columns have **different units of measure**. Be careful not to plot parts of a data set together if those parts do not have the same units or are otherwise incomparable.

Another way to declutter a plot is to use subplots. To quickly plot several columns in separate subplots, use `subplots=True` and specify a shape tuple as the `layout` for the plots. Subplots automatically share the same x-axis. Set `sharey=True` to force them to share the same y-axis as well.

```
>>> budget.plot(y=["Dining Out", "Gas", "Out With Friends", "Netflix"],
...     subplots=True, layout=(2, 2), sharey=True,
...     style=['-', "--", "-.", ':'], title="Plots of Dollars Spent for ↩
    Different Budgets")
```



As mentioned previously, the `plot()` method can be used to plot different kinds of plots. One possible kind of plot is a histogram. Since plots made by the `plot()` method share an x-axis by default, histograms turn out poorly whenever there are columns with very different data ranges or when more than one column is plotted at once.

```
# Plot three histograms together.
>>> budget.plot(kind="hist", y=["Gas", "Dining Out", "Out With Friends"],
...     alpha=0.7, bins=10, title="Frequency of Amount (in dollars) Spent")

# Plot three histograms, stacking one on top of the other.
>>> budget.plot(kind="hist", y=["Gas", "Dining Out", "Out With Friends"],
...     bins=10, stacked=True, title="Frequency of Amount (in dollars) Spent")
```

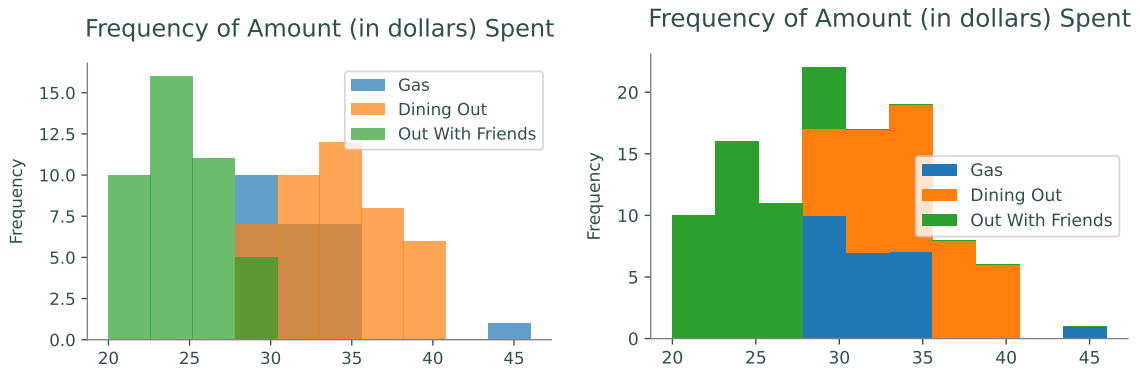


Figure 13.2: Two examples of histograms that are difficult to understand because multiple columns are plotted.

Thus, histograms are good for examining the distribution of a **single** column in a data set. For histograms, use the `hist()` method of the `DataFrame` instead of the `plot()` method. Specify the number of bins with the `bins` parameter. Choose a number of bins that accurately represents the data; the wrong number of bins can create a misleading or uninformative visualization.

```
>>> budget[["Dining Out", "Gas"]].hist(grid=False, bins=10)
```

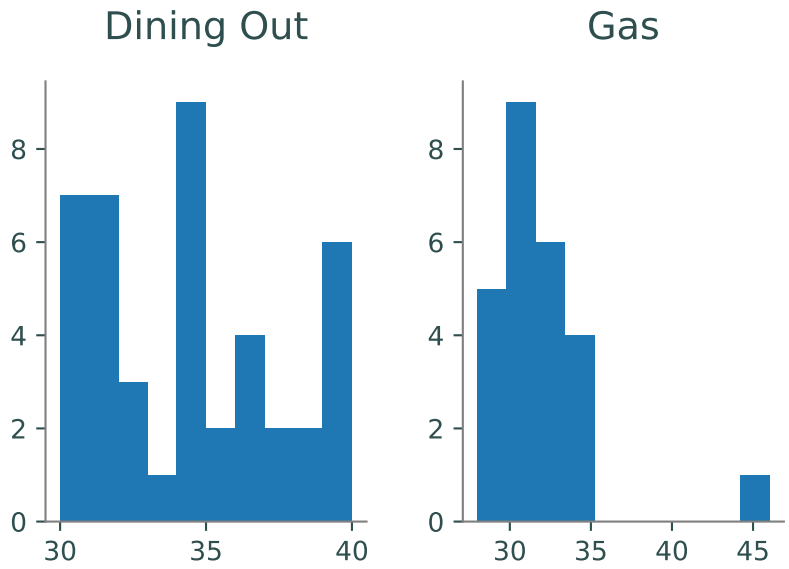


Figure 13.3: Histograms of `"Dining Out"` and `"Gas"`.

Problem 1. Create 3 visualizations for the data in `crime_data.csv`. Make one of the visualizations a histogram. The visualizations should be well labeled and easy to understand.

Patterns and Correlations

After visualizing the entire data set initially, a good next step is to closely compare related parts of the data. This can be done with different types of visualizations. For example, Figure 13.1b suggests that the `"Dining Out"` and `"Out With Friends"` columns are roughly on the same scale. Since this data is sequential (indexed by time), start by plotting these two columns against the index. Next, create a scatter plot of one of the columns versus the other to investigate correlations that are independent of the index. Unlike other types of plots, using `kind="scatter"` requires both `x` and `y` columns as arguments.

```
# Plot "Dining Out" and "Out With Friends" as lines against the index.
>>> budget.plot(y=["Dining Out", "Out With Friends"], title="Amount Spent on ↩
    Dining Out and Out with Friends per Day")

# Make a scatter plot of "Dining Out" against "Out With Friends"
>>> budget.plot(kind="scatter", x="Dining Out", y="Out With Friends",
...     alpha=0.8, xlim=(0, max(budget["Dining Out"]) + 1),
...     ylim=(0, max(budget["Out With Friends"]) + 1),
...     title="Correlation between Dining Out and Out with Friends")
```

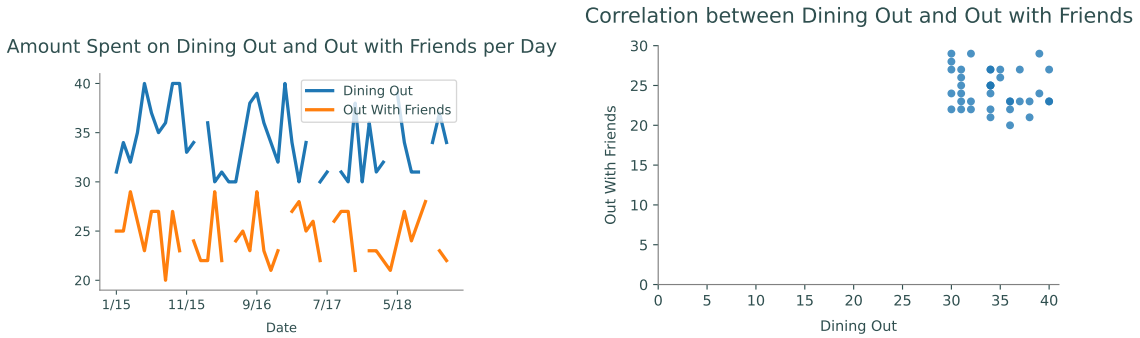


Figure 13.4: Correlations between `"Dining Out"` and `"Out With Friends"`.

The first plot shows us that more money is spent on dining out than being out with friends overall. However, both categories stay in the same range for most of the data. This is confirmed in the scatter plot by the block in the upper right corner, indicating the common range spent on dining out and being out with friends.

Achtung!

When analyzing data, especially while searching for patterns and correlations, **always** ask yourself if the data makes sense and is trustworthy. What lurking variables could have influenced the data measurements as they were being gathered?

The crime data set from Problem 1 is somewhat suspect in this regard. The number of murders is likely accurate across the board, since murder is conspicuous and highly reported. However,

the increase of rape incidents is probably caused both by the general rise in crime as well as an increase in the percentage of rape incidents being reported. Be careful about drawing conclusions for sensitive or questionable data.

Another useful visualization used to understand correlations in a data set is a scatter matrix. The function `pd.plotting.scatter_matrix()` produces a table of plots where each column is plotted against each other column in separate scatter plots. The plots on the diagonal, instead of plotting a column against itself, displays a histogram of that column. This provides a very quick method for an initial analysis of the correlation between different columns.

```
>>> pd.plotting.scatter_matrix(budget[["Living Expenses", "Other"]])
```

Figure 13.5: Scatter matrix comparing `"Living Expenses"` and `"Other"`.

Bar Graphs

Different types of graphs help to identify different patterns. Note that the data set `budget` gives monthly expenses. It may be beneficial to look at one specific month. Bar graphs are a good way to compare small portions of the data set.

As a general rule, horizontal bar charts (`kind="barh"`) are better than the default vertical bar charts (`kind="bar"`) because most humans can detect horizontal differences more easily than vertical differences. If the labels are too long to fit on a normal figure, use `plt.tight_layout()` to adjust the plot boundaries to fit the labels in.

```
# Plot all data for the last month in the budget
>>> budget.iloc[-1, :].plot(kind="barh")
>>> plt.tight_layout()

# Plot all data for the last month without "Rent" and "Living Expenses"
>>> budget.drop(["Rent", "Living Expenses"], axis=1).iloc[-1, :].plot(kind="↩
    barh")
```

```
>>> plt.tight_layout()
```

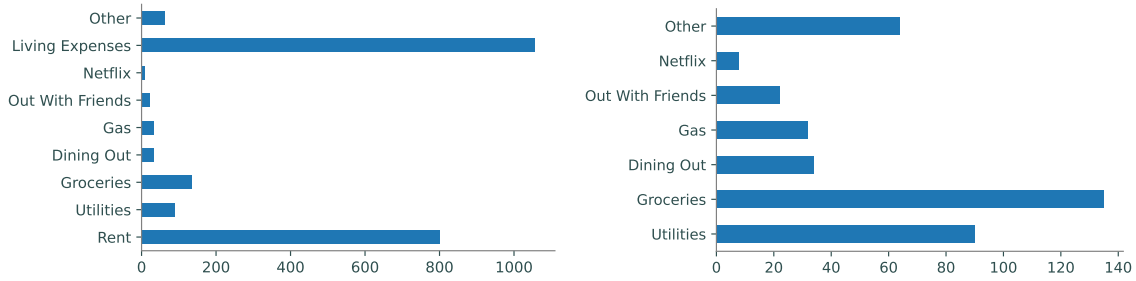


Figure 13.6: Bar graphs showing expenses paid in the last month of `budget`.

Problem 2. Using `crime_data.csv`, plot the trends between `Larceny` and the following variables:

1. `Violent`
2. `Burglary`
3. `Aggravated Assault`

Make sure each graph is clearly labeled and readable. One of these variables does *not* have a linear trend with `Larceny`. Return a string identifying this variable (You're welcome to hard-code this value after observing the data).

Distributional Visualizations

While histograms are good at displaying the distributions for one column, a different visualization is needed to show the distribution of an entire set. A *box plot*, sometimes called a "cat-and-whisker" plot, shows the five number summary: the minimum, first quartile, median, third quartile, and maximum of the data. Box plots are useful for comparing the distributions of relatable data. However, box plots are a basic summary, meaning that they are susceptible to miss important information such as how many points were in each distribution.

```
# Compare the distributions of four columns.
>>> budget.plot(kind="box", y=["Gas", "Dining Out", "Out With Friends", "Other"↩
    ])

# Compare the distributions of all columns but "Rent" and "Living Expenses".
>>> budget.drop(["Rent", "Living Expenses"], axis=1).plot(kind="box",
...     vert=False)
```

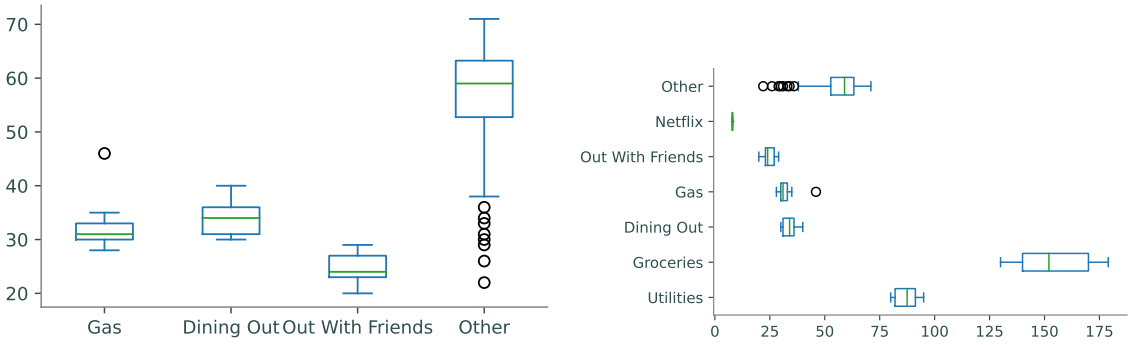


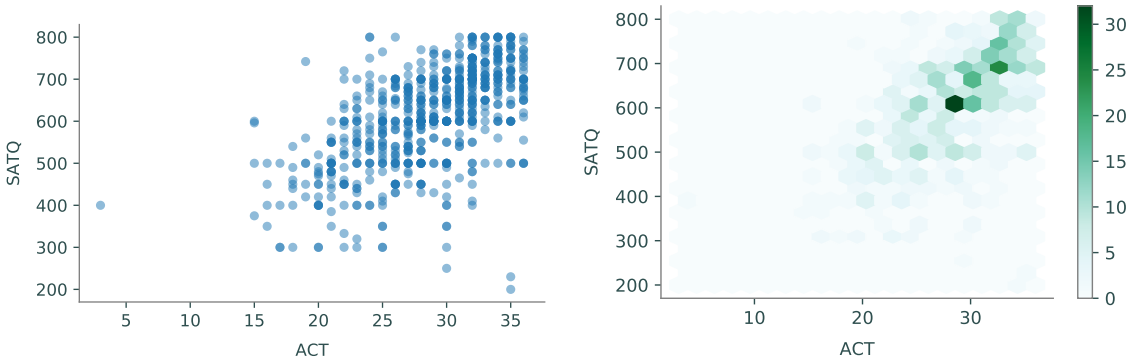
Figure 13.7: Vertical and horizontal box plots of `budget` dataset.

Hexbin Plots

A scatter plot is essentially a plot of samples from the joint distribution of two columns. However, scatter plots can be uninformative for large data sets when the points in a scatter plot are closely clustered. *Hexbin plots* solve this problem by plotting point density in hexagonal bins—essentially creating a 2-dimensional histogram.

The file `sat_act.csv` contains 700 self reported scores on the SAT Verbal, SAT Quantitative and ACT, collected as part of the Synthetic Aperture Personality Assessment (SAPA) web based personality assessment project. The obvious question with this data set is "how correlated are ACT and SAT scores?" The scatter plot of ACT scores versus SAT Quantitative scores, Figure 13.8a, is highly cluttered, even though the points have some transparency. A hexbin plot of the same data, Figure 13.8b, reveals the **frequency** of points in binned regions.

```
>>> satact = pd.read_csv("sat_act.csv", index_col="ID")
>>> list(satact.columns)
['gender', 'education', 'age', 'ACT', 'SATV', 'SATQ']

# Plot the ACT scores against the SAT Quant scores in a regular scatter plot.
>>> satact.plot(kind="scatter", x="ACT", y="SATQ", alpha=.8)

# Plot the densities of the ACT vs. SATQ scores with a hexbin plot.
>>> satact.plot(kind="hexbin", x="ACT", y="SATQ", gridsize=20)
```



(a) ACT vs. SAT Quant scores.

(b) Frequency of ACT vs. SAT Quant scores.

Figure 13.8: Scatter plots and hexbin plot of SAT and ACT scores.

Just as choosing a good number of `bins` is important for a good histogram, choosing a good `gridsize` is crucial for an informative hexbin plot. A large `gridsize` creates many small bins and a small `gridsize` creates fewer, larger bins.

> **Note**
>
> Since hexbins are based on frequencies, they are prone to being misleading if the dataset is not understood well. For example, when plotting information that deals with geographic position, increases in frequency may be results in higher populations rather than the actual information being plotted.

See `http://pandas.pydata.org/pandas-docs/stable/visualization.html` for more types of plots available in Pandas and further examples.

Problem 3. Use `crime_data.csv` to display the following distributions.

1. The distributions of `Burglary`, `Violent`, and `Vehicle Theft` as box plots,
2. The distribution of `Vehicle Thefts` against `Robbery` as a hexbin plot.

As usual, all plots should be labeled and easy to read.
Hint: To get the x-axis label to display, you might need to set the `sharex` parameter of `plot()` to `False`.

Principles of Good Data Visualization

Data visualization is a powerful tool for analysis and communication. When writing a paper or report, the author must make many decisions about how to use graphics effectively to convey useful information to the reader. Here we will go over a simple process for making deliberate, effective, and efficient design decisions.

Attention to Detail

Consider the plot in Figure 13.9. It is a scatter plot of positively correlated data of some kind, with `temp`–likely temperature–on the x axis and `cons` on the y axis. However, the picture is not really communicating anything about the dataset. It has not specified the units for the x or the y axis, nor does it tell what `cons` is. There is no title, and the source of the data is unknown.

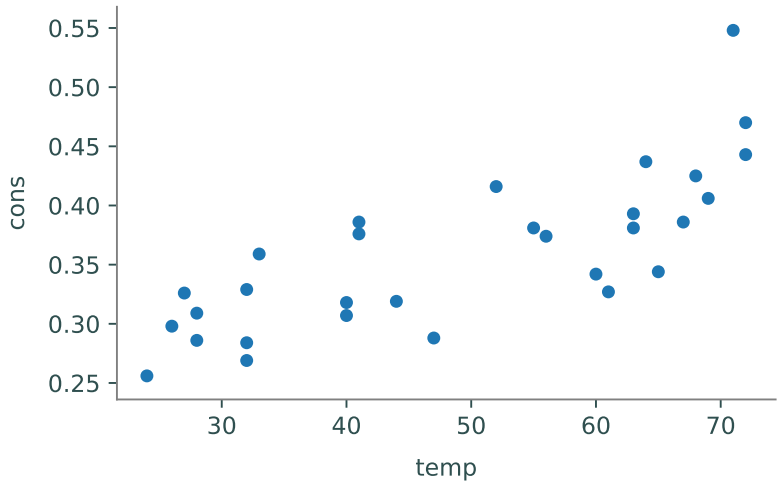


Figure 13.9: Non-specific data.

Labels and Citations

In a homework or lab setting, we sometimes (mistakenly) think that it is acceptable to leave off appropriate labels, legends, titles, and sourcing. In a published report or presentation, this kind of carelessness is confusing at best and, when the source is not included, even plagiaristic. Data needs to be explained in a useful manner that includes all of the vital information.

Consider again Figure 13.9. This figure comes from the `Icecream` dataset within the `pydataset` package, which we store here in a dataframe and then plot:

```
>>> from pydataset import data
>>> icecream = data("Icecream")
>>> icecream.plot(kind="scatter", x="temp", y="cons")
```

This code produces the rather substandard plot in Figure 13.9. Examining the source of the dataset can give important details to create better plots. When plotting data, make sure to understand what the variable names represent and where the data was taken from. Use this information to create a more effective plot.

The ice cream data used in Figure 13.9 is better understood with the following information:

1. The dataset details ice cream consumption via 30 four-week periods from March 1951 to July 1953 in the United States.
2. `cons` corresponds to "consumption of ice cream per capita" and is measured in pints.
3. `income` is the family's weekly income in dollars.

4. `price` is the price of a pint of ice cream.

5. `temp` corresponds to temperature, degrees Fahrenheit.

6. The listed source is: "Hildreth, C. and J. Lu (1960) *Demand relations with autocorrelated disturbances*, Technical Bulletin No 2765, Michigan State University."

This information gives important details that can be used in the following code. As seen in previous examples, pandas automatically generates legends when appropriate. Pandas also automatically labels the x and y axes, however our data frame column titles may be insufficient. Appropriate titles for the x and y axes must also list appropriate units. For example, the y axis should specify that the consumption is in units of *pints per capita*, in place of the ambiguous label `cons`.

```
>>> icecream = data("Icecream")
# Set title via the title keyword argument
>>> icecream.plot(kind="scatter", x="temp", y="cons",
...     title="Ice Cream Consumption in the U.S., 1951-1953")
# Override pandas automatic labeling using xlabel and ylabel
>>> plt.xlabel("Temp (Fahrenheit)")
>>> plt.ylabel("Consumption per capita (pints)")
```

To add the necessary text to the figure, use either `plt.annotate()` or `plt.text()`. Alternatively, add text immediately below wherever the figure is displayed. The first two parameters of `plt.text` are the x and y coordinates to place the text. The third parameter is the text to write. For instance, using `plt.text(0.5, 0.5, "Hello World")` will center the `Hello World` string in the axes.

```
>>> plt.text(20, .1, r"Source: Hildreth, C. and J. Lu (1960) \emph{Demand"
...     "relations with autocorrelated disturbances}\nTechnical Bulletin No"
...     "2765, Michigan State University.", fontsize=7)
```

Both of these methods are imperfect but can normally be easily replaced by a caption attached to the figure. Again, we reiterate how important it is that you source any data you use; failing to do so is plagiarism.

Finally, we have a clear and demonstrative graphic in Figure 13.10.

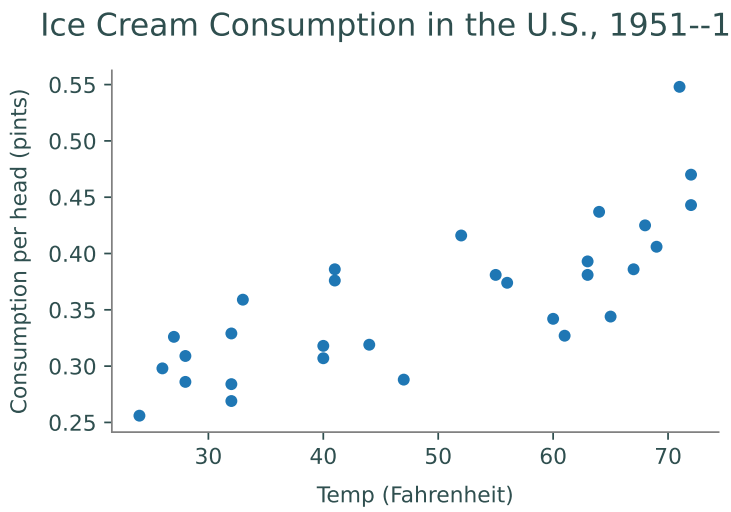


Figure 13.10: Source: Hildreth, C. and J. Lu (1960) *Demand relations with autocorrelated disturbances*, Technical Bulletin No 2765, Michigan State University.

Achtung!

Visualizing data can inherit many biases of the visualizer and as a result can be intentionally misleading. Examples of this include, but are not limited to, visualizing subsets of data that do not represent the whole of the data and having purposely misconstrued axes. Every data visualizer has the responsibility to avoid including biases in their visualizations to ensure data is being represented informatively and accurately.

Problem 4. The dataset `college.csv` contains information from 1995 on universities in the United States. To access information on variable names, go to `https://cran.r-project.org/web/packages/ISLR/ISLR.pdf`. Create 3 plots that compare variables or universities. These plots should answer questions about the data (e.g. What is the distribution of graduation rates? Do schools with lower student to faculty ratios have higher tuition costs? etc.). These three plots should be easy to understand and have clear titles and variable names. In addition, the dataset should be cited in your first plot as follows:

James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013) An Introduction to Statistical Learning with applications in R, www.StatLearning.com, Springer-Verlag, New York

This problem is intended to give you flexibility to create your own visualizations that answer your own questions. As long as your plots are clear and include all the information listed above, you will be graded generously!

14 Pandas 3: Grouping

Lab Objective: *Many data sets contain categorical values that naturally sort the data into groups. Analyzing and comparing such groups is an important part of data analysis. In this lab we explore pandas tools for grouping data and presenting tabular data more compactly, primarily through groupby and pivot tables.*

Groupby

The file `mammal_sleep.csv`[1] contains data on the sleep cycles of different mammals, classified by order, genus, species, and diet (carnivore, herbivore, omnivore, or insectivore). The `"sleep_total"` column gives the total number of hours that each animal sleeps (on average) every 24 hours. To get an idea of how many animals sleep for how long, we start off with a histogram of the `"sleep_total"` column.

```
>>> import pandas as pd
>>> from matplotlib import pyplot as plt

# Read in the data and print a few random entries.
>>> msleep = pd.read_csv("mammal_sleep.csv")
>>> msleep.sample(5)
      name     genus   vore         order  sleep_total  sleep_rem  sleep_cycle
51  Jaguar  Panthera  carni     Carnivora         10.4        NaN          NaN
77  Tenrec    Tenrec   omni  Afrosoricida         15.6        2.3          NaN
10    Goat     Capri  herbi  Artiodactyla          5.3        0.6          NaN
80   Genet   Genetta  carni     Carnivora          6.3        1.3          NaN
33   Human      Homo   omni      Primates          8.0        1.9          1.5

# Plot the distribution of the sleep_total variable.
>>> msleep.plot(kind="hist", y="sleep_total", title="Mammalian Sleep Data")
>>> plt.xlabel("Hours")
```

[1] Proceedings of the National Academy of Sciences, 104 (3):1051–1056, 2007. Updates from V. M. Savage and G. B. West, with additional variables supplemented by Wikipedia. Available in `pydataset` (with a few more columns) under the key `"msleep"`.

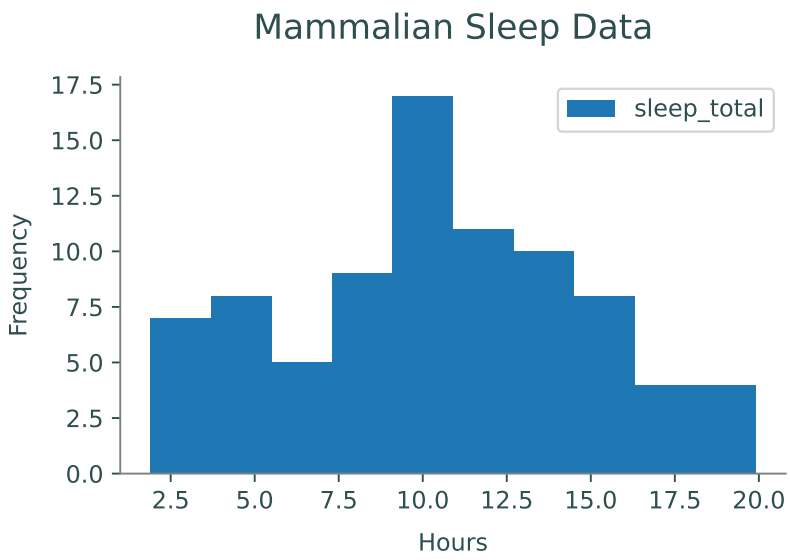


Figure 14.1: `"sleep_total"` frequencies from the mammalian sleep data set.

While this visualization is a good start, it doesn't provide any information about how different kinds of animals have different sleeping habits. How long do carnivores sleep compared to herbivores? Do mammals of the same genus have similar sleep patterns?

A powerful tool for answering these kinds of questions is the `groupby()` method of the pandas `DataFrame` class, which partitions the original `DataFrame` into groups based on the values in one or more columns. The `groupby()` method does **not** return a new `DataFrame`; it returns a pandas `GroupBy` object, an interface for analyzing the original `DataFrame` by groups.

For example, the columns `"genus"`, `"vore"`, and `"order"` in the mammal sleep data all have a discrete number of categorical values that could be used to group the data. Since the `"vore"` column has only a few unique values, we start by grouping the animals by diet.

```
# List all of the unique values in the "vore" column.
>>> set(msleep["vore"])
{nan, 'herbi', 'omni', 'carni', 'insecti'}

# Group the data by the "vore" column.
>>> vores = msleep.groupby("vore")
>>> list(vores.groups)
['carni', 'herbi', 'insecti', 'omni']       # NaN values for vore were dropped.

# Get a single group and sample a few rows. Note vore="carni" in each entry.
>>> vores.get_group("carni").sample(5)
       name     genus   vore      order  sleep_total  sleep_rem  sleep_cycle
80    Genet   Genetta  carni  Carnivora          6.3        1.3          NaN
50    Tiger  Panthera  carni  Carnivora         15.8        NaN          NaN
8       Dog     Canis  carni  Carnivora         10.1        2.9        0.333
0   Cheetah  Acinonyx  carni  Carnivora         12.1        NaN          NaN
82  Red fox    Vulpes  carni  Carnivora          9.8        2.4        0.350
```

As shown above, `groupby()` is useful for filtering a `DataFrame` by column values; the command `df.groupby(col).get_group(value)` returns the rows of `df` where the entry of the `col` column is `value`. The real advantage of `groupby()`, however, is how easily it compares groups of data. Standard `DataFrame` methods like `describe()`, `mean()`, `std()`, `min()`, and `max()` all work on `GroupBy` objects to produce a new data frame that describes the statistics of each group.

```
# Get averages of the numerical columns for each group.
>>> vores.mean(numeric_only=True)
         sleep_total  sleep_rem  sleep_cycle
vore
carni         10.379      2.290        0.373
herbi          9.509      1.367        0.418
insecti       14.940      3.525        0.161
omni          10.925      1.956        0.592

# Get more detailed statistics for "sleep_total" by group.
>>> vores["sleep_total"].describe()
         count    mean    std  min   25%   50%     75%   max
vore
carni     19.0  10.379  4.669  2.7  6.25  10.4  13.000  19.4
herbi     32.0   9.509  4.879  1.9  4.30  10.3  14.225  16.6
insecti    5.0  14.940  5.921  8.4  8.60  18.1  19.700  19.9
omni      20.0  10.925  2.949  8.0  9.10   9.9  10.925  18.0
```

Multiple columns can be used simultaneously for grouping. In this case, the `get_group()` method of the `GroupBy` object requires a tuple specifying the values for each of the grouping columns.

```
>>> msleep_small = msleep.drop(["sleep_rem", "sleep_cycle"], axis=1)
>>> vores_orders = msleep_small.groupby(["vore", "order"])
>>> vores_orders.get_group(("carni", "Cetacea"))
                    name          genus   vore    order  sleep_total
30           Pilot whale  Globicephalus  carni  Cetacea          2.7
59       Common porpoise       Phocoena  carni  Cetacea          5.6
79  Bottle-nosed dolphin       Tursiops  carni  Cetacea          5.2
```

Problem 1. Read in the data `college.csv` containing information on various United States universities in 1995. To access information on variable names, go to `https://cran.r-project.org/web/packages/ISLR/ISLR.pdf`. Use a `groupby` object to group the colleges by private and public universities. Read in the data as a `DataFrame` object and use `groupby` and `describe` to examine the following columns by group:

1. Student to faculty ratio
2. Percent of students from the top 10% of their high school class
3. Percent of students from the top 25% of their high school class

Determine whether private or public universities have a higher mean for each of these columns.

For the type of university with the higher mean, save the values of the describe function on said column as an array using `.values`. Return a tuple with these arrays in the order described above.

For example, if we were comparing whether the average number of professors with PhDs was higher at private or public universities, we would find that public universities have a higher average, and we would return the following array:

```
array([212., 76.83490566, 12.31752531, 33., 71., 78.5 , 86., 103.])
```

Visualizing Groups

There are a few ways that `groupby()` can simplify the process of visualizing groups of data. First of all, `groupby()` makes it easy to visualize one group at a time using the `plot` method. The following visualization improves on Figure 14.1 by grouping mammals by their diets.

```
# Plot histograms of "sleep_total" for two separate groups.
>>> vores.get_group("carni").plot(kind="hist", y="sleep_total", legend="False",
                                  title="Carnivore Sleep Data")
>>> plt.xlabel("Hours")
>>> vores.get_group("herbi").plot(kind="hist", y="sleep_total", legend="False",
                                  title="Herbivore Sleep Data")
>>> plt.xlabel("Hours")
```

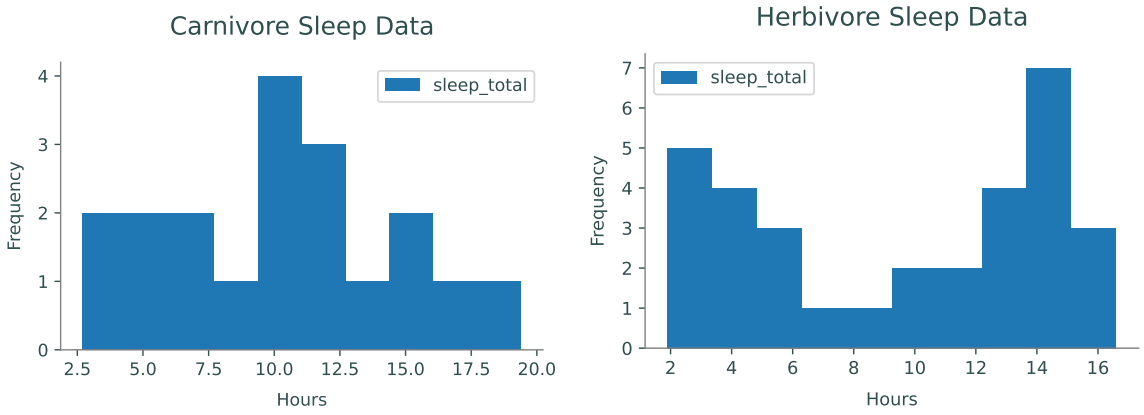
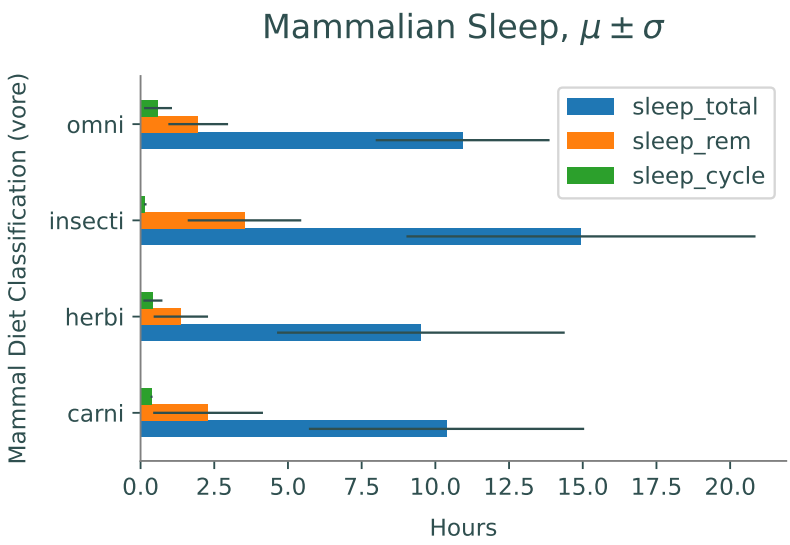


Figure 14.2: `"sleep_total"` histograms for two groups in the mammalian sleep data set.

The statistical summaries from the `GroupBy` object's `mean()`, `std()`, or `describe()` methods also lend themselves well to certain visualizations for comparing groups.

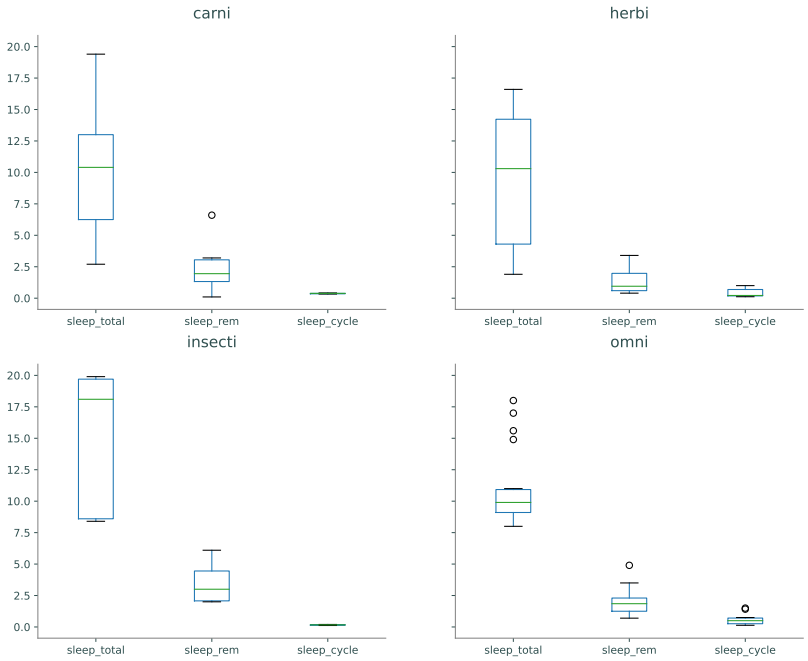
```
>>> cols = ["sleep_total", "sleep_rem", "sleep_cycle"]
>>> vores[cols].mean().plot(kind="barh", xerr=vores[cols].std(),
... title="Mammalian Sleep, $\mu\pm\sigma$")
>>> plt.xlabel("Hours")
```

```
>>> plt.ylabel("Mammal Diet Classification (vore)")
```



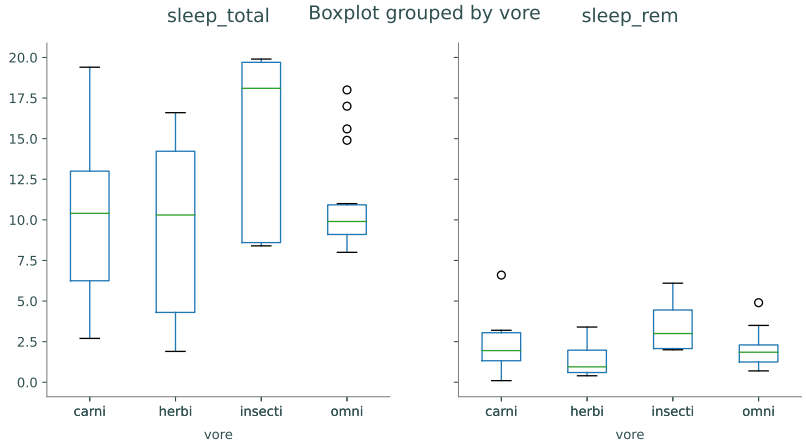
Box plots are well suited for comparing similar distributions. The `boxplot()` method of the `GroupBy` class creates one subplot **per group**, plotting each of the columns as a box plot.

```
# Use GroupBy.boxplot() to generate one box plot per group.
>>> vores.boxplot(grid=False)
>>> plt.tight_layout()
```



Alternatively, the `boxplot()` method of the `DataFrame` class creates one subplot **per column**, plotting each of the columns as a box plot. Specify the `by` keyword to group the data appropriately.

```
# Use DataFrame.boxplot() to generate one box plot per column.
>>> msleep.boxplot(["sleep_total", "sleep_rem"], by="vore", grid=False)
```



Like `groupby()`, the `by` argument can be a single column label or a list of column labels. Similar methods exist for creating histograms (`GroupBy.hist()` and `DataFrame.hist()` with `by` keyword), but generally box plots are better for comparing multiple distributions.

Problem 2. Create visualizations that give relevant information answering the following questions (using `college.csv`):

1. How do the number of applicants, number of accepted students, and number of enrolled students compare between private and public universities?
2. How does the range of money spent on room and board compare between private and public universities?

Pivot Tables

One of the downfalls of `groupby()` is that a typical `GroupBy` object has too much information to display coherently. A *pivot table* intelligently summarizes the results of a `groupby()` operation by aggregating the data in a specified way. The standard tool for making a pivot table is the `pivot_table()` method of the `DataFrame` class. As an example, consider the `"HairEyeColor"` data set from `pydataset`.

```
>>> from pydataset import data
>>> hec = data("HairEyeColor")              # Load and preview the data.
>>> hec.sample(5)
     Hair    Eye     Sex  Freq
3     Red  Brown    Male    10
```

```
1   Black  Brown    Male    32
14  Brown  Green    Male    15
31    Red  Green  Female     7
21  Black   Blue  Female     9

>>> for col in ["Hair", "Eye", "Sex"]:      # Get unique values per column.
...     print(f"{col}: {", ".join(set(str(x) for x in hec[col])})")
...
Hair: Brown, Black, Blond, Red
Eye: Brown, Blue, Hazel, Green
Sex: Male, Female
```

There are several ways to group this data with `groupby()`. However, since there is only one entry per unique hair-eye-sex combination, the data can be completely presented in a pivot table.

```
>>> hec.pivot_table(values="Freq", index=["Hair", "Eye"], columns="Sex")
Sex          Female  Male
Hair  Eye
Black Blue        9    11
      Brown      36    32
      Green       2     3
      Hazel       5    10
Blond Blue       64    30
      Brown       4     3
      Green       8     8
      Hazel       5     5
Brown Blue       34    50
      Brown      66    53
      Green      14    15
      Hazel      29    25
Red   Blue        7    10
      Brown      16    10
      Green       7     7
      Hazel       7     7
```

Listing the data in this way makes it easy to locate data and compare the female and male groups. For example, it is easy to see that brown hair is more common than red hair and that about twice as many females have blond hair and blue eyes as males.

Unlike `"HairEyeColor"`, many data sets have more than one entry in the data for each grouping. An example in the previous dataset would be if there were two or more rows in the original data for females with blond hair and blue eyes. To construct a pivot table, data of similar groups must be *aggregated* together in some way.

By default entries are aggregated by averaging the non-null values. You can use the keyword argument `aggfunc` to choose among different ways to aggregate the data. For example, if you use `aggfunc="min"`, the value displayed will be the minimum of all the values. Other arguments include `"max"`, `"std"` for standard deviation, `"sum"`, or `"count"` to count the number of occurrences. You also may pass in any function that reduces to a single float, like `np.argmax` or even `np.linalg.norm` if you wish. A list of functions can also be passed into the `aggfunc` keyword argument.

Consider the Titanic data set found in `titanic.csv`[2]. For this analysis, take only the `"Survived"`, `"Pclass"`, `"Sex"`, `"Age"`, `"Fare"`, and `"Embarked"` columns, replace null age values with the average age, then drop any rows that are missing data. To begin, we examine the average survival rate grouped by sex and passenger class.

```
>>> titanic = pd.read_csv("titanic.csv")
>>> titanic = titanic[["Survived", "Pclass", "Sex", "Age", "Fare", "Embarked"]]
>>> titanic["Age"] = titanic["Age"].fillna(titanic["Age"].mean(),)

>>> titanic.pivot_table(values="Survived", index="Sex", columns="Pclass")
Pclass    1.0    2.0    3.0
Sex
female  0.965  0.887  0.491
male    0.341  0.146  0.152
```

Note

The `pivot_table()` method is a convenient way of performing a potentially complicated `groupby()` operation with aggregation and some reshaping. The following code is equivalent to the previous example.

```
>>> titanic.groupby(["Sex", "Pclass"])["Survived"].mean().unstack()
Pclass    1.0    2.0    3.0
Sex
female  0.965  0.887  0.491
male    0.341  0.146  0.152
```

The `stack()`, `unstack()`, and `pivot()` methods provide more advanced shaping options.

Among other things, this pivot table clearly shows how much more likely females were to survive than males. To see how many entries fall into each category, or how many survived in each category, aggregate by counting or summing instead of taking the mean.

```
# See how many entries are in each category.
>>> titanic.pivot_table(values="Survived", index="Sex", columns="Pclass",
...                     aggfunc="count")
Pclass  1.0  2.0  3.0
Sex
female  144  106  216
male    179  171  493

# See how many people from each category survived.
>>> titanic.pivot_table(values="Survived", index="Sex", columns="Pclass",
...                     aggfunc="sum")
Pclass    1.0   2.0    3.0
```

[2]There is a `"Titanic"` data set in `pydataset`, but it does not contain as much information as the data in `titanic.csv`.

```
Sex
female  139.0  94.0  106.0
male     61.0  25.0   75.0
```

Problem 3. The file `Ohio_1999.csv` contains data on workers in Ohio in the year 1999. Use pivot tables to answer the following questions:

1. Which race/sex combination has the highest `Usual Weekly Earnings` in total?
2. Which race/sex combination has the lowest cumulative `Usual Hours Worked`?
3. What race/sex combination has the highest average `Usual Hours Worked`?

Return a tuple for each question (in order of the questions) where the first entry is the numerical code corresponding to the race and the second entry is corresponding to the sex. You may hard code each tuple as long as you also provide the working code that gave you the solutions.

Some useful keys in understanding the data are as follows:

1. In column `Sex`, {1: `male`, 2: `female`}.
2. In column `Race`, {1: `White`, 2: `African-American`, 3: `Native American/Eskimo`, 4: `Asian`}.

Discretizing Continuous Data

In the Titanic data, we examined survival rates based on sex and passenger class. Another factor that could have played into survival is age. Were male children as likely to die as females in general? We can investigate this question by *multi-indexing*, or pivoting, on more than just two variables, by adding in another index.

In the original dataset, the `"Age"` column has a floating point value for the age of each passenger. If we add `"Age"` as another pivot, then the table would create a new row for **each** age present. Instead, we partition the `"Age"` column into intervals with `pd.cut()`, thus creating a categorical that can be used for grouping. Notice that when creating the pivot table, the index uses the categorical `age` instead of the column name `"Age"`.

```
# pd.cut() maps continuous entries to discrete intervals.
>>> pd.cut([1, 2, 3, 4, 5, 6, 7], [0, 4, 8])
[(0, 4], (0, 4], (0, 4], (0, 4], (4, 8], (4, 8], (4, 8]]
Categories (2, interval[int64, right]): [(0, 4] < (4, 8]]

# Partition the passengers into 3 categories based on age.
>>> age = pd.cut(titanic["Age"], [0, 12, 18, 80])

>>> titanic.pivot_table(values="Survived", index=["Sex", age],
                        columns="Pclass", aggfunc="mean")
Pclass             1.0    2.0    3.0
Sex    Age
```

```
female (0, 12]   0.000  1.000  0.467
       (12, 18]  1.000  0.875  0.607
       (18, 80]  0.969  0.871  0.475
male   (0, 12]   1.000  1.000  0.343
       (12, 18]  0.500  0.000  0.081
       (18, 80]  0.322  0.093  0.143
```

From this table, it appears that male children (ages 0 to 12) in the 1st and 2nd class were very likely to survive, whereas those in 3rd class were much less likely to. This clarifies the claim that males were less likely to survive than females. However, there are a few oddities in this table: zero percent of the female children in 1st class survived, and zero percent of teenage males in second class survived. To further investigate, count the number of entries in each group.

```
>>> titanic.pivot_table(values="Survived", index=["Sex", age],
                        columns="Pclass", aggfunc="count")
Pclass           1.0  2.0  3.0
Sex    Age
female (0, 12]     1   13   30
       (12, 18]   12    8   28
       (18, 80]  131   85  158
male   (0, 12]     4   11   35
       (12, 18]    4   10   37
       (18, 80]  171  150  421
```

This table shows that there was only 1 female child in first class and only 10 male teenagers in second class, which sheds light on the previous table.

Achtung!

The previous pivot table brings up an important point about partitioning datasets. The Titanic dataset includes data for about 1300 passengers, which is a somewhat reasonable sample size, but half of the groupings include less than 30 entries, which is **not** a healthy sample size for statistical analysis. Always carefully question the numbers from pivot tables before making any conclusions.

Pandas also supports multi-indexing on the columns. As an example, consider the price of a passenger tickets. This is another continuous feature that can be discretized with `pd.cut()`. Instead, we use `pd.qcut()` to split the prices into 2 equal quantiles. Some of the resulting groups are empty; to improve readability, specify `fill_value` as the empty string or a dash.

```
# pd.qcut() partitions entries into equally populated intervals.
>>> pd.qcut([1, 2, 5, 6, 8, 3], 2)
[(0.999, 4.0], (0.999, 4.0], (4.0, 8.0], (4.0, 8.0], (4.0, 8.0], (0.999, 4.0]]
Categories (2, interval[float64]): [(0.999, 4.0] < (4.0, 8.0]]

# Cut the ticket price into two intervals (cheap vs expensive).
>>> fare = pd.qcut(titanic["Fare"], 2)
>>> titanic.pivot_table(values="Survived",
```

```
                        index=["Sex", age], columns=[fare, "Pclass"],
                        aggfunc="count", fill_value='-')
Fare             (-0.001, 14.454]           (14.454, 512.329]
Pclass                       1.0 2.0  3.0               1.0 2.0 3.0
Sex    Age
female (0, 12]                 -   -    7                 1  13  23
       (12, 18]                -   4   23                12   4   5
       (18, 80]                -  31  101               131  54  57
male   (0, 12]                 -   -    8                 4  11  27
       (12, 18]                -   5   26                 4   5  11
       (18, 80]                8  94  350               163  56  70
```

Not surprisingly, most of the cheap tickets went to passengers in 3rd class.

Problem 4. Use the employment data from Ohio in 1999 to answer the following questions:

1. The column `Educational Attainment` contains numbers 0-46. Any number less than 39 means the person did not get any form of degree. 39-42 refers to either a high-school or associate's degree. A number greater than or equal to 43 means the person got at least a bachelor's degree. Out of these categories, which degree type is the most common among the workers in this dataset?
2. Partition the `Age` column into 6 equally-sized groups using `pd.qcut()`. Which interval has the highest average `Usual Hours Worked`?
3. Using the partitions from the first two parts, what age/degree combination has the lowest yearly salary on average?

Return the answer to each question (in order) as an `Interval`. For part three, the answer should be a tuple where the first entry is the `Interval` of the age and the second is the `Interval` of the degree.

An `Interval` is the object returned by `pd.cut()` and `pd.qcut()`. These can also be obtained from a pivot table, as in the example below.

```
>>> # Create pivot table used in last example with titanic dataset
>>> table = titanic.pivot_table(values="Survived",
                        index=[age], columns=[fare, "Pclass"],
                        aggfunc="count")
>>> # Get index of maximum interval
>>> table.sum(axis=1).idxmax()
Interval(0, 12, closed='right')
```

Problem 5. Examine the college dataset using `pivot tables` and `groupby` objects. Determine the answer to the following questions. If the answer is yes, save the answer as `True`. If the answer the no, save the answer as `False`. For the last question, save the answer as a string

giving your explanation. Return a tuple containing your answers to the questions in order.

1. Partition `perc.alumni` into evenly spaced intervals of 20% using `pd.cut()`. Does the number of both private and public universities decrease as the percentage of alumni that donate increases, as expected?

2. Partition `Grad.Rate` into evenly spaced intervals of 20% using `pd.cut()`. Is the partition with the greatest number of schools the same for private and public universities?

3. Create a column that gives the acceptance rate of each university (using columns `Accept` and `Apps`). Partition this column into evenly spaced intervals of 25% using `pd.cut()`. Is it true that the partition with the least average number of students from the top 10 percent of their high school class is the same for both private and public universities?

4. Use the same partition as part 3. The average percentage of students admitted from the top 10 percent of their high school class is very high in private universities with the lowest acceptance rates ($< 25\%$ acceptance rate). Why is this *not* a good conclusion to draw solely from this dataset? Use only the data to explain why; do not extrapolate.

15 Data Cleaning

Lab Objective: *The quality of a data analysis or model is limited by the quality of the data used. In this lab we learn techniques for cleaning data, creating features, and determining feature importance.*

Data cleaning is the process of identifying and correcting bad data. This could be data that is missing, duplicated, irrelevant, inconsistent, incorrect, in the wrong format, or otherwise does not make sense. Though it can be tedious, data cleaning is the most important step of data analysis. Without accurate and legitimate data, any results or conclusions are suspect and may be incorrect.

We will demonstrate common issues with data and how to correct them using the following dataset. It consists of family members and some basic details.

```
# Example dataset
>>> df = pd.read_csv('toy_dataset.csv')

>>> df
          Name  Age        name         DOB Marital_Status
0     John Doe   30        john  01/01/2010       Divorcee
1     Jane Doe   29        jane  12/02/1990       Divorced
2   Jill smith   40         NaN  03/04/1980        married
3   Jill smith   40        jill  03/04/1980        married
4   jack smith  100        jack    4/4/1980       marrieed
5  Jenny Smith    5         NaN  05/05/2015            NaN
6  JAmes Smith    2         NaN  20/06/2018         single
7        Rover    2         NaN  05/05/2018            NaN



   Height  Weight    Marriage_Len      Spouse
0    72.0     175               5         NaN
1     5.5     125               5    John Doe
2    64.0     120              10  Jack Smith
3    64.0     120             NaN  jack smith
4     1.8     220              10  jill smith
5   105.0      40             NaN         NaN
```

```
6    27.0      25  Not Applicable         NaN
7    36.0      50             NaN         NaN
```

Data Type

We can check the data type in Pandas using `dtype`. A `dytpe` of `object` means that the data in that column contains either strings or mixed `dtypes`. These fields should be investigated to determine if they contain mixed datatypes. In our toy example, we would expect that `Marriage_Len` is numerical, so an object `dtype` is suspicious. Looking at the data, we see that `James` has `Not Applicable`, which is a string.

```
# Check validity of data
# Check Data Types
>>> df.dtypes
Name               object
Age                 int64
name               object
DOB                object
Marital_Status     object
Height            float64
Weight              int64
Marriage_Len       object
Spouse             object
dtype: object
```

Duplicates

Duplicates can be easily identified in Pandas using the `duplicated()` function. When no parameters are passed, it returns a DataFrame of the first duplicates. We can identify rows that are duplicated in only some columns by passing in the column names. The `keep` parameter has three possible values, first, last, and False. False keeps all duplicated values, while first and last keep only the first and last instances, respectively.

```
# Display duplicated rows
>>> df[df.duplicated()]
Empty DataFrame
Columns: [Name, Age, name, DOB, Marital_Status, Height, Weight, Marriage_Len, ↩
    Spouse]
Index: []

# Display rows that have duplicates in some columns
>>> df[df.duplicated(['Name', 'DOB', 'Marital_Status'], keep=False)]
         Name  Age  name         DOB Marital_Status  Height  Weight ↩
            Marriage_Len      Spouse
2  Jill smith   40   NaN  03/04/1980        married    64.0     120          ↩
    10  Jack Smith
```

```
3  Jill smith   40  jill  03/04/1980        married    64.0     120          ↩
    NaN  jack smith
```

Range

We can check the range of values in a numeric column using the `min` and `max` attributes. If a column corresponds to the temperature in Salt Lake City, measured in degrees Farenheit, then a value over 110 or below 0 should make you suspicious, since those would be extreme values for Salt Lake City. In fact, checking the all-time temperature records for Salt Lake shows that the values in this column should never be more than 107 and never less than -30. Any values outside that range are almost certainly errors and should probably be reset to NaN, unless you have special information that allows you to impute more accurate values.

Other options for looking at the values include line plots, histograms, and boxplots. Some other useful Pandas commands for evaluating the breadth of a dataset include `df.nunique()` (which returns a series giving the name of each column and the number of unique values in each column), `pd.unique()` (which returns an array of the unique values in a series), and `value_counts()` (which counts the number of instances of each unique value in a column, like a histogram).

```
# Count the number of unique values in each column
>>> df.nunique()
Name              7
Age               6
name              4
DOB               7
Marital_Status    5
Height            7
Weight            7
Marriage_Len      3
Spouse            4
dtype: int64


# Print the unique Marital_Status values
>>> pd.unique(df['Marital_Status'])
array(['Divorcee', 'Divorced', 'married', 'marrieed', nan, 'single'],
      dtype=object)


# Count the number of each Marital_Status values
>>> df['Marital_Status'].value_counts()
Marital_Status
married     2
Divorcee    1
Divorced    1
marrieed    1
single      1
Name: count, dtype: int64
```

Missing Data

The percentage of missing data is the completeness of the data. All uncleaned data will have missing values, but datasets with large amounts of missing data, or lots of missing data in key columns, are not going to be as useful. Pandas has several functions to help identify and count missing values. In Pandas, all missing data is considered a `NaN` and does not affect the dtype of a column. `df.isna()` returns a boolean DataFrame indicating whether each value is missing. `df.notnull()` returns a boolean DataFrame with `True` where a value is not missing. Information on how to deal with missing data is described in later sections.

```
# Count number of missing data points in each column
>>> df.isna().sum()
Name              0
Age               0
name              4
DOB               0
Marital_Status    2
Height            0
Weight            0
Marriage_Len      3
Spouse            4
dtype: int64
```

Consistency

Consistency measures how cohesive the data is, both within the dataset and across multiple datasets. For example, in our toy dataset `Jack Smith` is 100 years old, but his birth year is 1980. Data is inconsistent across datasets when the data points should be the same and are different. This could be due to incorrect entries or syntax errors.

It is also important to be consistent in units of measure. Looking at the `Height` column in our dataset, we see values ranging from 1.8 to 105. This is likely the result of different units of measure. It is also important to be consistent across multiple datasets.

All features should also have a consistent type and standard formatting (like capitalization). Syntax errors should be fixed, and white space at the beginning and ends of strings should be removed. Some data might need to be padded so that it's all the same length.

Method	Description
series.str.lower()	Convert to all lower case
series.str.upper()	Convert to all upper case
series.str.strip()	Remove all leading and trailing white space
series.str.lstrip()	Remove leading white space
series.str.replace(" ","")	Remove all spaces
series.str.pad()	Pad strings

Table 15.1: Pandas String Formatting Methods

Problem 1. The `g_t_results.csv` file is a set of parent-reported scores on their child's Gifted and Talented tests. The two tests, `OLSAT` and `NNAT`, are used by NYC to determine if children are qualified for gifted programs. The OLSAT Verbal has 16 questions for Kindergarteners and 30 questions for first, second, and third graders. The NNAT has 48 questions. Each test assigns 1 point to each question asked (so there are no non integer scores). Using this dataset, answer the following questions.

1. What column has the highest number of null values and what percent of its values are null? Print the answer as a tuple with (column name, percentage). Make sure the second value is a percent.
2. List the columns that should be numeric that aren't. Print the answer as a tuple.
3. How many third graders have scores outside the valid range for the OLSAT Verbal Score? Print the answer
4. How many data values are missing (NaN)? Print the number.

Cleaning

There are many aspects and methods of cleaning; here are a few ways of doing so.

Unwanted Data

Removing unwanted data typically falls into two categories, duplicated data and irrelevant data. Irrelevant data consists of observations that don't fit the specific problem you are trying to solve or don't have enough variation to affect the model. We can drop duplicated data using the `duplicated()` function described above with `drop()` or `drop_duplicates()`.

Missing Data

Some commonly suggested methods for handling data are removing the missing data and setting the missing values to some value based on other observations. However, missing data can be informative and removing or replacing missing data erases that information. Removing missing values from a dataset might result in losing significant amounts of data or even in a less accurate model. Retaining the missing values can help increase accuracy.
We have several options to deal with missing data:

- Dropping missing data is the easiest method. Dropping rows or columns should only be done if there is less than $90\% - 95\%$ of the data available. If dropping missing data is inappropriate, you may instead choose to estimate the missing values which can be done by solving the mean, mode, median, randomly choosing from a distribution, linear regression, or hot-decking, to name a few.
- Hot-decking is when you fill in the data based on similar observations. It can be applied to numerical and categorical data, unlike many of the other options listed above. Sequential hot-decking sorts the column with missing data based on an auxiliary column and then fills in the data with the value from the next available data point. K-Nearest Neighbors can also be used to identify similar data points.

- The last option is to flag the data as missing. This retains the information from missing data and removes the missing data (by replacing it). For categorical data, simply replace the data with a new category. For numerical data, we can fill the missing data with 0, or some value that makes sense, and add an indicator variable for missing data.

```
## Replace missing data
import numpy as np

# Add an indicator column based on missing Marriage_Len
>>> df['missing_ML'] = df['Marriage_Len'].isna()

# Fill in all missing data with 0
>>> df['Marriage_Len'] = df['Marriage_Len'].fillna(0)

# Change all other NaNs to missing
>>> df = df.fillna('missing')

# Change Not Applicable row to NaNs
>>> df = df.replace('Not Applicable', np.nan)

# Drop rows with NaNs
>>> df = df.dropna()


>>> df
          Name  Age     name         DOB Marital_Status
0     John Doe   30     john  01/01/2010       Divorcee
1     Jane Doe   29     jane  12/02/1990       Divorced
2   Jill smith   40  missing  03/04/1980        married
3   Jill smith   40     jill  03/04/1980        married
4   jack smith  100     jack    4/4/1980       marrieed
5  Jenny Smith    5  missing  05/05/2015        missing
7        Rover    2  missing  05/05/2018        missing

    Height  Weight  Marriage_Len       Spouse  missing_ML
0     72.0     175             5      missing       False
1     5.5      125             5     John Doe       False
2     64.0     120            10   Jack Smith       False
3     64.0     120             0   jack smith        True
4      1.8     220            10   jill smith       False
5    105.0      40             0      missing        True
7     36.0      50             0      missing        True
```

Nonnumerical Values Misencoded as Numbers

Missing data should always be stored in a form that cannot accidentally be incorporated into the model. This is typically done by storing missing values as NaN. Some algorithms will not run on data with NaN values, in which case you may choose to fill missing data with a string `'missing'`.

Many datasets have recorded missing values with a 0 or some other number. You should verify that this does not occur in your dataset.

Categorical data are also often encoded as numerical values. These values should not be left as numbers that can be computed with. For example, postal codes are shorthand for locations, and there is no numerical meaning to the code. It makes no sense to add, subtract, or multiply postal codes, so it is important to not do so. It is good practice to convert this kind of non-numeric data into strings or other data types that cannot be computed with.

Ordinal Data

Ordinal data is data that has a meaningful order but the differences between the values aren't consistent or meaningful. For example, a survey question might ask about your level of education, with 1 being high-school graduate, 2 bachelor's degree, 3 master's degree, and 4 doctoral degree. These values are called ordinal data because it is meaningful to talk about an answer of 1 being less than an answer of 2, but the difference between 1 and 2 is not necessarily the same as the difference between 3 and 4. Subtracting, taking the average, and other algebraic methods do not make a lot of sense with ordinal data.

Problem 2. `imdb.csv` contains a small set of information about 99 movies. Valid movies for this dataset should be longer than 30 minutes long, should have a positive `imdb_score`, and have a `title_year` after 2000.

Clean the data set by doing the following in order:

1. Remove duplicate rows by dropping the first or last. Print the shape of the dataframe after removing the rows.
2. Drop all rows that contain missing data. Print the shape of the dataframe after removing the rows.
3. Remove rows that have data outside of the valid data ranges described above and explain briefly how you determined your ranges for each column.
4. Identify and drop columns with three or fewer different values. Print a tuple with the names of the columns dropped.
5. Convert the titles to all lower case.

Print the first five rows of your dataframe.

Feature Engineering

Constructing new features is called *feature engineering*. Once new features are created, we can analyze how much a model depends on each feature. Features with low importance probably do not contribute much and could potentially be removed.

Discrete Fourier transforms and wavelet decomposition often reveal important properties of data collected over time (*called time-series*), like sound, video, economic indicators, etc. In many such settings it is useful to engineer new features from a wavelet decomposition, the DFT, or some other function of the data.

Engineering for Categorical Variables

Categorical features are those that take only a finite number of values, and usually no categorical value has a numerical meaning, even if it happens to be number. For example in an election dataset, the names of the candidates in the race are categorical, and there is no numerical meaning (neither ordering nor size) to numbers assigned to candidates based soley on their names.
Consider the following election data.

Ballot number	For Governor	For President
001	Herbert	Romney
002	Cooke	Romney
003	Cooke	Obama
004	Herbert	Romney
005	Herbert	Romney
006	Cooke	Stein

A common mistake occurs when someone assigns a number to each categorical entry (say 1 for Cooke, 2 for Herbert, 3 for Romney, etc.). While this assignment is not, in itself, inherently incorrect, it is incorrect to use the value of this number in a statistical model. Whenever you encounter categorical data that is encoded numerically like this, immediately change it to non-numerical form ("Cooke," "Herbert," "Romney,"...) or apply *one-hot encoding* or *dummy variable encoding*. [1] To do this construct a new feature for every possible value of the categorical variable, and assign the value 1 to that feature if the variable takes that value and zero otherwise. Pandas makes one-hot encoding simple:

```
# one-hot encoding
df = pd.get_dummies(df, columns=['For President'])
```

The previous dataset, when the presidential race is one-hot encoded, becomes

Ballot number	Governor	Romney	Obama	Stein
001	Herbert	1	0	0
002	Cooke	1	0	0
003	Cooke	0	1	0
004	Herbert	1	0	0
005	Herbert	1	0	0
006	Cooke	0	0	1

Note that the sum of the terms of the one-hot encoding in each row is 1, corresponding to the fact that every ballot had exactly one presidential candidate.
When the gubernatorial race is also one-hot encoded, this becomes

Ballot number	Cooke	Herbert	Romney	Obama	Stein
001	0	1	1	0	0
002	1	0	1	0	0
003	1	0	0	1	0
004	0	1	1	0	0
005	0	1	1	0	0
006	1	0	0	0	1

[1] Yes, these are silly names, but they are the most common names for it. Unfortunately, it is probably too late to change these now.

Now the sum of the terms of the one-hot encodings in each row is 2, corresponding to the fact that every ballot had two names—one gubernatorial candidate and one presidential candidate.

Summing the columns of the one-hot-encoded data gives the total number of votes for the candidate of that column. So the numerical values in the one-hot encodings are actually numerically meaningful, and summing the entries gives meaningful information. One-hot encoding also avoids the pitfalls of incorrectly using numerical proxies for categorical data.

The main disadvantage of one-hot encoding is that it is an inefficient representation of the data. If there are C categories and n datapoints, a one-hot encoding takes an $n \times 1$-dimensional feature and turns it into an $n \times C$ sparse matrix. But there are ways to store these data efficiently and still maintain the benefits of the one-hot encoding.

Achtung!

When performing linear regression, it is good practice to add a constant column to your dataset and to remove one column of the one-hot encoding of each categorical variable. This is done because the constant column is a linear combination of the one-hot encoded columns causing the matrix to fail to be invertible and can cause identifiability problems.

The standard way to deal with this is to remove one column of the one-hot embedding for each categorical variable. For example, with the elections dataset above, we could remove the Cooke (governor variable) and Romney (presidential variable) columns. Doing that means that in the new dataset a row sum of 0 corresponds to a ballot with a vote for Cooke and a vote for Romney, while a 1 in any column indicates how the ballot differed from the base choice of Cooke and Romney.

When using pandas, you can drop the first column of a one-hot encoding by passing in `drop_first=True`.

Problem 3. Load `housing.csv` into a dataframe with `index_col=0`. Descriptions of the features are in `housing_data_description.txt` for your convenience. The goal is to construct a regression model that predicts SalePrice using the other features of the dataset. Do this as follows:

1. Identify and handle the missing data. Hint: Dropping every row with some missing data is not a good choice because it gives you an empty dataframe. What can you do instead?
2. Create two new features:
   (a) `Remodeled`: Whether or not a house has been remodeled with a `Y` if it has been remodeled, or a `N` if it has not.
   (b) `TotalPorch`: Using the 5 different porch/deck columns, create a new column that provides the total square footage of all the decks and porches for each house.
3. Identify the variable with nonnumerical values that are misencoded as numbers. One-hot encode it. (Hint: don't forget to remove one of the encoded columns to prevent collinearity with the constant column).
4. Add a constant column to the dataframe.

5. Save a copy of the dataframe.
6. Choose four categorical features that seem very important in predicting SalePrice. One-hot encode these features, and remove all other categorical features.
7. Run an OLS (Ordinary Least Squares) regression on your model.

Print a list of the ten features that have the highest coefficients in your model. Print the summary for the dataset as well.

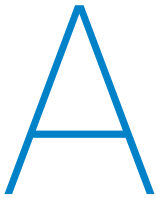
To run an OLS model in python, use the following code.

```
import statsmodels.api as sm

# In our case, y is SalesPrice, and X is the rest of the dataset.
>>> results = sm.OLS(y, X).fit()

# Print the summary
>>> results.summary()

# Convert the summary table to a dataframe
>>> results_as_html = results.summary().tables[1].as_html()
>>> result_df = pd.read_html(results_as_html, header=0, index_col=0)[0]
```

Problem 4. Using the copy of the dataframe you created in Problem 3, one-hot encode all the categorical variables. Print the shape of your database, and Run OLS.
Print the ten features that have the highest coefficient in your model and the summary. Write a couple of sentences discussing which model is better and why.

Part I

Appendices



NumPy Visual Guide

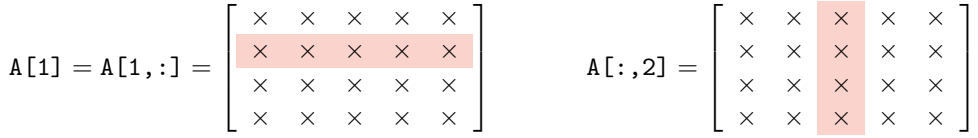
Lab Objective: *NumPy operations can be difficult to visualize, but the concepts are straightforward. This appendix provides visual demonstrations of how NumPy arrays are used with slicing syntax, stacking, broadcasting, and axis-specific operations. Though these visualizations are for 1- or 2-dimensional arrays, the concepts can be extended to n-dimensional arrays.*

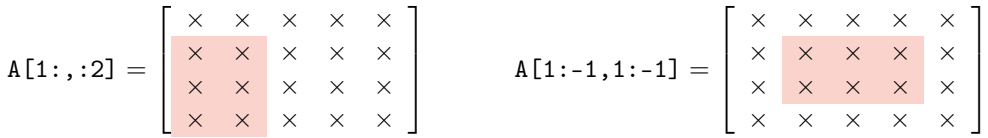
Data Access

The entries of a 2-D array are the rows of the matrix (as 1-D arrays). To access a single entry, enter the row index, a comma, and the column index. Remember that indexing begins with 0.

Slicing

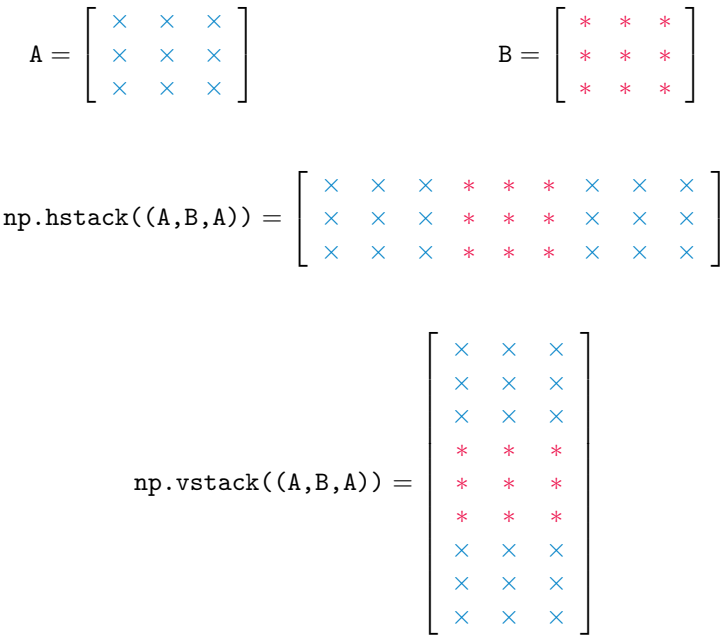
A lone colon extracts an entire row or column from a 2-D array. The syntax `[a:b]` can be read as "the ath entry up to (but not including) the bth entry." Similarly, `[a:]` means "the ath entry to the end" and `[:b]` means "everything up to (but not including) the bth entry."



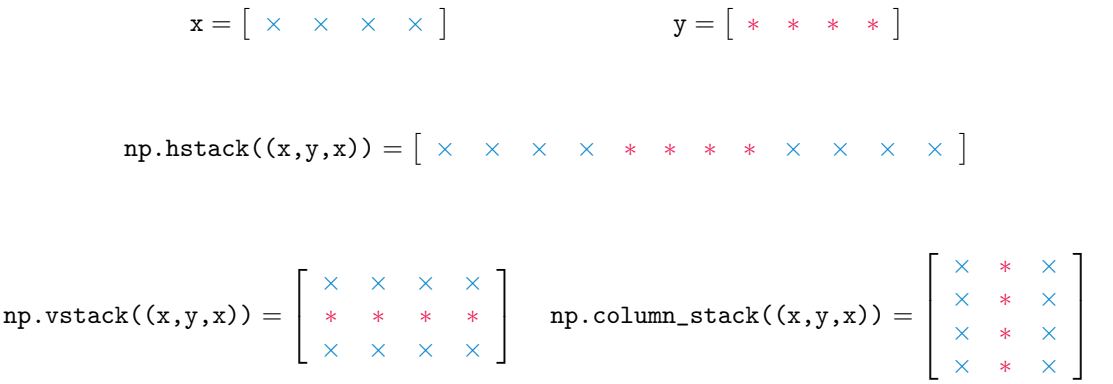


Stacking

`np.hstack()` stacks sequence of arrays horizontally and `np.vstack()` stacks a sequence of arrays vertically.



Because 1-D arrays are flat, `np.hstack()` concatenates 1-D arrays and `np.vstack()` stacks them vertically. To make several 1-D arrays into the columns of a 2-D array, use `np.column_stack()`.



The functions `np.concatenate()` and `np.stack()` are more general versions of `np.hstack()` and `np.vstack()`, and `np.row_stack()` is an alias for `np.vstack()`.

Broadcasting

NumPy automatically aligns arrays for component-wise operations whenever possible. See `http://docs.scipy.org/doc/numpy/user/basics.broadcasting.html` for more in-depth examples and broadcasting rules.

$$\texttt{A} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} \qquad \texttt{x} = \begin{bmatrix} 10 & 20 & 30 \end{bmatrix}$$

$$\texttt{A + x} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 11 & 22 & 33 \\ 11 & 22 & 33 \end{bmatrix}$$

$$\texttt{A + x.reshape((1,-1))} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} = \begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{bmatrix}$$

Operations along an Axis

Most array methods have an `axis` argument that allows an operation to be done along a given axis. To compute the sum of each column, use `axis=0`; to compute the sum of each row, use `axis=1`.

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

$$\texttt{A.sum(axis=0)} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 12 & 16 \end{bmatrix}$$

$$\texttt{A.sum(axis=1)} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 10 & 10 & 10 & 10 \end{bmatrix}$$

B Matplotlib Syntax and Customization Guide

Lab Objective: *The documentation for Matplotlib can be a little difficult to maneuver and basic information is sometimes difficult to find. This appendix condenses and demonstrates some of the more applicable and useful information on plot customizations. It is not intended to be read all at once, but rather to be used as a reference when needed. For an interative introduction to Matplotlib, see the Introduction to Matplotlib lab in Python Essentials. For more details on any specific function, refer to the Matplotlib documentation at* `https://matplotlib.org/`.

Matplotlib Interface

Matplotlib plots are made in a `Figure` object that contains one or more `Axes`, which themselves contain the graphical plotting data. Matplotlib provides two ways to create plots:

1. Call plotting functions directly from the module, such as `plt.plot()`. This will create the plot on whichever `Axes` is currently active.

2. Call plotting functions from an `Axes` object, such as `ax.plot()`. This is particularly useful for complicated plots and for animations.

Table B.1 contains a summary of functions that are used for managing `Figure` and `Axes` objects.

Function	Description
`add_subplot()`	Add a single subplot to the current figure
`axes()`	Add an axes to the current figure
`clf()`	Clear the current figure
`figure()`	Create a new figure or grab an existing figure
`gca()`	Get the current axes
`gcf()`	Get the current figure
`subplot()`	Add a single subplot to the current figure
`subplots()`	Create a figure and add several subplots to it

Table B.1: Basic functions for managing plots.

`Axes` objects are usually managed through the functions `plt.subplot()` and `plt.subplots()`. The function `subplot()` is used as `plt.subplot(nrows, ncols, plot_number)`. Note that if the inputs for `plt.subplot()` are all integers, the commas between the entries can be omitted. For example, `plt.subplot(3,2,2)` can be shortened to `plt.subplot(322)`.

The function `subplots()` is used as `plt.subplots(nrows, ncols)`, and returns a `Figure` object and an array of `Axes`. This array has the shape `(nrows, ncols)`, and can be accessed as any other array. Figure B.1 demonstrates the layout and indexing of subplots.

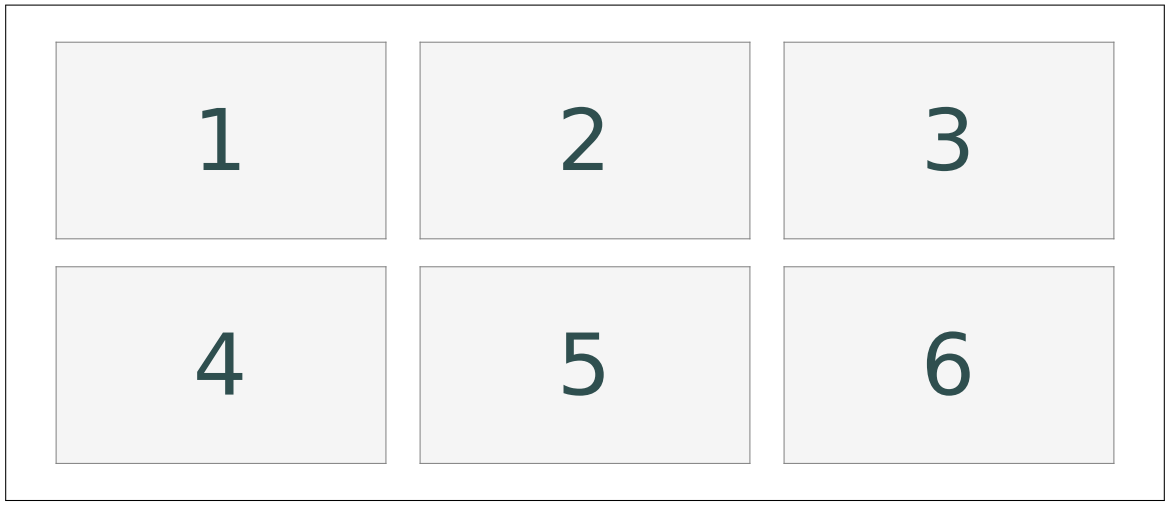


Figure B.1: The layout of subplots with `plt.subplot(2,3,i)` (2 rows, 3 columns), where `i` is the index pictured above. The outer border is the figure that the axes belong to.

The following example demonstrates three equivalent ways of producing a figure with two subplots, arranged next to each other in one row:

```
>>> x = np.linspace(-5, 5, 100)

# 1. Use plt.subplot() to switch the current axes.
>>> plt.subplot(121)
>>> plt.plot(x, 2*x)
>>> plt.subplot(122)
>>> plt.plot(x, x**2)

# 2. Use plt.subplot() to explicitly grab the two subplot axes.
>>> ax1 = plt.subplot(121)
>>> ax1.plot(x, 2*x)
>>> ax2 = plt.subplot(122)
>>> ax2.plot(x, x**2)

# 3. Use plt.subplots() to get the figure and all subplots simultaneously.
>>> fig, axes = plt.subplots(1, 2)
>>> axes[0].plot(x, 2*x)
>>> axes[1].plot(x, x**2)
```

ACHTUNG!

Be careful not to mix up the following similarly-named functions:

1. `plt.axes()` creates a new place to draw on the figure, while `plt.axis()` or `ax.axis()` sets properties of the x- and y-axis in the current axes, such as the x and y limits.
2. `plt.subplot()` (singular) returns a single subplot belonging to the current figure, while `plt.subplots()` (plural) creates a new figure and adds a collection of subplots to it.

Plot Customization

Styles

Matplotlib has a number of built-in styles that can be used to set the default appearance of plots. These can be used via the function `plt.style.use()`; for instance, `plt.style.use("seaborn")` will have Matplotlib use the "seaborn" style for all plots created afterwards. A list of built-in styles can be found at
`https://matplotlib.org/stable/gallery/style_sheets/style_sheets_reference.html`.
The style can also be changed only temporarily using `plt.style.context()` along with a `with` block:

```
with plt.style.context('dark_background'):
    # Any plots created here use the new style
    plt.subplot(1,2,1)
    plt.plot(x, y)
    # ...
# Plots created here are unaffected
plt.subplot(1,2,2)
plt.plot(x, y)
```

Plot layout

Axis properties

Table B.2 gives an overview of some of the functions that may be used to configure the axes of a plot.
The functions `xlim()`, `ylim()`, and `axis()` are used to set one or both of the x and y ranges of the plot. `xlim()` and `ylim()` each accept two arguments, the lower and upper bounds, or a single list of those two numbers. `axis()` accepts a single list consisting, in order, of
`xmin, xmax, ymin, ymax`. Passing `None` instead of one of the numbers to any of these functions will make it not change the corresponding value from what it was. Each of these functions can also be called without any arguments, in which case it will return the current bounds. Note that `axis()` can also be called directly on an `Axes` object, while `xlim()` and `ylim()` cannot.
`axis()` also can be called with a string as its argument, which has several options. The most common is `axis('equal')`, which makes the scale of the x- and y-scales equal (i.e. makes circles circular).

Function	Description
`axis()`	set the x- and y-limits of the plot
`grid()`	add gridlines
`xlim()`	set the limits of the x-axis
`ylim()`	set the limits of the y-axis
`xticks()`	set the location of the tick marks on the x-axis
`yticks()`	set the location of the tick marks on the y-axis
`xscale()`	set the scale type to use on the x-axis
`yscale()`	set the scale type to use on the y-axis
`ax.spines[side].set_position()`	set the location of the given spine
`ax.spines[side].set_color()`	set the color of the given spine
`ax.spines[side].set_visible()`	set whether a spine is visible

Table B.2: Some functions for changing axis properties. `ax` is an `Axes` object.

To use a logarithmic scale on an axis, the functions `xscale("log")` and `yscale("log")` can be used.

The functions `xticks()` and `yticks()` accept a list of tick positions, which the ticks on the corresponding axis are set to. Generally, this works the best when used with `np.linspace()`. This function also optionally accepts a second argument of a list of labels for the ticks. If called with no arguments, the function returns a list of the current tick positions and labels instead.

The spines of a Matplotlib plot are the black border lines around the plot, with the left and bottom ones also being used as the axis lines. To access the spines of a plot, call `ax.spines[side]`, where `ax` is an `Axes` object and `side` is `'top'`, `'bottom'`, `'left'`, or `'right'`. Then, functions can be called on the `Spine` object to configure it.

The function `spine.set_position()` has several ways to specify the position. The two simplest are with the arguments `'center'` and `'zero'`, which place the spine in the center of the subplot or at an x- or y-coordinate of zero, respectively. The others are a passed as a tuple `(position_type, amount)`:

- `'data'`: place the spine at an x- or y-coordinate equal to `amount`.
- `'axes'`: place the spine at the specified `Axes` coordinate, where 0 corresponds to the bottom or left of the subplot, and 1 corresponds to the top or right edge of the subplot.
- `'outward'`: places the spine `amount` pixels outward from the edge of the plot area. A negative value can be used to move it inwards instead.

`spine.set_color()` accepts any of the color formats Matplotlib supports. Alternately, using `set_color('none')` will make the spine not be visible. `spine.set_visible()` can also be used for this purpose.

The following example adjusts the ticks and spine positions to improve the readability of a plot of $\sin(x)$. The result is shown in Figure B.2.

```
>>> x = np.linspace(0,2*np.pi,150)
>>> plt.plot(x, np.sin(x))
>>> plt.title(r"$y=\sin(x)$")

#Set the ticks to multiples of pi/2, make nice labels
>>> ticks = np.pi / 2 * np.array([0,1,2,3,4])
```

```
>>> tick_labels = ["$0$", r"$\frac{\pi}{2}$", r"$\pi$", r"$\frac{3\pi}{2}$",
...                 r"$2\pi$"]
>>> plt.xticks(ticks, tick_labels)

#Move the bottom spine to zero, remove the top and right ones
>>> ax = plt.gca()
>>> ax.spines['bottom'].set_position('zero')
>>> ax.spines['right'].set_color('none')
>>> ax.spines['top'].set_color('none')

>>> plt.show()
```

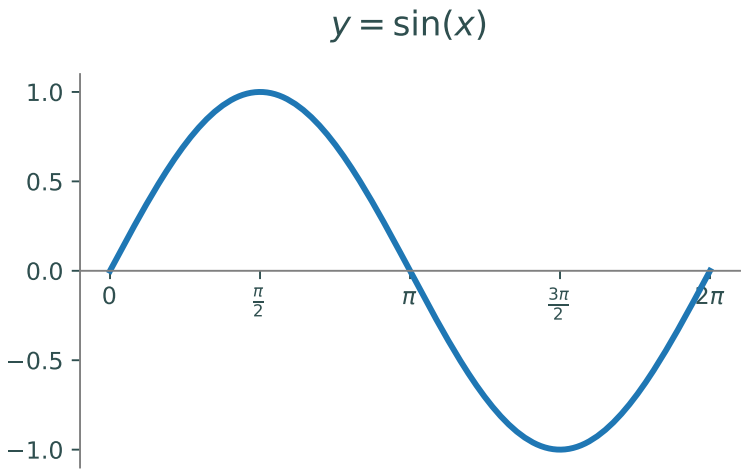


Figure B.2: Plot of $y = \sin(x)$ with axes modified for clarity

Plot Layout

The position and spacing of all subplots within a figure can be modified using the function `plt.subplots_adjust()`. This function accepts up to six keyword arguments that change different aspects of the spacing. `left`, `right`, `top`, and `bottom` are used to adjust the rectangle around all of the subplots. In the coordinates used, 0 corresponds to the bottom or left edge of the figure, and 1 corresponds to the top or right edge of the figure. `hspace` and `wspace` set the vertical and horizontal spacing, respectively, between subplots. The units for these are in fractions of the average height and width of all subplots in the figure. If more fine control is desired, the position of individual `Axes` objects can also be changed using `ax.get_position()` and `ax.set_position()`. The size of the figure can be configured using the `figsize` argument when creating a figure:

```
>>> plt.figure(figsize=(12,8))
```

Note that many environments will scale the figure to fill the available space. Even so, changing the figure size can still be used to change the aspect ratio as well as the relative size of plot elements. The following example uses `subplots_adjust()` to create space for a legend outside of the plotting space. The result is shown in Figure B.3.

```
#Generate data
>>> x1 = np.random.normal(-1, 1.0, size=60)
>>> y1 = np.random.normal(-1, 1.5, size=60)
>>> x2 = np.random.normal(2.0, 1.0, size=60)
>>> y2 = np.random.normal(-1.5, 1.5, size=60)
>>> x3 = np.random.normal(0.5, 1.5, size=60)
>>> y3 = np.random.normal(2.5, 1.5, size=60)

#Make the figure wider
>>> fig = plt.figure(figsize=(5,3))

#Plot the data
>>> plt.plot(x1, y1, 'r.', label="Dataset 1")
>>> plt.plot(x2, y2, 'g.', label="Dataset 2")
>>> plt.plot(x3, y3, 'b.', label="Dataset 3")

#Create a legend to the left of the plot
>>> lspace = 0.35
>>> plt.subplots_adjust(left=lspace)
#Put the legend at the left edge of the figure
>>> plt.legend(loc=(-lspace/(1-lspace),0.6))
>>> plt.show()
```

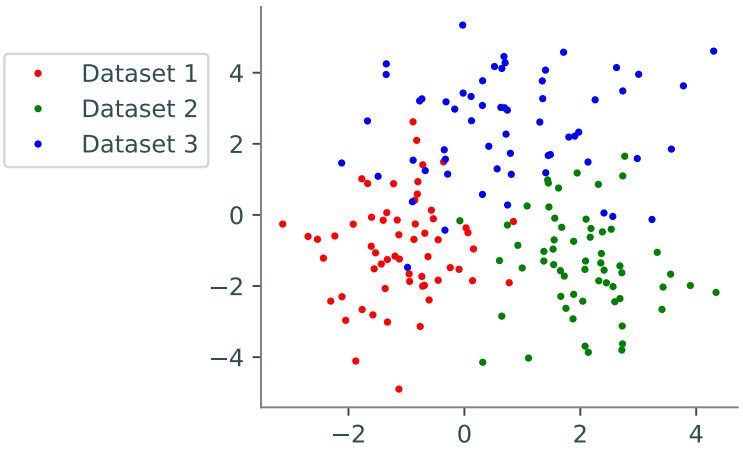


Figure B.3: Example of repositioning axes.

Colors

The color that a plotting function uses is specified by either the `c` or `color` keyword arguments; for most functions, these can be used interchangeably. There are many ways to specific colors. The most simple is to use one of the basic colors, listed in Table B.3. Colors can also be specified using an RGB tuple such as `(0.0, 0.4, 1.0)`, a hex string such as `"0000FF"`, or a CSS color name like `"DarkOliveGreen"` or `"FireBrick"`. A full list of named colors that Matplotlib supports can be found at `https://matplotlib.org/stable/gallery/color/named_colors.html`. If no color is specified for a plot, Matplotlib automatically assigns it one from the default color cycle.

Code	Color
`'b'`	blue
`'g'`	green
`'r'`	red
`'c'`	cyan
`'m'`	magenta

Code	Color
`'y'`	yellow
`'k'`	black
`'w'`	white
`'C0'` - `'C9'`	Default colors

Table B.3: Basic colors available in Matplotlib

Plotting functions also accept an `alpha` keyword argument, which can be used to set the transparency. A value of `1.0` corresponds to fully opaque, and `0.0` corresponds to fully transparent.

The following example demonstrates different ways of specifying colors:

```
#Using a basic color
>>> plt.plot(x, y, 'r')
#Using a hexadecimal string
>>> plt.plot(x, y, color='FF0080')
#Using an RGB tuple
>>> plt.plot(x, y, color=(1, 0.5, 0))
#Using a named color
>>> plt.plot(x, y, color='navy')
```

Colormaps

Certain plotting functions, such as heatmaps and contour plots, accept a colormap rather than a single color. A full list of colormaps available in Matplotlib can be found at `https://matplotlib.org/stable/gallery/color/colormap_reference.html`. Some of the more commonly used ones are `"viridis"`, `"magma"`, and `"coolwarm"`. A colorbar can be added by calling `plt.colorbar()` after creating the plot.

Sometimes, using a logarithmic scale for the coloring is more informative. To do this, pass a `matplotlib.colors.LogNorm` object as the `norm` keyword argument:

```
# Create a heatmap with logarithmic color scaling
>>> from matplotlib.colors import LogNorm
>>> plt.pcolormesh(X, Y, Z, cmap='viridis', norm=LogNorm())
```

Function	Description	Usage
`annotate()`	adds a commentary at a given point on the plot	annotate('text',(x,y))
`arrow()`	draws an arrow from a given point on the plot	arrow(x,y,dx,dy)
`colorbar()`	Create a colorbar	colorbar()
`legend()`	Place a legend in the plot	legend(loc='best')
`text()`	Add text at a given position on the plot	text(x,y,'text')
`title()`	Add a title to the plot	title('text')
`suptitle()`	Add a title to the figure	suptitle('text')
`xlabel()`	Add a label to the x-axis	xlabel('text')
`ylabel()`	Add a label to the y-axis	ylabel('text')

Table B.4: Text and annotation functions in Matplotlib

Text and Annotations

Matplotlib has several ways to add text and other annotations to a plot, some of which are listed in Table B.4. The color and size of the text in most of these functions can be adjusted with the `color` and `fontsize` keyword arguments.

Matplotlib also supports formatting text with LaTeX, a system for creating technical documents.[1] To do so, use an `r` before the string quotation mark and surround the text with dollar signs. This is particularly useful when the text contains a mathematical expression. For example, the following line of code will make the title of the plot be $\frac{1}{2}\sin(x^2)$:

```
>>> plt.title(r"$\frac{1}{2}\sin(x^2)$")
```

The function `legend()` can be used to add a legend to a plot. Its optional `loc` keyword argument specifies where to place the legend within the subplot. It defaults to `'best'`, which will cause Matplotlib to place it in whichever location overlaps with the fewest drawn objects. The other locations this function accepts are `'upper right'`, `'upper left'`, `'lower left'`, `'lower right'`, `'center left'`, `'center right'`, `'lower center'`, `'upper center'`, and `'center'`. Alternately, a tuple of `(x,y)` can be passed as this argument, and the bottom-left corner of the legend will be placed at that location. The point (0,0) corresponds to the bottom-left of the current subplot, and (1,1) corresponds to the top-right. This can be used to place the legend outside of the subplot, although care should be taken that it does not go outside the figure, which may require manually repositioning the subplots.

The labels the legend uses for each curve or scatterplot are specified with the `label` keyword argument when plotting the object. Note that `legend()` can also be called with non-keyword arguments to set the labels, although it is less confusing to set them when plotting.

The following example demonstrates creating a legend:

```
>>> x = np.linspace(0,2*np.pi,250)

# Plot sin(x), cos(x), and -sin(x)
# The label argument will be used as its label in the legend.
>>> plt.plot(x, np.sin(x), 'r', label=r'$\sin(x)$')
>>> plt.plot(x, np.cos(x), 'g', label=r'$\cos(x)$')
>>> plt.plot(x, -np.sin(x), 'b', label=r'$-\sin(x)$')
```

[1] See `http://www.latex-project.org/` for more information.

```
# Create the legend
>>> plt.legend()
```

Line and marker styles

Matplotlib supports a large number of line and marker styles for line and scatter plots, which are listed in Table B.5.

character	description	character	description
-	solid line style	3	tri_left marker
--	dashed line style	4	tri_right marker
-.	dash-dot line style	s	square marker
:	dotted line style	p	pentagon marker
.	point marker	*	star marker
,	pixel marker	h	hexagon1 marker
o	circle marker	H	hexagon2 marker
v	triangle_down marker	+	plus marker
^	triangle_up marker	x	x marker
<	triangle_left marker	D	diamond marker
>	triangle_right marker	d	thin_diamond marker
1	tri_down marker	|	vline marker
2	tri_up marker	_	hline marker

Table B.5: Available line and marker styles in Maplotlib.

The function `plot()` has several ways to specify this argument; the simplest is to pass it as the third positional argument. The `marker` and `linestyle` keyword arguments can also be used. The size of these can be modified using `markersize` and `linewidth`. Note that by specifying a marker style but no line style, `plot()` can be used to make a scatter plot. It is also possible to use both a marker style and a line style. To set the marker using `scatter()`, use the `marker` keyword argument, with `s` being used to change the size.

The following code demonstrates specifying marker and line styles. The results are shown in Figure B.4.

```
#Use dashed lines:
>>> plt.plot(x, y, '--')
#Use only dots:
>>> plt.plot(x, y, '.')
#Use dots with a normal line:
>>> plt.plot(x, y, '.-')
#scatter() uses the marker keyword:
>>> plt.scatter(x, y, marker='+')

#With plot(), the color to use can also be specified in the same string.
#Order usually doesn't matter.
#Use red dots:
>>> plt.plot(x, y, '.r')
```

```
#Equivalent:
>>> plt.plot(x, y, 'r.')

#To change the size:
>>> plt.plot(x, y, 'v-', linewidth=1, markersize=15)
>>> plt.scatter(x, y, marker='+', s=12)
```

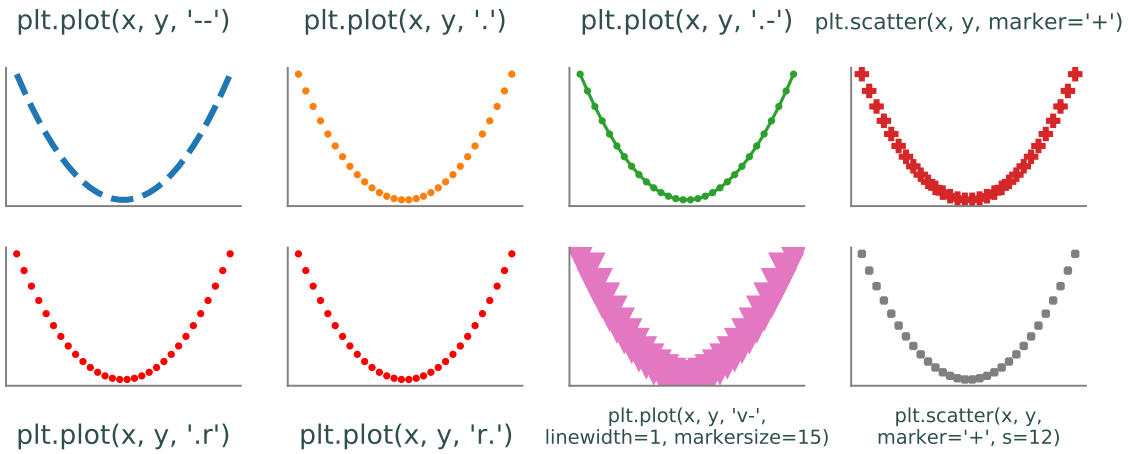


Figure B.4: Examples of setting line and marker styles.

Plot Types

Matplotlib has functions for creaing many different types of plots, many of which are listed in Table B.6. This section gives details on using certain groups of these functions.

Function	Description	Usage
`bar`	makes a bar graph	bar(x,height)
`barh`	makes a horizontal bar graph	barh(y,width)
`boxplots`	makes one or more boxplots	boxplots(data)
`contour`	makes a contour plot	contour(X,Y,Z)
`contourf`	makes a filled contour plot	contourf(X,Y,Z)
`imshow`	shows an image	imshow(image)
`fill`	plots lines with shading under the curve	fill(x,y)
`fill_between`	plots lines with shading between two given y values	fill_between(x,y1, y2=0)
`hexbin`	creates a hexbin plot	hexbin(x,y)
`hist`	plots a histogram from data	hist(data)
`pcolormesh`	makes a heatmap	pcolormesh(X,Y,Z)
`pie`	makes a pie chart	pie(x)
`plot`	plots lines and data on standard axes	plot(x,y)
`plot_surface`	plot a surface in 3-D space	plot_surface(X,Y,Z)
`polar`	plots lines and data on polar axes	polar(theta,r)
`loglog`	plots lines and data on logarithmic x and y axes	loglog(x,y)
`scatter`	plots data in a scatterplot	scatter(x,y)
`semilogx`	plots lines and data with a log scaled x axis	semilogx(x,y)
`semilogy`	plots lines and data with a log scaled y axis	semilogy(x,y)
`specgram`	makes a spectogram from data	specgram(x)
`spy`	plots the sparsity pattern of a 2D array	spy(Z)
`triplot`	plots triangulation between given points	triplot(x,y)

Table B.6: Some basic plotting functions in Matplotlib.

Line plots

Line plots, the most basic type of plot, are created with the `plot()` function. It accepts two lists of x- and y-values to plot, and optionally a third argument of a string of any combination of the color, line style, and marker style. Note that this method only works with the single-character color codes; to use other colors, use the `color` argument. By specifying only a marker style, this function can also be used to create scatterplots.

There are a number of functions that do essentially the same thing as `plot()` but also change the axis scaling, including `loglog()`, `semilogx()`, `semilogy()`, and `polar`. Each of these functions is used in the same manner as `plot()`, and has identical syntax.

Bar Plots

Bar plots are a way to graph categorical data in an effective way. They are made using the `bar()` function. The most important arguments are the first two that provide the data, `x` and `height`. The first argument is a list of values for each bar, either categorical or numerical; the second argument is a list of numerical values corresponding to the height of each bar. There are other parameters that may be included as well. The `width` argument adjusts the bar widths; this can be done by choosing a single value for all of the bars, or an array to give each bar a unique width. Further, the argument `bottom` allows one to specify where each bar begins on the y-axis. Lastly, the `align` argument can be set to 'center' or 'edge' to align as desired on the x-axis. As with all plots, you can use the `color` keyword to specify any color of your choice. If you desire to make a horizontal bar graph, the syntax follows similarly using the function `barh()`, but with argument names `y`, `width`, `height` and `align`.

Box Plots

A box plot is a way to visualize some simple statistics of a dataset. It plots the minimum, maximum, and median along with the first and third quartiles of the data. This is done by using `boxplot()` with an array of data as the argument. Matplotlib allows you to enter either a one dimensional array for a single box plot, or a 2-dimensional array where it will plot a box plot for each column of the data in the array. Box plots default to having a vertical orientation but can be easily laid out horizontally by setting `vert=False`.

Scatter and hexbin plots

Scatterplots can be created using either `plot()` or `scatter()`. Generally, it is simpler to use `plot()`, although there are some cases where `scatter()` is better. In particular, `scatter()` allows changing the color and size of individual points within a single call to the function. This is done by passing a list of colors or sizes to the `c` or `s` arguments, respectively.

Hexbin plots are an alternative to scatterplots that show the concentration of data in regions rather than the individual points. They can be created with the function `hexbin()`. Like `plot()` and `scatter()`, this function accepts two lists of x- and y-coordinates.

Heatmaps and contour plots

Heatmaps and contour plots are used to visualize 3-D surfaces and complex-valued functions on a flat space. Heatmaps are created using the `pcolormesh()` function. Contour plots are created using `contour()` or `contourf()`, with the latter creating a filled contour plot.

Each of these functions accepts the x-, y-, and z-coordinates as a mesh grid, or 2-D array. To create these, use the function `np.meshgrid()`:

```
>>> x = np.linspace(0,1,100)
>>> y = np.linspace(0,1,80)
>>> X, Y = np.meshgrid(x, y)
```

The z-coordinate can then be computed using the x and y mesh grids.

Note that each of these functions can accept a colormap, using the `cmap` parameter. These plots are sometimes more informative with a logarithmic color scale, which can be used by passing a `matplotlib.colors.LogNorm` object in the `norm` parameter of these functions.

With `pcolormesh()`, it is also necessary to pass `shading='auto'` or `shading='nearest'` to avoid a deprecation error.

The following example demonstrates creating heatmaps and contour plots, using a graph of $z = (x^2 + y)\sin(y)$. The results is shown in Figure B.5

```
>>> from matplotlib.colors import LogNorm

>>> x = np.linspace(-3,3,100)
>>> y = np.linspace(-3,3,100)
>>> X, Y = np.meshgrid(x, y)
>>> Z = (X**2+Y)*np.sin(Y)

#Heatmap
>>> plt.subplot(1,3,1)
>>> plt.pcolormesh(X, Y, Z, cmap='viridis', shading='nearest')
>>> plt.title("Heatmap")

#Contour
>>> plt.subplot(1,3,2)
>>> plt.contour(X, Y, Z, cmap='magma')
>>> plt.title("Contour plot")

#Filled contour
>>> plt.subplot(1,3,3)
>>> plt.contourf(X, Y, Z, cmap='coolwarm')
>>> plt.title("Filled contour plot")
>>> plt.colorbar()

>>> plt.show()
```

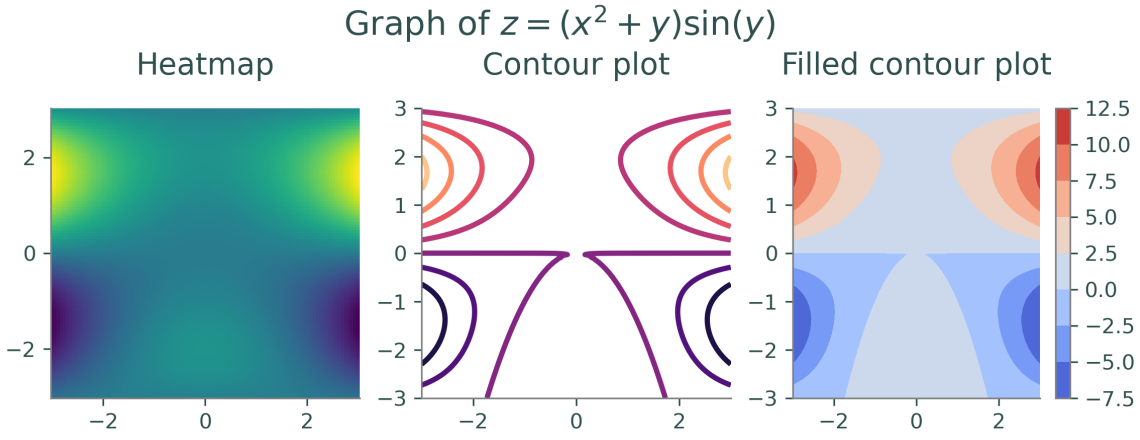


Figure B.5: Example of heatmaps and contour plots.

Showing images

The function `imshow()` is used for showing an image in a plot, and can be used on either grayscale or color images. This function accepts a 2-D $n \times m$ array for a grayscale image, or a 3-D $n \times m \times 3$ array for a color image. If using a grayscale image, you also need to specify `cmap='gray'`, or it will be colored incorrectly.

It is best to also use `axis('equal')` alongside `imshow()`, or the image will most likely be stretched. This function also works best if the images values are in the range $[0, 1]$. Some ways to load images will format their values as integers from 0 to 255, in which case the values in the image array should be scaled before using `imshow()`.

3-D Plotting

Matplotlib can be used to plot curves and surfaces in 3-D space. In order to use 3-D plotting, you need to run the following line:

```
>>> from mpl_toolkits.plot3d import Axes3D
```

The argument `projection='3d'` also must be specified when creating the subplot for the 3-D object:

```
>>> plt.subplot(1,1,1, projection='3d')
```

Curves can be plotted in 3-D space using `plot()`, by passing in three lists of x-, y-, and z-coordinates. Surfaces can be plotted using `ax.plot_surface()`. This function can be used similar to creating contour plots and heatmaps, by obtaining meshes of x- and y- coordinates from `np.meshgrid()` and using those to produce the z-axis. More generally, any three 2-D arrays of meshes corresponding to x-, y-, and z-coordinates can be used. Note that it is necessary to call this function from an Axes object.

The following example demonstrates creating 3-D plots. The results are shown in Figure B.6.

```
#Create a plot of a parametric curve
ax = plt.subplot(1,3,1, projection='3d')
t = np.linspace(0, 4*np.pi, 160)
x = np.cos(t)
y = np.sin(t)
z = t / np.pi
plt.plot(x, y, z, color='b')
plt.title("Helix curve")

#Create a surface plot from np.meshgrid
ax = plt.subplot(1,3,2, projection='3d')
x = np.linspace(-1,1,80)
y = np.linspace(-1,1,80)
X, Y = np.meshgrid(x, y)
Z = X**2 - Y**2
ax.plot_surface(X, Y, Z, color='g')
plt.title(r"Hyperboloid")
```

```
#Create a surface plot less directly
ax = plt.subplot(1,3,3, projection='3d')
theta = np.linspace(-np.pi,np.pi,80)
rho = np.linspace(-np.pi/2,np.pi/2,40)
Theta, Rho = np.meshgrid(theta, rho)
X = np.cos(Theta) * np.cos(Rho)
Y = np.sin(Theta) * np.cos(Rho)
Z = np.sin(Rho)
ax.plot_surface(X, Y, Z, color='r')
plt.title(r"Sphere")

plt.show()
```

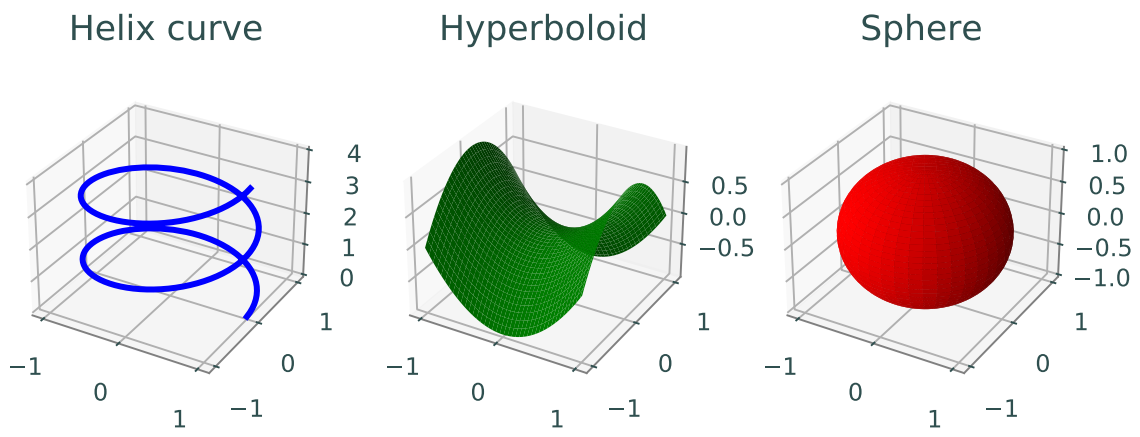


Figure B.6: Examples of 3-D plotting.

Additional Resources

rcParams

The default plotting parameters of Matplotlib can be set individually and with more fine control than styles by using `rcParams`. `rcParams` is a dictionary that can be accessed as either `plt.rcParams` or `matplotlib.rcParams`.
For instance, the resolution of plots can be changed via the `"figure.dpi"` parameter:

```
>>> plt.rcParams["figure.dpi"] = 600
```

A list of parameters that can set via `rcParams` can be found at `https://matplotlib.org/stable/api/matplotlib_configuration_api.html#matplotlib.RcParams`.

Animations

Matplotlib has capabilities for creating animated plots. The Animations lab in Volume 4 has detailed instructions on how to do so.

Matplotlib gallery and tutorials

The Matplotlib documentation has a number of tutorials, found at `https://matplotlib.org/stable/tutorials/index.html`. It also has a large gallery of examples, found at `https://matplotlib.org/stable/gallery/index.html`. Both of these are excellent sources of additional information about ways to use and customize Matplotlib.

C Using Google Colab

Lab Objective: *Google Colab is an environment similar to Jupyter Notebooks that is run remotely on Google's servers. It has some advantages over other environments, including easy package management and installation and the ability to run on a machine that doesn't have python installed. This appendix details how Colab can be used for ACME labs, as well as some issues to be aware of and relevant workarounds.*

Setup

Google Colab can be started by going to the Colab website `https://colab.research.google.com/` or by opening a `.ipynb` file from your Google Drive. When Colab starts, you can upload a `.ipynb` file by selecting the `Upload` tab and choosing the `.ipynb` file you wish to upload. Colab can also be used with `.py` files, but the process is somewhat more involved and is detailed below.

Once your file is open in Colab, it can be run and edited like a normal Jupyter notebook. To submit your lab for grading, you will need to download the file by selecting `File > Download` and then either `Download .ipynb` or `Download .py`. You will then need to move your file into the repository clone on your machine in the correct lab folder, where you can then add, commit, and push it like normal.

> ACHTUNG!
>
> Remember to keep the name of the file ***IDENTICAL*** to the original name of the lab spec! Failure to do this will cause the test driver to skip over your file and give you an automatic zero for the lab!

Using .py Files with Colab

Google Colab only works with `.ipynb` files, but many ACME labs use `.py` files. Since the former filetype has additional formatting, there isn't a way to directly import a `.py` file into Colab. The simplest way around this is to create a new file in Colab and copy-paste the lab file's contents into it. This way, it can be edited and used as a normal Jupyter notebook. Then, when you're finished, download the notebook as a `.py` file instead of a `.ipynb` file.

However, under these circumstances some care must be taken in order for your work to be graded properly. In particular, before you download your file, make sure that the following are satisfied:

- All Colab-specific code to upload files is commented out; as the packages used do not exist outside of Google Colab, the test driver will raise an error when it attempts to import them.
- There are no calls to the lab's functions, for testing purposes or otherwise, outside of a typical `if __name__=="__main__"` statement; otherwise, they will be run when the file is imported by the test driver, which will certainly cause issues.

Using Data Files

To use a data file in Colab, you must first upload it by running the following in a code cell in Colab:

```
from google.colab import files
files.upload()
```

When run, this will create a prompt for you to upload your files. After doing so, the data files can be accessed from the notebook as normal.

The best way to upload an entire folder is to first zip it, then upload the zipped folder to Colab, and then unzip it in the Colab notebook:

```
# To upload a folder called 'data', first zip it, and then upload 'data.zip'
files.upload()
# Then, it can be unzipped
!unzip data.zip
```

If the files do not need to be in the same folder, you can always simply upload multiple files at once with the usual `files.upload()` code.

Installing Packages

One advantage of Google Colab is that a very large percentage of the python packages used in the ACME labs are already pre-installed. However, there are a few packages that are not installed, and must be installed each session. Packages can be installed by running `pip` in a code cell, which is done by putting an exclamation point before the usual command. For example, to install the GeoPandas package, you would run:

```
!pip install geopandas
```